ORIENTACJA WYDANIA R3 R3-F01

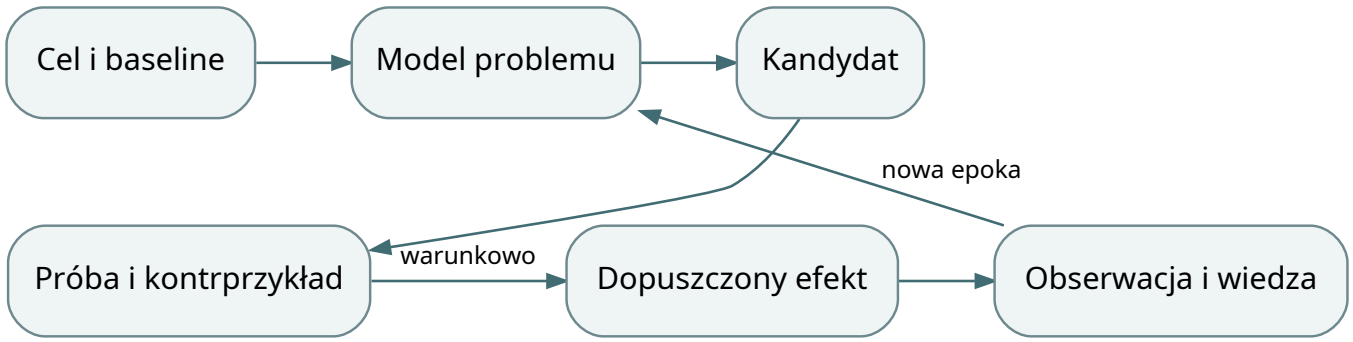
LION
Architektura w działaniu

Natura systemu wyprowadzona ze źródeł. Rozwinięcie atlasu ewolucyjnego i komunikacji holomozaikowej.

Wydanie R3 dodaje czterdziestostronicowy wstęp źródłowy oraz 102 rozwinięcia problemowe, zachowując wszystkie 137 stron otrzymanego atlasu R2. Nowe studia prowadzą od konkretnego pliku przez jego kontrakt i ograniczenie do możliwej formalizacji. Oryginały pozostają odrębne od nowych interpretacji.

Przyszłość generuje cele i warianty. Teraźniejszość rozwija problem, modele i dowody. Przeszłość zachowuje świadectwa oraz otrzymuje nowe interpretacje. Stable jest lokalnym profilem danej epoki, nie końcem ewolucji.

Stan badania ekosystemu pozostaje PARTIAL. TO-BE jest warunkowym projektem. Nie wykonano migracji repozytoriów, aktywacji misji, inferencji SaaS/LOCAL ani zmiany authority.



R3-G01 · Epizod rozliczonej zmiany. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Wydanie	R3 · 14 października 2026 · 300 stron
Zachowane	137/137 stron załączonego R2; oryginał także osadzony
Dodane	40 stron wstępu + 102 rozwinięcia + orientacja i aneksy
Nowy dowód	18 lokalnych prób trzech natywnych modułów kontraktowych

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

ORIENTACJA WYDANIA R3 R3-F02

Mapa całego tomu

Jedno wydanie, dwa rodzaje paginacji i jawna genealogia materiału.

Numery R3 w nawigacji oznaczają fizyczne strony całego PDF. Oryginalny blok R2 zachowuje własne stopki 1–137 oraz identyfikatory AT2. Nie naniesiono na niego nowych nagłówków ani numerów, aby nie zmieniać świadectwa. Strona R2 n znajduje się na fizycznej stronie R3 n+46.

Nowe strony mają identyfikatory R3-I001–I040, R3-D001–D102 i R3-A01–A15. Powiązania do AT2 są relacjami dokumentacyjnymi, nie nowymi schematami produktu. Zakładki PDF prowadzą do obu warstw oraz wszystkich rozwinięć.

Źródła i ograniczenia należy czytać razem z tezą. Liczba stron, diagramów lub referencji nie zwiększa sama siły dowodu. Modele R2 i pakiet komunikacyjny nie są niezależnymi obserwacjami względem przywoływanych przez nie źródeł.

Strony 1–6	Orientacja wydania, metoda i nawigacja
7–46	40 studiów źródłowych: od materialnych plików do natury LION
47–183	Pełny oryginalny atlas R2, bez usuwania lub zmiany stron
184–285	17 rozdziałów po 6 rozwinięć problemowych
286–300	Źródła, próby, mapowania, ograniczenia i checkpoint

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

ORIENTACJA WYDANIA R3 R3-F03

Co oznacza „analiza realnych zasobów”

Materiał pierwotny, źródło historyczne i projekt komunikacyjny pozostają rozróżnione.

W R3 przeanalizowano oba załączniki, ich modele i manifesty, dostępny historyczny pakiet 268 plików oraz wybrane ponowne odczyty GitHub. Dokładny ref master ai_platform wskazał d367e47cf4f72e450cc212908b2cdfed14ceb805. Nie jest to nowe pełne pobranie wszystkich gałęzi ani obserwacja aktywnych hostów.

Historia snapshotu ma bazę ai_platform 0e24fd48… . Cztery lokalne pliki porównano z blobami aktualnego pinned master. Pozostałych nie przemianowano automatycznie na bieżące. Wykorzystanie deklaracji z pliku historycznego jest oznaczone jako analiza takiego źródła, nie live currentness.

Pakiet komunikacji bazuje przede wszystkim na kandydacie PR #456, HEAD 3aaaf2e9… . Jego zapisy o kolejce i sesji są historycznymi obserwacjami autora pakietu. W R3 nie uruchamiano żadnego turnu i nie odnawiano tych stanów. Brakujący pełny korpus writeups i niezamknięte zależności pozostają widoczne.

Odczyt źródła	Kod, dokument, model lub artefakt o rozpoznanej tożsamości.
Inferencja R3	Uzasadniony wniosek z podanego materiału.
Projekt R3	Postulowana organizacja, test albo migracja.
Runtime	Nowy odbiór materialnego systemu: niewykonywany w R3.

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

ORIENTACJA WYDANIA R3 R3-F04

Nowy wynik: 18 prób kontraktowych

Test może przejść właśnie dlatego, że poprawnie wykazuje granicę istniejącego walidatora.

W odizolowanej lokalnej kopii wykonano 18 prób CommunicationEnvelope, SemanticScaffoldIR i SemanticRelevance. Wynik 18/18 oznacza zgodność obserwacji z oczekiwaniami instrumentu badawczego. Nie oznacza pełnej kwalifikacji bezpieczeństwa ani działania Factory.

Najważniejsze przypadki ograniczające: walidator koperty nie rozwiązuje referencji payloadu; SemanticScaffoldIR lokalnie nie wymaga przynależności końców relacji do entity_refs. Nie wykazano, że każdy wyższy konsument pomija te kontrole. To konkretne pytania do domknięcia producent–konsument.

Pozytywne kontrole obejmują kanonizację odbiorców, wykrywanie duplikatu, podmiany digestu, niedozwolonego efektu i rozszerzenia RelevanceProjection. Wyniki Filter/SHD w R1 oraz 45 testów Code Perception w R2 zachowano jako poprzednie wykonania. Nie dopisano ich do nowego licznika.

T01–T09	Koperta: null mission, kanonizacja, granice pól i digestów
T10	Referencja payloadu nie jest w tej funkcji pobierana
T11–T15	Korzenie evidence i ograniczanie projekcji
T16–T18	Granica endpoint closure i zmiana digestu scaffoldingu

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

ORIENTACJA WYDANIA R3 R3-F05

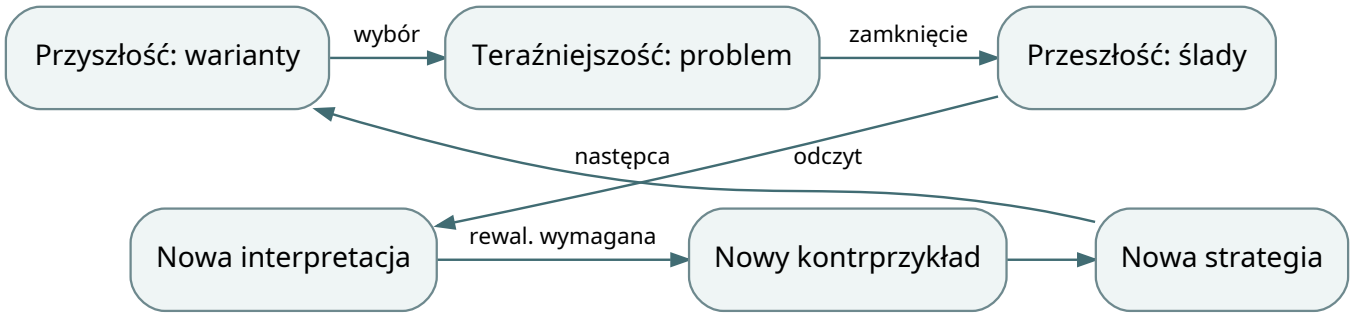
Teza, którą można już formalizować

LION organizuje rozliczalne przejście od możliwości do działania i od działania do wiedzy.

Rzeczywiste kontrakty powtarzają rozróżnienia: źródło i jego interpretacja, intencja i wykonywalność, propozycja i authority, efekt i obserwacja, pamięć robocza i przyjęta wiedza. To fundament obecnej syntezy. Nie jest to dowód, że cały obieg jest już produkcyjnie domknięty.

R3 proponuje formalizację przez epizod, typowane porty, zobowiązania dowodowe oraz trzy horyzonty informacji. Efektem ewolucji może być nowy artefakt albo lepiej uzasadniona granica starej zdolności. W obu przypadkach należy zachować oryginalne źródła i warunki przyjęcia wyniku.

Najważniejsze warunki zachowania natury systemu to brak niejawnej promocji statusów, kontrola krytycznych zależności, zachowanie UNKNOWN oraz odrębne dopuszczenie materialnych skutków. Zmiana formy organizacji jest możliwa, jeżeli nie usuwa tych rozróżnień.



R3-G03 · Trzy role informacji. Model źródłowo-analityczny; nie nowa obserwacja runtime.

```
źródło -> model problemu -> kandydat -> falsyfikacja
-> osobne dopuszczenie -> artefakt -> obserwacja
-> rekonsyliacja -> wersjonowana wiedza -> nowy problem
```

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

ORIENTACJA WYDANIA R3 R3-F06

Nawigacja rozwinięć

Każdy rozdział pogłębia część atlasu, zamiast zastępować jego poprzednie karty.

Oryginalny R2 jest zachowany w całości w części środkowej. Nowe rozwinięcia mają odnośnik do rodzica AT2 oraz własną numerację, aby dotychczasowe cytowania nie traciły znaczenia. Diagramy nowych rozdziałów są modelami analitycznymi i projektowymi, chyba że podpis wskazuje dokładny status źródłowego przepływu.

Aneks źródeł rozwiązuje identyfikatory R3-S... do plików, refów i zakresów odczytu. Pełne hashe i maszynowe mapowania są osadzone w PDF oraz dostarczone w pakiecie źródłowym.

184–189	NATURA SYSTEMU I PĘTLA EWOLUCJI
190–195	CZAS, HISTORIA I ZMIENNOŚĆ PROBLEMU
196–201	POZIOMY I ZACHOWANIE ZNACZENIA
202–207	GIT JAKO PAMIĘĆ I PERCEPCJA
208–213	AST I GRANICE PERCEPCJI KODU
214–219	ONTOLOGIA, HIPOTEZY I RELEWANCJA
220–225	PAMIĘĆ JAKO UKŁAD MAGAZYNÓW I PROJEKCJI
226–231	PROCES, FAZY I WYKONYWALNOŚĆ
232–237	FABRYKA ARTEFAKTÓW I KOMPOZYCJA
238–243	FEDERACJA I RZECZYWISTA INTEROPERACYJNOŚĆ
244–249	KOMUNIKACJA: TOŻSAMOŚĆ PRZED TRANSPORTEM
250–255	HOLOMOZAIKA JAKO ORGANIZACJA ZNACZENIA
256–261	DELIVERY, BOOTSTRAP I RECOVERY
262–267	AUTHORITY, DOPUSZCZENIE I OBSERWACJA
268–273	NAZEWNICTWO, NORMALIZACJA I CORE
274–279	TRANSFORMACJA: OD BRAKU DO PRZYROSTU
280–285	INTERFEJS, DOKUMENTACJA I NASTĘPNA ITERACJA

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

WSTĘP ŹRÓDŁOWY R3-I001

01 · Przedmiotem badania są bajty, nie nazwy wydań

Fundamentalna natura LION zaczyna się od możliwości rozpoznania, o jakim dokładnie obiekcie mówimy.

Załączony atlas ma 137 stron i 134 karty w osadzonym modelu. Jego nazwa nadal zawiera R2, ale hash całego pliku różni się od hasha wcześniej podanego w rozmowie dla 133-stronicowego nośnika. To nie przesądza o przyczynie różnicy. Ustanawia natomiast nową, jednoznaczną podstawę obecnego rozwinięcia: konkretne bajty otrzymanego załącznika, a nie domyślne wyobrażenie o jego poprzedniku.

Wydanie R3 zachowuje każdą stronę tego egzemplarza. Nowy wstęp i rozwinięcia mają osobne identyfikatory, dlatego późniejszy czytelnik może odróżnić zdanie autora R2 od interpretacji powstałej teraz. Ten sam problem występuje w repozytorium: etykieta branch nie identyfikuje na zawsze jednej zawartości, a nazwa dokumentu nie przesądza o aktualności jego twierdzeń.

Właściwa jednostka porównania zależy od pytania. Dla integralności nośnika potrzebny jest hash bajtów; dla struktury źródeł commit, tree i blob; dla sensu zmiany dodatkowo model relacji i interpretacja. Rozwój polega na dodaniu nowej wiedzy o wskazanym obiekcie, nie na podmienieniu jego tożsamości wygodniejszą nazwą. Z tej obserwacji wynika pierwsza reguła formalizacji: każda ocena musi mieć jawny przedmiot i zakres.

Rozwinięcie analizy źródłowej

Przykład odczytu jest materialny: poprzednia wiadomość wymieniała 133 strony, a dostarczony nośnik ma 137. Zamiast dopasować dokument do komunikatu, odczytujemy page_count i osadzone metadane. Zachowujemy też hash całego wejścia. Rozbieżność może wynikać z innej kopii lub późniejszego złożenia; bez właściwej genealogii nie wybieramy arbitralnie jednej przyczyny. Tak właśnie powinna działać percepcja LION: rozpoznać różnicę, zachować oba świadectwa i dopiero później ocenić jej znaczenie. Nie chodzi o kult liczb SHA, lecz o możliwość rozstrzygnięcia, czy dwaj wykonawcy analizowali ten sam materiał. Gdy przedmiot jest niejednoznaczny, najbardziej elegancka argumentacja nie daje odtwarzalnego wyniku.

```
ocena = przedmiot + wersja + zakres + dowód
```

Przedmiot R3	Dokładny załącznik R2: 137 stron, 134 karty.
Zachowanie	Oryginalne strony oraz osadzony oryginał PDF.
Ograniczenie	Numer wydania nie jest dowodem identyczności ani prawdziwości.

Źródła: R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I002

02 · Dwa pakiety, dwie granice obserwacji

Atlas i opracowanie komunikacyjne dotyczą tego samego ekosystemu, lecz nie opisują jednej atomowej chwili.

Pakiet komunikacji holomozaikowej zawiera dwa teksty, dwie wersje grafu, plik diagramów, manifest i zapis walidacji. Wersja R2 grafu wymienia 29 węzłów implementacyjnych, 49 relacji i dziewięć ścieżek. Jej główna baza implementacyjna to kandydat PR #456, podczas gdy atlas wiąże znaczną część opisu z master d367e47c. Już te metadane zabraniają automatycznego zlania obu opisów w jedno AS-IS.

Węzeł oznaczony AS_IS_R7_SOURCE reprezentuje źródło kandydata odczytane przez autora pakietu. Węzeł OBSERVED_PREVIOUS_RUNTIME odwołuje się do wcześniejszej obserwacji hosta. TARGET_NOT_IMPLEMENTED pozostaje projektem, nawet jeśli ma ten sam styl graficzny. W R3 zachowane są wszystkie te rozróżnienia, a wspólny model służy porównaniu, nie nadpisaniu statusów.

Wynik statycznej walidacji grafu potwierdza, że jego odnośniki można przejść. Nie potwierdza, że wiadomość faktycznie przeszła przez broker, DOM i zewnętrzną sesję. Dlatego integracja dokumentacyjna powinna dodawać krawędzie pochodzenia i zależności między opisami. Nie może z samej zgodności nazw wytworzyć brakującego dowodu wykonania.

Rozwinięcie analizy źródłowej

Istotny jest przypadek dwóch plików o tej samej nazwie conversation_chat.py. W master i w PR #456 mogą mieć różne bloby, zakresy metod i zależności. Diagram z pakietu komunikacji odnosi się do swojego pinned kandydata; atlas nie powinien przez to automatycznie dziedziczyć statusu deploymentu. W nowym indeksie należy przejść od nazwy modułu do repository/ref/path/blob i dopiero wtedy odtworzyć relację. Taki zabieg zachowuje wartość obu modeli: jeden wyjaśnia ogólną naturę systemu, drugi konkretny tor komunikacji. Porównanie ujawnia też, które proponowane elementy trzeba dopiero skonfrontować z kanoniczną gałęzią, zamiast przypisywać im od razu obecność w produkcie.

```
SOURCE_R7 != SOURCE_MASTER != PREVIOUS_RUNTIME != TARGET
```

Pakiet	7 członków archiwum; manifest obejmuje pięć nośników treści.
Graf komunikacji	29 węzłów / 49 relacji / 9 przebiegów.
W R3	Walidacja struktury nie jest nowym testem delivery.

Źródła: R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I003

03 · RAG przechowuje drogę do źródeł

Wersjonowana wiedza jest częścią percepcji systemu, ale nie zastępuje świata, którego dotyczy.

Pakiet lion-rag30-v1.5-r1 rozdziela instrukcję startową, routing, stan kontynuacji, indeks rekordów oraz mapę właścicieli. Manifest obejmuje 29 payloadów, a sam jest trzydziestym plikiem. Indeks 24 zachowuje lokalizatory historycznych rekordów zamiast udawać, że zawiera pełne oryginalne payloady. To celowa kompresja orientacyjna, nie utrata znaczenia pochodzenia.

Bieżący odczyt RAG_BOOTSTRAP.json nadal wskazuje preferred_release z rodziny R9 i zarazem posiada odnośniki do źródeł Semantic Cloud v1.5. Wybrany przez operatora kontekst wiedzy oraz stan integracji bootstrapu są odrębnymi relacjami. Zmiana dokumentu R3 nie daje podstaw do samoczynnego przestawienia żadnej z nich.

Naturalną funkcją RAG jest zmniejszenie kosztu znalezienia właściwego ownera, ograniczeń i wersji. Nie może natomiast udzielać aktualności uruchomionemu procesowi. Gdy agent otrzymuje fragment wiedzy, powinien wiedzieć, jakie źródło ma sprawdzić i kiedy stara informacja przestaje wystarczać. W ten sposób pamięć nie zamyka eksploracji, lecz organizuje jej następny krok.

Rozwinięcie analizy źródłowej

W tej architekturze skrót jest poprawny, jeżeli pozwala odzyskać konieczny kontekst i nie usuwa jego klasy dowodowej. Rekord mówiący „test istnieje” prowadzi do pliku testu; rekord mówiący „test przeszedł” wymaga ponadto logu i tożsamości wykonania. Indeks może zawierać oba odnośniki, ale nie powinien scalać ich w jedno ogólne zapewnienie jakości. Dlatego retrieval powinien zwracać fragment wraz ze statusem, epoką i granicą. Pytanie o aktualny stan usługi musi trafić do obserwacji runtime, nawet jeśli najbardziej podobny semantycznie tekst RAG opisuje dawną awarię. Dobra pamięć organizuje ponowny odczyt, zamiast zastępować go autorytatywnym brzmieniem.

```
wiedza wersjonowana -> routing -> źródło -> ponowna ocena
```

Pamięć orientacyjna	Kompaktowe znaczenie i identyfikowalne odnośniki.
Dowód aktualności	Odrębny odczyt odpowiedniego źródła.
Promocja	Nowe wydanie wiedzy wymaga odrębnego przyjęcia.

Źródła: R3-S029. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I004

04 · AGENTS wyznacza granice interpretacji

Instrukcja wejścia nie jest silnikiem wykonywania, ale wpływa na to, jak system rozpoznaje własne obowiązki.

AGENTS.md kieruje badacza do bootstrapu, właściwych ownerów i Process Contract Plane. Jawnie odróżnia fazę, kontrakt wykonania, capability binding, Action IR, efekt i completion. Dokument zakresowy LION/AGENTS.md zabrania cichego przepisywania historycznej pamięci. Dokument cyber_lion/AGENTS.md nakazuje usuwać przyczynę defektu, a nie osłabiać test wykrywający defekt.

Te pliki pełnią funkcję metakontraktu pracy nad projektem. Nie opisują tylko stylu kodowania: wyznaczają, które rodzaje twierdzeń wymagają jakich źródeł. Są dlatego elementami informacyjnej architektury aplikacji. Ich obecność w repozytorium nie oznacza jednak, że każdy działający worker automatycznie je odczytuje; to wymaga osobnego dowodu bootstrapu i rzeczywiście użytego kontekstu.

Formalizacja powinna zachować różnicę między normą, jej wczytaniem i egzekwowaniem. Nawet doskonale sformułowane polecenie może nie dotrzeć do konsumenta. Trwałym CORE może być więc kontrakt źródłowo rozliczanego wejścia, a nie tylko treść instrukcji. Dalszy projekt bootstrapu rozmów w pakiecie komunikacji dotyczy właśnie takiego brakującego powiązania.

Rozwinięcie analizy źródłowej

Rozważmy fragment repozytorium zawierający dawny prompt „uruchom flotę”. Dla obecnego badania jest on dowodem historycznej procedury, nie bieżącym poleceniem. Literalne przeczytanie wszystkich znaków nie wystarcza: konieczne jest przypisanie ich do roli w kontekście. AGENTS wskazuje tę granicę, natomiast techniczny consumer musi ją faktycznie zachować. W przyszłym teście warto umieścić takie instrukcje w analizowanym materiale i sprawdzić, czy pozostają danymi. To nie nowa zasada oparta wyłącznie na dobrej woli modelu. To wymaganie wobec całej ścieżki źródło–kontekst–propozycja, której efekty są ograniczane także poza interpretacją językową.

```
norma != załadowany kontekst != egzekwowanie
```

Właściciel normy	Dokładny plik i jego obowiązujący zakres.
Dowód użycia	Wiązanie kontekstu / wersji / odbiorcy.
Dowód działania	Test lub obserwacja właściwej granicy.

Źródła: R3-S004. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I005

05 · Manifest repozytorium opisuje rolę

Repozytorium jest jednostką własności źródła, nie automatycznie jedną usługą ani jednym agentem.

Manifest ai_platform wymienia równolegle EnterpriseControlPlane, AgentFoundry, SwarmPlanner, FederationRegistry i AuthorityPolicyEngine. Wskazuje płaszczyzny INF, SEM i MAND oraz eksportowane zdolności. Ta wielość ról obala prosty model „jeden katalog = jeden moduł runtime”. Ten sam zbiór źródeł może dostarczać kontrakty, generatory, serwer i instrumenty weryfikacji.

Manifest określa również semantic_exports, semantic_imports i invalidation_triggers. Dzięki nim czytelnik może zidentyfikować zależności znaczeniowe, zanim zacznie porównywać importy Python. Są to deklaracje właściciela. Dopiero właściwy adapter i jego konsument dowodzą, że eksportowana zdolność jest używana przez aplikację.

Podczas normalizacji nie należy więc rozpoczynać od fizycznego łączenia repozytoriów. Pierwszym zadaniem jest rozdzielenie odpowiedzialności, portów i kryteriów odbioru. Manifest staje się wówczas czytelnym kontraktem odkrywania. Wspólna federacja pozostaje wspólnym modelem kooperacji, nie wymogiem jednej przestrzeni adresowej lub jednego release train.

Rozwinięcie analizy źródłowej

Pole maximum_level w manifeście opisuje sufit deklarowanego zakresu, a nie gotowy grant dla każdego wywołania. Podobnie capability action.propose oznacza istnienie lub deklarację roli proponowania, nie prawo do wykonania dowolnego Action IR. Gdy dokumentację buduje się z manifestów, te różnice powinny przechodzić do grafu jako typy krawędzi. W przeciwnym przypadku reprezentacja „repozytorium ma capability” staje się błędnie „repozytorium wykonało efekt”. Kontrakt odkrywania powinien zatem kończyć się wskazaniem właściwych ownerów i konsumentów. Następna warstwa osobno ocenia dostępność implementacji, jej przyjęcie oraz uprawnienia konkretnego runtime.

```
repozytorium -> właściciel źródeł -> role -> kontrakty -> realizacje
```

Deklaracja	Role i zdolności manifestu.
Integracja	Rzeczywisty provider oraz consumer.
Wykonanie	Osobny release, konfiguracja i instancja.

Źródła: R3-S033. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I006

06 · Federacja jest strukturą współzależności

Lokalne specjalizacje zyskują wspólny sens przez jawne relacje, nie przez listę dziesięciu nazw.

Historyczny pakiet źródeł zawiera manifesty i README dziewięciu repozytoriów oraz jawny błąd pobrania archiwum writeups z powodu limitu rozmiaru. Nie wolno zmieniać tego zapisu w twierdzenie, że cały korpus badań był dostępny. Nowszy pakiet komunikacyjny opisuje writeups na osobnym pinned HEAD i dodaje referencję do badania Semantic Cloud. Każde wejście zachowuje własny zakres.

Role specjalistyczne układają się w czytelną całość: chunk-chunk dostarcza semantykę procesu, GlitchLab analizę delty, HA2D snapshot kontekstu, repozytorium hipotez opis falsyfikacji, mosaic projekcję struktury, sbom tożsamość bajtów, swarm granicę runtime, a symulator kontrolowane środowisko porównawcze. Nie wynika z tego, że każda inferencja wywołuje wszystkie te komponenty.

Fundamentalna cecha federacji to możliwość selektywnej kompozycji. Wymagany podgraf zależy od problemu. Przeniesienie jednej roli do innej implementacji jest dopuszczalnym kierunkiem polimorfizacji tylko wtedy, gdy zachowuje kontrakt, wersje i ograniczenia. Liczba repozytoriów pozostaje metryką inwentarza, nie miarą inteligencji.

Rozwinięcie analizy źródłowej

Przykładowe pytanie o zmianę semantyki pliku może potrzebować trzech perspektyw: dokładnego diffu Git, projekcji strukturalnej i oceny kontraktu konsumenta. Nie potrzebuje automatycznie uruchomienia modeli wszystkich uczestników. Jeżeli zaś zmiana dotyczy izolacji materialnego workera, sama analiza tekstowa nie wystarczy; należy przejść do jego domeny wykonania. Selektywność nie jest redukowaniem ekosystemu do wygodnego fragmentu. Oznacza świadome wyznaczenie zależnego podgrafu i zapisanie wyłączeń. Przy pytaniu o całą federację mianownik musi pozostać szeroki, a niewczytane elementy muszą być widoczne. Przy pytaniu lokalnym zbędna materializacja floty zwiększa koszty bez zwiększenia adekwatności dowodu.

```
potrzeba -> wymagany podgraf federacji, nie zawsze cała flota
```

Źródło historyczne	SOURCE_SNAPSHOT_REPORT.json w source_transfer_small.zip.
Brak	Pełne bytes writeups nie były częścią tego snapshotu.
Kierunek	Selektywny traversal producentów i konsumentów.

Źródła: R3-S023. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I007

07 · Własność semantyczna stabilizuje język

System może zmieniać implementacje, zachowując jednoznaczny adres odpowiedzi na pytanie o znaczenie.

semantic_owners.json przypisuje pojęciom primary i secondary sources. Dla architecture primary wskazuje model projekcji; dla authority wskazuje źródło uprawnień; dla SemanticScaffoldIR wskazuje jego kontrakt. Ten słownik jest mapą kompetencji informacyjnej. Rozwiązuje problem, w którym podobne słowo występuje w kilku dokumentach z różnym zakresem.

Wersja mapy w pakiecie RAG jest zamrożoną projekcją, nie drugim ownerem. Ta różnica ma znaczenie przy zmianie architektury: nowy opis nie przejmuje odpowiedzialności za kontrakt tylko dlatego, że jest nowszy i obszerniejszy. R3 korzysta z mapy do lokalizowania źródeł, zachowując natywne nazwy typów oraz scope ich własności.

Można już sformalizować pojęcie „normalizacji nazwy” jako jawnego aliasu między etykietą dokumentacyjną a istniejącym symbolem. Renaming źródła jest inną operacją: może zmienić importy, identyfikatory percepcyjne i piny. Stabilizacja języka poprzedza więc refaktoryzację katalogów, a nie zastępuje analizę jej wpływu.

Rozwinięcie analizy źródłowej

Praktyczny konflikt ownership pojawia się wtedy, gdy dwa dokumenty nadają tej samej nazwie odmienne znaczenie. Rejestr primary owner wskazuje miejsce rozstrzygnięcia, ale nie usuwa automatycznie konkurencyjnego opisu z historii. Należy zachować relację supersedes lub conflict i określić, którego konsumenta dotyczy migracja. Jeżeli peer publikuje własny ContextSnapshot, nie musi kopiować globalnej definicji ProjectReality. Powinien wskazać kontrakt lokalnego eksportu oraz mapowanie do reprezentacji konsumenta. Takie rozdzielenie umożliwia federację wielu implementacji, a jednocześnie chroni przed powstaniem kilku nieuzgodnionych właścicieli tego samego pojęcia i niejawnych zmian semantyki w jednym z repozytoriów.

```
alias dokumentu != zmiana publicznego kontraktu != migracja symbolu
```

Primary	Źródło znaczenia i odpowiedzialności.
Secondary	Kontekst, implementacje i konsumenci.
Projekt R3	Alias ma relację do natywnego ID; nie tworzy nowego ownera.

Źródła: R3-S028. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I008

08 · Źródło, model i obraz architektury

Projekcja jest wynikiem przekształcenia źródła według określonej metody.

W atlasie R2 rozdzielono piętnaście natywnych domen, dziesięć canonical flows i siedem poziomów dokumentacji. Poziomy L0–L6 opisują ziarnistość wyjaśnienia. Nie są nowymi komponentami runtime. Oryginalny FullArchitectureModel zawiera deklaratywne przypisania symboli do warstw; weryfikuje także istnienie wskazanego symbolu. Nie jest zatem wyłącznie automatycznym odkryciem dowolnej architektury.

Ta obserwacja pozwala wyjaśnić organiczny charakter projektu bez mistyfikacji. Część struktury wyłania się z analizowanych relacji, część jest hipotezą interpretacyjną, a część staje się świadomie przyjętym kontraktem. Wspólny graf powinien zachowywać te klasy. Diagram o zamkniętej geometrii może nadal przedstawiać niezamknięty system dowodowy.

R3 traktuje każdą wizualizację jako widok modelu z własnym celem. Widok roli pomija szczegóły transportu, lecz nie może usunąć granicy propozycja–efekt. Poprawne rozwinięcie dodaje implementacyjne szczegóły przy zachowaniu znaczenia rodzica. Dzięki temu kolejny dokument może być dokładniejszy bez konieczności ponownego projektowania najwyższej warstwy.

Rozwinięcie analizy źródłowej

Konstruktor modelu architektury może sprawdzać, że źródłowy symbol istnieje w zadanym pliku i że reprezentowane są wszystkie wymagane domeny. To wartościowa kontrola referencji. Nie dowodzi jednak, że opisana relacja działa na produkcyjnym hoście. Ustalony ręcznie status IMPLEMENTED w katalogu elementów jest świadectwem klasyfikacji autora i powinien być porównany z kodem oraz testem. Ten detal jest ważny dla przyszłej generacji dokumentacji: model może być spójny z własną specyfikacją, lecz nieaktualny względem świata. Potrzebny jest więc osobny proces rekonsyliacji, a nie samo ponowne renderowanie tego samego JSON do większego diagramu.

```
źródło + metoda + interpretacja -> model -> widok
```

15 domen	Natywna klasyfikacja architecture layers.
7 poziomów	Skala przybliżenia dokumentacyjnego.
3 horyzonty	Rola informacji w przyszłości, teraźniejszości i historii.

Źródła: R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I009

09 · Goal, World i System nie są jednym stanem

Cel wyznacza oczekiwanie, świat dostarcza warunków, a system ujawnia własne możliwości.

evolutionary_state.py rozdziela kontrakt celu od snapshotów świata i systemu. Dzięki temu brak zdolności wewnętrznej nie musi być mylony z zewnętrznym zdarzeniem, a zmiana celu z awarią implementacji. To dokładniejsza realizacja analogii podróży: cel pozostaje punktem odniesienia, lecz stan dróg i stan pojazdu są innymi klasami danych.

Gap jest pochodnym opisem różnicy. Nie należy traktować go jako dowolnej narracji modelu. Powinien być związany z wersją celu, source digests i warunkami, których nie spełniono. Niepewny obraz świata nie daje „pewnej luki”; niewiedza o zdolności nie jest dowodem jej nieistnienia.

Formalny model najwyższego poziomu może już przyjąć trzy niezależnie wersjonowane wejścia. Pozwala to aktualizować interpretację sytuacji bez przepisywania celu wcześniejszego epizodu. Taka separacja umożliwia porównanie, czy poprawa wyniku wynikała z lepszego rozwiązania, łatwiejszego problemu czy zmiany kryterium oceny. To podstawa uczciwej ewolucji, a nie dodatkowy estetyczny podział klas.

Rozwinięcie analizy źródłowej

Załóżmy, że celem jest uzyskanie artefaktu możliwego do ponownego użycia, a stan systemu zawiera generator specyfikacji, lecz nie posiada kwalifikowanego wykonawcy. Gap nie powinien wtedy brzmieć „brakuje całej fabryki”. Powinien wskazać właściwą brakującą zdolność i oddzielić ją od już istniejącej logiki reprezentacji. Ta precyzja wpływa na kolejność prac: najpierw można dopracować kontrakt przyszłego wykonawcy i test, później go zmaterializować. Niewiedza o runtime pozostaje osobną właściwością obserwacji. Nie należy zastępować jej pesymistycznym stwierdzeniem o globalnym braku kodu ani optymistyczną deklaracją, że wszystkie części muszą już działać.

```
Gap = interpretowana różnica (Goal, World, System)
```

Goal	Co ma zostać osiągnięte i jak rozpoznać spełnienie.
World	Warunki zewnętrzne w określonej epoce.
System	Zdolności i ograniczenia badanego układu.

Ślad implementacyjny · SystemSnapshot

snapshot_id	str; pole wymagane
observed_at	str; pole wymagane
captured_at	str; pole wymagane
epistemic_state	str; pole wymagane
repository	str; pole wymagane

Wybrane pola z evolutionary_state.py, klasa w liniach 143–188; całość deklaracji: 13 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S045. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I010

10 · CapabilityNeed wiąże brak z konkretnym celem

Potrzeba nowego modułu musi wynikać z luki, a nie z atrakcyjnej nazwy technologii.

derive_capability_needs porównuje przekazany goal_digest z gap.goal_digest. Każda wymagana zdolność musi należeć do jawnego zbioru missing_capabilities; duplikat lub pominięcie powoduje błąd. Kontrakt przenosi warunki akceptacji, falsyfikacji, obserwowalności i provenance. Powstaje zatem materiał do projektowania rozwiązania, nie polecenie uruchomienia dowolnego buildera.

Warto zauważyć, że funkcja nie wybiera jeszcze sposobu rozwiązania. Sprowadza wcześniej zadeklarowaną różnicę do typowanych wymagań. Warstwa modelowa może proponować warianty, a resolver sprawdzać istniejące zdolności, ale musi zachować powiązanie z tą potrzebą. Inaczej liczba wytworzonych artefaktów mogłaby rosnąć bez poprawy realizacji celu.

Trwałą regułą jest rozdzielenie „czego brakuje” od „jak to zbudować”. Daje ono naturalną pętlę eksperymentowania: kilka kandydatów może odpowiadać na tę samą potrzebę i być ocenianych według wspólnego kontraktu. Negatywny wynik jednego wariantu nie wymaga usunięcia potrzeby ani unieważnienia wszystkich pozostałych.

Rozwinięcie analizy źródłowej

W source-bound CapabilityNeed żądanie musi odpowiadać konkretnemu Goal i Gap. Jeżeli projektant usuwa niewygodną brakującą capability z listy wymaganej tylko po to, by kompozycja przeszła, zmienia przedmiot zadania. Nie jest to optymalizacja realizacji, lecz osłabienie postcondition. Odwrotnie, dopisanie zbędnej zdolności może powodować niepotrzebną materializację i szerszy zakres ryzyka. Dlatego formalizacja powinna porównywać zbiory wymagań i wskazywać różnicę. Taki test jest deterministyczny: nie potrzebuje głosowania modeli, aby rozpoznać, że dwa konkretne zbiory nie są równe. Wnioskowanie probabilistyczne jest potrzebne wcześniej, do zaproponowania trafnego modelu luki.

```
required capabilities == jawny zbiór braków związany z Gap
```

Ochrona tożsamości	goal_digest musi odpowiadać celowi źródłowego Gap.
Kompletność	Zakaz cichego pominięcia missing capability.
Granica	authority_effect / execution_effect / external_effect = NONE.

Ślad implementacyjny · CapabilityNeed

need_id	str; pole wymagane
gap_digest	str; pole wymagane
goal_digest	str; pole wymagane
required_capability	str; pole wymagane
required_inputs	Tuple[str, ...]; pole wymagane

Wybrane pola z capability_need.py, klasa w liniach 25–51; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S040. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I011

11 · BeanSpec jest projektem zdolności

Jednostką kompozycji jest opisana odpowiedzialność, nie sam proces modelu.

bean.py definiuje różne typy Bean, między innymi observer, builder, verifier, adapter, deterministic_service i provider. Specyfikacja zawiera purpose, wejścia, wyjścia, interfejsy, wymagane i dostarczane capabilities oraz warunki powodzenia i zatrzymania. Taki kontrakt pozwala porównywać heterogeniczne realizacje według funkcji, nie według tego, czy są napisane jako „agent”.

W tym samym pliku odróżniono BeanInstance. Instancja wiąże specyfikację z konkretną tożsamością i lifecycle, podczas gdy admission pozostaje zewnętrzne. Uruchomienie procesu nie tworzy automatycznie specyfikacji, a specyfikacja nie dowodzi istnienia procesu. Właśnie to umożliwia organizacyjny polimorfizm: jedną rolę można powierzyć innej realizacji bez zmiany jej znaczenia.

W toku stabilizacji warto utrwalić publiczne wejścia, wyjścia i niezmienniki. Nie trzeba zamrażać wszystkich algorytmów wewnętrznych. Eksperymentalny następca może zmieniać implementację przy zachowaniu kontraktu, a zmiana kontraktu wymaga jawnej migracji konsumentów. To inny model rozwoju niż nieograniczone dopisywanie kolejnych klas o zbliżonych nazwach.

Rozwinięcie analizy źródłowej

BeanSpec zawiera nie tylko inputs i outputs. Zapisuje success, stop i defer conditions, politykę pamięci, obserwowalność, zasoby, ryzyko oraz zasady zastępowania. To pokazuje, że zdolność jest kontraktem zachowania w środowisku, a nie tylko nazwą funkcji. BeanInstance dodaje rzeczywiste powiązanie runtime i generację. Jeżeli dwie instancje korzystają z tej samej specyfikacji, nadal mogą mieć różne źródła dowodów, workspace i dostępność. Dlatego registry specyfikacji i registry instancji powinny mieć odrębne znaczenie nawet przy wspólnym technicznym magazynie. W kolejnej epoce specyfikacja może przetrwać, podczas gdy konkretna instancja zostanie rozwiązana.

```
BeanSpec != BeanInstance != runtime admission
```

Specyfikacja	Znaczenie zdolności i jej wymagania.
Kandydat	Możliwa implementacja specyfikacji.
Instancja	Konkretna realizacja z własnym stanem.

Ślad implementacyjny · BeanSpec

bean_id	str; pole wymagane
bean_type	str; pole wymagane
version	str; pole wymagane
purpose	str; pole wymagane
goal_digest	str; pole wymagane

Wybrane pola z bean.py, klasa w liniach 84–170; całość deklaracji: 39 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S037. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I012

12 · Kandydat zachowuje rodowód rozwiązania

Wytworzenie pliku rozpoczyna kwalifikację, a nie kończy procesu dowodowego.

BeanCandidate oddziela opis specyfikacji od digestu implementacji i referencji weryfikacji. Kandydat może powstać przez istniejący bridge do ograniczonego buildera. Jest to postać materializacji pomysłu w źródle, ale nadal nie utożsamia się z integracją do master ani uruchomieniem w runtime.

W procesie generatywnym najcenniejsza staje się relacja między kolejnymi kandydatami. Należy wiedzieć, który kontrprzykład wywołał poprawkę, jakie wejście model faktycznie zobaczył i który fragment kontraktu pozostał bez zmian. Bez tego kolejna generacja może wyglądać na nowy sukces, choć po prostu zmieniono oczekiwany wynik testu.

R3 formalizuje tę potrzebę jako dossier wersji: kandydat, jego baseline, próby, zakres wyniku i jawny następca. Dossier jest widokiem istniejących źródeł, a nie koniecznie nową bazą. Utrzymanie odrzuconych wariantów pozwala później stwierdzić, dlaczego wybrana architektura jest taka, a nie inna, oraz kiedy dawne ograniczenie przestało obowiązywać.

Rozwinięcie analizy źródłowej

Kandydat przechowuje spec_digest, implementation_digest, builder_identity_digest oraz osobne evidence związane z przyjęciem i weryfikacją. Wypełnienie tych pól poprawnymi formatami nie potwierdza automatycznie treści zewnętrznych dowodów. Ich rozwiązywanie jest odpowiedzialnością właściwego procesu odbioru. W nowej dokumentacji warto dlatego wskazać zarówno minimalny poprawny rekord, jak i brakującego rodzica. Przykład „verified candidate” musi prowadzić do dokładnej implementacji i testu. Kiedy powstaje następca, previous_candidate_digest zachowuje genealogię. Nie powinno się zastępować starego kandydata przez nadpisanie pliku tak, aby dawna nieudana próba straciła własną tożsamość.

```
kandydat = specyfikacja + implementacja + lineage + stan weryfikacji
```

Przeszłość	Niezmienny zapis poprzedniej generacji.
Teraźniejszość	Nowe dowody dotyczące tej generacji.
Przyszłość	Następca związany z konkretną przyczyną zmiany.

Ślad implementacyjny · BeanCandidate

candidate_id	str; pole wymagane
bean_id	str; pole wymagane
spec_digest	str; pole wymagane
implementation_digest	str; pole wymagane
builder_identity_digest	str; pole wymagane

Wybrane pola z bean_candidate.py, klasa w liniach 45–90; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S038. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I013

13 · Kompozycja wymaga czegoś więcej niż podobieństwa

Zgodne znaczenie komponentów musi przejść w zgodność ich rzeczywistych portów.

CompositionContract zachowuje listę BeanBinding, z których każdy wiąże spec_digest z implementation_digest. Wynik opisuje capabilities wymagane i rozwiązane, interface_bindings, dependency_edges oraz role weryfikacji i obserwacji. Te pola pokazują, że kompozycja nie powinna być zbiorem etykiet dobranych przez podobieństwo wektorowe.

Kontrakt lokalnie kontroluje m.in. niepustość i unikalność bindings oraz obecność verifier i observer IDs w wybranym składzie. Nie wynika z tego automatyczny dowód wszystkich zachowań kompozytora. Źródło danych, algorytm wyboru i późniejszy executor pozostają oddzielnymi przedmiotami badania. Ta różnica pozwala stabilizować format wyniku bez uznawania każdego producenta tego formatu za poprawnego.

W modelu architektury krawędź pomiędzy dwiema capabilities powinna wskazywać nie tylko „zależy od”, lecz także konkretny typ wymiany i warunek, który musi przejść dalej. Polimorfizm jest wtedy zastępowalnością kontraktową. Sama zgodność pól JSON lub identyczny tytuł roli nie daje takiej gwarancji.

Rozwinięcie analizy źródłowej

CompositionRequest zawiera required_capabilities, mission_inputs, max_resource_units, max_cost_units oraz observability_quorum. CompositionContract zapisuje resolved_capabilities, interface_bindings, dependency_edges i wybrane role verifiera oraz obserwatora. Ten podział pozwala odtworzyć, jak rozwiązanie zostało wybrane względem żądania. Nie wystarczy więc sprawdzić, czy każdy Bean ma poprawny hash. Trzeba rozliczyć całość zestawu i jego porty. Przy dwóch dopuszczalnych kompozycjach można porównywać koszt, ale wynik rankingu nie ma prawa osłabiać twardego warunku. Tę zasadę należy przenosić do następnych algorytmów optymalizacji, nawet gdy sposób doboru komponentów stanie się probabilistyczny.

```
skład = wybrane spec/impl + porty + zależności + obserwowalność
```

BeanBinding	Rozdziela wersję specyfikacji i realizacji.
Role	Verifier i observer muszą należeć do wskazanego składu.
Odbiór	Wymaga także zachowania implementacji i konsumentów.

Ślad implementacyjny · CompositionRequest

composition_id	str; pole wymagane
mission_id	str; pole wymagane
goal_digest	str; pole wymagane
required_capabilities	Tuple[str, ...]; pole wymagane
external_allowed_capabilities	Tuple[str, ...]; pole wymagane

Wybrane pola z bean_composition.py, klasa w liniach 49–74; całość deklaracji: 14 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S039. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I014

14 · Mosaic organizuje czas życia współpracy

Mozaika jest ograniczoną w czasie organizacją zdolności, nie nazwą jednej niekończącej się floty.

MosaicCell posiada sekwencję FORM, ATTEST, OPERATE, OBSERVE, RECONCILE i DISSOLVE. advance_mosaic dopuszcza tylko następny krok; przeskok lub replay jest odrzucany. Późniejsze etapy wymagają wcześniejszych referencji evidence, a rozwiązanie składu wymaga powodu. Ten kod wyraża czasową odpowiedzialność organizacji.

form_mosaic sprawdza, czy zadeklarowani członkowie dokładnie odpowiadają kompozycji. Ogranicza to ciche podstawienie lub pominięcie uczestnika. Nadal jest to walidacja modelu organizacji, nie dowód, że kontener powstał, posiada właściwy lease albo wykonał zadanie. Z tego powodu obecność OPERATE w rekordzie wymaga osobnego powiązania z faktyczną obserwacją.

Organiczna architektura nie musi mieć permanentnego, maksymalnego składu. Może formować niezbędny zestaw zdolności na czas epizodu, po czym zachować wyniki i rozwiązać organizację. Trwałe są kontrakty i pamięć przyczyn, a zmienny pozostaje konkretny układ uczestników. To praktyczny sens polimorfizmu organizacyjnego.

Rozwinięcie analizy źródłowej

MosaicCell zachowuje composition_digest, listę members i oddzielne referencje attestation, operation, observation oraz reconciliation. Dzięki temu nazwa stanu nie musi być jedynym nośnikiem informacji o postępie. Przejście do OBSERVE bez właściwego operation ref albo do DISSOLVE bez rozliczenia celu powinno być ocenione w kontekście kontraktu. Wspólna topologia logiczna może zostać zrealizowana przez różne kombinacje procesów i usług. Nie należy więc liczyć komórek Mosaic jako materialnych kontenerów. Właściwy widok runtime musi pokazać odrębne przypisanie każdej roli do instancji, a archiwum zachować tę mapę po rozwiązaniu organizacji roboczej.

```
FORM -> ATTEST -> OPERATE -> OBSERVE -> RECONCILE -> DISSOLVE
```

Skład	Związany z digestem CompositionContract.
Postęp	Bez pomijania etapów i ich evidence.
Koniec	Rozwiązanie organizacji nie usuwa jej historii.

Ślad implementacyjny · MosaicCell

mosaic_id	str; pole wymagane
mission_id	str; pole wymagane
composition_digest	str; pole wymagane
members	Tuple[MosaicMember, ...]; pole wymagane
lifecycle_state	str; pole wymagane

Wybrane pola z mosaic.py, klasa w liniach 24–57; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S055. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I015

15 · MissionIntent zachowuje pochodzenie celu

Formalna intencja jest czymś więcej niż krótkim poleceniem modelowi.

MissionIntent zawiera source_envelope_ref i source_envelope_digest, goal_ref i goal_digest, a także wymagane capabilities, constraints, preconditions oraz wymagania dowodowe i aktualności. Dzięki temu można rozpoznać, do której wersji wypowiedzi i celu należy dalszy plan. Pole next_representation opisuje następny oczekiwany artefakt, nie prawo do wykonania.

Istotna granica pojawia się przy rozmowie bez misji. CommunicationEnvelope dopuszcza mission_ref=None, natomiast odczytana MissionIntent wymaga poprawnego identyfikatora mission_ref. Nie należy po cichu wpisywać fikcyjnej aktywnej misji, żeby zaspokoić typ. Rozmowa projektowa może potrzebować jawnie określonego kandydata misji lub innej zgodnej projekcji. Rozstrzygnięcie należy do ownera kontraktu i jego konsumentów.

W ten sposób formalizacja ujawnia nie tylko gotowe połączenia, ale także konkretne punkty wymagające projektu. Jednolita narracja o „wspólnej przestrzeni rozmów” nie usuwa różnicy pomiędzy swobodnym dialogiem, intencją misji i zarejestrowanym wykonaniem. R3 zachowuje ten kontrapunkt jako zależność do wyjaśnienia, a nie uzupełnia go automatycznie.

Rozwinięcie analizy źródłowej

Konkretna różnica nullable jest dobrym testem granicy pojęć. Koperta komunikacyjna może nie mieć mission_ref; MissionIntent nie może być poprawną instancją bez swojego wymaganego odniesienia. To nie błąd składni, który należy obejść magiczną nazwą „general”. To informacja, że rozmowa ad hoc i intencja misji należą do różnych etapów. Projekt przejścia powinien dopiero wtedy utworzyć mission identity, gdy rzeczywiście powstaje program pracy. Wcześniejsza rozmowa zachowuje swoją tożsamość i może zostać źródłem propozycji. Dzięki temu nie zaciera się granicy między analizą świata a aktywacją procesu oddziałującego na ten świat.

```
źródłowa koperta -> intencja związana z celem -> następna reprezentacja
```

Wymagane	evidence_requirements i currentness_requirements.
UNBOUND	Nie podstawiać automatycznie aktywnej mission identity.
Niezmiennik	authority_effect = NONE; effect = NONE.

Ślad implementacyjny · MissionIntent

intent_id	str; pole wymagane
mission_ref	str; pole wymagane
conversation_ref	str | None; pole wymagane
source_envelope_ref	str; pole wymagane
source_envelope_digest	str; pole wymagane

Wybrane pola z mission_intent.py, klasa w liniach 36–84; całość deklaracji: 21 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S053. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I016

16 · QueryPlan nadaje badaniu warunek końca

Nieograniczone poszukiwanie źródeł nie jest jeszcze dobrze zorganizowaną percepcją.

QueryPlan wiąże pytania z konkretną MissionIntent i jej digestem. Wymaga niepustych question_refs, retrieval_semantics, scope, stop_conditions i wymagań dowodowych. budget_limit musi być dodatnią liczbą całkowitą; bool jest odrzucany. To mała, ale znacząca reguła: parametr pozornie liczbowy nie może zmienić znaczenia przez właściwość języka Python.

Plan porządkuje badanie, zanim powstanie kontekst modelu. Pozwala odpowiedzieć, dlaczego analizowano dany plik, jakie źródło miało rozstrzygnąć hipotezę i co oznacza zakończenie. Samo istnienie planu nie gwarantuje jednak pełności źródeł; rzeczywisty odczyt i jego brak powinny zostać osobno rozliczone.

W LION generatywność nie polega więc wyłącznie na zwiększaniu liczby propozycji. Obejmuje również zmienianie pytań tak, aby kolejna próba lepiej rozstrzygała problem. Nowy QueryPlan powinien wskazać, które pytanie zostało zamknięte, a które otwarto po kontrprzykładzie. Zachowana wersja celu umożliwia ocenę, czy proces rzeczywiście zbliża się do rozwiązania.

Rozwinięcie analizy źródłowej

QueryPlan wiąże konkretny rodzaj pytań z budżetem i warunkami zatrzymania. Budżet ogranicza rozpoznanie, ale nie nadaje wynikowi automatycznej kompletności. Jeżeli odczyt zakończył się przed rozliczeniem wszystkich producentów, plan powinien zachować frontier. Podobnie brak trafienia w wyszukiwarce nie jest dowodem nieistnienia źródła. Można zaplanować alternatywne wyszukanie albo dokładny odczyt znanej ścieżki. W generatywnym systemie plan pytań również ewoluuje: nowy kontrargument może wymagać innego zakresu niż pierwotnie przewidziany. Zmiana planu powinna zachować rodzica i uzasadnienie, aby koszt oraz granice kolejnych iteracji pozostały odtwarzalne.

```
pytania + scope + budżet + warunek stop + pochodzenie
```

Relewancja	Wynika z pytania i zakresu, nie tylko z podobieństwa.
Zakończenie	Dowód albo jawnie rozliczona niewiedza.
Ewolucja	Nowy plan zachowuje związek z rodzicem.

Ślad implementacyjny · QueryPlan

plan_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
question_refs	Tuple[str, ...]; pole wymagane
retrieval_semantics	Tuple[str, ...]; pole wymagane

Wybrane pola z query_plan.py, klasa w liniach 25–55; całość deklaracji: 14 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S058. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I017

17 · Kontekst musi respektować czas dostępności

Informacja prawdziwa dzisiaj mogła być niedostępna modelowi podejmującemu wczorajszą decyzję.

RagContextSource rozróżnia observed_at i available_at, a walidacja odnosi je do input_cutoff. Źródło dostępne po cutoff jest odrzucane, podobnie jak available_at wcześniejsze od observed_at. RagContextEnvelope wiąże taki zestaw z QueryPlan i knowledge_release_ref. Dzięki temu reprezentacja kontekstu ma więcej treści niż sam hash listy dokumentów.

Znaczenie tego kontraktu ujawnia się przy ponownej interpretacji eksperymentu. Późniejszy kontrprzykład może zmienić ocenę starej hipotezy, ale nie może zostać wstawiony do historycznego wejścia. Bez tej granicy retrospektywna analiza przeceniałaby wiedzę poprzedniego wykonawcy albo błędnie oceniała jego decyzję.

W formalnym modelu archiwum potrzebne są dwa zapytania: „co było dostępne wtedy” i „co wiemy o tym teraz”. Oba korzystają z zachowanych źródeł, lecz mają inny cutoff. Tak rozumiana ewoluująca przeszłość nie jest cichą edycją historii. Jest powiększającym się zbiorem interpretacji związanych z tym samym świadectwem.

Rozwinięcie analizy źródłowej

RagContextEnvelope nie powinien być traktowany jako surowy worek tekstów. Jego rodzice i currentness basis pozwalają ustalić, dla jakiego pytania wybrano źródła i jakie ograniczenia przyjęto. Jeśli jeden tekst opisuje kandydata, a drugi aktualny master, połączenie bez etykiet może wytworzyć pozornie kompletną implementację, która nigdy nie istniała jako jeden stan. Poprawny kontekst zachowuje rozróżnienie epok i umożliwia odpowiedź warunkową. W tym właśnie miejscu dokumentacja pełni funkcję percepcyjną: nie tylko dostarcza danych, ale organizuje zakres poprawnych wniosków. Nie może jednak przepisać historycznej obserwacji jako bieżącego stanu hosta.

```
observed_at <= available_at <= input_cutoff
```

Wiedza wejściowa	Ograniczona czasem rzeczywistej dostępności.
Ocena późniejsza	Może konsumować późniejsze dowody jako późniejsze.
Granica	Timestamp pliku nie zastępuje całej osi czasowej.

Ślad implementacyjny · RagContextEnvelope

envelope_id	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane
knowledge_release_ref	str; pole wymagane
retrieval_time	str; pole wymagane

Wybrane pola z rag_context_envelope.py, klasa w liniach 50–81; całość deklaracji: 11 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S059. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I018

18 · Scaffold przechowuje problem, nie samą odpowiedź

Model roboczy obejmuje także konkurencyjne wyjaśnienia i elementy nierozstrzygnięte.

SemanticScaffoldIR posiada referencje hipotez, kontrhipotez, falsyfikatorów, ograniczeń, zależności, luki i potrzeb capability. Zachowuje też trzy rodzaje parent digest: intencji, planu pytań i kontekstu RAG. Ta konstrukcja jest dobrym miejscem utrwalania kształtu problemu w ewoluującej teraźniejszości.

Nowa próba R3-T16 wykazała konkretną granicę lokalnej walidacji: można uszczelnić scaffold, którego relacja wskazuje subject spoza entity_refs. Validator sprawdza kształt referencji i unikalność relacji, lecz nie pełne domknięcie wszystkich końców. Jest to fakt o tej funkcji, nie dowód obejścia całego systemu. Właściwy konsument musi rozstrzygnąć referencje lub zachować ich niewiedzę.

Wynik prowadzi do precyzyjnej formalizacji. Należy oddzielić integralność lokalnego rekordu od zgodności jego relacji z konkretnym grafem rodziców. Oba warunki mogą być potrzebne, ale nie wolno zastępować drugiego pierwszym. Dzięki temu model źródłowy odsłania dokładnie miejsce, w którym kolejny adapter powinien przenosić dowód, a nie tylko dodatkowe pola.

Rozwinięcie analizy źródłowej

R3-T16 nie jest dowodem, że grafy są bezużyteczne. Przeciwnie: pokazuje, gdzie deklarowana struktura wymaga mocniejszego związania. Można zaprojektować walidację lokalnych końców względem entity_refs i oddzielny resolver referencji zewnętrznych. Każde rozwiązanie musi rozróżniać brakujący obiekt od obiektu poprawnie istniejącego poza payloadem. Bez tego nadmiernie surowa naprawa mogłaby zniszczyć federacyjność przez wymuszenie kopiowania wszystkich encji. Odbiór powinien więc zawierać oba przypadki: rzeczywisty dangling ref ma zostać zatrzymany, a prawidłowa zewnętrzna referencja zachowana po odrębnym potwierdzeniu. To przykład powstawania dokładniejszej logiki przez rozstrzygający kontrprzykład.

```
sealed rekord != rozstrzygnięte zewnętrzne relacje
```

Nowy wynik	R3-T16: dopuszczony niezwiązany koniec relacji.
Zachowane	unknown_refs wpływają na payload i digest — R3-T17.
Projekt	Zależny walidator parent/edge closure, nie drugi graf prawdy.

Ślad implementacyjny · SemanticScaffoldIR

scaffold_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane

Wybrane pola z semantic_scaffold.py, klasa w liniach 33–76; całość deklaracji: 25 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S066. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I019

19 · Atom semantyczny zachowuje rodowód twierdzenia

To samo zdanie może mieć różną wartość dowodową w zależności od źródła.

SemanticAtom zawiera proposition_core, relation_signature, scope, temporal_state i instancje evidence. Każda EvidenceInstance ma root_ref. Funkcja independent_root_count liczy różne wartości tego pola, a nie liczbę plików lub odpowiedzi. Próba R3-T11 potwierdza, że dwa dowody o wspólnym root_ref są traktowane jako jeden korzeń.

Ten mechanizm ogranicza prosty błąd wielokrotnego liczenia tej samej przesłanki, ale nie ustanawia samodzielnie rzeczywistej niezależności. Dwa różne tekstowe root_ref mogą nadal oznaczać wspólne źródło fizyczne lub epistemiczne. Ich wiarygodne przypisanie wymaga producenta i sposobu weryfikacji genealogii.

W konsekwencji architektura pamięci powinna rozdzielać treść tezy od ścieżki, która ją wspiera. Synteza modeli może dostarczyć nowej interpretacji, lecz nie dodatkowego eksperymentu. W przyszłym CORE warto utrwalić taką reprezentację pochodzenia; reguła promocji do wiedzy musi uwzględniać zarówno kontrdowody, jak i wspólne zależności źródeł.

Rozwinięcie analizy źródłowej

W SemanticAtom proposition_core zachowuje treść, scope jej zakres, temporal_state kwalifikację czasu, a evidence listę instancji dowodowych. Dwie tezy o identycznym brzmieniu mogą mieć różny scope i inne źródła. Ich scalenie przez podobieństwo tekstu byłoby utratą informacji. Odwrotnie, dwie parafrazy mogą odwoływać się do tej samej tezy i jednego korzenia. Dlatego proces deduplikacji powinien badać relacje, a nie liczyć zgodne zdania. Nowa interpretacja może zmienić semantic_key lub status, lecz musi zachować powód tej zmiany. W przyszłym RAG potrzebna jest możliwość odtworzenia oryginalnego twierdzenia i jego późniejszego zawężenia.

```
liczba rekordów evidence != liczba niezależnych korzeni
```

Treść	proposition_core + scope + relation_signature.
Historia	temporal_state i source_representation_refs.
Dowód	EvidenceInstance z root_ref; niezależność wymaga weryfikacji przypisania.

Ślad implementacyjny · RelevanceProjection

projection_id	str; pole wymagane
graph_ref	str; pole wymagane
graph_digest	str; pole wymagane
consumer_ref	str; pole wymagane
required_capability_refs	Tuple[str, ...]; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 127–159; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I020

20 · Delta wiedzy nie jest deltą uprawnień

Rozwój modelu świata i zmiana władzy wykonawczej mają oddzielne reguły.

SemanticDelta dopuszcza ADD, UPDATE, SUPERSEDE, CONTRADICT, RETRACT i REVALIDATE. Wiąże operację z base_graph_digest oraz wymaganymi referencjami evidence i currentness. To język kontrolowanej zmiany interpretacji, przydatny zarówno do korygowania teraźniejszości, jak i ponownej oceny historii.

Kontrakt blokuje wybrane predykaty niosące credentials lub authority, a pola efektu muszą pozostać NONE. Ta walidacja ma określony zakres i nie powinna być przedstawiana jako kompletny detektor dowolnego ukrytego znaczenia. Jej funkcją jest zachowanie granicy między semantyczną reprezentacją a oryginalną ścieżką decyzji i admission.

Przeszłość może więc „ewoluować” przez nowy rekord CONTRADICT albo SUPERSEDE. Nie trzeba zmieniać oryginalnych bytes starego raportu. System zachowuje dwa fakty: co utrwalono wcześniej i dlaczego obecnie oceniamy to inaczej. Ten mechanizm jest trwalszą podstawą pamięci niż nadpisywany plik z jedną najnowszą wersją prawdy.

Rozwinięcie analizy źródłowej

SemanticDelta ma base_graph_digest, operation, subject, predicate i object. To umożliwia odróżnienie dodatku od retrakcji lub sprzeczności. Operacja CONTRADICT nie musi usuwać wcześniejszego węzła: może zachować konkurencyjny zapis i źródło różnicy. Dla wiedzy naukowej jest to ważniejsze niż wymuszenie jednego natychmiastowego zwycięzcy. Podczas aktualizacji trzeba jednak sprawdzić, czy delta dotyczy właściwego baseline. Zastosowanie jej do innej wersji grafu może prowadzić do błędnego przypisania relacji. Poprawność transformacji wymaga zatem zarówno zgodnego typu operacji, jak i tożsamości przedmiotu. Żaden z tych warunków nie tworzy prawa do działania poza reprezentacją.

```
nowa interpretacja -> nowa delta + base digest + evidence
```

Supersession	Zmiana obowiązującej interpretacji z zachowaniem poprzednika.
Retraction	Jawne wycofanie tezy, nie skasowanie dowodu.
Authority	Nadal odrębny owner i oryginalny grant.

Ślad implementacyjny · RelevanceProjection

projection_id	str; pole wymagane
graph_ref	str; pole wymagane
graph_digest	str; pole wymagane
consumer_ref	str; pole wymagane
required_capability_refs	Tuple[str, ...]; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 127–159; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I021

21 · RelevanceProjection jest ograniczonym widokiem

Istotność informacji musi być związana z konkretnym odbiorcą i grafem.

RelevanceProjection przechowuje consumer_ref, widoczne węzły i krawędzie, capabilities, excluded_classes oraz resource_budget. validate_against sprawdza zgodność graph_ref i graph_digest, a następnie podzbiory węzłów, krawędzi i dopuszczonych capabilities. To silniejsze wiązanie niż samo uszczelnienie lokalnego rekordu.

Próby R3-T12–T15 pokazały poprawną projekcję, odrzucenie obcego węzła, niedopuszczalnej capability oraz podmienionego digestu grafu. Wyniki dotyczą zadanego, niewielkiego wejścia. Nie dowodzą optymalności wyboru informacji ani kompletności zachowania każdego ograniczenia semantycznego w dowolnej projekcji.

Można już wyodrębnić dwa osobne problemy badawcze: czy projekcja nie wychodzi poza dozwoloną domenę oraz czy zachowuje informacje niezbędne do poprawnego rozwiązania. Pierwszy ma tu konkretny mechanizm sprawdzający. Drugi wymaga testów z negacją, konkurencyjną hipotezą i ograniczeniem, którego pominięcie zmieniłoby decyzję. Relewancja nie jest zatem ani prawdziwością, ani permission.

Rozwinięcie analizy źródłowej

W projekcji istnieją dwa odmienne pytania: czy widok jest dozwolonym podzbiorem oraz czy jest wystarczający dla zadania. Native validate_against odpowiada na pierwsze. Drugie wymaga kryterium zachowania semantyki. Jeżeli ograniczenie bezpieczeństwa zapisano w węźle niewłączonym do widoku, poprawny podzbiór może nadal być mylący. Dlatego proponowany test powinien porównywać odpowiedź przed i po projekcji dla kontrolnych zmian scope, negacji i warunków. Nie trzeba zachowywać wszystkich danych, lecz trzeba wiedzieć, których nie wolno pominąć bez zmiany klasy wyniku. Tę różnicę widać dopiero przy analizie rzeczywistego konsumenta, nie samego schematu.

```
visible nodes/edges ⊆ source graph; capabilities ⊆ allowed set
```

Integralność	Zgodność tożsamości i podzbiorów.
Użyteczność	Osobna ocena wystarczalności informacji.
Nowe próby	R3-T12–T15; brak nowej inferencji modelowej.

Ślad implementacyjny · RelevanceProjection

projection_id	str; pole wymagane
graph_ref	str; pole wymagane
graph_digest	str; pole wymagane
consumer_ref	str; pole wymagane
required_capability_refs	Tuple[str, ...]; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 127–159; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I022

22 · Git stanowi źródło percepcji

Odczyt wersji i odczyt zawartości muszą zostać związane w jedną obserwację.

CodePerceptionGitBoundary dopuszcza tylko pięć klas operacji: rozpoznanie commitu, rozpoznanie tree, listę drzewa, odczyt bloba i hash wejściowych bytes. Konstruuje argv po własnej stronie i nie udostępnia operacji zapisu Git. Ogranicza timeout oraz rozmiar odebranych danych, a każdy wynik otrzymuje odrębną obserwację.

Istotne jest miejsce tej granicy w architekturze. Git nie jest „umysłem systemu”; dostarcza sprawdzalnych faktów o wersji źródeł. Następny indekser zamienia je w rekordy plików i symboli. Dopiero późniejszy model semantyczny może interpretować, jaka odpowiedzialność została zmieniona i jak to wpływa na przyszły cel.

Formalizacja powinna rozróżniać zgodność formatu SHA od zgodności konkretnej treści. Podany expected_tree musi zostać porównany z zaobserwowanym tree, a blob z jego bytes. Ta praktyka chroni przed użyciem poprawnie wyglądającej referencji do innej rzeczywistości. Jednocześnie nie ustanawia fizycznej niezależności procesu odczytu ani zaufania do całego hosta.

Rozwinięcie analizy źródłowej

Granica Git przechowuje obserwację returncode, rozmiarów i digestów stdout/stderr. To pozwala związać rezultat procesu z żądaniem. Użycie shell=False i zamkniętego zestawu operacji ogranicza możliwość pomylenia danych z poleceniem. Nie należy jednak na tej podstawie ogłaszać pełnej izolacji sandboxa. Minimalne środowisko nadal potrzebuje rzeczywistego executable i systemu plików. Z kolei pojedynczy read_blob nie weryfikuje sam całej genealogii commitu. Właściwy łańcuch od resolve i list_tree do porównania bytes tworzy mocniejszy dowód niż odosobniona metoda. Dokumentacja powinna przedstawiać właśnie tę kompozycję odpowiedzialności.

```
ref -> commit -> tree -> blob -> bytes -> odczytana tożsamość
```

Git	Genealogia i zawartość źródła.
AST	Pochodna reprezentacja struktury programu.
Runtime	Osobna obserwacja zachowania konkretnej realizacji.

Źródła: R3-S021. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I023

23 · AST nie jest pełną semantyką wykonania

Struktura programu opisuje możliwości i wiązania źródłowe, nie dowodzi przyszłego zachowania.

Natywny Code Perception używa ast i symtable. Odczytane źródło jawnie określa CALLS jako STATIC_CALL_EVIDENCE. Wywołania atrybutowe, anonymous scopes i niejednoznaczne bindingi mogą pozostać nierozwiązane. Istnienie symbolu pod warunkiem if False nie oznacza, że runtime kiedykolwiek go zwiąże.

Wynik jest użyteczny właśnie dlatego, że nie udaje wszechwiedzy. Pozwala odnaleźć definicje, importy, dziedziczenie i potencjalne wywołania oraz skierować obserwatora do brakującej części. W atlasie Git odpowiada na pytanie o materiał, AST o jego strukturę, a dowód runtime o konkretny przebieg.

W przyszłej logice AST warto zachować wersję parsera, konfigurację i dokładne source identities. Hash AST nie jest twierdzeniem o równoważności programów. Zmiana komentarza może nie zmienić AST, podczas gdy zmiana stałej zmienia zachowanie i strukturę. Refaktoryzacja nazwy może z kolei zmienić identyfikator symbolu bez zmiany istotnego algorytmu. Potrzebne są osobne relacje aliasu, podobieństwa i następstwa.

Rozwinięcie analizy źródłowej

Wywołanie target() może zostać rozpoznane jako kandydat, jeśli model zakresu wskazuje jedną bezwarunkową definicję. Ten sam token wewnątrz caller(target) odnosi się jednak do parametru. Wywołanie self.target() wymaga dodatkowej wiedzy o instancji i dispatch. Źródło LION zachowuje te różnice zamiast zamieniać wszystkie nazwy w pozornie dokładny graf wykonania. Taka ostrożność jest istotną cechą systemu poznającego własny kod: musi umieć reprezentować granicę własnej percepcji. Wzrost precyzji przyszłego indeksera powinien być kwalifikowany przez nowe przypadki negatywne, a nie przez sam spadek liczby nierozwiązanych krawędzi.

```
Git bytes -> AST/symtable -> static evidence; runtime target osobno
```

Zachowane	UNRESOLVED jest informacją, nie brakiem wypełnienia wykresu.
Nieudowodnione	Wybór callable przez przyszły runtime.
Właściwy następca	RuntimeCallTargetProof powiązany z dokładnym grafem.

Źródła: R3-S020. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I024

24 · Testy zapisują granice rozumienia kodu

Kontrprzykład może stać się trwalszą częścią architektury niż rozwiązanie pojedynczego zadania.

test_code_perception.py zawiera przypadki przesłaniania parametrów, lokalnych przypisań, nonlocal, mutable attributes, lambda i comprehensions. Sprawdza także niepoprawny Python, podmianę bloba i duplikację ścieżki. Testy nie mówią, że indekser rozwiązuje każdy program; określają sytuacje, w których powinien odmówić nadmiernie mocnego wniosku.

Dwa szczególnie użyteczne przypadki pokazują trwałość i zmienność identyfikatora. Zmiana return 1 na return 2 zachowuje node_id przy zmianie semantic_digest. Renaming ścieżki zmienia node_id. Wynika z tego, że „tożsamość funkcji” nie jest jedną uniwersalną relacją: zależy od polityki identyfikacji stosowanej przez dany model.

W R3 wynik 45 testów z R2 pozostaje wynikiem historycznym tamtego wykonania. Nowe próby dotyczą innych kontraktów i nie są z nim sumowane w jedną miarę bezpieczeństwa. Taki porządek jest sam w sobie przykładem ewoluującej przeszłości: stare testy zachowują zakres, a kolejne uzupełniają mapę właściwości bez retrospektywnego rozszerzania dawnych sukcesów.

Rozwinięcie analizy źródłowej

Plik testowy zawiera również próby malformed Python, blob substitution, duplicate path i rename. Razem pokazują, że percepcja dotyczy zarówno struktury programu, jak i integralności wejścia. Błędny plik ma pozostać w inwentarzu ze stanem PARSE_ERROR, nie zniknąć. Zmiana ścieżki zmienia tożsamość symbolu, a zmiana ciała przy zachowanej nazwie może zmienić tylko digest struktury. To zestaw użytecznych rozróżnień do normalizacji repozytorium. R3 zachowuje historyczny wynik 45 testów z R2 i osobno wykonuje 18 nowych prób innych kontraktów. Nie sumuje ich w jeden nieopisany certyfikat bezpieczeństwa aplikacji.

```
kontrprzykład -> brakująca reguła -> regresja o określonym zakresie
```

Stary wynik	45 testów Code Perception w osadzonym dossier R2.
Nowa delta	18 prób koperty i reprezentacji wykonanych dla R3.
Zakaz agregacji	PASS różnych serii nie jest procentem globalnej gotowości.

Źródła: R3-S022. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I025

25 · Aktualność jest relacją do przedmiotu

Zielony status bez source binding nie określa, czego dotyczy.

DOCUMENTATION_CURRENTNESS_MODEL rozdziela aspekty aktualności projekcji, dokumentacji i źródeł. W źródłach truth plane istnieje osobna koncepcja subject_digest, która oddziela właściwy przedmiot od plików przenoszących deklarację jego digestu. Ten mechanizm przeciwdziała bezproduktywnej rekurencji, w której zapis własnego hasha zawsze zmieniałby hash przedmiotu.

Nie oznacza to, że można dowolnie wyłączać pliki z oceny. Zestaw carriers jest kontraktowy i ograniczony. Jeśli zmienia się inne źródło, zależna projekcja może stać się stara. Jednocześnie zmiana genealogii commitów przy niezmienionym tree nie musi zmienić struktury kodu. Dokumentacja potrzebuje dlatego więcej niż jednego wymiaru aktualności.

W modelu czasu R3 każda teza otrzymuje przedmiot, wersję i warunek rewalidacji. Można zachować prawdziwy raport historyczny oraz odmówić użycia go do decyzji dzisiejszej. Rozdzielenie tych dwóch zdań zapobiega zarówno kasowaniu dorobku, jak i przenoszeniu starych kwalifikacji na nowe, nieprzebadane źródło.

Rozwinięcie analizy źródłowej

Wzorzec carrier-last rozwiązuje konkretny problem samoodniesienia. Jeśli plik opisujący aktualność należałby bezpośrednio do swojego własnego subject, jego aktualizacja zmieniałaby przedmiot deklaracji. Źródło definiuje zamknięty zbiór carrier paths i jawnie oblicza subject z pozostałych elementów. Nie wolno rozszerzać wykluczeń według wygody kandydata. Różne rodzaje aktualności mogą przy tym reagować inaczej na tę samą zmianę. Historia commitów zmieniająca tylko nośnik reprezentacji nie jest identyczna z materialną zmianą kodu. Poprawna kwalifikacja wymaga dowodu ancestry i zakresu zmienionych ścieżek, a nie tylko porównania etykiet branch.

```
CURRENT(ocena, przedmiot, epoka, rodzaj dowodu)
```

Historycznie poprawne	Może być nadal wartościowym źródłem.
Aktualnie wystarczające	Wymaga zgodności z obecnym przedmiotem decyzji.
Carrier-last	Domknięcie reprezentacji następuje po stabilizacji źródeł.

Źródła: R3-S027. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I026

26 · Process IR porządkuje możliwe przejścia

Proces musi opisać nie tylko cel fazy, lecz także warunki jej wyboru i rozliczenia.

process_ir.py zawiera natywną reprezentację procesu, podczas gdy Process Contract Plane dodaje odrębny kontrakt klasy wykonania fazy. Te dwa mechanizmy odpowiadają na inne pytania. IR organizuje semantykę przebiegu; kontrakt fazy określa, co może zaspokoić wymaganie, jakie evidence jest potrzebne i jaki maksymalny zakres efektu jest dopuszczalny.

W generatywnym systemie planowanie może być probabilistyczne, ale przejścia wykonania wymagają jawnych warunków. Model może zaproponować nową kolejność, lecz nie powinien przeprowadzać bieżącej misji na zmieniony kontrakt przez samą wypowiedź. Taki skok zatarłby, które kryterium obowiązywało dla już wykonanego kroku.

Proces jest więc elementem teraźniejszości i jednocześnie pamięcią przyczyn. Zachowane fazy, wersje i wyniki pozwalają wyjaśnić, dlaczego następny krok stał się możliwy. Gdy warunek jest nierozstrzygnięty, trwałe WAIT lub UNKNOWN może być poprawną informacją, o ile istnieje jawna polityka dalszego postępowania i nie udaje się completion.

Rozwinięcie analizy źródłowej

W process IR istotne są warunki przejść i ograniczenia kroku. Sekwencja „analizuj, napraw, potwierdź” pozostaje zbyt ogólna, jeśli nie wiadomo, co znaczy potwierdzenie. Kompilator semantyczny powinien wyprowadzić konkretne predykaty, a wykonawca otrzymać aktualną capability. W przeciwnym przypadku model może zwrócić przekonującą narrację zamiast artefaktu. To nie problem stylu odpowiedzi, lecz brak formalnego odbioru. Język procesu ma więc organizować zobowiązania, a nie imitować naturalny język poleceń. Możliwość generowania LPCL jest wartościowa dopiero wtedy, gdy jego ograniczenia są rozpoznawalne przez późniejszych konsumentów i nie zależą od domysłu modelu.

```
plan procesu != kontrakt fazy != binding wykonawcy
```

Cel	Semantycznie opisane oczekiwanie.
Przejście	Warunek oparty na właściwym evidence.
Zakończenie	Rozliczenie predykatów, nie heartbeat procesu.

Ślad implementacyjny · CanonicalProcessIR

canonical_bytes	bytes; pole wymagane
process_digest	str; pole wymagane

Wybrane pola z process_ir.py, klasa w liniach 333–360; całość deklaracji: 2 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S057. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I027

27 · PhaseExecutionContract powstał z realnego kontrprzykładu

Znacząca faza może być niewykonalna, jeśli nie istnieje związany mechanizm jej realizacji.

Process Contract Plane zachowuje PCD-0001: faza REPAIR_EXECUTION_BINDER dotarła do CAPABILITY_NOT_AVAILABLE mimo poprawnego semantycznie LPCL. Odpowiedzią było odróżnienie intencji od kontraktu i dynamicznego bindingu. To przykład, w którym praktyczny brak został przekształcony w trwałe rozróżnienie ontologiczne.

PhaseExecutionContract określa execution_class, capability_classes, effect_ceiling, politykę brakującej zdolności oraz completion_predicates. LPCL/1.1 może pozostać czytelny przez LEGACY_INFERRED_SAFE z ceiling NONE. Słowa REPAIR czy MIGRATE nie uzyskują wtedy przez domysł prawa do mutacji.

Fundamentalna własność wyłaniającej się architektury polega na tym, że błąd staje się nowym rozróżnieniem. System nie potrzebuje tylko dłuższego promptu „napraw”. Potrzebuje reprezentacji, która potrafi powiedzieć: cel jest ważny, kontrakt poprawny, ale binding nierozwiązany. Takie rozpoznanie prowadzi do budowy brakującej zdolności zamiast ślepego retry lub fałszywego sukcesu.

Rozwinięcie analizy źródłowej

W PCD-0001 semantyczna faza była sensowna, lecz nie miała dostępnego REPAIR_EXECUTION_BINDER. To właśnie rodzaj doświadczenia, z którego można wyprowadzić nowy byt architektoniczny. Kontrakt fazy nie musi znać stałego workera, ale musi określić klasę wykonania i warunki jego poprawności. Dynamiczny binding pozwala znaleźć inną realizację bez zmiany celu. Jeżeli zaś cel wymaga nowej klasy skutku, nie wystarczy przepiąć wykonawcę: zmienia się kontrakt i zakres dopuszczenia. W ten sposób system może być elastyczny organizacyjnie, zachowując stabilne granice znaczenia oraz odpowiedzialności.

```
PHASE_INTENT != CONTRACT != BINDING != EFFECT != COMPLETION
```

Historia	PCD-0001 opisany w kanonicznym dokumencie procesu.
Mechanika	Preflight rozdziela INVALID i VALID_UNBOUND_WAITING.
Odbiór	Wymaga aktualnego evidence dla predykatów fazy.

Ślad implementacyjny · PhaseExecutionContract

mission_id	str; pole wymagane
phase_id	str; pole wymagane
ordinal	int; pole wymagane
execution_class	str; pole wymagane
capability_classes	tuple[str, ...]; pole wymagane

Wybrane pola z phase_execution_contract.py, klasa w liniach 70–158; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S056. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I028

28 · CommunicationEnvelope oddziela znaczenie od transportu

Wspólna koperta jest kontraktem tożsamości i pochodzenia, nie nowym brokerem.

CommunicationEnvelope wiąże nadawcę, zbiór odbiorców, konwersację, misję, korelację, causation i zewnętrzny payload. Jego parser wymaga dokładnego zestawu pól. Nowa próba R3-T04 pokazuje, że dopisanie pola hmk9d do poprawnej koperty jest odrzucane. Semantyczne rozszerzenie należy projektować przez referencjonowany payload lub odrębny adapter, a nie przez niejawne poszerzenie schematu.

Pozostałe próby potwierdzają kanonizację kolejności odbiorców, odrzucenie duplikatów, self-causation i deklarowanego efektu. Jednocześnie R3-T10 pokazuje granicę: poprawnie uformowana referencja do nieodczytanego payloadu przechodzi lokalną walidację. Funkcja nie pobiera bytes, więc nie może sama poświadczyć ich dostępności ani treści.

Ten kontrapunkt dobrze pokazuje fundamentalną naturę LION. Każdy artefakt ma określone zadanie dowodowe i nie powinien przejmować cudzych obowiązków. Koperta zachowuje tożsamość wymiany. Transport dostarcza. Provider poznawczy odpowiada. Odrębny mechanizm autoryzuje materialny skutek, a jeszcze inny go obserwuje.

Rozwinięcie analizy źródłowej

R3-T02 i T03 sprawdzają różne przypadki recipient set: build normalizuje kolejność, ale duplikat jest odrzucany. To przykład kanonizacji, która nie ukrywa sprzeczności. Dwa różne porządki tej samej poprawnej listy mogą otrzymać identyczną reprezentację; powtórzony adresat nie staje się poprawny przez ciche usunięcie. Podobnie parser mappingu odrzuca nieznane pola. Takie szczegóły decydują o deterministyczności digestu i o przewidywalności interfejsu. W architekturze federacyjnej brak wspólnej kanonizacji może prowadzić do różnych hashy tej samej wiadomości, a nadmierna normalizacja może zgubić błąd wymagający interwencji.

```
envelope != payload bytes != transport != delivery != cognition
```

Nowe testy	R3-T01–T10: ograniczone próby natywnego kontraktu.
Przejście HMK	Osobny semantic payload / adapter.
Niezmiennik	Koperta nie tworzy authority i nie raportuje effect.

Ślad implementacyjny · CommunicationEnvelope

schema_id	str; pole wymagane
envelope_id	str; pole wymagane
sender_ref	str; pole wymagane
recipient_refs	tuple[str, ...]; pole wymagane
conversation_ref	str | None; pole wymagane

Wybrane pola z communication_envelope.py, klasa w liniach 115–218; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S043. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I029

29 · Invocation wiąże konkretną pracę poznawczą

Tożsamość rozmowy nie określa automatycznie modelu, który obsłużył dany krok.

cognitive_invocation.py rozdziela sesję, intencję wywołania, plan i receipt. Pakiet komunikacji dodaje ich rolę w trasach LOCAL, SAAS i DUAL. Jeden model lokalny może obsłużyć wiele participantów, ale wynik musi pozostać związany z właściwym lane, requestem i zakresem. Nie powstaje z tego automatycznie wiele niezależnych epistemicznie umysłów.

Istotne są co najmniej trzy skróty: wspólnego kontekstu, projekcji dla providera i rzeczywiście wysłanych bytes. Pierwszy pozwala porównać wspólną podstawę, drugi odróżnia lokalne reprezentacje, trzeci opisuje materialny input. Prepared payload i sent payload mogą być różnymi stanami. Bez ich rozdzielenia analiza nie rozpozna, czy błąd powstał przy doborze kontekstu, serializacji czy wysłaniu.

Formalizacja powinna więc opisywać poznanie jako adresowany proces, a nie abstrakcyjny punkt „LLM”. Provider jest realizacją zdolności; uczestnik i cel pozostają identyfikowane niezależnie. Dzięki temu można zastąpić model albo porównać dwa modele bez przepisywania tożsamości całej misji.

Rozwinięcie analizy źródłowej

CognitiveSessionBinding ma scope_type, opcjonalną mission, thread, endpoint, session, epoch, model_release_ref oraz input_context_digest. Ta kombinacja pokazuje, że „ten sam model” nie wystarcza do ciągłości pracy. Nowa sesja może mieć inny kontekst, a to samo conversation identity może przetrwać zmianę backendu tylko po jawnym rebind. InvocationIntent, Plan, Attempt, Lease i Receipt rozdzielają kolejne stadia. Dzięki temu timeout jednej próby nie usuwa intencji ani nie uzasadnia automatycznej zmiany providera. W przyszłym teście warto zmieniać kolejno każdą z tych tożsamości, zamiast ograniczyć się do sprawdzenia, że backend zwrócił dowolny niepusty tekst.

```
participant != provider != session != invocation != response
```

Wspólne	shared_context_digest.
Odrębne	projection_digest i provider_session_ref.
Materialne	Hash bytes faktycznie przekazanych właściwym torem.

Ślad implementacyjny · InvocationIntent

invocation_id	str; pole wymagane
parent_request_ref	str; pole wymagane
message_ref	str; pole wymagane
payload_digest	str; pole wymagane
binding_ref	str; pole wymagane

Wybrane pola z cognitive_invocation.py, klasa w liniach 56–62; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S042. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I030

30 · ModelRelease i ModelCall chronią reprodukowalność

Nazwa „LOCAL” nie identyfikuje dokładnej realizacji poznawczej.

ModelRelease stanowi osobny kontrakt wersji modelu. ModelCallV2 wiąże wywołanie z jego tożsamościami i provenance. Właściwe przypisanie wyniku wymaga także wersji adaptera, kontekstu oraz parametrów istotnych dla danego wykonania. Stwierdzenie, że endpoint odpowiada, nie dostarcza tych informacji.

Przy ewolucji aplikacji model może zmienić się szybciej niż kontrakt procesu. Przed wykorzystaniem poprzednich testów trzeba więc ustalić, co dokładnie pozostaje stałe. Test parsera nie traci ważności tylko dlatego, że zmienił się model; benchmark jakości odpowiedzi może natomiast wymagać nowej kwalifikacji. Wspólna etykieta „AI działa” zacierałaby tę różnicę.

R3 traktuje release i call jako elementy dowodowej pamięci teraźniejszości. Po archiwizacji epizodu pozwalają odtworzyć użyty mechanizm, ale nie oznaczają automatycznego uczenia parametrów. Nowy model może konsumować dawny wynik jako kontekst lub regułę dopiero przez właściwą projekcję i politykę użycia.

Rozwinięcie analizy źródłowej

ModelRelease obejmuje nie tylko base_weights_digest, ale też adapter, tokenizer, chat_template, prompt_profile, tool_schema, runtime i knowledge_release. Zmiana któregokolwiek z tych składników może zmienić interpretację tego samego tekstu wejściowego. Dlatego nazwa modelu nie jest wystarczającą tożsamością eksperymentu. ModelReleaseRef może zachować stan IDENTITY_ONLY_UNRESOLVED; samo istnienie ref nie dowodzi pełnego rozpoznania wydania. W dokumentacji należy oddzielić znany identyfikator usługowy od dokładnego release, którego skład rzeczywiście ustalono. Takie rozróżnienie jest szczególnie ważne przy porównywaniu wyników SaaS i lokalnego modelu.

```
wynik poznawczy -> dokładny model release + input + invocation + receipt
```

Tożsamość	Nie tylko nazwa routingu LOCAL / SAAS.
Reprodukcja	Zgodny zakres wersji i warunków.
Pamięć	Użycie wyniku nie oznacza zmiany wag modelu.

Ślad implementacyjny · ModelCallV2

model_call_id	str; pole wymagane
mission_id	str; pole wymagane
phase_id	str; pole wymagane
task_id	str; pole wymagane
assignment_id	str; pole wymagane

Wybrane pola z model_call_v2.py, klasa w liniach 46–132; całość deklaracji: 35 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S054. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I031

31 · GroupChannel jest kanałem dowodowym

Komunikacja zespołu i zlecenie materialnego działania nie są tym samym procesem.

Protokół SWARM_COMMUNICATION rozróżnia adresy mission, drone, swarm i group. Dla grup architecture, security i runtime opisuje governowany workflow z receipt evidence-only. group_channel.py utrwala kopertę i receipt w osobnych kontraktach. Nierozwiązany adres ma prowadzić do odmowy, nie do automatycznego broadcastu.

Ten kanał pozwala przekazać informację między uczestnikami federacji przy zachowaniu konkretnego źródła, targetu i currentness. Nie jest natomiast dowodem, że lokalny model otrzymał treść, ani pozwoleniem na wykonanie naprawy opisanej w wiadomości. Także poprawnie pobrany artefakt workflow wymaga interpretacji w swoim zakresie.

Naturalną formalizacją jest model portów o różnych semantykach dostarczenia. Jeden port może kończyć się receipt dokumentu, drugi odpowiedzią modelu, trzeci osobnym Action IR. Polimorfizacja transportu jest bezpieczna tylko wtedy, gdy zachowuje znaczenie odbioru. Zastąpienie komentarza w Issue wywołaniem modelu nie jest neutralną zmianą medium, jeśli zmienia końcowy predykat sukcesu.

Rozwinięcie analizy źródłowej

Kanał group:security w źródłowym protokole jest evidence-only. Odbiór wymaga konkretnego runu, nazwanego artefaktu i sprawdzenia bindingów, a nie samego komentarza „wysłano”. Jednocześnie historyczne issues pozostają jako materiał koordynacyjny i nie powinny być automatycznie usuwane. To przykład, w którym ewolucja transportu zachowuje historię bez pozostawiania jej jako aktywnego ownera. W nowej dokumentacji należy wskazać, który adres routuje do którego mechanizmu w danej epoce. Nierozpoznany adres powinien dać błąd routingu, nie broadcast do całego roju ani próbę użycia dowolnego dostępnego narzędzia.

```
dostarczony dowód != odpowiedź modelu != wykonana akcja
```

Źródło normy	LION/protocols/SWARM_COMMUNICATION.md, odczyt pinned.
Adres	Rozwiązany przez właściwy rejestr.
Efekt	Kanał grupowy pozostaje evidence-only.

Ślad implementacyjny · GroupChannelEnvelope

schema_version	str; pole wymagane
repository	str; pole wymagane
message_id	str; pole wymagane
target	str; pole wymagane
expected_master_head	str; pole wymagane

Wybrane pola z group_channel.py, klasa w liniach 99–179; całość deklaracji: 10 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S052. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I032

32 · Rola uczestnika nie jest procesem

Logiczna organizacja zadania i fizyczna organizacja wykonania przecinają się, ale nie są identyczne.

Kontrakty agent_registry, workload_identity i runtime_attestation opisują różne tożsamości. Pakiet komunikacji odróżnia logical_drone, material_worker i model_provider. Ten podział jest niezbędny, gdy wiele ról korzysta z jednego modelu, a jeden worker obsługuje różne przypisania w kolejnych chwilach.

Sama liczba identyfikatorów nie jest miarą niezależności. Wspólny proces, model, kontekst lub host może tworzyć wspólnego przodka błędu. Stary raport R1 pokazał tę potrzebę także na poziomie WSL: różne nazwy instancji nie ustanawiają automatycznie niezależnych domen fizycznych. R3 zachowuje to jako historyczną obserwację, bez nowego odczytu hostów.

Przy projektowaniu CORE należy więc opisać profil wymaganej niezależności dla konkretnego skutku. Dla jednego zadania wystarczy rozdzielenie ról i implementacji, dla innego potrzebne są odrębne domeny zaufania. Formalizacja nie powinna przyznawać tej własności samym prefiksem MD lub LD.

Rozwinięcie analizy źródłowej

Rozdzielenie LD i MD pozwala projektować organizację niezależnie od materialnego substratu. Logiczny dron może pełnić rolę recenzenta, ale jego inferencja może zostać wykonana przez współdzielony provider. Worker materialny może mieć ograniczoną możliwość zapisu bez własnego modelu. Z punktu widzenia niezależności dowodowej konieczne jest więc rozpoznanie wspólnych przodków: danych, kodu, modelu i hosta. Dwie role o innych nazwach nie stanowią automatycznie dwóch niezależnych świadectw. Nie oznacza to, że taki układ jest bezwartościowy; oznacza, że jego profil kwalifikacji musi uczciwie opisywać zakres separacji.

```
rola -> przypisanie -> instancja -> domena awarii; każda relacja jawna
```

Logiczne	Participant i jego obowiązek wobec misji.
Wykonawcze	Worker, sandbox, lease i runtime identity.
Poznawcze	Provider oraz źródła wiedzy danej odpowiedzi.

Ślad implementacyjny · WorkloadIdentityProof

schema_version	str; pole wymagane
proof_id	str; pole wymagane
subject_id	str; pole wymagane
trust_domain	str; pole wymagane
tenant_id	str; pole wymagane

Wybrane pola z workload_identity.py, klasa w liniach 35–86; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S067. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I033

33 · Action IR materializuje ograniczony zamiar

Droga do skutku wymaga bardziej precyzyjnego obiektu niż naturalnojęzykowe „zrób”.

CanonicalActionIR jest źródłową reprezentacją ograniczonej akcji. ActionRuntimeBinding odróżnia ją od decyzji authority i konkretnej instancji wykonawczej. To miejsce, w którym semantyczny cel zostaje zawężony do resource, klasy działania, wejść i warunków. Nie wolno przeskoczyć go bezpośrednim tłumaczeniem tekstu modelu na shell.

W ewolucyjnym procesie wiele propozycji może współistnieć. Dopiero wybrany wariant i aktualne dopuszczenie wiążą konkretny skutek. Zmiana implementacji, modelu lub celu po kwalifikacji może unieważnić wcześniejsze bindings. Dlatego sprawdzenie przed effect nie jest biurokratycznym dodatkiem, lecz ochroną tożsamości tego, co faktycznie zatwierdzono.

Formalny wniosek jest następujący: szeroka przestrzeń generowania i wąska przestrzeń efektów mogą istnieć równocześnie. System może swobodniej badać hipotezy, nie rozszerzając przy tym prawa do zmiany środowiska. To rozróżnienie jest osią najwyższego modelu L0, a nie szczegółem do ukrycia dopiero w rozdziale bezpieczeństwa.

Rozwinięcie analizy źródłowej

Action IR powinien pozwolić deterministycznie sprawdzić, jaki zasób i jaki efekt są przedmiotem konkretnego żądania. Narracja modelu może uzasadniać wybór, ale nie powinna być jedynym miejscem zapisania scope. Jeżeli target, payload lub klasa operacji zmieniają się po decyzji, zmienił się przedmiot dopuszczenia. Właściwy hash i currentness pomagają to wykryć. Ten model jest zarazem generatywny i ograniczony: można swobodnie projektować nowe kandydatury, lecz każda musi zostać przełożona na jawny kontrakt przed skutkiem. Ograniczenie nie polega na enumeracji wszystkich możliwych myśli, tylko na kontroli przejścia do działania.

```
propozycja -> ograniczona akcja -> odrębna decyzja -> admission
```

Znaczenie	Cel i rationale pozostają w lineage.
Konkretyzacja	Dokładny zasób oraz dopuszczalna klasa efektu.
Kontrola	Zgodność aktualnej tożsamości przed wykonaniem.

Ślad implementacyjny · CanonicalActionIR

canonical_bytes	bytes; pole wymagane
payload_digest	str; pole wymagane

Wybrane pola z action_ir.py, klasa w liniach 286–313; całość deklaracji: 2 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S034. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I034

34 · Admission jest zdarzeniem zależnym od stanu

Raz poprawny kontrakt nie nadaje nieograniczonej gotowości na wszystkie przyszłe wykonania.

Kontrakty runtime_currentness i runtime_authority_binding wiążą stan wykonawcy z właściwym źródłem oraz kontekstem uprawnień. Ich zadaniem jest rozliczenie tego konkretnego użycia capability. Rejestr zdolności może opisywać, co system potrafi, podczas gdy aktualny lease, stan revocation lub nowa generacja może ograniczać, co wolno wykonać teraz.

W propozycji federacyjnej adapter swarm tworzy reprezentację PDPResultBinding z decyzją ALLOW przy założeniu wcześniejszego governed wejścia. Odczyt kodu nie dowodzi samodzielnej weryfikacji oryginalnego grantu przez ten adapter. Powstaje konkretne wymaganie kontraktu upstream: trzeba rozpoznać, kto dostarcza i sprawdza dowód decyzji, zanim jej reprezentacja stanie się wejściem admission.

To nie jest wykazany bypass runtime. Jest to przykład, dlaczego deklarowane „nie tworzy authority” nie może zastąpić analizy producenta i konsumenta pola. Oryginalna decyzja, jej serializowana reprezentacja i dopuszczenie executora są trzema różnymi obiektami. R3 zachowuje tę zależność jako warunek przyszłej kwalifikacji.

Rozwinięcie analizy źródłowej

W adapt_to_swarm_request warto przeczytać linię konstruującą PDPResultBinding, a nie poprzestać na komentarzu. Ustawione ALLOW reprezentuje założenie o już governowanym wejściu. Poprawny opis adaptera musi wskazać, kto dostarcza i weryfikuje to wejście. Jeśli żaden upstream consumer nie zostanie odnaleziony, pozostaje luka dowodowa. Nie wolno jednak z samego konstruktora wywnioskować, że wykonano nieautoryzowaną operację. To przykład naukowego rozdzielenia ryzykownej powierzchni, kontraktu zależności i rzeczywistej ścieżki efektu. Właśnie ta precyzja pozwala zaprojektować rozstrzygający test zamiast ogólnego alarmu.

```
oryginalna decyzja != reprezentacja ALLOW != runtime admission
```

Źródło dodatkowe	swarm/federation/ai_platform_runtime_adapter.py, pinned odczyt R3.
Ustalenie	Adapter materializuje strukturę decyzji, nie dowód issuer.
Otwarta zależność	Weryfikacja oryginalnego upstream authority i jego konsumenta.

Ślad implementacyjny · RuntimeEvidenceReference

runtime_evidence_digest	str; pole wymagane
runtime_instance_id	str; pole wymagane
repository	str; pole wymagane
base_sha	str; pole wymagane
head_sha	str; pole wymagane

Wybrane pola z runtime_authority_binding.py, klasa w liniach 39–79; całość deklaracji: 10 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S060. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I035

35 · Receipt nie zamyka jeszcze epizodu

Raport wykonawcy, zaobserwowany skutek i rozliczenie celu są odrębnymi etapami.

runtime_reconciliation.py reprezentuje warunki uzgodnienia wyniku, zamiaru i obserwacji. Ta rola nie sprowadza się do kopiowania pola SUCCESS z odpowiedzi workera. Materialny artefakt może nie odpowiadać deklarowanemu digestowi, może zostać zapisany w innym miejscu lub należeć do innej generacji.

W komunikacji podobna zasada rozdziela PERSISTED od DELIVERED. Wniosek z samego zapisu wiadomości dotyczy trwałości intencji wysłania, nie dostarczenia do modelu. Pakiet holomozaikowy opisuje historyczną kolejkę bez pełnego receipt. R3 nie zamyka tych turnów ani nie przenosi tego starego licznika na nową obserwację.

Zamknięcie epizodu powinno zatem podawać, które dokładnie predykaty osiągnięto. Wynik negatywny może być poprawnie rozliczony, a częściowy sukces może pozostawić aktywną lukę. Ten model umożliwia ewolucję opartą na błędach bez udawania, że każde rozpoczęte działanie musi zakończyć się pozytywną etykietą.

Rozwinięcie analizy źródłowej

Niezależny readback powinien porównać faktyczny cel z wymaganym postcondition, nie tylko powtórzyć raport executora. Dla pliku oznacza to dokładną ścieżkę, bajty, digest i związek z kandydatem. Dla wiadomości — odpowiedź w właściwej epokowej relacji, nie dowolny tekst w oknie przeglądarki. Dla zmiany stanu — odrębny odczyt po działaniu. Bez tego można otrzymać łańcuch receipts, który opisuje poprawne przekazywanie deklaracji bez potwierdzenia świata. LION już posiada język rozdzielający te warstwy; następna formalizacja musi wykazać konkretnych producentów i konsumentów tego języka.

```
reported effect -> observed effect -> reconciled outcome
```

Receipt	Co zgłosiła konkretna ścieżka.
Obserwacja	Co odczytano z właściwego zasobu.
Rekonsyliacja	Czy rezultat spełnia dokładny zamiar i warunki.

Ślad implementacyjny · RuntimeObserverTrustBinding

source_id	str; pole wymagane
source_instance_id	str; pole wymagane
source_implementation_digest	str; pole wymagane
trust_anchor_id	str; pole wymagane
trust_anchor_digest	str; pole wymagane

Wybrane pola z runtime_reconciliation.py, klasa w liniach 35–41; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S063. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I036

36 · Epizod uczenia jest widokiem dowodów

Pamięć organizacyjna może się rozwijać bez założenia, że zmieniają się parametry modelu.

EvidenceBoundLearningEpisode wiąże message_refs, invocation_refs, assignment_refs, model_call_refs, wyniki, obserwacje i rekonsyliacje. Potwierdzony sukces lub porażka wymagają silniejszej klasy evidence niż sam teacher output albo receipt. Kontrakt zabrania także prostego przemianowania UNKNOWN na pozytywnego lub negatywnego kandydata.

Warto dostrzec podwójny poziom znaczenia: można zaobserwować, że rekord epizodu istnieje, bez dowiedzenia prawdziwości każdej tezy zawartej w tym rekordzie. Konwersja do EvidenceObservation musi zachować ten zakres. To samo dotyczy PDF: istnienie dokumentu jest faktem, ale jego propozycje nie stają się przez to stanem świata.

Formalizacja pamięci powinna określać dozwolone użycie wyniku. Epizod może służyć do retrieval, regression fixture, uzasadnienia nowej hipotezy lub planu eksperymentu. Promocja do normy albo aktualizacja parametrów modelu wymaga osobnego procesu i właściwych praw. Dzięki temu rozwój wiedzy pozostaje rozliczalny, nawet gdy różne rodzaje pamięci mają różne czasy i właścicieli.

Rozwinięcie analizy źródłowej

EvidenceBoundLearningEpisode stanowi naturalny łącznik między komunikacją i ewolucją, ponieważ wiąże konkretną pracę poznawczą z wynikami. Jego użycie nie powinno wymagać przejęcia canonical conversation store ani scheduler state. Referencje pozwalają zestawić ślady pod jednym epizodem badawczym. Trzeba jednak sprawdzić zgodność czasu, model release i zakresu źródeł. Samo zapisanie wielu refs nie gwarantuje, że wszystkie dotyczyły tej samej generacji. W docelowym verifierze epizodu niezbędne są więc kontrole tożsamości i cutoff, a także zachowanie wyników negatywnych. Pamięć, która przechowuje wyłącznie sukcesy, traci najważniejsze granice generalizacji.

```
obserwacja rekordu != potwierdzenie każdej zawartej w nim tezy
```

Silniejszy label	Wymaga właściwego evidence i reconciliation_refs.
Czas	Input cutoff oddziela wejścia od późniejszych wyników.
Pamięć	Working context nie jest automatycznie committed memory.

Ślad implementacyjny · EvidenceBoundLearningEpisode

episode_id	str; pole wymagane
model_release_ref	str; pole wymagane
causal_group_ref	str; pole wymagane
task_family	str; pole wymagane
episode_class	str; pole wymagane

Wybrane pola z evidence_bound_learning_episode.py, klasa w liniach 39–65; całość deklaracji: 26 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S044. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I037

37 · Formalizacja określa zasięg zmiany

Zmiana jednego modułu może wymagać aktualizacji wielu zależnych powierzchni.

ArchitectureFormalizationManifest opisuje materialny zakres delty i powiązane wymagania. RequiredFormalizationSet oraz registry wskazują, jakie dokumenty, kontrakty, testy i projekcje wymagają rozliczenia. Celem nie jest mechaniczne wygenerowanie kolejnych plików, lecz zachowanie zgodności znaczenia między źródłami a ich konsumentami.

Przy zmianie identyfikatora albo schematu nie wystarcza poprawny test jednostkowy w pliku producenta. Trzeba wiedzieć, czy istnieje konsument starszego formatu, czy dokumentacja wskazuje prawidłowy owner i czy RAG nadal prowadzi do aktualnej ścieżki. Taki graf wpływu jest konkretną formą samoobserwacji systemu.

R3 rozwija projekt dokumentacyjny, ale sam nie ustanawia FormalizationClosureRecord dla całej federacji. Nowe karty są kandydatem wiedzy z odnośnikami do źródeł. Ich przyjęcie powinno respektować istniejącego ownera i odrębne piny. W przeciwnym razie obszerny atlas mógłby stać się nieformalnym konkurentem kontraktów, które miał wyjaśniać.

Rozwinięcie analizy źródłowej

ArchitectureFormalizationManifest powinien wskazywać powierzchnie dotknięte zmianą. Nie każda delta wymaga aktualizacji każdego repozytorium, ale żadna istotna zależność nie powinna pozostać pominięta tylko dlatego, że jej plik nie był bezpośrednio edytowany. Zmiana publicznego pola może unieważnić parser w innym module. Zmiana znaczenia statusu może wymagać korekty panelu i RAG. Formalizacja porządkuje te zobowiązania, lecz nie przyznaje merge lub runtime authority. Dzięki temu można ocenić kompletność projektu niezależnie od tego, czy operator już dopuszcza jego zastosowanie. To ważna separacja między gotowością decyzji i samą decyzją o skutku.

```
delta -> powierzchnie wpływu -> wymagane dowody -> closure zakresu
```

Nie tylko kod	Interfejsy, testy, dokumenty, piny i konsumenci.
Nie authority	Closure formalizacji nie przyznaje uprawnień.
R3	Kandydat referencyjnego opisu, bez promocji do bootstrapu.

Ślad implementacyjny · ArchitectureFormalizationManifest

manifest_id	str; pole wymagane
baseline	BaselineIdentity; pole wymagane
source_evolution_delta	EvolutionDeltaRef; pole wymagane
change_class	str; pole wymagane
affected_concepts	Tuple[str, ...]; pole wymagane

Wybrane pola z architecture_formalization_manifest.py, klasa w liniach 13–47; całość deklaracji: 20 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S036. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I038

38 · FederatedFormalizationBinding zamyka wymiar między repozytoriami

Globalny wpływ wymaga wskazania, co zmienia się w każdym zależnym repozytorium.

FederatedFormalizationBinding wiąże lokalny AFM z dokładnym baseline federacji i dyspozycjami repozytoriów. UPDATE, VALIDATE_ONLY i NOT_APPLICABLE mają różne konsekwencje. Nie każdy peer musi otrzymać patch, ale dla każdego trzeba rozliczyć, czy jego kontrakt lub wiedza zależą od danej delty.

Taki model pozwala uniknąć dwóch skrajności: zmieniania całej federacji przy lokalnej korekcie oraz ignorowania konsumenta poza aktualnym checkoutem. Odpowiedni poziom abstrakcji nie jest tu maksymalną centralizacją. Jest zgodną reprezentacją relacji między niezależnymi źródłami i organizacjami wykonania.

Przyszła stabilizacja może więc mieć lokalne okna kompatybilności. Nowy provider współistnieje przez określony czas ze starym adapterem, a wycofanie jest jawnie rozliczone. Nie należy jednak mechanicznie wprowadzać dual-write dla stanów władzy lub postępu misji. Każdy taki wzorzec musi zachować jednego właściwego właściciela rozstrzygnięcia danej domeny.

Rozwinięcie analizy źródłowej

FederatedFormalizationBinding wiąże lokalny manifest ze wspólnym baseline i dyspozycjami repozytoriów. UPDATE oznacza wymagany przyrost, VALIDATE_ONLY sprawdzenie zachowanej zgodności, a NOT_APPLICABLE jawne wyłączenie. Ten trzeci stan nie powinien być używany jako wygodny sposób zmniejszenia mianownika; potrzebuje uzasadnienia wynikającego z zależności. Dzięki takiemu zapisowi federacja może ewoluować etapami. Jedno repozytorium dodaje provider, inne tylko potwierdza kompatybilność, a jeszcze inne pozostaje bez zmian. Wspólny model zachowuje pełną mapę wpływu bez wymuszania monorepo lub jednoczesnego deploymentu wszystkich części.

```
jedna delta lokalna + exact federation baseline + dyspozycje peerów
```

UPDATE	Wymagana zmiana w danym repozytorium.
VALIDATE_ONLY	Sprawdzenie zgodności bez wymuszania patcha.
NOT_APPLICABLE	Jawnie uzasadniony brak wpływu w tym zakresie.

Ślad implementacyjny · RepositoryFormalizationDisposition

repository	str; pole wymagane
baseline_head	str; pole wymagane
baseline_tree	str; pole wymagane
disposition	str; pole wymagane
required_artifact_ids	Tuple[str, ...]; pole wymagane

Wybrane pola z federated_formalization_binding.py, klasa w liniach 30–58; całość deklaracji: 5 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S048. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I039

39 · Holomozaikowość jest kompozycją znaczeń

Warstwa semantyczna może organizować wiadomość, ale nie może zastąpić jej tożsamości i dostarczenia.

Pakiet komunikacyjny proponuje LCCP jako logiczny agregat istniejących ownerów. Nie opisuje gotowej nowej usługi ani kolejnej kolejki. HMK-9D jest draftem semantyki procesu, a CommunicationEnvelope istniejącym, ścisłym kontraktem. Te dwie role można zestawić przez referencje i adapter bez utożsamienia ich formatów.

Źródłowy ProcessSemanticsSpec zachowuje dziewięć osi w ustalonej kolejności oraz runtime_identifier_policy=INTERNAL_ONLY. Wartości osi i bridge'ów w draftowym protokole nie są pomiarami rzeczywistej energii lub formalnym PEP. Nadają modelowi język opisu kompresji, skali, relacji i punktów przejścia; ich skuteczność wymaga własnej kwalifikacji.

Fundamentalny sens tego projektu polega na łączeniu różnych projekcji tej samej intencji bez utraty ich odrębności. LOCAL i SaaS mogą otrzymywać różne reprezentacje wspólnego kontekstu. Dron i worker mogą mieć inne obowiązki wobec jednego fanoutu. Warunkiem kompozycji jest zachowanie identyfikatorów, ograniczeń i lineage — nie wytworzenie jednej wszechogarniającej koperty.

Rozwinięcie analizy źródłowej

Nowy pakiet komunikacyjny wnosi szczegółowe granice: prepare nie oznacza send, claim nie oznacza odczytu przez model, zewnętrzny thread nie jest kanoniczną rozmową, a participant nie jest materialnym workerem. Każda z nich może być sformalizowana i negatywnie testowana. To właśnie przejście od trafnej metafory mozaiki do sprawdzalnego kontraktu. R3 nie usuwa metafory, lecz nadaje jej punkty odniesienia w plikach, stanach i digestach. Gdy fragment pozostaje projektowany, oznacza go jako TARGET. Związek nazwany holomozaikowym nie staje się przez to automatycznie działającą usługą albo nowym właścicielem transportu.

```
semantyczna soczewka + koperta + routing + ledger; bez zlania ownerów
```

Istniejące	CommunicationEnvelope i ProcessSemanticsSpec.
Projektowane	HolomosaicRoutingFrame, bootstrap i adaptery.
Historyczne	Dziewięć pending requests nie jest nowym odczytem R3.

Źródła: R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

WSTĘP ŹRÓDŁOWY R3-I040

40 · Natura systemu, którą można formalizować

LION wyłania się jako układ rozliczanej zmiany zdolności, a nie tylko generator kolejnych plików.

Przekrój źródeł ujawnia powtarzalny wzorzec: najpierw powstaje reprezentacja przedmiotu i problemu, potem wariant działania, a następnie osobno kwalifikowany skutek. Wynik wraca przez obserwację i rekonsyliację do wiedzy. To synteza z rozpoznanych kontraktów, nie dowód, że cała pętla została już spięta w produkcyjnym runtime.

Można sformalizować stałe rozróżnienia, choć konkretne implementacje pozostają zmienne. Źródło różni się od projekcji, intencja od wykonania, capability od authority, receipt od niezależnej obserwacji, a historyczny zapis od aktualnej interpretacji. Dzięki tym granicom system może rozwijać przyszłość i korygować teraźniejszość bez wymazywania przeszłości.

Proponowany najwyższy model jest więc temporalną federacją kontraktów, zdolności i dowodów. CORE powinien utrwalać te mechanizmy, które zachowują znaczenie przez kolejne implementacje i kontrprzykłady. Wartość eksperymentu nie kończy się na poprawnym artefakcie: pozostaje również lepszy opis problemu, nowy falsyfikator i bardziej precyzyjna reguła następnego działania. To uzasadniony kierunek formalizacji wyprowadzony z plików, z zachowaniem otwartych granic integracji.

Rozwinięcie analizy źródłowej

Synteza nie polega na udowodnieniu, że wszystkie części już tworzą autonomiczny organizm. Polega na rozpoznaniu powtarzalnego sposobu organizacji: jawne źródło, ograniczona projekcja, kandydat, test, osobne dopuszczenie, obserwacja i powrót do wiedzy. Ten wzorzec znajduje się w wielu niezależnych powierzchniach kodu i dokumentacji, choć ich pełne połączenie nadal wymaga odbioru. Można więc formalizować naturę systemu jako projektową zasadę i rozpoznaną strukturę kontraktową, nie ogłaszając produkcyjnego E2E. Następny dokument powinien rozwijać właśnie konkretne brakujące przejścia, a nie ponownie opisywać od zera całą ideę federacji.

```
rozliczona ewolucja = zachowana tożsamość + zmiana + dowód + nowa wiedza
```

Możliwe teraz	Formalizacja ról, niezmienników i zakresów.
Nadal osobne	Pełne E2E, runtime admission, deployment i promocja RAG.
Następna iteracja	Eksperyment korzysta z utrwalonej podstawy i zachowuje kontrprzykłady.

Źródła: R3-S024. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

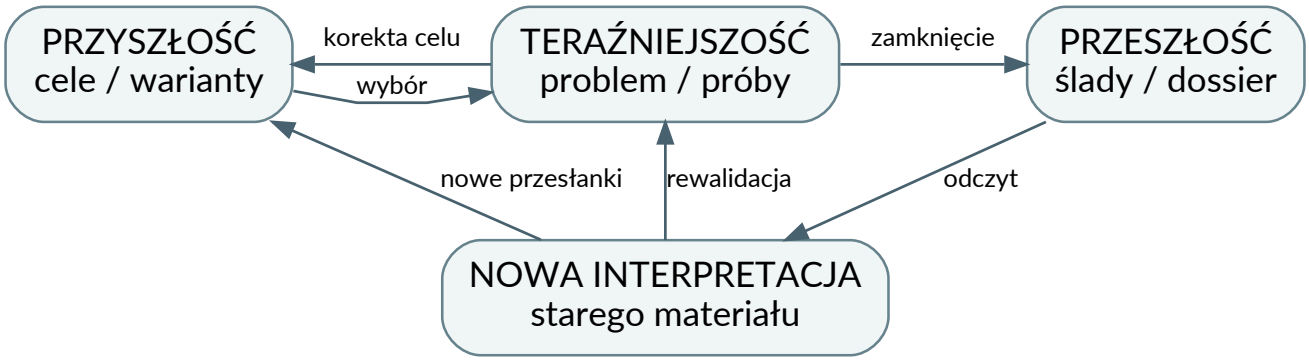
Atlas architektury ewolucyjnej

Pętla · poziomy · Git · AST · trzy horyzonty czasu

137 stron | 7 poziomów rozwinięcia | 41 widoków

Przyszłość generuje warianty. Teraźniejszość konfrontuje je z rzeczywistością. Przeszłość zachowuje ślady i otrzymuje kolejne interpretacje.

Projekt nadrzędnej logiki LION i jej rozwinięcia od domen odpowiedzialności przez kontrakty oraz procesy do artefaktów, kodu i obserwacji. Uporządkowana podstawa następnego dokumentu szczegółowego.



Status: AS-IS PARTIAL. TO-BE jest warunkowym projektem. Nie zakończono pełnego odczytu wszystkich gałęzi, dokumentów i środowisk. Nie wykonano migracji produktu, uruchomienia misji ani promocji wiedzy do RAG.

Nowa delta badawcza: 45 testów Code Perception oraz 12 jawnych prób Git–AST–czas. Rozpoznany przykład zachowania drzewa przy zmianie genealogii commitów writeups. Źródłowy baseline ai_platform: d367e47cf4f72e450cc212908b2cdfed14ceb805.

Mapa dokumentu

Czytaj od logiki L0; potem wybierz domenę i przejdź do kontraktów, procesu, artefaktów, kodu oraz obserwacji. Zakładki PDF prowadzą także do poszczególnych kart.

I ORIENTACJA 4

II PĘTLA NADRZĘDNA 7

III CZAS ORGANICZNY 13

IV HIERARCHIA ROZWINIĘCIA 22

V GIT JAKO PERCEPCJA 31

VI AST I LOGIKA KODU 43

VII SEMANTYKA I PAMIĘĆ 55

VIII PROCES I FABRYKA 64

IX FEDERACJA 72

X KANONICZNE PRZEPŁYWY 83

XI NORMALIZACJA I CORE 93

XII TRANSFORMACJA 99

XIII WIZUALIZACJA JAKO DOKUMENTACJA 112

XIV DOWODY I GRANICE 116

XV REJESTR ŹRÓDEŁ 121

XVI MAPA OBIEKTÓW 131

Trzy osie i granica dowodu

Szczegółowość, czas i status są niezależne. Nie wolno zamienić siedmiu poziomów dokumentu w siedem nowych warstw runtime.

L0 · Pętla ewolucyjna — Dlaczego system się zmienia i kiedy obieg można uznać za rozliczony?

L1 · Domeny odpowiedzialności — Jakie funkcje muszą współpracować bez dublowania swoich właścicieli?

L2 · Kontrakty i niezmienniki — Co dokładnie może przekraczać granicę domeny?

L3 · Procesy i współpraca — W jakiej kolejności i pod jakim warunkiem używane są kontrakty?

L4 · Artefakty i magazyny — Gdzie znajduje się dana wersja modelu, programu i dowodu?

L5 · Kod, AST i semantyka statyczna — Jak implementacja realizuje kontrakt i gdzie kończy się dowód ze źródeł?

L6 · Wykonanie i obserwacja — Co naprawdę działało i co rzeczywiście się wydarzyło?

Horyzonty F / P / H

F: cele i możliwe realizacje. P: aktywny problem, modele, workery i dowody. H: zachowane oryginały oraz kolejne interpretacje. Archiwalne bajty nie zmieniają się wraz z nową oceną.

Dowód i wizualizacja

Linia ciągła pokazuje nazwaną relację w klasie modelu danego widoku; nie oznacza automatycznie obserwowanego wywołania. Linia przerywana wyróżnia przejście warunkowe, propozycyjne lub wymagające odrębnego wiązania. Katalogi SOURCE, TARGET i HISTORY należy czytać wraz z tekstem ograniczenia.

AT2-Pxxx oznacza kartę. AT2-Gxx i AT2-Fxx oznaczają diagram. G01–G03 bez prefiksu AT2 są identyfikatorami odczytów Git w rejestrze źródeł. C01–C37 zachowują tożsamość obiektów R1; C38–C42 są dodatkami modelu dokumentacyjnego, nie nowo utworzonymi modułami produktu.

Najpierw pętla, potem komponenty

Najwyższą jednostką projektowanej architektury jest rozliczona zmiana zdolności, a nie pojedynczy model, repozytorium albo liczba workerów.

W istniejących źródłach LION występuje rozdział rozumowania, reprezentacji, propozycji, authority, admission, skutku, obserwacji i rekonsyliacji. Raport R1 nie wykazał zamknięcia wszystkich przejść w jedną produkcyjną fabrykę. R2 zachowuje ten wynik i rozwija go w hierarchiczny atlas: najpierw opisuje sens pętli, następnie obowiązki jej części, kontrakty, procesy, artefakty, kod i runtime.

W tym projekcie kolejna generacja rozpoczyna się nie od wymazania poprzedniej, lecz od nowej interpretacji rozpoznanego problemu. Cel może ulec zmianie, kandydat może zostać odrzucony, a dowód utracić aktualność. Ciągłość zapewnia tożsamość epizodu, źródłowe odniesienia oraz jawne relacje następstwa. Stabilizacja jest lokalnym rezultatem kwalifikacji, nie końcem przyszłości.

Atlas służy przygotowaniu następnego dokumentu szczegółowego. Każda karta wskazuje granicę rozwinięcia i źródła, aby można było powiększyć wybrany fragment bez projektowania całej federacji ponownie. Nowa hierarchia opisowa nie dodaje schedulera ani nie zmienia natywnych enumów LION.

Rezultat	Hierarchiczny model i kontrakty rozwinięcia.
Niezastosowane	Migracja kodu, zmiany hostów, aktywacja misji i promocja RAG.
Najważniejszy warunek	Projekt nie może podawać nieodtworzonego runtime jako faktu.

Źródła: R1, S01, S02, S03 | PROJEKT R2 / AS-IS OGRANICZONY

Co zostało odzyskane i co zbadano teraz

Wznowienie zachowuje wcześniejszy dorobek, lecz nie dziedziczy niewidocznych wyników nieukończonego wykonania.

Po przerwaniu poprzedniej próby dostępne były podglądy stron atlasu, lecz nie gotowy plik PDF ani pełny tekst jego wersji roboczej. Niniejsze wydanie jest ponownym złożeniem na podstawie zachowanych załączników, raportu R1, źródeł pokazanych przez GitHub oraz nowych prób offline. Podglądy nie zostały zamienione w rzekomo pełny, indeksowalny dokument przez powiększenie bitmap.

Odzyskano pięć załączników osadzonych w R1, w tym model i pakiet dowodowy. Ponownie sprawdzono integralność wejściowych archiwów. Z hosta pobrano istniejący pakiet 268 plików źródłowych z historycznego badania; nie jest to pełne bieżące repozytorium. Pięć plików potrzebnych do testów Code Perception porównano z dokładnymi identyfikatorami blobów GitHub na d367e47c… .

Nowy wynik wykonania obejmuje 45 oryginalnych testów Code Perception oraz 12 jawnych prób Git–AST–czas. Dziesięć prób korzysta z rzeczywistego buildera grafu na syntetycznych wejściach; dwie sprawdzają wyłącznie proponowany model temporalny i hierarchię. Nie wykonywano inferencji SaaS/LOCAL ani produkcyjnego efektu.

Aktualne wykonanie	45 testów źródłowych + 12 prób badawczych; osobne logi.
Dziedziczone wyniki	Testy Filter/SHD z R1 pozostają historycznymi wynikami R1.
Status pełnego rozpoznania	PARTIAL; literalne „cały ekosystem” nie zostało zamknięte.

Źródła: R1, T01, T02, A01 | PROJEKT R2 / AS-IS OGRANICZONY

Jak czytać atlas

Poziom szczegółowości, horyzont czasu i rodzaj dowodu są trzema niezależnymi osiami każdego widoku.

Czytaj od karty L0, następnie wybierz odpowiedzialność w L1 i przejdź do jej kontraktów oraz procesu. Rozdziały Git i AST wyjaśniają, jak zakotwiczyć model w konkretnych bajtach. Karty federacji pokazują lokalne specjalizacje; karty FLOW zachowują nazwy istniejących przepływów. Mapę transformacji traktuj jako zbiór propozycji następnych przyrostów.

ŹRÓDŁO oznacza rozpoznaną definicję lub mechanikę w dokładnym źródle. OBSERWACJA dotyczy konkretnego odczytu z jego czasem. WYNIK TESTU wymaga rzeczywistego wykonania wskazanej próby. PROJEKT oznacza organizację postulowaną w R2. HISTORYCZNE nie jest synonimem fałszu: określa epokę, do której należy zapis. UNKNOWN nie może zostać usunięte tylko po to, aby graf wyglądał na zamknięty.

Numery L0–L6 to skala rozwinięcia dokumentacji. Nie zastępują piętnastu warstw ARCHITECTURE_LAYERS ani płaszczyzn INF/SEM/MAND. Diagram przedstawia relacje nazwane w swojej legendzie; sama strzałka nie jest obserwacją wywołania, a umieszczenie komponentu w docelowej pętli nie dowodzi jego wdrożenia.

Identyfikatory	AT2-P…: karty; AT2-G…: widoki; D…: plan; P…: próby.
Nawigacja	Nagłówki PDF, indeks źródeł i model JSON opisują ten sam zbiór kart.
Czytelnik RAG	Zachowaj status, źródło, epokę i ograniczenie razem z tezą.

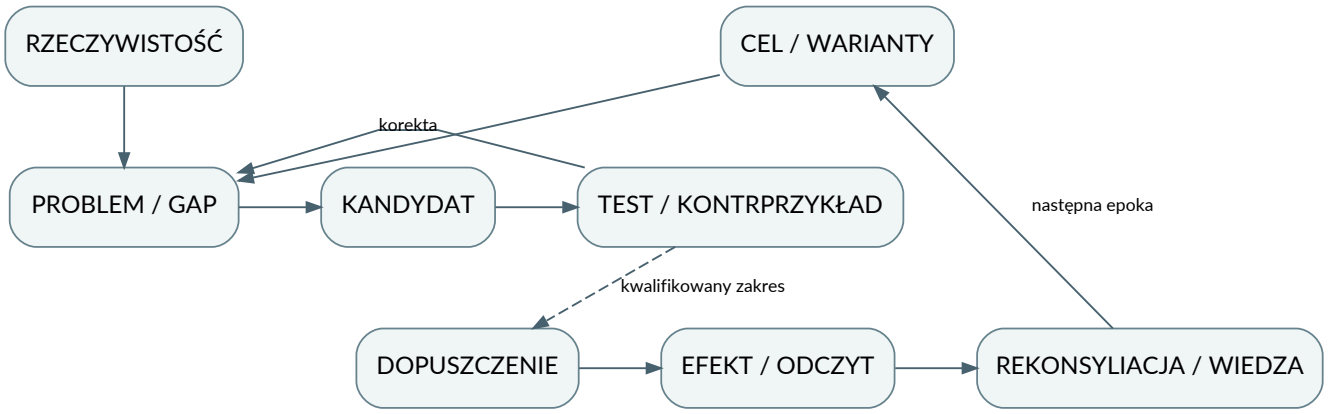
Źródła: R1, S02, S04 | PROJEKT R2 / AS-IS OGRANICZONY

L0 · obieg rozliczonej ewolucji

Pętla jest zamknięta dopiero wtedy, gdy rezultat działania zmienia źródłowo uzasadnioną wiedzę o problemie.

GENERATION_EVOLUTION_PROTOCOL opisuje przejście OBSERVE → STRUCTURE → HYPOTHESISE → PROPOSE CHANGE, następnie analizę delty, kompatybilności, bezpieczeństwa, test i ograniczone wykonanie. Po receipt pojawiają się odrębne obserwacja wyniku oraz kandydat pamięci. Ten porządek jest podstawą syntezy L0, nie nowym harmonogramem uruchamianym obok Mission Control.

Przyszłość dostarcza celu i wariantów, teraźniejszość konfrontuje je z warunkami, a przeszłość przechowuje rozliczone ślady. Wynik negatywny również może zamknąć epizod: usuwa błędną hipotezę lub wyznacza granicę capability. Sam komunikat SUCCESS nie wystarcza, jeśli nie wiadomo, jaki artefakt powstał, kto go zweryfikował i do której generacji należy.



AT2-G01 · Pętla rozliczonej ewolucji. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Wejście	Cel + aktualny obraz rzeczywistości + zachowane ograniczenia.
Wyjście	Rozliczona delta zdolności lub źródłowo utrwalony kontrprzykład.
Sprzężenie	Nowy problem, nowa interpretacja lub kwalifikowana stabilizacja.

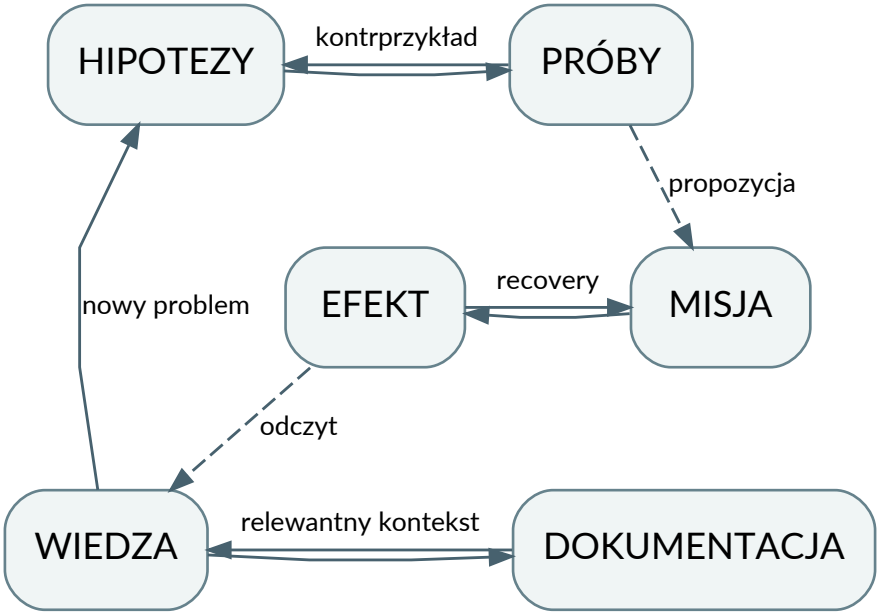
Źródła: S03, S05, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L0 · trzy sprzężone obiegi

Badanie hipotezy, wykonanie misji i utrzymanie architektury nie mają tego samego tempa ani tych samych warunków zakończenia.

Obieg poznawczy może produkować wiele konkurencyjnych interpretacji bez materialnego efektu. Obieg wykonawczy realizuje skończony kontrakt w dokładnym zakresie. Obieg homeostazy aktualizuje projekcje, dokumentację, formalizację i wiedzę po zmianie źródeł. Ich wzajemne wyniki są wejściami, lecz żaden z nich nie może po cichu zastąpić pozostałych.

W R2 proponuje się wspólną tożsamość epizodu ponad tymi obiegami oraz odrębne stany ich postępu. Dzięki temu można badać nową hipotezę, gdy materialne wdrożenie czeka na zgodę, i można odświeżyć opis starego wyniku bez ponownego uruchamiania tego efektu. Opóźnienie jednego procesu nie powinno bez uzasadnienia zatrzymywać niezależnej pracy.



AT2-G02 · Trzy sprzężone obiegi. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Poznanie	Hipoteza → falsyfikator → wynik w zadeklarowanym zakresie.
Wykonanie	Kontrakt → admission → efekt → niezależny readback.
Homeostaza	Delta źródeł → wpływ → projekcje → odnowiona wiedza.

Źródła: S03, S05, S06, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Co rozpoczyna kolejny obieg

Nowy cel, kontrprzykład i zmiana rzeczywistości są różnymi wyzwalaczami; sam upływ czasu nie dowodzi potrzeby kolejnego efektu.

Źródłowa ścieżka Project Reality rozpoczyna się od HUMAN GOAL / SYSTEM ALERT i ponownego pozyskania obserwacji. Odczytany entrypoint Mission Control natomiast migruje bazę, rekoncyliuje kontrakty i uruchamia pętle obsługi. Zależność logiczna nowego badania nie jest zatem identyczna z kolejnością startu procesów systemowych.

W projektowanej dokumentacji każdy trigger otrzymuje producenta, warunek aktywacji, sposób deduplikacji oraz właściciela obsługi. Alert o starej projekcji może uruchomić odczyt, nie musi uruchamiać mutacji. Zmiana celu tworzy nową wersję intencji, a nie retroaktywną podmianę celu wcześniejszego eksperymentu. Brak capabilities może prowadzić do programu ich budowy, zamiast do nieskończonego WAIT.

Cel operatora	Nowa intencja, scope, kryterium przyjęcia.
Kontrprzykład	Rewizja hipotezy; nie kasowanie poprzedniego wyniku.
Drift źródła	Rewalidacja zależnych tez i bindings.
Restart	Odtworzenie i reconciliation, nie automatyczny replay.

Źródła: S05, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Jednostka ewolucji: epizod i delta

Epizod pozwala zachować ciągłość problemu; delta ogranicza rozmiar konkretnej zmiany.

Protokół generowania zawiera regułę jednej pętli prowadzącej do jednego głównego artefaktu lub spójnej zmiany kontraktowej. EvidenceBoundLearningEpisode wiąże natomiast referencje wiadomości, invocations, assignments, wyników, obserwacji i rekonsyliacji bez posiadania ich kolejek. Są to uzupełniające się role: jednostka poznawcza i jednostka kontrolowanego przyrostu.

Projekt R2 nie narzuca jednego pliku na każdą misję. Spójna zmiana kontraktu może wymagać testu, adaptera i aktualizacji dokumentacji. Musi jednak posiadać jeden sens odbioru i rozliczalny zakres zależności. Dekompozycja ogranicza przypadkowe sprzężenia: poprawa semantyki nie powinna przy okazji zmieniać grantu, topologii floty i historii rozmowy.

Epizod	Dlaczego i na jakich przesłankach podjęto pracę.
Delta	Co dokładnie ma się zmienić względem baseline.
Wydanie	Które kompatybilne delty tworzą stabilizowaną podstawę.
Tożsamość	Każdy następca ma jawnych rodziców i własną generację.

Źródła: S03, S07, S08 | PROJEKT R2 / AS-IS OGRANICZONY

Selekcja nie jest wykonaniem

Wybór najlepszego kandydata nie nadaje mu uprawnień i nie powoduje powstania materialnego wyniku.

Dokument Project Reality opisuje adapter obserwacji do Dynamic Evolution Fitness. Hard gates poprzedzają wybór kandydata; presja i scoring nie zmieniają FAIL ani UNKNOWN w PASS. ArchitectureCompiler wytwarza deltę i wymagania formalizacji, zatrzymując się przed governed admission. To rozdział odpowiedzialności, który model najwyższego poziomu musi zachować.

W przyszłej wizualizacji należy rozróżnić kandydatów możliwych, dopuszczalnych strukturalnie, wybranych do dalszej pracy oraz dopuszczonych do konkretnego skutku. Różne rankingi mogą współistnieć dla różnych celów, ale nie mogą istnieć dwa nieuzgodnione kursory wykonujące tę samą misję. Wyjaśnienie wyboru powinno wskazywać odrzucone alternatywy i brakujące dowody.

Ocena	Źródłowe bramki i jawnie wybrana funkcja celu.
Wybór	Jeden ograniczony kandydat do dalszego procesu.
Dopuszczenie	Oddzielne aktualne authority oraz runtime binding.
Wynik	Obserwacja, nie deklaracja wygranego wariantu.

Źródła: S05, S09, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Stabilizacja jako lokalny przedział

Stable jest profilem przyjętego zakresu w określonej epoce, a nie ontologicznym końcem rozwoju.

LION rozdziela lifecycle, currentness, authority, execution, verification i deployment. Dlatego zintegrowany kontrakt może być stabilną bazą testów, podczas gdy jego provider nie został wdrożony. Działający proces może z kolei mieć niepełne wejścia lub niespójny kontrakt. Globalny kolor zielony zaciera oba przypadki.

Projektowana stabilizacja wymaga jawnego ownera, wersji, konsumentów, testów negatywnych i warunków rewizji. Okno stabilności kończy nowy kontrprzykład albo zmiana zależności, nie arbitralna liczba dni. Nie wszystkie komponenty muszą stabilizować się jednocześnie. Eksperyment może korzystać z wersjonowanego rdzenia, nie wprowadzając własnej niepewności do całego systemu.



AT2-G03 · Stabilizacja zakresu. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Stabilny kontrakt	Znana semantyka i zachowana kompatybilność.
Stabilne źródło	Przypięty commit/tree oraz zakres testu.
Stabilny runtime	Oddzielnie odebrana konfiguracja i skutki.
Rewizja	Nowa delta oraz relacja supersession.

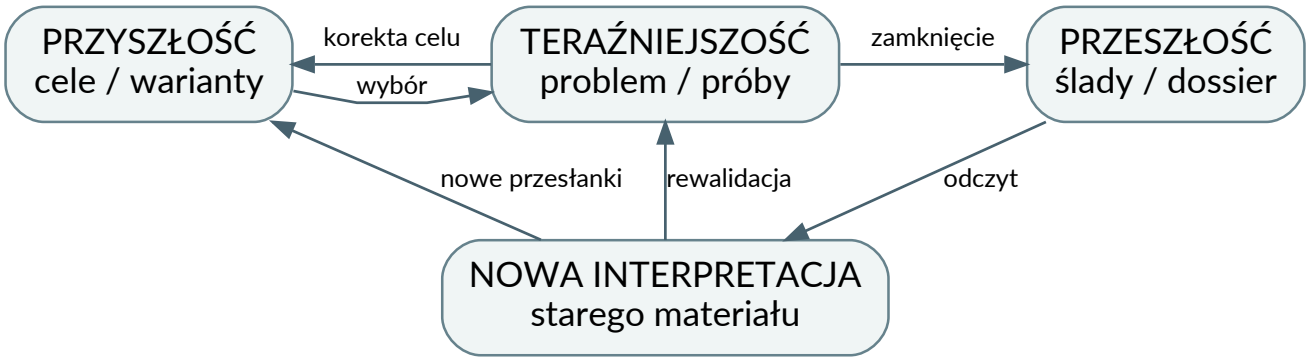
Źródła: S10, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Trzy horyzonty jednej architektury

Przyszłość, teraźniejszość i przeszłość są rolami informacji; nie muszą być trzema katalogami ani trzema niezależnymi bazami.

Przyszłość obejmuje cele, hipotezy kierunku i projekty architektury. Teraźniejszość jest przestrzenią aktywnego rozstrzygania: powstają modele, wyniki prób, kandydaci i dowody. Przeszłość zawiera utrwalone źródła oraz ślady zakończonych epizodów. W kolejnej iteracji te same artefakty mogą uczestniczyć w nowej roli, lecz ich pierwotne bajty i tożsamość nie zmieniają się wstecz.

„Ewoluująca przeszłość" jest w R2 interpretowana jako przyrost wiedzy o historii: odkrywanie brakujących powiązań, nowe kontrdowody, reindeksacja i wersjonowana reinterpretacja. Nie oznacza cichego poprawiania starego logu testu. Właśnie rozdział niezmiennego świadectwa od zmiennego modelu jego znaczenia pozwala zachować organiczny charakter systemu bez utraty rozliczalności.



AT2-G04 · Trzy horyzonty czasu. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Przyszłość	Możliwe struktury i warunki ich akceptacji.
Teraźniejszość	Aktywny problem, praca i jeszcze nierozstrzygnięte ślady.
Przeszłość	Oryginały + narastający graf interpretacji historii.

Źródła: S03, S07, S10, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Przyszłość: cele i strategia budowy

Cel jest ograniczeniem poszukiwania, a nie gotową trasą ani obietnicą osiągniętego stanu.

GoalContract, MissionIntent, roadmap oraz CandidateDesign zajmują różne miejsca w istniejącej terminologii. Cel opisuje oczekiwanie, intencja zachowuje zakres pracy, roadmap organizuje zależności, a kandydat reprezentuje konkretną możliwą zmianę. Zlanie tych obiektów w jeden „plan" utrudnia wykrycie, czy zmieniono wymaganie, rozwiązanie, czy jedynie kolejność implementacji.

W atlasie horyzont przyszłości jest wersjonowanym zbiorem alternatyw. Każda ma przesłanki, ograniczenia, falsyfikator i warunek przejścia do aktywnego badania. Strategia może dostosowywać się do kosztu i obserwacji bez przepisywania zakończonych epizodów. Podczas rewalidacji aktualizuje się zależności kandydatów; nie odrzuca się całej historii tylko dlatego, że zmienił się docelowy układ modułów.

Cel	Co ma stać się możliwe lub poprawione.
Strategia	Które luki rozwiązać wcześniej i dlaczego.
Alternatywa	Konkretny wariant z kosztem i ryzykiem.
Przejście	Wybrany zakres staje się obecnym problemem badawczym.

Źródła: S03, S05, S09 | PROJEKT R2 / AS-IS OGRANICZONY

Teraźniejszość: ewoluujący problem

W teraźniejszości zmienia się nie tylko rozwiązanie; zmienia się również rozumienie tego, jaki problem rzeczywiście rozwiązujemy.

SemanticScaffoldIR zawiera gap_refs, dependency_refs, constraints, hypotheses, counter_hypotheses, falsifiers i unknowns. Taki układ umożliwia utrzymanie kilku konkurencyjnych opisów przy zachowaniu rodziców intencji i kontekstu. Nie dowodzi samodzielnej ekstrakcji dowolnego języka, ale dostarcza istniejącego miejsca na zapis procesu rozumowania.

Praktyczny model teraźniejszości powinien wskazywać aktywne pytanie, jego bieżące brzmienie, poprzednią wersję i dowód uzasadniający zmianę. Worker otrzymuje ograniczony wycinek problemu, nie prawo do redefinicji celu całej federacji. Wyniki prowizoryczne pozostają rozpoznawalne jako prowizoryczne. Dopiero rozstrzygnięty epizod może stanowić podstawę kandydackiej reguły lub dokumentu referencyjnego.

Hipoteza	Testowalne wyjaśnienie, nie zaakceptowany fakt.
Kontrargument	Powód konkurencyjnego wyjaśnienia.
Dowód roboczy	Ślad z producentem, czasem i zakresem.
Zmiana problemu	Nowa wersja z jawną przyczyną i rodzicem.

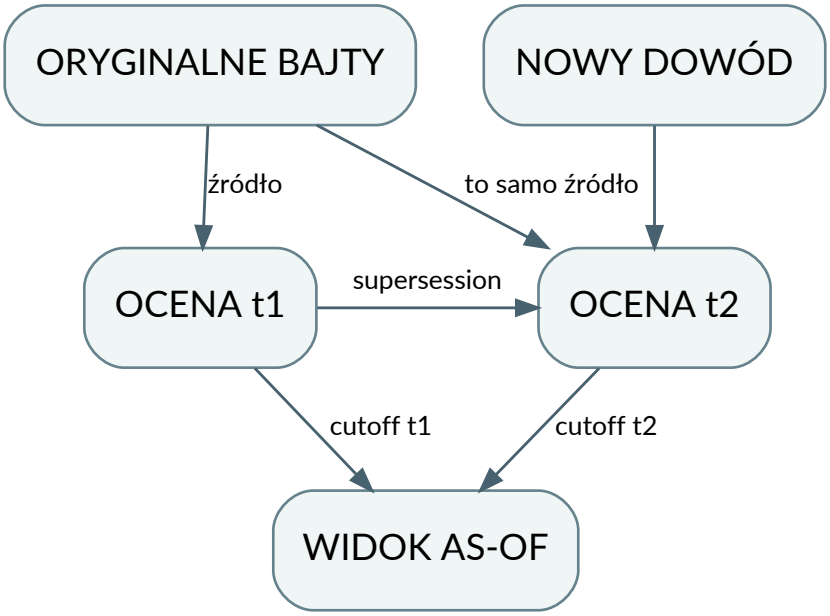
Źródła: S08, A-SHD, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Przeszłość: zapis i reinterpretacja

Niezmienność świadectwa i zmienność wiedzy o świadectwie nie są sprzecznością.

Git przechowuje historyczne obiekty, a kontrakty semantyczne dopuszczają SUPERSEDE, CONTRADICT, RETRACT i REVALIDATE. Można więc dodać nowy rekord, który zawęża poprzedni wniosek, nie edytując starego wyniku. R1 pokazał to dla testów SHD: wcześniejszy zielony suite zachowuje swój zakres, a nowe mutacje ujawniają nieobjętą właściwość.

W R2 archiwum ma dwa widoki: „co zapisano wówczas” oraz „co wiemy o tym teraz”. Oba wskazują te same oryginalne bytes, lecz korzystają z innego cutoff wiedzy. Nowy model nie może wstawić późniejszego dowodu do historycznego wejścia agenta. Reinterpretacja może zmienić status aktualnej tezy, a nie sprawić, że wcześniejszy badacz posiadał informację, której jeszcze nie miał.



AT2-G05 · Niezmienny ślad i zmienna interpretacja. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Warstwa dowodowa	Oryginał i jego tożsamość pozostają zachowane.
Warstwa interpretacyjna	Nowe relacje, kontrdowody i zakres zastosowania.
Widok historyczny	Tylko informacja dostępna w danym cutoff.
Widok dzisiejszy	Historia plus późniejsze, jawnie datowane rozstrzygnięcia.

Źródła: S07, S08, R1, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Sześć czasów jednego dowodu

Czas zdarzenia, odczytu i dostępności modelowi nie mogą zostać zastąpione jednym timestampem pliku.

COGNITIVE_EVOLUTION_INTEGRATION wymienia event_time, observation_time, ingestion_time, available_to_model_at, input_cutoff i decision_time. Są potrzebne do odtworzenia tego, co rzeczywiście mogło wpływać na decyzję. Data commitu albo czas utworzenia PDF nie ustalają wszystkich tych wartości.

Projekt temporalnej pamięci musi zachować rozdzielenie osi. Wynik uzyskany po decyzji może służyć do oceny rezultatu, ale nie do rekonstrukcji wejściowej wiedzy. Dane spóźnione powinny tworzyć nowe projekcje z informacją o opóźnieniu. Porządek przyczynowy nie może wynikać wyłącznie z zegarów różnych hostów, których synchronizacji nie sprawdzono.

event_time	Kiedy zaszło opisywane zdarzenie.
observation / ingestion	Kiedy je odczytano i wprowadzono do magazynu.
available_to_model_at	Od kiedy dany konsument miał do niego dostęp.
cutoff / decision_time	Granica wejścia i moment decyzji.

Źródła: S07, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Dossier przed archiwizacją

Archiwizacja jest domknięciem opisu epizodu, nie zastępstwem bieżącego gromadzenia dowodów.

W teraźniejszości dossier łączy wymaganie, warianty, specyfikację, testy, wynik, kontrargumenty i otwarte ograniczenia. Źródła LION posiadają artefakty kandydatów i referencje evidence, ale pełna operacyjna retencja wszystkich stores pozostawała otwarta w R1. R2 projektuje zatem kontrakt dossier, nie ogłasza istnienia nowej bazy.

Przed zamknięciem należy ustalić, które rezultaty są potwierdzone, które symulowane, które odrzucone i które nierozstrzygnięte. Archiwum nie powinno gubić nieudanego kandydata, bo właśnie on wyznacza granicę generalizacji. Brak końcowego verdictu może być poprawnym zakończeniem badania z ograniczonym zakresem; nie jest zgodą na zakodowanie niewiedzy jako PASS.

Praca aktywna	Referencje robocze, mutacje i kolejne wyniki.
Przygotowanie zamknięcia	Dokładne bytes, wersje, zakres i wynik rewalidacji.
Archiwum	Niezmienny nośnik oraz odwołania następnych interpretacji.
Promocja	Osobna decyzja o użyciu jako wiedzy lub normy.

Źródła: R1, S03, S07 | PROJEKT R2 / AS-IS OGRANICZONY

Stan przejściowy nie jest błędem

WAIT, UNKNOWN i częściowy wynik mają odrębne znaczenia; ich ukrycie usuwa informację potrzebną do dalszej ewolucji.

Process Contract Plane rozdziela intencję fazy od capability binding i completion. W R1 wskazano, że działający scheduler może nie wykonać docelowej misji, jeśli nie została zarejestrowana, aktywowana lub związana z właściwym providerem. To nie dowodzi, że cały system niczego nie wykonuje.

Projektowany widok teraźniejszości musi wyjaśniać, co blokuje postęp i czy blokada jest informacyjna, implementacyjna, operacyjna czy uprawnieniowa. Te kategorie prowadzą do innych działań: odczytu źródła, stworzenia adaptera, poprawy deployu albo uzyskania osobnego dopuszczenia. Nie każdą blokadę rozwiązuje retry i nie każda wymaga nowej floty.

UNKNOWN	Brak wystarczającej obserwacji lub rozstrzygnięcia.
WAIT	Znany warunek oczekiwania w skończonym procesie.
FAIL	Niespełnione wymaganie w zadanym teście lub kontrakcie.
PARTIAL	Poprawny podzakres, bez twierdzenia o całości.

Źródła: R1, S10 | PROJEKT R2 / AS-IS OGRANICZONY

Nieprzemienność transformacji wiedzy

Kolejność interpretacji, agregacji i walidacji może zmieniać wynik; sam graf nie gwarantuje zachowania znaczenia.

Przykład projektowy: najpierw odrzucenie nieaktualnych źródeł, a potem zbudowanie relewantnej projekcji nie musi dać tego samego wyniku co projekcja, która wcześniej usunęła informację o aktualności. Podobnie złączenie równobrzmiących tez przed analizą genealogii może ukryć wspólny korzeń dowodowy.

R2 proponuje opisywanie operatorów projekcji przez zachowywane własności. Diagram przejścia powinien wyjaśniać, które ograniczenia są przenoszone bezstratnie, a które są agregowane. Nie jest to nowy dowód matematyczny działania LION; jest to warunek przyszłego testowania adapterów. Najważniejszym kontrprzykładem jest otrzymanie tej samej końcowej etykiety po utracie zasadniczego zakazu.

Najpierw	Tożsamość, cutoff, provenance i zakres.
Potem	Selekcja informacji oraz reprezentacja dla konsumenta.
Test	Porównaj przebiegi przy przestawieniu operatorów.
Wymaganie	Nie utracono constraints ani nie zwiększono authority.

Źródła: S08, A-SHD, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Rekurencja bez wymazywania rodziców

Nowa generacja ma być następcą, nie zamiennikiem udającym, że zawsze istniał.

Źródła projektu wymagają zachowania generation, source identity, conversation i mission bindings. Zmiana modelu, celu, źródła albo zakresu nie dziedziczy dopuszczenia automatycznie. W R2 ta sama zasada obejmuje dokumentację: nowy atlas ma relację do R1, ale nie zmienia historycznego ledgeru jego odczytów ani wyników.

W pętli samoewolucyjnej fabryka staje się jednocześnie przedmiotem i narzędziem zmiany. Trzeba więc oceniać kandydata na znanej podstawie, zachować różnicę między użytym mechanizmem i jego następcą oraz zapisać warunki replay. Nowa reguła może skorygować dawną kwalifikację, lecz nie może stanowić jedynego dowodu własnej poprawności.

Rodzic	Źródła i mechanizm użyte do utworzenia kandydata.
Następca	Nowy digest, zakres oraz zachowana genealogia.
Odbiór	Dowód odnoszący się do właściwej wersji.
Powrót	Wynik zmienia model rzeczywistości, nie oryginalny zapis.

Źródła: S03, S05, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L0 · Pętla ewolucyjna

Dlaczego system się zmienia i kiedy obieg można uznać za rozliczony?

L0 przedstawia wspólny sens wszystkich realizacji: problem zostaje skonfrontowany ze źródłami, kandydat poddany próbie, a wynik wraca jako wiedza. Model nie wymaga jednego monolitycznego agenta. Oddziela też trwanie organizacji od skończonych misji, których wyniki mogą mieć status negatywny lub częściowy.

Rozwinięcie L0 ma wskazać odpowiedzialności L1 i zachować granice epistemiczne. Nie wolno na tej skali pominąć kontroli skutku jako „detalu implementacyjnego". Pominięcie w diagramie technicznego providera jest dopuszczalną abstrakcją; pominięcie odrębności propozycji i uprawnienia zmienia znaczenie systemu.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

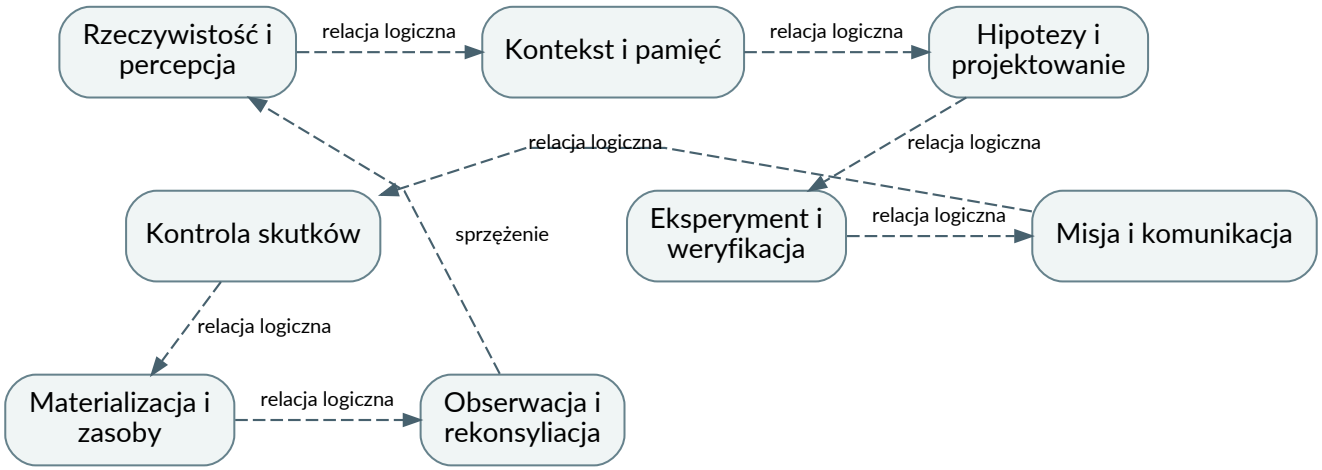
L1 · Domeny odpowiedzialności

Jakie funkcje muszą współpracować bez dublowania swoich właścicieli?

Wyróżniamy rozpoznanie rzeczywistości, kontekst i pamięć, generowanie wariantów, eksperyment i weryfikację, organizację misji, kontrolę skutków, materializację i rekonsyliację wiedzy. Są to domeny analityczne, które mogą obejmować więcej niż jeden moduł lub repozytorium.

Dla każdej domeny należy rozliczyć ownera, zakres i punkty styku. Nie przypisujemy automatycznie jednego repozytorium do jednego pudełka. ai_platform może pełnić wiele ról, a swarm pozostaje lokalną realizacją ograniczonych obowiązków. Tę samą domenę można pokazać z perspektywy przyszłego kontraktu i obecnego runtime.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.



AT2-G06 · Domeny odpowiedzialności. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L2 · Kontrakty i niezmienniki

Co dokładnie może przekraczać granicę domeny?

Na tym poziomie zachowujemy natywne typy i ich wersje: SourceIdentity, MissionIntent, SemanticScaffoldIR, PhaseExecutionContract, RuntimeAdmission i inne rozpoznane kontrakty. Nazwa pola nie wystarcza; potrzebne są preconditions, postconditions, pochodzenie, czas oraz sposób kwalifikacji błędu.

Dokument szczegółowy powinien pokazać minimalny poprawny rekord i rozstrzygające negatywne warianty. Jeśli kontrakt tylko sprawdza kształt, nie wolno przypisać mu weryfikacji zewnętrznego źródła. Podobieństwo JSON w peerach wymaga testu zgodności, a nie automatycznej zamienności.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L3 · Procesy i współpraca

W jakiej kolejności i pod jakim warunkiem używane są kontrakty?

Sekwencje, automaty stanów, porty komunikacji i skończone programy opisują przebieg. Istniejące FLOW-01–FLOW-10 oraz LPCL stanowią materiał do projekcji, ale sama lista kroków nie udowadnia ich runtime composition. Dla ścieżki krytycznej trzeba odnaleźć producenta i konsumenta każdego przejścia.

Poziom L3 zawiera warunki zakończenia, retry i recovery. Pętla hipotez może być cykliczna, natomiast DAG konkretnej implementacji musi rozliczać zależności. Opis procesu rozróżnia oczekiwanie na wiedzę od oczekiwania na zasób lub zgodę.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L4 · Artefakty i magazyny

Gdzie znajduje się dana wersja modelu, programu i dowodu?

Repozytoria, ścieżki, bloby, rekordy baz, epizody i pakiety są materializacjami wiedzy lub pracy. W jednym katalogu mogą współistnieć aktualne normy, kandydaci i historyczne wyniki. Dlatego struktura plików jest mapą przechowywania, nie gotową ontologią.

Każdy artefakt otrzymuje relacje produced_by, derived_from i consumed_by oraz sposób aktualizacji. Zachowujemy identyfikator oryginału, nawet gdy normalizujemy nazwę widoku. Decyzja o przeniesieniu pliku wymaga mapy importów, konfiguracji, pinów i możliwości odtworzenia poprzedniej wersji.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L5 · Kod, AST i semantyka statyczna

Jak implementacja realizuje kontrakt i gdzie kończy się dowód ze źródeł?

Natywny Code Perception wykorzystuje ast oraz symtable. Tworzy symbole i relacje, a wywołania klasyfikuje jako statyczne kandydatury. Część zależności pozostaje nierozwiązana z powodu dynamiki języka. FullSymbolCensus jest osobnym mechanizmem spisu, nie zamiennikiem dowodu zachowania.

Poziom L5 wiąże symbole z blobem, zakresem, wersją generatora i testami. Hash AST informuje o strukturze w ramach konkretnej metody, nie o dowiedzionej równoważności programów. Zmiana ścieżki może zmienić tożsamość symbolu mimo niezmienionych bajtów.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

L6 · Wykonanie i obserwacja

Co naprawdę działało i co rzeczywiście się wydarzyło?

Proces, kontener, host, przydział, wysłane bytes i odczytany skutek należą do oddzielnej płaszczyzny dowodowej. Health nie dowodzi pełnej zdolności misji. Dostępny endpoint nie dowodzi inferencji. Role buildera i verifiera nie ustalają automatycznie fizycznej niezależności.

Dokument L6 musi związać źródło, release, konfigurację, tożsamość wykonawcy, admission i obserwację. Brak takiego połączenia jest luką, nie prawem do uzupełnienia wykresu hipotezą. Zakończony test lokalny nie uzyskuje statusu produkcyjnego przez umieszczenie go na tym poziomie.

Każda karta rozwijająca ten poziom powinna zachować odniesienie do rodzica, granice zakresu i źródła. Przejście na niższy poziom dodaje szczegół, a nie nową władzę wykonawczą. Poziomy przecinają wszystkie trzy horyzonty czasu.

Rodzic	Odpowiedź na pytanie o szerszą całość.
Rozwinięcie	Składowe, porty i dowody zachowujące znaczenie rodzica.
Test spójności	Brak nowej niejawnej odpowiedzialności lub utraty ograniczeń.

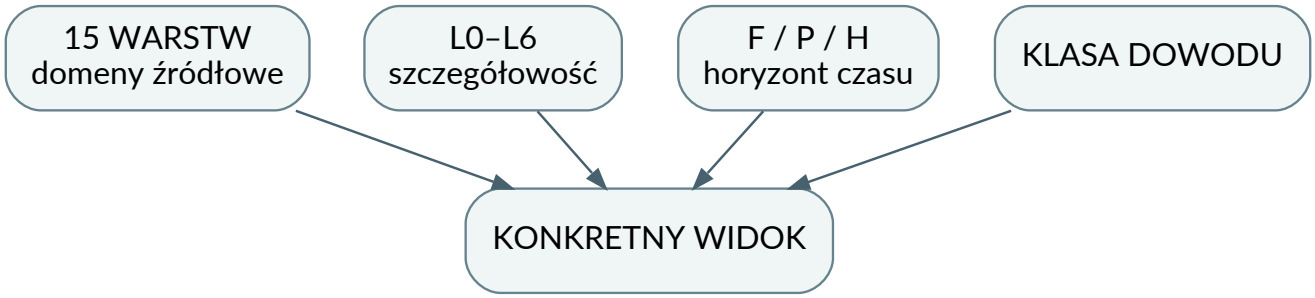
Źródła: S02, S04, S05, S08, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Siedem poziomów a piętnaście warstw

Nowa skala L0–L6 organizuje dokumentację; istniejące ARCHITECTURE_LAYERS organizują domeny modelu źródłowego.

flows.py deklaruje piętnaście warstw, między innymi EVIDENCE_AND_EPISTEMIC_PLANE, CODE_PERCEPTION, AUTHORITY_AND_EFFECT i ARCHITECTURE_PROJECTION. FullArchitectureModel wymaga reprezentacji wszystkich warstw oraz dziesięciu canonical flows. R2 nie dopisuje szesnastej warstwy i nie przekształca siedmiu poziomów powiększenia w konkurencyjną architekturę runtime.

Każdą warstwę natywną można rozwinąć od celu przez kontrakty i proces aż po kod oraz obserwację. Na przykład CODE_PERCEPTION jest w L1 odpowiedzialnością rozpoznania źródeł, w L2 rodziną kontraktów, w L5 konkretnym indekserem AST, a w L6 odrębnym wykonaniem tego indeksera. Przypisanie warstwy do jednego poziomu byłoby utratą informacji.



AT2-G07 · Niezależne osie widoku. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Warstwy natywne	Domeny i kategorie istniejącego modelu.
Poziomy L0–L6	Ziarnistość opisu każdej domeny.
INF / SEM / MAND	Płaszczyzny funkcjonalne z manifestów.
Horyzonty	Rola temporalna danej wersji informacji.

Źródła: S02, S04, S16 | PROJEKT R2 / AS-IS OGRANICZONY

Kontrakt rozwinięcia widoku

Rozwinięcie jest poprawne, gdy można przejść w obie strony między elementem nadrzędnym a jego składowymi.

Atlas proponuje relację refines jako mechanizm dokumentacyjny, a nie nowe pole obowiązkowe w produkcyjnych kontraktach. Dziecko identyfikuje rodzica, rozbijaną odpowiedzialność, zachowywane niezmienniki i porty przekraczające granicę. Poziom nadrzędny wskazuje zbiór dzieci oraz dopuszczalny zakres agregacji ich stanów.

Nie wolno obliczyć gotowości rodzica samym głosowaniem zielonych dzieci. Brak jednego krytycznego producenta może blokować całą ścieżkę, mimo że większość modułów jest zaimplementowana. W drugą stronę brak odbioru całej aplikacji nie unieważnia testu konkretnego parsera. Reguła agregacji stanu musi zależeć od tezy i od krytycznych relacji.

Element	ID, znaczenie, owner i zakres.
Rozwinięcie	Składowe, porty, constraints oraz źródła.
Agregacja	Jawny predykat gotowości danej tezy.
Kontrpróba	Usuń krytyczne połączenie bez zmiany liczby komponentów.

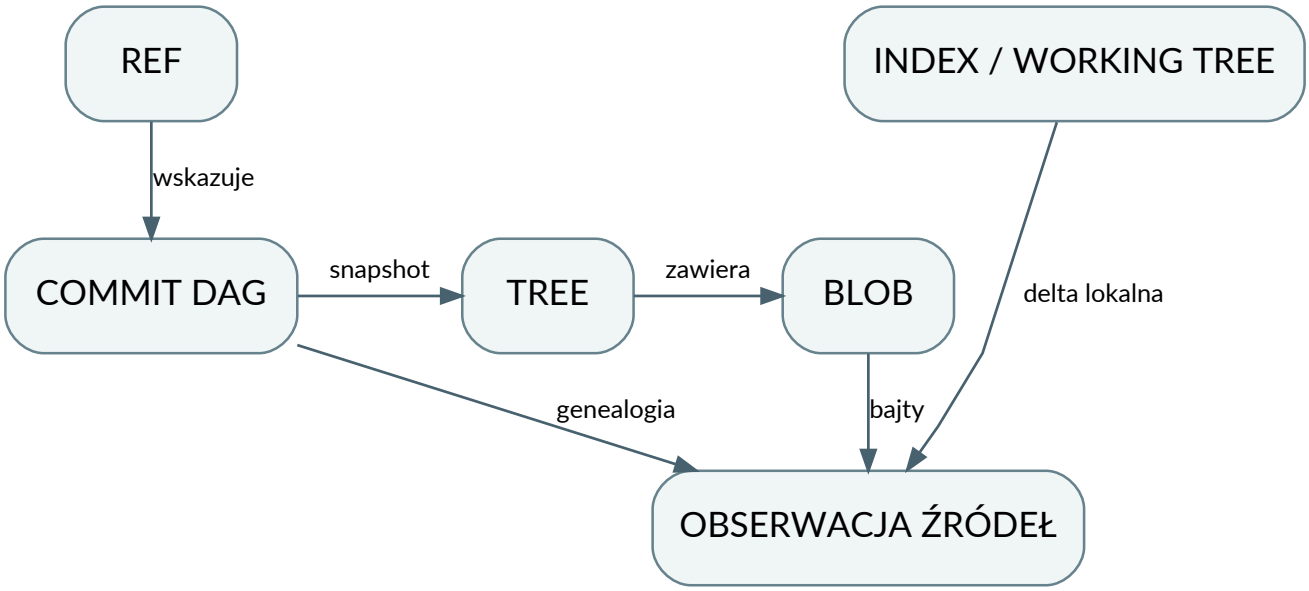
Źródła: R1, S04, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Cztery różne struktury Git

Drzewo plików, graf commitów, referencje i working tree odpowiadają na inne pytania o rzeczywistość projektu.

Git tree opisuje zawartość wybranej wersji wraz z nazwami, trybami i identyfikatorami obiektów. Commit wiąże tree z rodzicami i metadanymi. Ref wskazuje commit i może się zmienić. Index oraz working tree opisują przygotowywane i lokalne zmiany. Żadnego z tych obiektów nie wolno utożsamiać z topologią uruchomionej aplikacji.

Code Perception przyjmuje dokładne SourceIdentity i bloby. Granica Git rozwiązuje commit, tree, listę plików i bytes, a następnie sprawdza tożsamości. W projektowanym atlasie Git jest zatem źródłem obserwacji o wersjach i strukturach, a nie samym rozumowaniem. AST i semantyczna interpretacja dopiero nadają tym obserwacjom różne rodzaje relacji.



AT2-G08 · Git: cztery różne struktury. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Commit DAG	Genealogia zapisanych zmian.
Tree / blob	Zawartość i położenie obiektów wybranej wersji.
Refs	Ruchome nazwy wskazujące wersje.
Index / working tree	Przygotowywana i lokalna teraźniejszość.

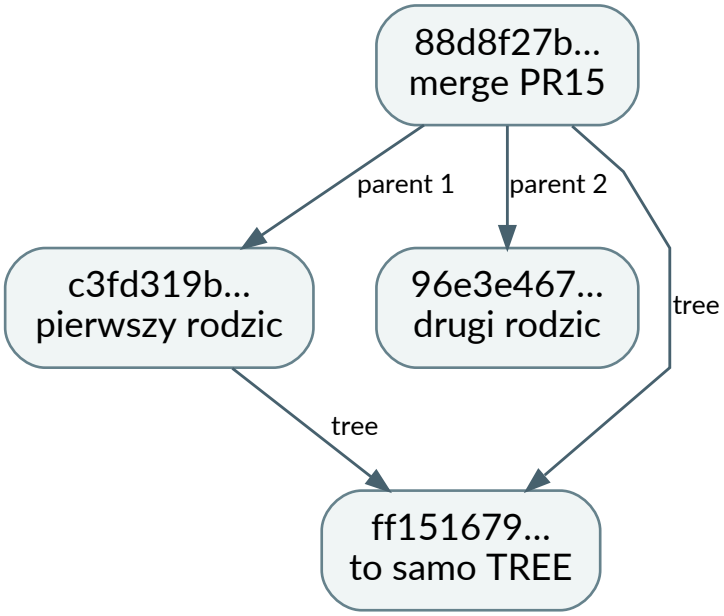
Źródła: S11, S12, S13, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Commit DAG jako genealogia

Nowy rodzic commitu może rozszerzyć osiągalną historię bez zmiany aktualnych plików.

W nowym odczycie writeups master wskazał 88d8f27b5fac48812ac266d990c7639fc5d1e93d. Pierwszym rodzicem jest c3fd319b205088df9fca3f9b1400e27e31834f17, drugim 96e3e467b85d5dcc780f8833f66bb097bde87d41. Commit i pierwszy rodzic mają ten sam tree ff15167945e1a7230ee8f8ddf41f00a748c6bcda. Odczytano obiekty Git, nie tylko opis PR.

To empiryczny przykład dwóch zmian na różnych osiach: genealogia się rozwinęła, ale zawartość drzewa względem pierwszego rodzica nie zmieniła się. Nie wynika z tego wdrożenie nowych funkcji ani ponowna prawdziwość wszystkich historycznych wyników. W atlasie horyzont przeszłości może więc zyskać nową osiągalną ścieżkę bez zmiany bieżącego kodu.



AT2-G09 · Writeups: inne HEAD, to samo tree. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Nowy commit	88d8f27b...
Pierwszy rodzic	c3fd319b...
Wspólne tree	ff151679...
Wniosek	Zmiana lineage i zmiana zawartości nie są równoważne.

Źródła: G02, G03 | ODCZYT GITHUB / DOKŁADNE OBIEKTY

Tree jako struktura informacyjna

Git tree daje dokładne położenie artefaktu, lecz nazwa katalogu nie ustala jego znaczenia ani aktualności.

Na d367e47c... korzeń ai_platform obejmuje między innymi LION, cyber_lion, browser_broker, tools, deploy, docs, tests, .github i manifest federacyjny. Są to rzeczywiste powierzchnie repozytorium. Kod, normy, przykłady, evidence i historia mogą występować obok siebie. Starszy numer w ścieżce nie wystarcza do klasyfikacji ARCHIVED.

R2 proponuje nakładkę metadanych nad dokładnym drzewem: rola artefaktu, owner, konsumenci, status źródłowy, horyzont, relacje następstwa oraz sposób regeneracji. Nakładka nie przenosi plików. Pozwala najpierw sprawdzić, które historyczne nazwy mają nadal aktywnych konsumentów i które wygenerowane pliki są poprawne jedynie dla konkretnego baseline.

Struktura fizyczna	Ścieżka i obiekt Git.
Rola informacyjna	Norma, implementacja, kandydat, projekcja lub dowód.
Aktualność	Relacja do wskazanego wejścia i epoki.
Zmiana	Najpierw mapa wpływu, dopiero później migracja.

Źródła: S01, S10, S16, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Blob, hash i pochodzenie

Integralność bajtów jest konieczna, lecz nie rozstrzyga autorstwa, prawdziwości ani użycia w konkretnym wykonaniu.

Code Perception oblicza Git-framed SHA-1 z długości i zawartości bloba oraz porównuje wynik z zadanym identyfikatorem. Przy podmienionych bajtach builder odrzuca wejście. Kod kontraktów dodatkowo wiąże rekordy plików z repozytorium, commitem i tree. W nowym badaniu pięć plików zostało w ten sposób porównanych z metadanymi GitHub.

Prawidłowy hash nie dowodzi, że tekst jest zgodny z rzeczywistością albo że został wysłany do modelu. Nie dowodzi również, że caller wybrał kompletny zbiór plików. W badaniu P06 pokazano, że czysty builder może przyjąć dwa różne poprawnie zahashowane wejścia pod tą samą deklarowaną SourceIdentity. Sprawdzenie członkostwa w prawdziwym tree należy do odrębnej granicy pozyskania źródła.

Sprawdzono	Bytes odpowiadają deklarowanemu blobowi.
Oddzielne zobowiązanie	Blob należy do dokładnego drzewa i zbiór jest kompletny.
Niewynikające z hasha	Prawda, autorstwo, wysłanie, wykonanie lub zgoda.

Źródła: S11, S12, S13, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Refs, gałęzie i przyszłe warianty

Gałąź może przechowywać projekt przyszłości, ale jej nazwa nie jest dowodem roli ani zgody na integrację.

Wersjonowane źródła LION rozdzielają źródłowego kandydata od integracji w master oraz od deploymentu. W R1 odczytano także stacked candidates i rozbieżność pomiędzy aktualnym HEAD PR456 a rewizją opisaną w jego body. Wznowienie nie przenosi historycznych wyników automatycznie na nowe referencje.

W projektowanej normalizacji gałęzie otrzymują dyspozycję: zachować, analizować kandydata, wskazać supersession, przygotować zamknięcie do osobnego dopuszczenia albo pozostawić UNKNOWN. Warianty przyszłości mogą być rozproszone po branches, dokumentach i bazach. Wspólna architektura powinna je wiązać przez tożsamość problemu, nie wymuszać jednego prefiksu branch jako uniwersalnego języka semantycznego.

Branch name	Wskazówka do odkrywania, nie kwalifikacja.
Exact HEAD	Tożsamość badanego wariantu.
Base i parents	Kontekst integracji oraz zależności.
Dyspozycja	Propozycja postępowania, nie wykonane zamknięcie.

Źródła: R1, S10, G01 | PROJEKT R2 / AS-IS OGRANICZONY

Working tree jako lokalna teraźniejszość

Aktywny checkout nie jest automatycznie kanoniczną wersją projektu.

Nowy SentinelX project snapshot głównego katalogu na MOON wskazał detached HEAD 95634907d2e7, 1268 śledzonych plików i 334 untracked. Osobny cooperative-publication-r4 wskazał 95f44213ca85, dziewięć staged zmian i 1635 śledzonych plików. Są to zakresowane obserwacje dwóch katalogów, a nie wynik pełnego rozpoznania wszystkich worktree.

Dlatego analiza źródeł nie może po prostu czytać dowolnego aktywnego folderu i podpisywać wyniku SHA master. W tym badaniu trzy pobrane pliki Code Perception pochodziły z lokalnego worktree, ale ich pełne bytes porównano z blobami kanonicznego d367e47c… . Dopiero równość obiektów pozwoliła przypisać te konkretne pliki do badanego źródła, bez przenoszenia takiej kwalifikacji na cały katalog.

Obserwacja lokalna	HEAD skrócony, zmiany i licznik śledzonych plików.
Kanoniczna tożsamość	Exact commit/tree ustalone przez GitHub.
Pomost dowodowy	Porównanie Git-framed blob dla każdego użytego pliku.

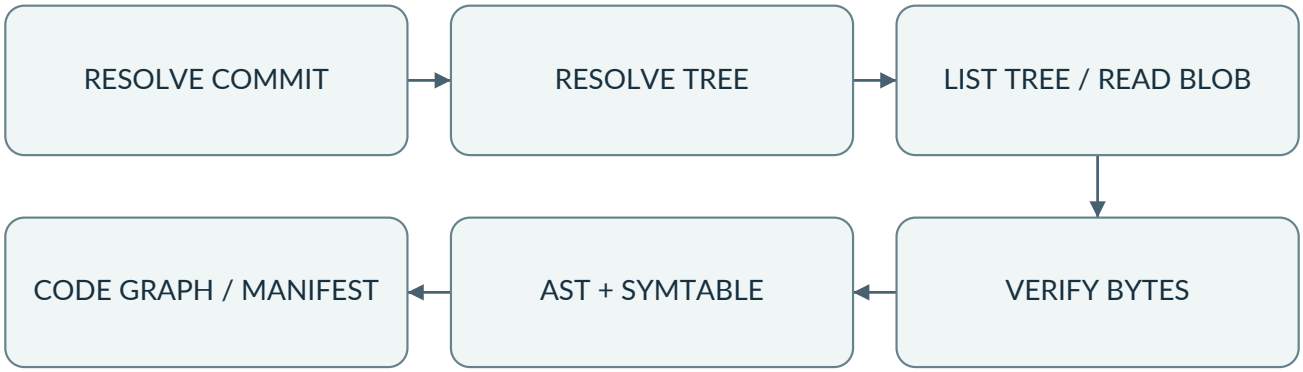
Źródła: H01, H02, T01, G01 | OBSERWACJA ZAKRESOWANA

Percepcja repozytorium krok po kroku

Percepcja kodu powinna zaczynać się od dokładnych obiektów, a kończyć na projekcji z jawnymi ograniczeniami.

Istniejące git_source_identity rozwiązuje commit i tree. git_blob_inputs odczytuje listę, rozmiary i zawartość obiektów oraz porównuje hashe. build_code_graph tworzy rekordy plików, symbole i krawędzie. manifest_for wiąże rezultat z wersją generatora, liczbami i digestem grafu. Jest to źródłowa ścieżka percepcyjna LION, a nie pomysł wprowadzony dopiero w R2.

Docelowy widok operatorski może pokazywać postęp tej sekwencji i odrębny mianownik: rozpoznane obiekty, odczytane bytes, sparsowane pliki, przeanalizowane symbole oraz rozstrzygnięte zależności. Nie wolno zastępować jednego mianownika innym. Brak błędów parsowania nie oznacza, że człowiek lub agent przeczytał całą semantykę kodu.



AT2-G10 · Git do percepcji kodu. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Pozyskanie	Commit → tree → bloby.
Projekcja	Files → symbols → typed edges.
Walidacja	Tożsamości, liczności, błędy i wersja generatora.
Interpretacja	Oddzielne hipotezy o funkcji i zachowaniu.

Źródła: S11, S12, S13, T01 | PROJEKT R2 / AS-IS OGRANICZONY

Aktualność przedmiotu a genealogia commitu

Nie każda zmiana HEAD oznacza taką samą zmianę treści, ale exact-head evidence nadal ma własną tożsamość.

truth_plane definiuje subject jako deterministyczną reprezentację dokładnych wpisów drzewa, z zamkniętym zbiorem dwóch carrier paths. Ich wyłączenie ogranicza samoodniesienie przy zapisie nośników aktualności. Równocześnie klasyfikacja kandydata wymaga dokładnych tożsamości i dowodu ancestry dla dopuszczanych przypadków zgodności.

R2 zachowuje oddzielnie commit, tree, subject digest, graph digest i wersję generatora. Równość tree nie pozwala bez kontroli przenieść historycznej zgody ani dowolnego wyniku CI. Może natomiast uzasadniać ponowne użycie konkretnej deterministycznej projekcji treści, jeśli cały jej kontrakt zależności jest zachowany. Reguła reuse musi wynikać z kontraktu danego dowodu.

HEAD	Tożsamość wersji w genealogii.
TREE	Tożsamość układu obiektów.
SUBJECT	Kontraktowo zdefiniowany przedmiot aktualności.
CI / grant	Własne bindings; brak automatycznego dziedziczenia.

Źródła: S14, S11, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Carrier-last i problem samoodniesienia

Źródło opisujące własną aktualność wymaga kontrolowanego punktu zamknięcia epoki.

LION wskazuje, że najpierw stabilizuje się źródła i zależne projekcje, a dopiero później zapisuje nośniki truth/currentness. Dotyczy to także dokumentacji, która uczestniczy w przedmiocie formalizacji. Wcześniejszy global-owner overlay rozwiązuje osobny problem: bieżący owner nie powinien oceniać siebie wyłącznie przez stary snapshot federacji.

W modelu temporalnym oznacza to krótkie, jawne okno stabilizacji. Dodanie kolejnego pliku po zamknięciu nie jest moralnym błędem systemu, lecz początkiem nowej epoki, której zależności trzeba ponownie przeliczyć. Nie wolno ukrywać takiej zmiany przez ręczną wymianę digestu albo przepisywanie historycznej kwalifikacji.



AT2-G11 · Carrier-last. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Przed zamknięciem	Kod, kontrakty, testy i opis wymaganych powierzchni.
Zamknięcie	Wektor i nośniki aktualności w wymaganej kolejności.
Po zmianie	Nowa epoka oraz unieważnienie zależnych tez.
Ograniczenie	Carrier nie jest issuerem authority.

Źródła: S01, S03, S14, S17 | PROJEKT R2 / AS-IS OGRANICZONY

Zmiana drzewa nie jest jeszcze zmianą zachowania

Różnica plików, różnica AST i różnica efektu tworzą trzy odrębne klasy delty.

Zmiana komentarza może zmienić blob i source span bez zmiany struktury badanej funkcji. Zmiana wartości w return zmienia strukturę, pozostawiając logiczny identyfikator symbolu. Przeniesienie pliku może zachować bytes, a zmienić identyfikatory zależne od ścieżki. Te przypadki zostały odtworzone w P02–P04 na rzeczywistym builderze Code Perception.

Projektowany klasyfikator wpływu powinien zachować wszystkie te osie zamiast wybierać jedną „semantyczną zmianę” bez kwalifikacji. Nawet niezmieniony kod może zachowywać się inaczej z inną konfiguracją lub modelem. Z kolei zmiana AST nie gwarantuje różnicy wyników dla każdego wejścia. Wniosek o zachowaniu wymaga odpowiedniego testu albo obserwacji.

Delta pliku	Ścieżka, mode, blob.
Delta struktury	Symbole, AST, imports i static edges.
Delta zachowania	Wynik dla konkretnych warunków i wejść.
Delta wiedzy	Nowa interpretacja zakresu i ograniczeń.

Źródła: S11, S12, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Mianownik całej federacji

Pełny spis zasobów i pełna analiza ich znaczenia są odrębnymi wynikami.

Historyczny FEDERATION_LIVE_SNAPSHOT zawiera listy plików dziesięciu repozytoriów. R1 odtworzył nowszy mianownik ai_platform: 1917 blobów. Wznowione badanie odzyskało podzbiór 268 plików i wybrane bieżące obiekty Git. Żadna z tych liczb nie może być podpisana jako pełny odczyt wszystkich gałęzi i źródeł federacji.

Projekt pełnej inwentaryzacji powinien rozdzielić snapshot każdej gałęzi, zbiór unikalnych blobów, historię decyzji, lokalne worktree, konfiguracje deploymentu i runtime. Te zbiory częściowo się przecinają, ale mają inne znaczenie. Pokrycie można liczyć dla dokładnie opisanej populacji, a nie dla nieograniczonego słowa „wszystko”. Brak dostępu musi pozostać widoczny.

Zbiór startowy	Dziesięć repozytoriów z jawnego wektora.
Frontier	Nowe referencje odkryte przez producentów i konsumentów.
Czytanie	Osobny status dla całego pliku i zakresu częściowego.
Kompletność	Nierozliczone gałęzie i dynamiczne zależności pozostają otwarte.

Źródła: R1, A01, A-RAG | PROJEKT R2 / AS-IS OGRANICZONY

Repozytorium jako narząd percepcji

Git nie jest mózgiem LION; jest jednym z precyzyjnych źródeł, przez które system może poznawać własną budowę.

W źródłach istnieją CodeGraph, projekcja architektury, census dokumentów i mechanizmy rekonsyliacji. Git dostarcza im tożsamości oraz treści. Semantyczne znaczenie powstaje dopiero wtedy, gdy te obserwacje zostają związane z celem, kontraktem i dowodem. Tak rozumiana funkcja percepcyjna nie wymaga metafory, że każda gałąź jest myślą.

Organiczny charakter projektu polega na sprzężeniu: repozytorium jest obserwowane, z jego analizy powstaje kandydat zmiany, a zastosowany i zweryfikowany przyrost zmienia następne obserwacje. Sprzężenie nie jest jednak automatycznie zamknięte. R2 wskazuje rzeczywistych właścicieli każdego przejścia i osobne kryteria tego, czego kod źródłowy jeszcze nie dowodzi.

Percepcja	Dokładna zawartość oraz wersja źródła.
Rozumienie robocze	Powiązanie z problemem i hipotezami.
Działanie	Osobny proces dopuszczonej zmiany.
Pamięć	Zachowany wynik i jego późniejsza reinterpretacja.

Źródła: S03, S05, S11, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Istniejący silnik zamiast nowego duplikatu

W źródłach LION istnieje model percepcji AST; rozwój powinien najpierw wykorzystać jego kontrakty i granice.

cyber_lion/enterprise/code_perception.py korzysta z ast i symtable, tworząc pliki, zakresy symboli, dziedziczenie, imports, dekoratory oraz relacje CALLS. Osobny kontrakt określa STATIC_CALL_EVIDENCE i wymaga pozostawienia runtime target jako UNRESOLVED bez odrębnego dowodu. To bardziej precyzyjna podstawa niż ogólny skan słów kluczowych.

Nowe 45 testów odtworzono na pięciu plikach zgodnych bajtowo z przypiętym master. Wynik dotyczy tej jednostkowej powierzchni, nie całego wywołania na wszystkich 982 plikach repozytorium. W R2 proponowane jest domknięcie związku Git–AST–architektura–obserwacja przy istniejącym ownerze, a nie utworzenie konkurencyjnego „super-AST” jako nowego systemu prawdy.

Istniejący owner	enterprise/code_perception.py + contracts/code_perception.py.
Osobny spis	architecture_projection/full_symbol_census.py.
Nowy dowód	45 testów jednostkowych, returncode 0.
Otwarta granica	Pełny graph całej federacji i dynamiczna kompozycja.

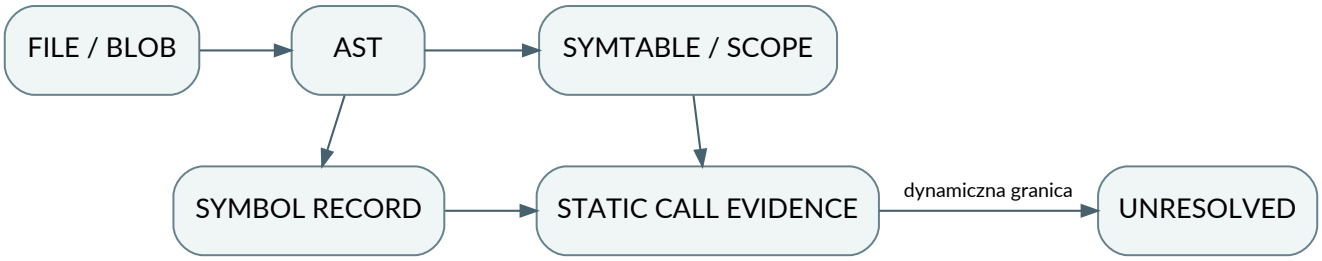
Źródła: S11, S12, S15, T01 | ŹRÓDŁO + NOWY TEST OFFLINE

AST, symbole i graf relacji

AST opisuje składnię programu, graf symboli opisuje wybrane relacje; żaden z nich nie jest kompletnym modelem całego zachowania.

SymbolRecord zachowuje qualified_name, kind, path, blob_sha, pozycję, signature i semantic_digest. CodeEdge rozróżnia CONTAINS, DEFINES, IMPORTS, CALLS, INHERITS i DECORATED_BY. Pochodzenie relacji wskazuje konkretny blob oraz source span. Nazwa semantic_digest w kontrakcie oznacza hash reprezentacji AST, nie dowód logicznej równoważności programów.

W atlasie należy pokazywać osobno drzewo składni, graf zależności oraz semantyczną interpretację roli modułu. Na przykład relacja IMPORTS może istnieć w gałęzi warunkowej; nie oznacza to, że import nastąpił w badanym wykonaniu. Z kolei brak zidentyfikowanego target nie dowodzi, że wywołanie jest niemożliwe. Tożsamość obserwacji runtime pozostaje osobnym rekordem.



AT2-G12 · AST i graf symboli. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

AST	Struktura składniowa w danej wersji parsera.
Symbol graph	Wybrane relacje statyczne związane z kodem.
Architecture view	Interpretacja odpowiedzialności i kontraktów.
Runtime trace	Oddzielny dowód rzeczywistych przejść.

Źródła: S11, S12 | PROJEKT R2 / AS-IS OGRANICZONY

Logiczny symbol a wersja implementacji

Ta sama funkcja jako rola może zachować identyfikator, choć jej implementacja i kwalifikacja ulegają zmianie.

W kodzie node_id zależy od ścieżki, qualified_name i kind. semantic_digest wyliczany jest z AST symbolu. P03 zmieniło return 1 na return 2: node_id pozostał równy, a digest struktury się zmienił. To pomaga śledzić rozwój danego miejsca w systemie, ale wymaga przechowania wersji obok logicznej tożsamości.

Nie wolno korzystać z samego node_id jako identyfikatora przetestowanej implementacji. Rekord odbioru powinien wskazywać konkretny blob, wersję generatora, warunki i rezultat. W przeciwieństwie do starej wersji symbolu, nowy wariant nie dziedziczy semantycznego zakresu wszystkich testów bez rewalidacji. Taki podział umożliwia ewolucję roli przy zachowaniu jej czytelnej historii.

Logiczny slot	path + qualified_name + kind.
Wersja	blob + structural digest + source identity.
Test	Zakres badanej implementacji i środowiska.
Następstwo	Nowa wersja zachowuje relację do poprzedniej.

Źródła: S11, S12, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Komentarz, dokumentacja i hash struktury

Nie każda zmiana źródła zmienia badane AST, ale nawet komentarz może wymagać odnowienia odwołań do źródła.

W P02 dodano zwykły komentarz przed funkcją. Blob i początkowa linia symbolu się zmieniły; jego node_id i digest AST pozostały te same. To wynik dla konkretnego fixture. Nie rozszerzamy go na docstring, type comment, zmianę kodowania albo dowolne przekształcenie tekstu.

Dokumentacja architektury nie może ignorować takiej delty tylko dlatego, że test funkcjonalny daje ten sam wynik. Źródłowe lokalizatory i hashe wymagają aktualizacji. Z drugiej strony nie ma uzasadnienia do ogłaszania zmiany zachowania funkcji, jeżeli zaobserwowano jedynie zmianę komentarza. R2 proponuje jawnie typowaną klasyfikację impactu, zamiast jednego zbiorczego „zmieniło się”.

Zmienione	Git blob i source span w fixture.
Niezmienione	AST danej funkcji przy tej mutacji.
Rewalidacja	Odnośniki źródłowe i claim-level currentness.
Brak wniosku	Powszechna równoważność zachowania dowolnego programu.

Źródła: T02, S11 | PROJEKT R2 / AS-IS OGRANICZONY

Nazwy, aliasy i koszt refaktoryzacji

Normalizacja nazwy zmienia również identyfikatory i relacje, które na tę nazwę wskazują.

P04 przeniosło identyczny plik z a.py do b.py. Blob się nie zmienił, lecz zmienił się logiczny node_id i qualified_name. Własność ta jest spójna z konstrukcją aktualnego Code Perception. Dlatego estetyczne ujednolicenie katalogów nie jest operacją bez wpływu na wiedzę.

Docelowy rejestr migracji powinien łączyć stary i nowy symbol, zachowując source identity i zakres semantycznej zgodności. Alias nie oznacza automatycznie tożsamości zachowania; może dotyczyć tylko powierzchni importu. Trzeba odtworzyć konsumujące testy, konfiguracje, dokumenty, narzędzia pakowania i historyczne odnośniki. Oryginalnych dowodów nie przenumerowuje się wstecz.

Przed zmianą	Stara ścieżka i wszyscy rozpoznani konsumenci.
Przejście	Adapter albo jawne mapowanie aliases/supersession.
Po zmianie	Nowe identyfikatory, testy i refreshed projections.
Rollback	Możliwa poprzednia wersja i zgodne dane.

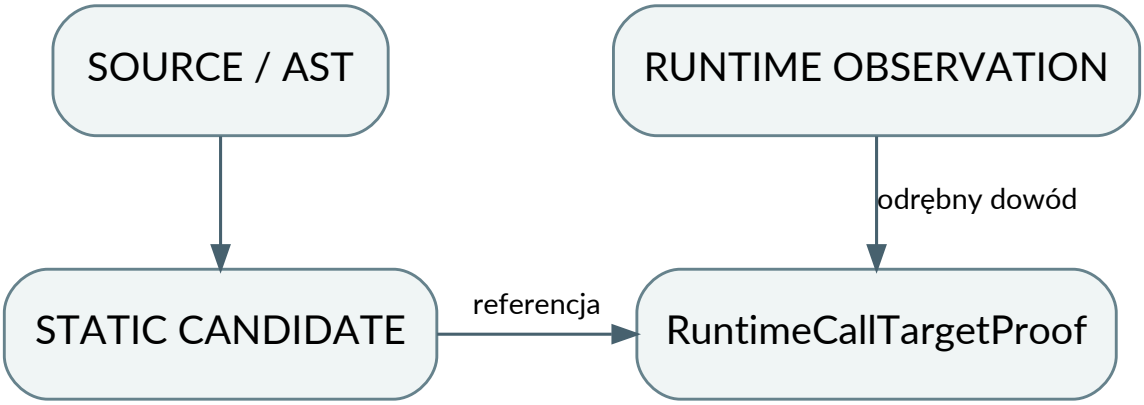
Źródła: S10, S11, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Statyczny target nie jest wykonanym targetem

Właściwie rozpoznana funkcja źródłowa nadal nie dowodzi, że to ją wybrał przyszły runtime.

W P08 prosty caller otrzymał target_node_id wskazujący znaną funkcję. Mimo to runtime_target_state pozostał UNRESOLVED, zgodnie z kontraktem CodeEdge. Źródło rozdziela w ten sposób lexical candidate od exact runtime target. Takiej ostrożności nie należy usuwać dla lepszego wyglądu diagramu.

R2 proponuje wizualnie odróżniać relację statyczną od zaobserwowanego przebiegu. Dopiero RuntimeCallTargetProof związany z dokładnym grafem, krawędzią i zweryfikowanym zewnętrznym dowodem może rozszerzyć zakres twierdzenia. Sama poprawna konstrukcja rekordu proof nie uwierzytelnia obserwatora; odpowiedzialność za to pozostaje u właściwej warstwy zaufania.



AT2-G13 · Statyczny target nie jest runtime proof. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Statycznie	Znany kandydat callable w dokładnym źródle.
Dynamicznie	Zaobserwowany callable w konkretnym wykonaniu.
Binding	Commit, tree, graf, edge, target i dowód obserwacji.
Granica	Proof record nie jest grantem ani własnym issuerem.

Źródła: S11, S12, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Cień nazw i dynamiczne wywołania

Konserwatywne pozostawienie zależności nierozwiązanej chroni przed fałszywą pewnością.

Przeczytany i odtworzony suite obejmuje parameter shadowing, nonlocal, global reassignment, lambdy, comprehensions, self/cls dispatch, dziedziczenie i modyfikację atrybutów. Dla tych klas przypadków nie wolno wywodzić dokładnego runtime target z podobieństwa nazwy. To realne zachowanie testowanego indeksera, nie ogólna rada o języku Python.

Docelowy call graph powinien rozdzielać unresolved-but-present od braku wywołania. W przyszłym rozwinięciu można uzupełnić tę wiedzę przez rejestr providerów, konfigurację, dependency injection albo obserwację runtime, ale trzeba dodać pochodzenie nowej relacji. Nie należy przepisywać pierwotnej krawędzi statycznej tak, jakby źródłowy indekser sam wykrył wynik runtime.

Przykłady barier	Parametr, mutable attribute, lambda, nonlocal.
Wynik	Nierozwiązana relacja zachowana w grafie.
Uzupełnienie	Osobny, źródłowo identyfikowalny model wiązania.
Błąd interpretacji	UNRESOLVED potraktowane jako ABSENT.

Źródła: S11, S12, T01 | PROJEKT R2 / AS-IS OGRANICZONY

Warunek obecności a obecność symbolu

Definicja pod if False może istnieć w AST, choć nie stanowi dostępnego runtime bindingu.

Próba P07 i oryginalny suite sprawdzają taki przypadek. Graf zachowuje symbol target, lecz wywołanie z caller pozostaje nierozwiązane. To rozróżnienie jest potrzebne w analizie całej federacji: znalezienie klasy lub funkcji nie dowodzi istnienia aktywnej ścieżki od entrypointu do jej użycia.

Podobny problem występuje dla importów pod try, loop, match lub with. R2 wymaga, aby wyższy widok architektury zachował warunki obecności i aktywacji. Skondensowany diagram może je pokazać jako port warunkowy z opisem, ale nie może zamienić źródłowego potencjału w fakt uruchomienia usługi. Ma to szczególne znaczenie przy diagnozowaniu brakującego composition root.

AST	Symbol znaleziony.
Binding	Nieudowodniony na każdej ścieżce.
Architektura	Relacja warunkowa, nie gotowy provider.
Weryfikacja	Konfiguracja, entrypoint i niezależny odczyt wykonania.

Źródła: S11, T01, T02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Przestrzeń nazw federacji

Lokalne identyfikatory symboli muszą zostać zakwalifikowane kontekstem repozytorium i wersji grafu.

W P05 dwa grafy różnych repozytoriów z identyczną lokalną ścieżką i funkcją otrzymały równy local node_id, ale różny digest grafu. Konstrukcja lokalnego ID nie zawiera repozytorium; tożsamość źródła znajduje się w CodeGraph. Jest to granica zakresu identyfikatora, nie wykazane naruszenie izolacji produkcyjnej.

W R2 klucz federacyjnego widoku powinien zachować co najmniej repozytorium, source identity, lokalny identyfikator i generator. Nie zmieniamy natywnego contract ID tylko po to, aby zbudować atlas. Tworzymy zewnętrzny klucz dokumentacyjny. Dzięki temu można zestawiać homologiczne symbole i implementacje bez ich niejawnego scalania w jeden byt.

Local ID	Znaczenie w obrębie grafu źródłowego.
Federated key	Repo + wersja grafu + local ID.
Podobieństwo	Osobna relacja semantyczna.
Zakaz	Flattening lokalnych IDs bez ich namespace.

Źródła: S11, S12, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Census a pokrycie zachowania

Zliczenie symboli, obecność docstringa i referencja testowa nie są metryką poprawności całej aplikacji.

FullSymbolCensus jawnie opisuje swoje wyniki jako deterministic static source evidence. Zbiera symbole, annotations, docstrings, statyczne wywołania i kandydatów referencji testowych. Nie wykonuje badanego programu i nie rozstrzyga runtime target. R1 odnotował 982 parsowalne pliki, nie 982 wykonane testy.

W atlasie potrzebne są odrębne mianowniki: wszystkie śledzone pliki, pliki obsługiwane przez parser, poprawnie sparsowane pliki, symbole, prześledzone interfejsy i rzeczywiście przetestowane właściwości. Wskaźnik dokumentacji nie może ukryć niezbadanego kodu. W przyszłym raporcie każda metryka musi podawać populację i metodę, aby nie utrwalać syntetycznej pewności w RAG.

PARSED	Parser przyjął określony plik.
REFERENCED	Test zawiera statyczne odniesienie.
EXECUTED	Test rzeczywiście uruchomiono.
QUALIFIED	Wykazano konkretną wymaganą właściwość.

Źródła: S15, R1, T01 | PROJEKT R2 / AS-IS OGRANICZONY

Wersja generatora jest częścią dowodu

To samo źródło może dać różne projekcje po zmianie sposobu analizy.

PerceptionManifest przechowuje generator_version i generator_digest. tree_semantic_digest normalizuje tożsamość commitu w określony sposób, zachowując pozostałą semantykę projekcji. Zmiana reguł call-resolution nie powinna być ukryta pod niezmienionym opisem, że „AST jest takie samo”.

Projekt porównania kolejnych epok musi rozróżniać deltę źródła od delty obserwatora. Jeśli zmieniono parser albo extractor, rosnąca liczba krawędzi nie musi oznaczać nowej funkcji systemu. Należy uruchomić kontrolę na tej samej zamrożonej próbce i zapisać różnicę metody. To analogiczny problem do zmiany instrumentu pomiarowego w trakcie eksperymentu.

Źródło	Commit/tree/blob badanego materiału.
Instrument	Parser, wersja i digest generatora.
Wynik	Graf wraz z regułami interpretacji.
Kontrola	Ten sam corpus przy dwóch wersjach instrumentu.

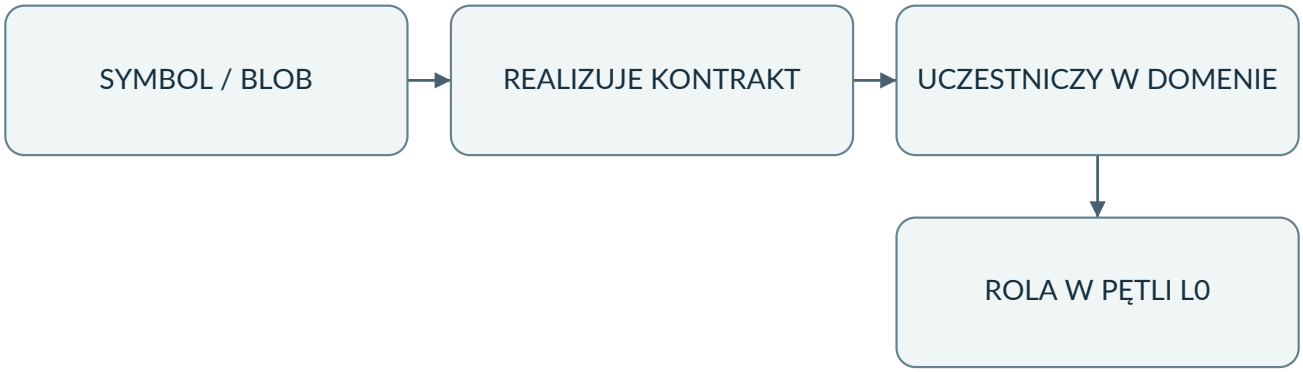
Źródła: S11, S12, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Od AST do modelu architektury

Najważniejszym zadaniem jest kontrolowane przejście między źródłowym symbolem a odpowiedzialnością systemową.

FullArchitectureModel zawiera źródłowo wskazane elementy, warstwy, statusy i target refs. Sprawdza obecność wskazanego symbolu oraz wymaga źródłowego digestu. Znaczenie warstwy i opis odpowiedzialności pochodzą jednak z deklaracji modelu, nie są dowolnie wywnioskowane przez ast.parse. Właściwe źródła rozdzielają kod-derived i document-derived projections.

R2 proponuje relację realizes pomiędzy wersją symbolu a kontraktem, relację belongs_to pomiędzy kontraktem a domeną oraz relację participates_in pomiędzy domeną a pętlą. Każda wymaga własnego rodzaju dowodu. Ten układ umożliwia hierarchiczne rozwijanie wizualizacji, zachowując to, co faktycznie ustalono, i nie zamieniając etykiety architektonicznej w ślad wywołania.



AT2-G14 · Od symbolu do roli. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Symbol → kontrakt	Implementacja, consumer i test.
Kontrakt → domena	Właściciel i semantyka odpowiedzialności.
Domena → L0	Rola w rozliczeniu pętli.
Diagram	Projekcja wspólnego modelu, nie osobna prawda.

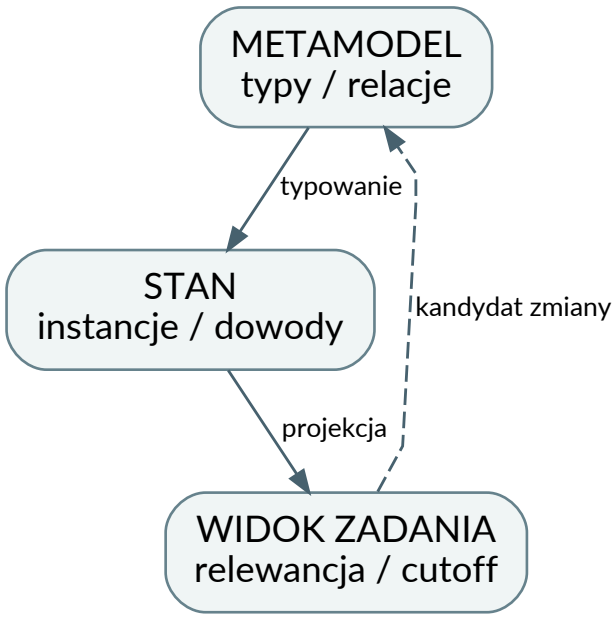
Źródła: S04, S11, S15 | PROJEKT R2 / AS-IS OGRANICZONY

Metamodel, stan świata i widok zadania

Ontologia nie jest jednym stale rosnącym workiem tekstu: ma różne role i różne częstotliwości zmian.

Metamodel definiuje dopuszczalne rodzaje bytów, relacji i ograniczeń. Model rzeczywistości wiąże konkretne źródła oraz obserwacje z ich wersjami. Widok zadania wybiera relewantny fragment dla jednego pytania, konsumenta i cutoff. R1 wyodrębnił te trzy poziomy na podstawie istniejących kontraktów; R2 wykorzystuje je jako poprzeczną oś hierarchii L0–L6.

Przyszły komponent percepcji może proponować nowe pojęcie, ale nie powinien sam zmieniać obowiązującego słownika wszystkich konsumentów. Potrzebny jest kandydat zmiany metamodelu oraz migracja. Z kolei nowa obserwacja tej samej instancji może zaktualizować graf stanu bez zmiany języka kontraktów. Rozdzielenie tych operacji umożliwia ewolucję bez utraty interpretowalności archiwum.



AT2-G15 · Trzy poziomy ontologii. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Metamodel	Wersjonowane typy i semantyka relacji.
Stan	Instancje, zdarzenia, wersje i dowody.
Widok	Projekcja pytania, zakresu i konsumenta.
Próba negatywna	Zmiana słownika nie może po cichu reinterpretować starego rekordu.

Źródła: R1, S08, S16 | PROJEKT R2 / AS-IS OGRANICZONY

Hipoteza ma strukturę problemu

Dobra reprezentacja hipotezy przechowuje również warunki, w których jej twierdzenie przestaje być zasadne.

W źródłowych kontraktach federacji występują hipotezy, założenia, obserwowalne konsekwencje, falsyfikatory i referencje dowodowe. SemanticScaffoldIR obejmuje także kontrhipotezy, constraints oraz unknowns. To podstawa do zachowania rzeczywistego przebiegu rozumowania problemowego bez potrzeby utrwalania prywatnych wewnętrznych rozważań modelu.

W R2 dossier problemu wiąże propozycję, alternatywne wyjaśnienie i rozstrzygającą obserwację. Przykładowo brak jawnego konstruktora w AST wspiera hipotezę luki integracyjnej, ale konkuruje z hipotezą dynamicznej rejestracji. Następną pracą nie jest automatycznie napisanie drugiego konstruktora. Jest nią zbadanie istniejącego composition root, aby rozstrzygnąć, czy zmiana jest potrzebna.

Teza	Twierdzenie w określonym zakresie.
Kontrhipoteza	Alternatywa prowadząca do odmiennego przewidywania.
Dowód	Obserwacja z właściwej płaszczyzny i epoki.
Rezultat	Ograniczenie zakresu, odrzucenie albo dalsza niewiedza.

Źródła: A01, A-SHD, S08, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Relewancja jest ograniczeniem widoku

Wybór kontekstu ma redukować niepotrzebną złożoność, nie usuwać warunków bezpieczeństwa ani niewiedzy.

RelevanceProjection ma źródłową metodę validate_against(graph, allowed_capabilities). Ten mechanizm kontroluje zgodność wybranych węzłów, krawędzi i capabilities z wejściowym grafem. Nie dowodzi jednak, że graf zawiera wszystkie istotne fakty świata lub że model otrzymał odpowiadające mu bajty. Te przejścia wymagają własnych wiązań.

Dla nowego poziomu wizualizacji potrzebny jest kontrakt streszczania: uproszczenie może ukryć szczegół, ale musi zachować granicę, od której zależy decyzja. Nierozpoznany zakaz nie może zniknąć dlatego, że nie pasuje do rozpoznanej gramatyki. R1 pokazał takie ryzyko interpretacyjne w SHD: PASS_LIMITED dotyczył części rozpoznanej, a nie kompletności całej instrukcji.

Wejście	Graf + dozwolony zakres + pytanie.
Wyjście	Źródłowo związana projekcja.
Obowiązek	Constraints i UNKNOWN istotne dla decyzji pozostają widoczne.
Test	Dodaj nierozpoznane ograniczenie; sprawdź każdy następny consumer.

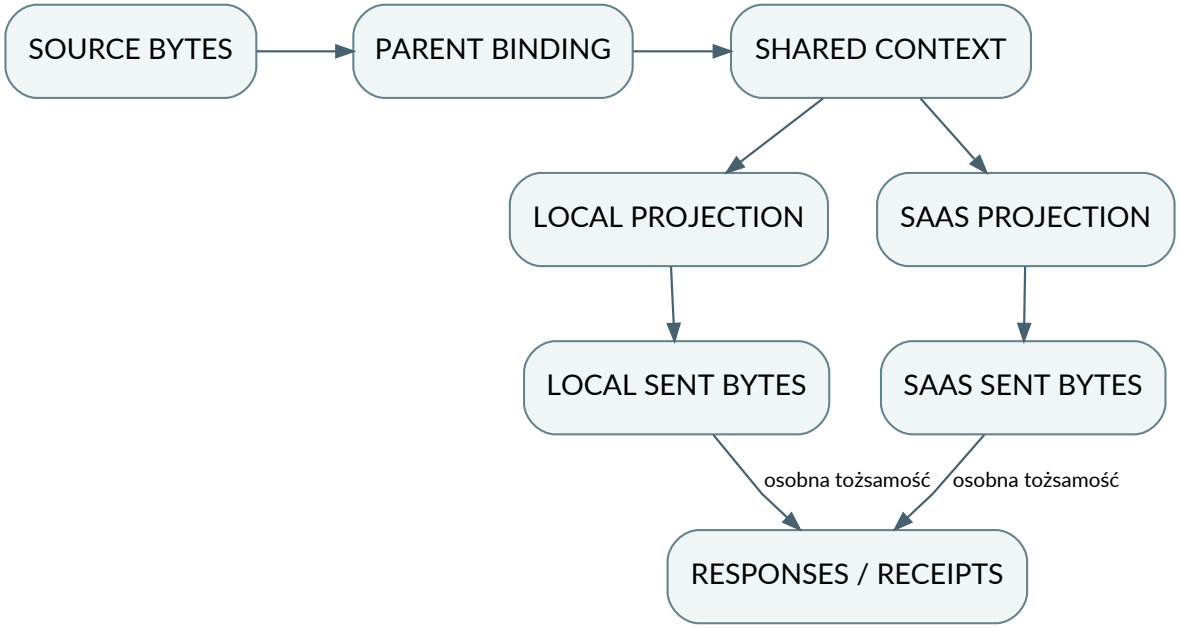
Źródła: S08, A-SHD, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Od źródłowych bajtów do wejścia modelu

Digest przygotowanego kontekstu i digest faktycznie wysłanej wiadomości odpowiadają na różne pytania.

Docelowy łańcuch powinien łączyć źródłowe bajty, ich rodziców, natywną instancję semantyczną, ograniczoną projekcję, przygotowany payload i rzeczywistą wysyłkę. R1 nie wykazał całego tego obiegu w runtime. Istnienie klas i serializatorów pozostaje istotną podstawą, ale nie zastępuje odczytu konkretnego invocation.

Przy współpracy SaaS i LOCAL ta sama wspólna baza kontekstu może dawać różne projekcje. Należy więc zachować shared_context_digest, projection_digest oraz actual_sent_bytes_digest osobno dla każdej ścieżki. Nowa wersja modelu albo zmiana cutoff otwiera nową kwalifikację wyniku. Nie wolno odtwarzać tożsamości tej pracy z aktywnej karty przeglądarki.



AT2-G16 · Źródło, kontekst i dwie ścieżki. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Źródło	Exact bytes i parent references.
Projekcja	Consumer, zakres i cutoff.
Invocation	Model release, session, request oraz rzeczywiste bajty.
Odbiór	Odpowiedź, jej digest i odrębny zapis wyniku.

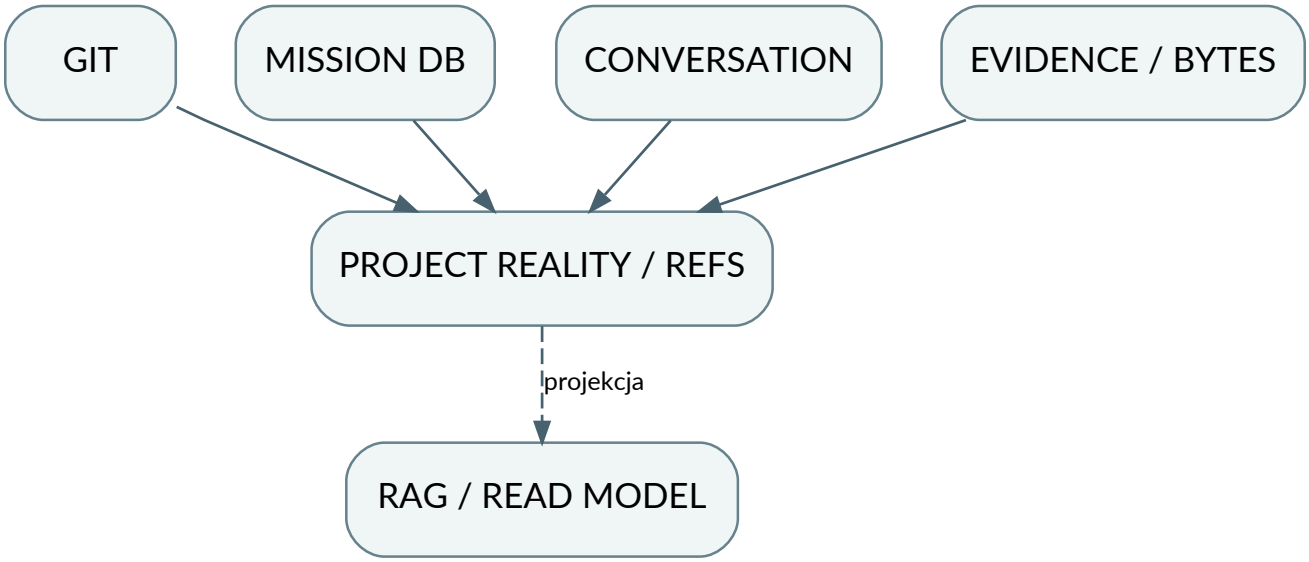
Źródła: R1, S05, S08 | PROJEKT R2 / AS-IS OGRANICZONY

Mapa pamięci rozproszonej

Pamięć systemu jest układem magazynów i relacji, a nie jedną bazą, którą należy utworzyć od nowa.

Git przechowuje źródła i ich genealogię. Mission Control zachowuje stan misji, fazy, przydziały i receipts. Conversation plane identyfikuje rozmowy, epoki i sesje providerów. RAG przechowuje wersjonowaną wiedzę. Graf semantyczny i ProjectReality tworzą powiązane widoki nad tymi nośnikami, nie przejmując automatycznie ich odpowiedzialności.

Architektura docelowa wymaga mapy: kto zapisuje, kto czyta, co jest kanonicznym obiektem, co cache, a co odtwarzalną projekcją. Przy awarii utrata cache może być naprawiona przez regenerację; utrata jedynego oryginału dowodu ma inne znaczenie. Kopia bazy i eksport RAG nie są równoważnymi backupami. Pełna operacyjna polityka retencji i odtworzenia pozostaje w otwartym obszarze U13.



AT2-G17 · Pamięć: oryginały i projekcje. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Git	Źródła, wersje i ancestry.
MC / conversation	Stan procesu i tożsamość pracy.
Evidence / artifacts	Oryginalne wyniki i ich lineage.
RAG / projekcje	Pochodne widoki wiedzy z aktualnością.

Źródła: R1, S05, S07 | PROJEKT R2 / AS-IS OGRANICZONY

Promocja wiedzy i archiwizacja

W teraźniejszości powstają dowody; zamknięcie epizodu dopiero określa, co wolno z nich utrwalić.

Working context i wygenerowane notatki nie stają się automatycznie committed organizational memory. GENERATION_EVOLUTION_PROTOCOL oddziela memory candidate od promote, reject i supersede. Dzięki temu negatywny wynik może zostać zachowany jako wiedza o granicy bez promowania odrzuconego artefaktu do wykonania.

R2 proponuje dossier z wersją problemu, wariantami, źródłami, testami, decyzją i odczytem efektu. Archiwizacja zamraża tę postać oraz wskazuje rodzica następnego epizodu. Późniejsza interpretacja może skorygować ocenę, ale nie zmienia historycznego logu. Wymaganie dotyczy także wyników zerowych, niewygodnych kontrprzykładów i przerwanych prób, inaczej pamięć byłaby obciążona wyłącznie historią sukcesów.

Kandydat wiedzy	Nowy wynik z określonym zakresem.
Kwalifikacja	Właściwy owner i warunek promocji.
Archiwum	Niezmienione źródła i zapis decyzji.
Reinterpretacja	Nowy rekord z supersession i uzasadnieniem.

Źródła: S03, S07, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Rewalidacja zamiast przepisywania przeszłości

Nowy dowód może zmienić aktualną ocenę starego artefaktu, zachowując prawdę o wcześniejszej ocenie.

SemanticDelta rozróżnia ADD, UPDATE, SUPERSEDE, CONTRADICT, RETRACT i REVALIDATE. Te operacje pozwalają opisać ewolucję wiedzy bez utożsamienia jej ze zmianą oryginalnych bajtów. Dla każdej operacji należy zachować podmiot, zakres czasu, źródło i zależne projekcje. Aktualizacja widoku nie musi oznaczać utraty historycznego rekordu.

Przykładem jest reinterpretacja wcześniejszego testu B0: wynik negatywnego holdoutu pozostaje taki sam, ale zmienia się zakres uprawnionego wniosku. Poprawne odrzucenie niezgodnego wyniku nie stanowi samodzielnego dowodu braku zapamiętywania. To ewolucja wiedzy o dowodzie, nie poprawianie testu po fakcie. W interfejsie obie oceny powinny być porównywalne przez ich epoki.

Oryginał	Bajty oraz wynik wcześniejszej próby.
Nowa teza	Skorygowana interpretacja zakresu.
Relacja	CONTRADICTS lub SUPERSEDES z uzasadnieniem.
Widok czasu	Co wiedziano wtedy i co wiadomo teraz.

Źródła: S08, R1, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Niezależność źródeł i wspólny przodek

Dwie odpowiedzi albo dwa procesy nie są automatycznie dwiema niezależnymi obserwacjami.

Źródłowy graph provenance powinien ujawniać wspólne korzenie. Raport, JSON, diagram i odpowiedź modelu wyprowadzone z tego samego testu stanowią różne projekcje jednego dowodu. Powielanie ich liczby nie zwiększa empirycznej siły twierdzenia. Podobnie dwie ścieżki modelowe nad wspólnym błędnym kontekstem mogą uzgodnić ten sam błąd.

R1 wykazał wspólny kernel i boot ID czterech instancji Linux. To ogranicza twierdzenie o niezależności fizycznej, choć nie usuwa wartości logicznego rozdziału buildera i verifiera. W R2 poziom wymaganej niezależności należy przypisać do konkretnej klasy skutku. Nie wolno automatycznie wymagać od każdego eksperymentu osobnego hosta ani przypisywać mu takiej niezależności bez dowodu.

Logiczna	Odrębna rola, implementacja lub ścieżka oceny.
Epistemiczna	Odrębny korzeń danych i obserwacji.
Operacyjna	Odrębne ograniczenia procesu i uprawnień.
Fizyczna	Rozliczona domena wspólnej awarii.

Źródła: R1, S03, S08 | PROJEKT R2 / AS-IS OGRANICZONY

RAG jako pamięć wersjonowana

Indeks powinien odtwarzać kontekst twierdzenia, nie tylko odnajdywać podobne zdanie.

Wybrane wydanie lion-rag30-v1.5-r1 i starszy preferred_release w repozytoryjnym bootstrapie są różnymi nośnikami wiedzy. R2 nie zmienia ich relacji integracyjnej. Pakiet RAG ma własny manifest i źródłowy cutoff; wynik retrieval nie jest świeżą obserwacją hosta, gałęzi ani grantu.

Proponowany fragment indeksowania obejmuje ID obiektu, źródło, epokę, status, poziom szczegółowości i powiązanie z parent view. Dzięki temu zapytanie o aktualny producer nie powinno zwracać historycznej deklaracji TARGET jako wdrożonej funkcji. Weryfikacja nowej dokumentacji powinna testować również odpowiedź UNKNOWN, a nie wyłącznie odnalezienie hasła, które na pewno występuje w pliku.

Jednostka	Twierdzenie razem z granicą dowodu.
Źródło	Ref, path, span, wersja dokumentu.
Test retrieval	Odróżnij historyczne, postulowane i zaobserwowane.
Promocja	Osobna decyzja istniejącego ownera RAG.

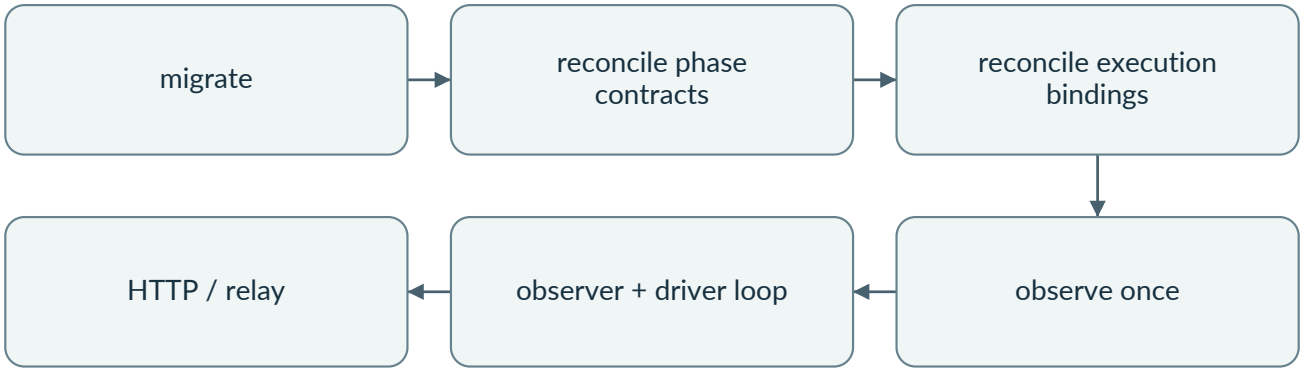
Źródła: A-RAG, R1, S01, S06 | PROJEKT R2 / AS-IS OGRANICZONY

Start aplikacji nie jest startem misji

Gotowy proces serwera i gotowa ścieżka konkretnego zadania to dwa osobne stany.

R1 odtworzył main() Mission Control: przygotowanie bazy, rekonsyliacja kontraktów faz, rekonsyliacja bindingów, pojedyncza obserwacja, uruchomienie observera i mission_driver_loop, a następnie serwer. Nie ma w tym porządku automatycznej rekonstrukcji ontologii całej federacji. AGENTS i bootstrap organizują wejście badacza, nie cały start fizycznych usług.

W najwyższym widoku trzeba więc pokazać dwa porty: gotowość infrastruktury oraz inicjację konkretnej pracy. Brak misji może być poprawnym stanem bezczynności; missing dependency może być błędem gotowości. Ich odróżnienie wymaga kontraktu, stanu rejestracji i diagnostyki import closure. Sam zielony health nie rozstrzyga, który przypadek zachodzi.



AT2-G18 · Start Mission Control. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Bootstrap usług	Stan i procesy istniejących właścicieli.
Bootstrap zadania	Cel, kontekst, źródła i kontrakt.
Aktywacja	Jawne uruchomienie związanej tożsamości.
Diagnostyka	Ready-for-mission odrębne od process-alive.

Źródła: R1, S01, S05 | PROJEKT R2 / AS-IS OGRANICZONY

Intencja, kontrakt fazy i zdolność

Nazwa fazy opisuje zamiar; nie tworzy wykonawcy zdolnego go zrealizować.

Process Contract Plane rozdziela PHASE_INTENT, PHASE_EXECUTION_CONTRACT, CAPABILITY_BINDING, ACTION_IR, EFFECT i COMPLETION. Przypadek PCD-0001 przywołany w R1 ilustruje, że poprawnie nazwana naprawa nie miała dostępnego bindera. Wniosek architektoniczny nie brzmi „dodaj jeszcze prompt”, lecz „sprawdź rzeczywistą ścieżkę producenta i konsumenta kontraktu”.

Przyszłe rozwinięcie fazy powinno zawierać typed input/output, klasę wykonania, ceiling, wymagania aktualności, postconditions, timeout, retry i politykę brakującej capability. VALID_UNBOUND_WAITING może być poprawne, gdy kontrakt jawnie dopuszcza późniejsze wiązanie. Nie może natomiast maskować bezterminowego oczekiwania na moduł, którego nikt nie buduje.

Zamiar	Co ma się zmienić lub zostać sprawdzone.
Kontrakt	Warunki wejścia, wyniku i błędu.
Binding	Rzeczywista implementacja i wersja capability.
Completion	Zaobserwowany predykat, nie tekst deklaracji.

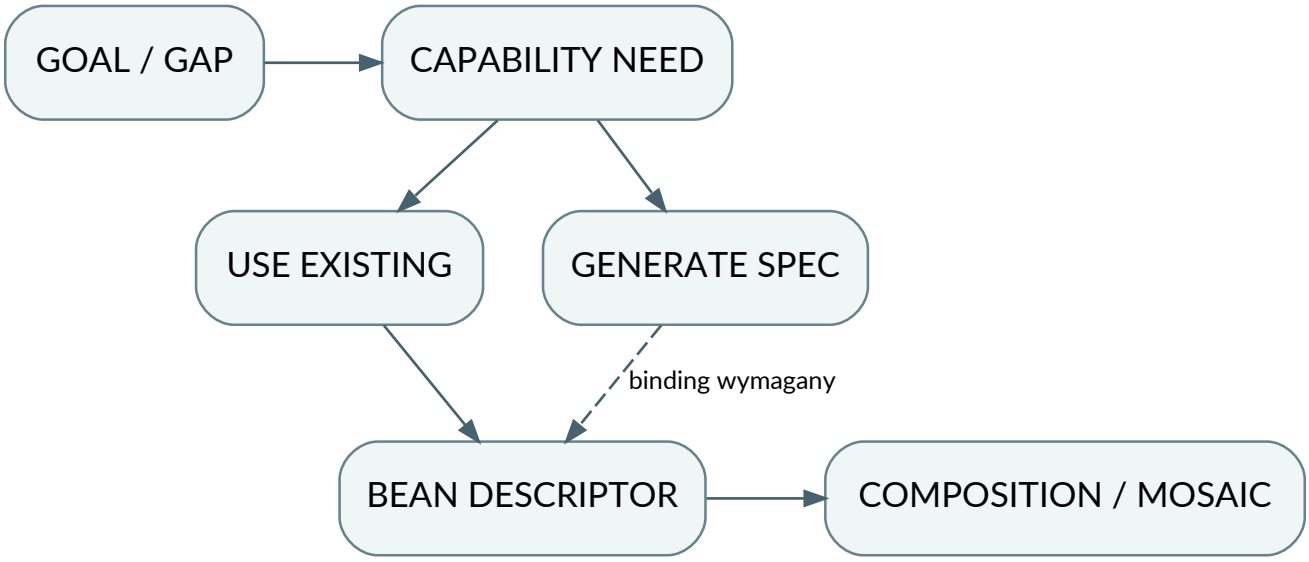
Źródła: R1, S18 | PROJEKT R2 / AS-IS OGRANICZONY

Od luki do kompozycji zdolności

Wspólny mechanizm fabryki oddziela potrzebę, specyfikację i faktyczną implementację.

GoalContract, obserwacje świata i systemu pozwalają odtworzyć Gap. CapabilityNeedResolver sprawdza istniejące specyfikacje lub zwraca GENERATE_SPEC. Taki wynik jest nowym opisem zdolności, nie gotowym kodem. CompositionEngine dobiera BeanDescriptor z uwzględnieniem zależności, ograniczeń oraz ról oceny. R1 skorygował skrót BeanCandidate → Composition: potrzebne jest właściwe wiązanie deskryptora.

Docelowa karta zdolności powinna zachować realizacje alternatywne. Ten sam kontrakt może zostać wykonany przez funkcję deterministyczną, model lub usługę, o ile nie zmienia to semantyki wyniku i granicy skutku. Factory ma wybierać rozwiązanie problemu, a nie maksymalizować liczbę generowanych plików. Reuse jest poprawnym rezultatem, gdy nowa implementacja nie jest potrzebna.



AT2-G19 · Od luki do kompozycji. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Gap	Rozpoznana różnica celu i stanu.
Spec	Opis potrzebnej zdolności.
Descriptor	Realizacja, interfejs i zależności.
Composition	Dopuszczalny zestaw builder / verifier / observer.

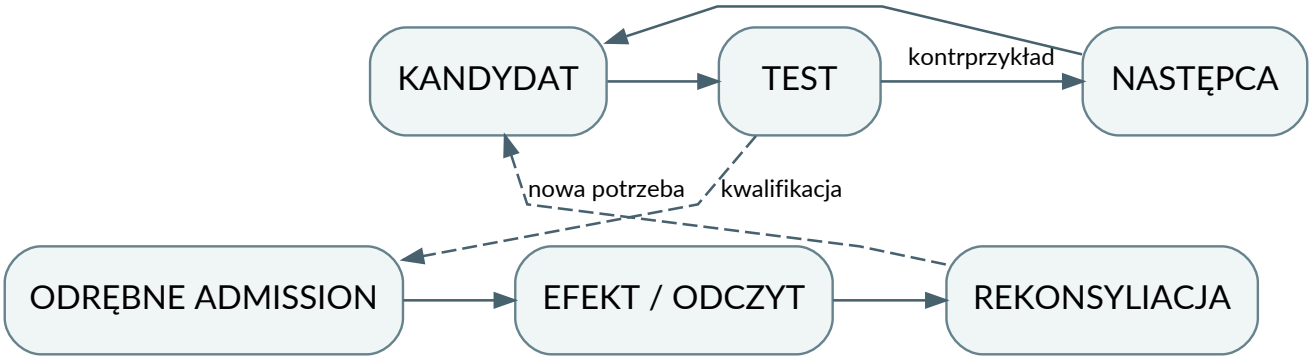
Źródła: R1, S05, S03 | PROJEKT R2 / AS-IS OGRANICZONY

Dwie pętle generowania artefaktu

Fabryka może iterować nad kandydatem bez wykonywania skutku w środowisku docelowym.

Pierwsza pętla obejmuje propozycję, lokalne budowanie kandydata, test i korektę. Druga obejmuje osobne dopuszczenie, rzeczywiste wykonanie, obserwację i rozliczenie. Te pętle mają wspólne lineage, ale inne granice. Lokalny kontrprzykład może zmienić kandydata bez konieczności cofania produkcyjnego wdrożenia, którego jeszcze nie wykonano.

W R2 każda generacja ma określony zakres i primary artifact albo spójną deltę kontraktu. Porównanie nowej generacji z poprzednią zachowuje zarówno wady, jak i poprawy. Powrót do eksperymentu po stabilizacji nie oznacza odtworzenia błędnej wersji w runtime. Oznacza otwarcie następnego problemu na znanej podstawie, z nowymi dowodami i limitem kosztu badania.



AT2-G20 · Candidate loop i effect loop. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Candidate loop	Budowa → test → kontrprzykład → następca.
Effect loop	Admission → skutek → odczyt → rekonsyliacja.
Łącznik	Exact candidate identity i zatwierdzony zakres.
Zakaz	PASS lokalny przeniesiony automatycznie na deployment.

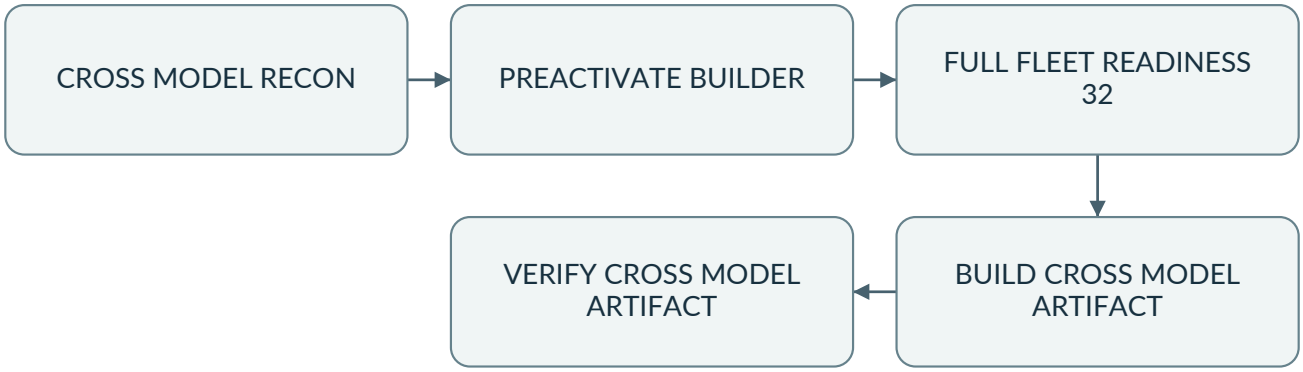
Źródła: S03, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Application Factory R1 jako odbiór skończony

Istniejący program ma własną skalę i postconditions, których dokument R2 nie zmienia.

Odczytany program zawiera CROSS_MODEL_RECON, PREACTIVATE_BUILDER, FULL_FLEET_PROVIDER_READINESS, BUILD_CROSS_MODEL_ARTIFACT oraz VERIFY_CROSS_MODEL_ARTIFACT. MATERIAL_TARGET=32 jest częścią tej wersji. Kwalifikacja pojedynczego workera przed bramką nie upoważnia do pominięcia pełnej gotowości w tej samej misji.

R1 odnotował zero rekordów exact Factory w sprawdzonej kanonicznej bazie, jednocześnie potwierdzając historię innych misji. To jest obserwacja z określonego czasu, nie dzisiejszy pomiar wszystkich kopii SQLite. R2 nie uruchamia tej misji. Proponuje doprecyzowanie jej package closure, lineage kontekstu i odbioru bajtów; mniejszy proof wymaga osobnego kontraktu albo formalnej zmiany obecnego programu.



AT2-G21 · Pięć faz programu Factory. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

RECON	Wiązanie materiału poznawczego.
PREACTIVATE / READINESS	Kwalifikacja i odrębna gotowość zasobu.
BUILD	Ograniczony materialny skutek po dopuszczeniu.
VERIFY	Odrębny odczyt i sprawdzenie artefaktu.

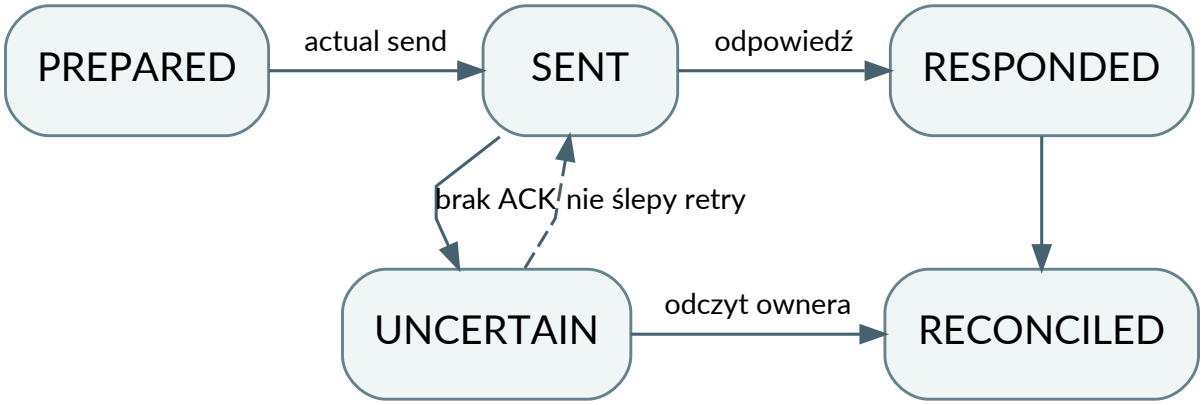
Źródła: R1, A-R6 | PROJEKT R2 / AS-IS OGRANICZONY

SaaS: przygotowane, wysłane, rozliczone

Niepewność dostarczenia wiadomości nie daje automatycznego prawa do jej ponownego wysłania.

Raport R1 opisuje historyczną sekwencję korekt PR451/452/454, rozdzielając claim, przygotowanie, materialny POST i odpowiedź. Nie odtwarzał zewnętrznej wysyłki ani nie kończył pending turns. Z tej historii wynika użyteczny niezmiennik: prepared ≠ sent ≠ responded ≠ reconciled.

W docelowym automacie brak ACK prowadzi do stanu niepewnego skutku. Następny krok to odczyt źródła odpowiedzialnego za dispatch i receipt, nie ślepy retry. Identyfikacja rozmowy, epoki, requestu i wysłanych bajtów ma pozostać niezmienna podczas rekonsyliacji. Nowy kontekst albo nowa sesja może tworzyć nową pracę, ale nie może udawać dokończenia poprzedniej.



AT2-G22 · Wiadomość i niepewny skutek. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

PREPARED	Gotowy payload, bez dowodu dostarczenia.
SENT	Potwierdzona rzeczywista wysyłka.
UNCERTAIN	Najpierw rekonsyliacja u ownera.
RECONCILED	Wynik związany z tą samą tożsamością i skutkiem.

Źródła: R1, S01 | PROJEKT R2 / AS-IS OGRANICZONY

Granica authority i runtime admission

Przestrzeń możliwości może rosnąć; prawo do skutku nie rośnie razem z nią.

LION rozdziela propozycję, decyzję polityki, aktualne uprawnienie, dopuszczenie wykonawcy oraz obserwację skutku. Specyfikacja zdolności, mapa relewancji lub trafna odpowiedź modelu nie stają się grantem. Istniejący proces pozostaje właścicielem tej granicy także po dodaniu nowej wizualizacji lub nowego adaptera federacyjnego.

R2 proponuje pokazywanie bramek na każdej ścieżce przechodzącej od reprezentacji do efektu. Nie chodzi o dodawanie identycznych blokad do wszystkich operacji analitycznych, lecz o wskazanie momentu, w którym zmienia się świat poza lokalnym badaniem. Wcześniejsze consumed albo expired uprawnienie nie jest dziedziczone przez successor identity. Source-only publisher nie dowodzi istnienia oryginalnego issuera.



AT2-G23 · Oddzielna droga uprawnionego skutku. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Propozycja	Kandydat i jego uzasadnienie.
Decyzja / grant	Oryginalny owner, zakres i aktualność.
Admission	Exact identity, zasób, replay i moment wykonania.
Skutek	Osobno obserwowany, nie tylko raportowany.

Źródła: R1, S01, S18 | PROJEKT R2 / AS-IS OGRANICZONY

Recovery zachowuje ciągłość problemu

Przywrócenie usługi i przywrócenie wiedzy o wykonaniu to dwa różne zadania.

Restart może odtworzyć proces z bazy, lecz pozostawić nierozstrzygnięty wynik ostatniego assignmentu. Jeśli wykonanie było nieidempotentne, ponowienie może stworzyć drugi artefakt albo drugi zewnętrzny skutek. Dlatego stan niepewny musi mieć odrębny właściciel i metodę rekonsyliacji, zanim scheduler dopuści następną próbę.

Projekt dokumentacji na L3 powinien dla każdego przejścia opisać: utratę procesu przed efektem, po efekcie przed receipt oraz po receipt przed obserwacją. Powrót do stable wymaga zamknięcia odpowiedniego predykatu, a nie samego odczytu health. Dla danych nieodwracalnych rollback może być niemożliwy; wtedy projekt musi określić kompensację i zachować rzeczywistą historię, zamiast deklarować powrót do świata sprzed zdarzenia.

Awaria przed efektem	Potwierdź brak efektu i dopuszczalność retry.
Awaria po efekcie	Odtwórz rzeczywisty wynik przed ponowieniem.
Rollback	Przywrócenie wersji nie odwraca każdego skutku.
Pamięć	Nowa obserwacja wskazuje oryginalną generację.

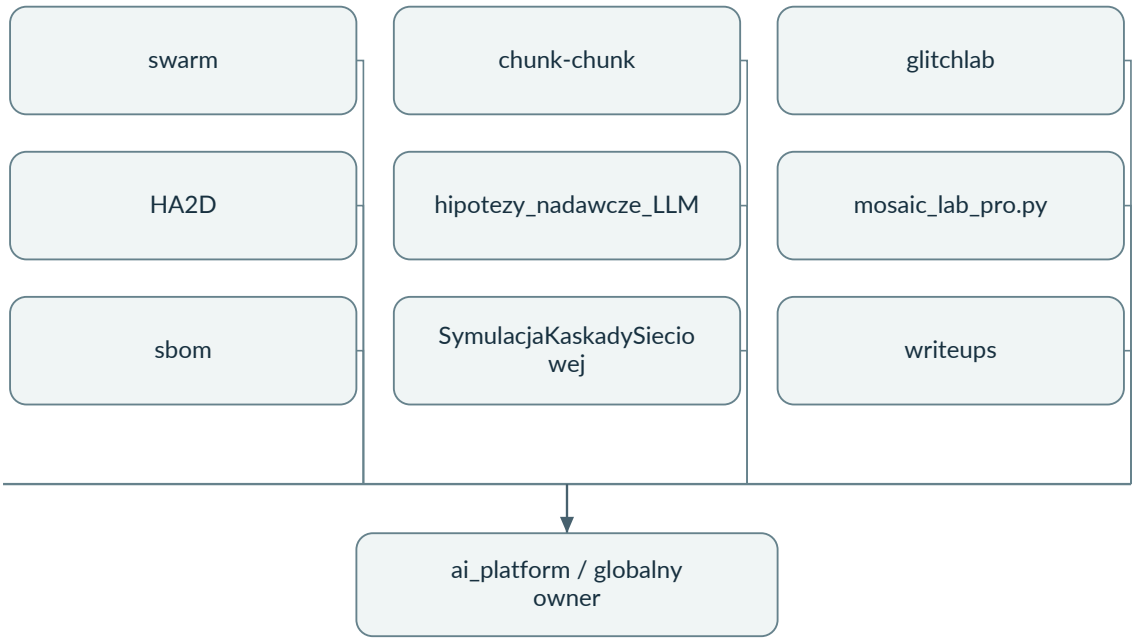
Źródła: R1, S03, S18 | PROJEKT R2 / AS-IS OGRANICZONY

Jedność kontraktów, wielość realizacji

Wspólny model federacyjny nie wymaga wspólnego katalogu ani jednego wdrożenia.

Znana federacja obejmuje dziesięć repozytoriów. Manifest ai_platform importuje semantykę procesu, analizę ewolucji, kontekst, hipotezy, grafy, provenance, execution mesh, symulację i corpus badań. To zależności odpowiedzialności i reprezentacji, nie automatyczny pipeline dziewięciu uruchomionych usług.

Karty tego rozdziału porządkują role na podstawie manifestów w odzyskanym pakiecie oraz źródeł R1. Pełne bieżące implementacje, wszystkie gałęzie i historyczne warianty peerów nie zostały odczytane. Kierunek normalizacji jest zatem kontraktowy: wskazać local owner, eksport, odbiorcę, wersję i warunek zastępowalności, a dopiero później kwalifikować konkretny adapter lub runtime.



AT2-G24 · Federacja: routing odpowiedzialności. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica	Globalny model przy ai_platform, lokalne specjalizacje u peerów.
Wspólny mianownik	Tożsamość, typ wyniku, currentness, lineage i błędy.
Zmienna realizacja	Algorytm, język, proces i domena wykonania.
Otwarte	Pełny frontier repozytoryjny oraz infrastruktura sąsiednia.

Źródła: A01, S16, R1 | PROJEKT R2 / AS-IS OGRANICZONY

ai_platform · Globalny owner i organizator fabryki

Sterowanie procesem, kontrakty, formalizacja i wspólna wiedza architektoniczna.

W manifestach i README ai_platform łączy EnterpriseControlPlane, AgentFoundry, SwarmPlanner, FederationRegistry i ProcessContractIntegration. Jego szczególna rola wynika z organizacji kontraktów i misji, nie z obowiązku lokalnego wykonywania wszystkich specjalistycznych analiz.

Najważniejszą transformacją jest domknięcie źródło → problem → kandydat → materialny odbiór → wiedza bez tworzenia równoległych właścicieli. Repozytorium jest także obiektem własnej percepcji: zmiana jego źródeł wymaga nowej projekcji i rewalidacji zależnych twierdzeń.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Kontrakty procesu, granice authority, tożsamość oraz formalization closure.
Do eksperymentowania	Natywne powiązania percepcji, kompozycji i następnej epoki.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: S16, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

swarm · Lokalna domena wykonania

Workload, telemetry i capability lease przy zachowaniu globalnego postępu misji.

Manifest definiuje ExecutionMesh, WorkloadRuntime, TelemetryPlane i CapabilityLeaseRuntime. R1 przeanalizował adapter mapujący dane ai_platform na lokalny RuntimeAdmissionRequest. Nie wykazał kompletnego materializera dowolnych klastrów i flot.

Proponowany kontrapunkt jest prosty: ai_platform określa sens i kolejność misji, a wyspecjalizowany provider realizuje ograniczony workload. Rozszerzenie swarm o lifecycle klastrów wymaga osobnego kontraktu zasobu, trwałości nonce, telemetrii i recovery. Pole ALLOW w mapowaniu danych nie staje się samodzielnym źródłem uprawnień.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Zastępowalny kontrakt runtime i właściwe pochodzenie decyzji.
Do eksperymentowania	Jawny provider klastrów dopiero po ustaleniu istniejących ownerów.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

chunk-chunk · Semantyka trajektorii procesu

Znaczenie przejścia nie jest tożsame z jego materialnym wykonaniem.

Manifest eksportuje process_semantics, hmk9d, semantic_transition_candidate i trajectory_semantics. Repozytorium jest przedstawiane jako ProcessSemanticsEngine oraz TransitionMicrocode. Te role informują, jak klasyfikować problem i trajektorię, nie przenoszą globalnej authority.

W R2 wynik peera powinien wskazywać aktualny stan procesu, proponowane przejście i źródłowe ograniczenia. Konsument w ai_platform musi rozliczyć zgodność z kontraktem fazy. Dobrym falsyfikatorem integracji jest ten sam opis przejścia użyty poza właściwym stanem lub przy zmienionym effect ceiling.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Semantyka wejścia i wyniku przejścia wraz z wersją.
Do eksperymentowania	Nowe trajektorie i klasy procesów bez zmiany globalnego ownera.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

glitchlab · Delty i niezmienniki

Zmiana staje się przedmiotem analizy strukturalnej przed przyjęciem skutku.

Manifest opisuje EvolutionCompiler, DeltaNormalizer, InvariantEvaluator i StructuralChangeAnalyzer. GENERATION_EVOLUTION_PROTOCOL wskazuje analizę zmian wykraczającą poza sam source code: również polityki, schematy, agentów i pamięć. Część tej integracji pozostaje docelową organizacją.

W kontrapunkcie Git–AST GlitchLab powinien otrzymywać nie tylko listę zmienionych plików, lecz źródłowo związaną deltę oraz naruszone relacje. Wynik structural analysis jest wejściem do decyzji, nie nowym grantem. Warto stabilizować format delty i uzasadnienie naruszenia, pozostawiając algorytmy klasyfikacji w kolejnych generacjach eksperymentu.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	DeltaObservation, fingerprint i zasady porównania struktur.
Do eksperymentowania	Nowe niezmienniki oraz odporność na pozornie bezpieczne migracje.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

HA2D · Kontekst, replay i pamięć kandydacka

Przeszłość wraca jako źródłowo ograniczony kontekst, nie jako automatycznie aktualna prawda.

Manifest wskazuje ContextMemoryLab, MemoryCandidateProvider, ReplayContextProvider oraz ProcessStateDiagnostics. Eksporty obejmują temporal_semantic_state_candidate i supersession_evidence. Są to podstawy do wiązania kolejnych interpretacji oraz kontekstów.

Przyszły kontrakt integracji powinien rozdzielać snapshot, bieżący store i wynik replay. Wczytanie starego kontekstu nie odtwarza uprawnień ani aktualności runtime. Przy migracji pamięci potrzebne są testy cutoff, brakujących rodziców i zachowania wycofanej hipotezy w historii. Pełna operacyjna retencja HA2D nie została zamknięta w tym atlasie.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Tożsamość ContextSnapshot, lineage i jawne zakresy replay.
Do eksperymentowania	Nowe strategie kontekstu i temporalnego wyszukiwania.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

hipotezy_nadawcze_LLM · Laboratorium hipotez i eksperymentów

Kontrargument jest pełnoprawnym artefaktem procesu, nie odpadem po nieudanej odpowiedzi.

Manifest wymienia HypothesisLab, ExperimentDesignProvider i FalsificationEvidenceProvider. Eksportowane są kandydaty hipotez i falsyfikatorów. Ta rola może zasilać teraźniejszą analizę problemu, zanim powstanie specyfikacja zmiany.

W R2 interfejs powinien wymagać twierdzenia, założeń, przewidywań, rozstrzygającej próby i genealogii źródeł. Nowa hipoteza może być cenna mimo braku potwierdzenia; jej status musi jednak pozostać widoczny. Zestaw hipotez nie powinien być oceniany wyłącznie liczbą pozytywnych wyników, ponieważ odrzucenie złego kierunku jest również użyteczną deltą wiedzy.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	HypothesisSpec, ExperimentSpec i warunki rozstrzygnięcia.
Do eksperymentowania	Nowe rodziny problemów, niezależne referencje i kontrpróby.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

mosaic_lab_pro.py · Grafy, topologie i projekcje

Wizualna organizacja relacji musi zachować semantykę wejściowego grafu.

Manifest przypisuje StructuralIntelligenceEngine, GraphProjectionProvider i TopologyRiskAnalyzer. Wśród eksportów są structure_graph, topology_analysis oraz relevance_graph_candidate. Graf authority może zostać zobrazowany, ale jego obraz nie jest źródłem uprawnień.

W atlasie proponujemy wykorzystanie takiej specjalizacji do rozwijania poziomów widoku. Każda agregacja powinna ujawniać, które węzły ukryto i jakie relacje reprezentuje krawędź zbiorcza. Testem poprawności nie jest estetyka diagramu, lecz zachowanie kierunku, tożsamości i granic. Plik lub nazwa repozytorium zakończona .py nie stanowi powodu do automatycznego rename.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	GraphProjection oraz kontrakt agregacji i pochodzenia widoku.
Do eksperymentowania	Nowe projekcje czasu i poziomów szczegółowości.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

sbom · Tożsamość i pochodzenie artefaktów

Gotowy plik potrzebuje genealogii składu, nie tylko nazwy i etykiety PASS.

Manifest opisuje SupplyChainEvidenceProvider, ArtifactIdentityProvider oraz CompositionIntelligence. Eksporty obejmują artifact_identity, lineage_binding i composition_risk_evidence. To wsparcie dla przejścia od kandydata do materialnego artefaktu i jego reużycia.

R2 proponuje wiązanie produktu z dokładnymi źródłami, zależnościami, parametrami budowania oraz odczytem wyniku. Taki zapis może umożliwić odtworzenie i analizę wpływu zmiany biblioteki. Nie dowodzi jednak samodzielnie bezpieczeństwa zależności ani prawdziwości każdego twierdzenia producenta. Potrzebne są osobne warstwy integralności, pochodzenia i kwalifikacji.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	ArtifactIdentityEvidence i rozdział identyfikacji od oceny.
Do eksperymentowania	Nowe klasy artefaktów i analiza ich złożonego lineage.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

SymulacjaKaskadySieciowej · Scenariusze i wrażliwość

Symulacja bada konsekwencje założeń; nie zastępuje obserwacji rzeczywistego systemu.

Manifest definiuje SimulationProvider, FalsificationEngine, StressScenarioProvider i SensitivityAnalyzer. Rezultat może wspierać ocenę wariantu topologii, ograniczeń zasobów albo skutków utraty obserwowalności.

W R2 każda symulacja musi zachować model, parametry, seed, zakres scenariusza i ograniczenia. Zmiana wyniku po modyfikacji założeń jest informacją o modelu, a nie automatyczną prognozą produkcji. Wspólny kontrakt federacyjny powinien oddzielać SimulationResult od ObservedEffect. Takie rozróżnienie chroni przyszłe plany przed przedstawieniem eksperymentu jako faktycznego deploymentu.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Odtwarzalny SimulationResult z jawną wrażliwością.
Do eksperymentowania	Nowe modele awarii i porównania polityk.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

Źródła: A01, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

writeups · Korpus badań i historyczna percepcja

Publikacja utrwala wiedzę o badaniu; merge historii nie musi zmieniać bieżących plików.

Raport R1 opisywał writeups jako corpus badawczy. Nowe odczyty GitHub w tej próbie pokazują HEAD 88d8f27b… oraz pierwszy parent c3fd319b… z tym samym TREE ff151679…. Jest to konkretny przykład rozdziału genealogii od zawartości snapshotu.

W R2 materiał opublikowany pozostaje powiązany z oryginalnym eksperymentem, jego wynikiem i ograniczeniami. Dodanie publikacji do Git nie promuje jej automatycznie do normy CORE ani aktualnego RAG. Historyczne opracowania można reinterpretować i wiązać ze świeższymi dowodami, ale kopiowanie tego samego raportu do repozytorium nie tworzy nowej niezależnej obserwacji.

Rola czasowa	Może wspierać F/P/H; repo nie jest przypisane do jednego czasu.
Do stabilizacji	Identyfikacja publikacji, źródeł i relacji supersession.
Do eksperymentowania	Nowe wyjaśnienia historii oraz badania nad kolejną architekturą.
Odbiór integracji	Rzeczywisty producer, consumer, wersja i negatywny test kontraktu.

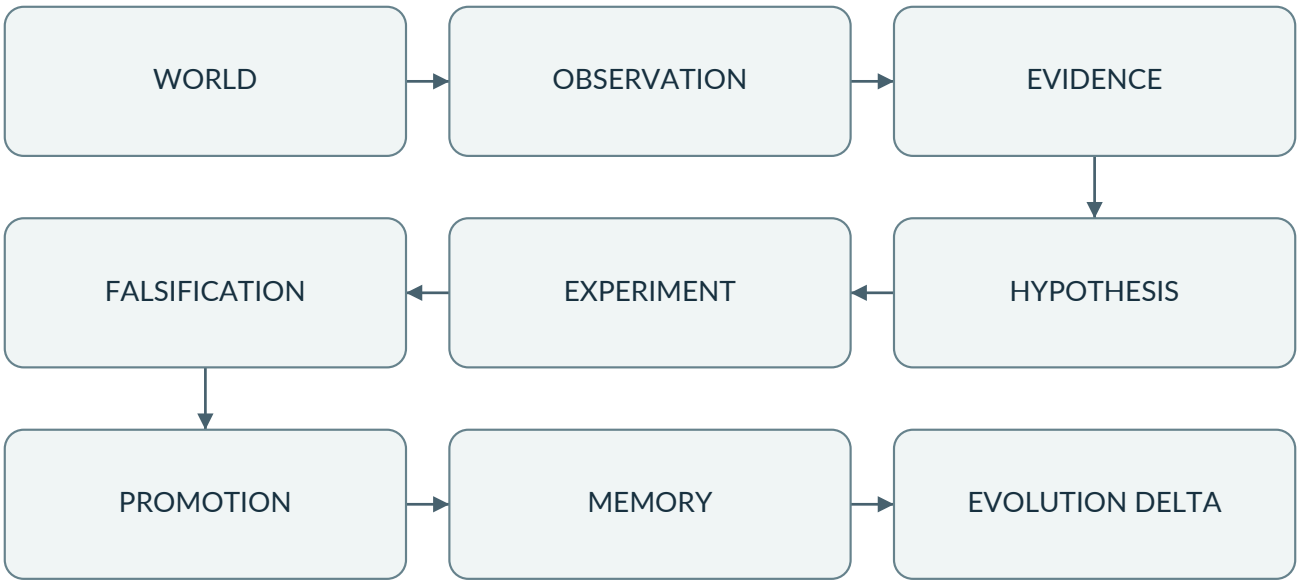
Źródła: G02, R1, S16 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-01 · Badanie i pamięć

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Źródłowy przepływ zaczyna się od świata i obserwacji, a kończy deltą ewolucji. Jego kluczową granicą jest przejście od wyniku eksperymentu do pamięci: promocja nie może być zastąpiona samym zapisem notatki. To makrologika badania, nie dowód, że każda strzałka jest obecnie wykonywana automatycznie.

W rozwinięciu L3 należy pokazać źródło hipotezy, kontrhipotezę i interpretację falsyfikacji. W L4 potrzebne są konkretne artefakty badania, a w L5–L6 faktyczni producenci i miejsca zapisu. Negatywny rezultat może zakończyć epizod bez powstania wdrożonego modułu.



AT2-F01 · FLOW-01 / Badanie i pamięć. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

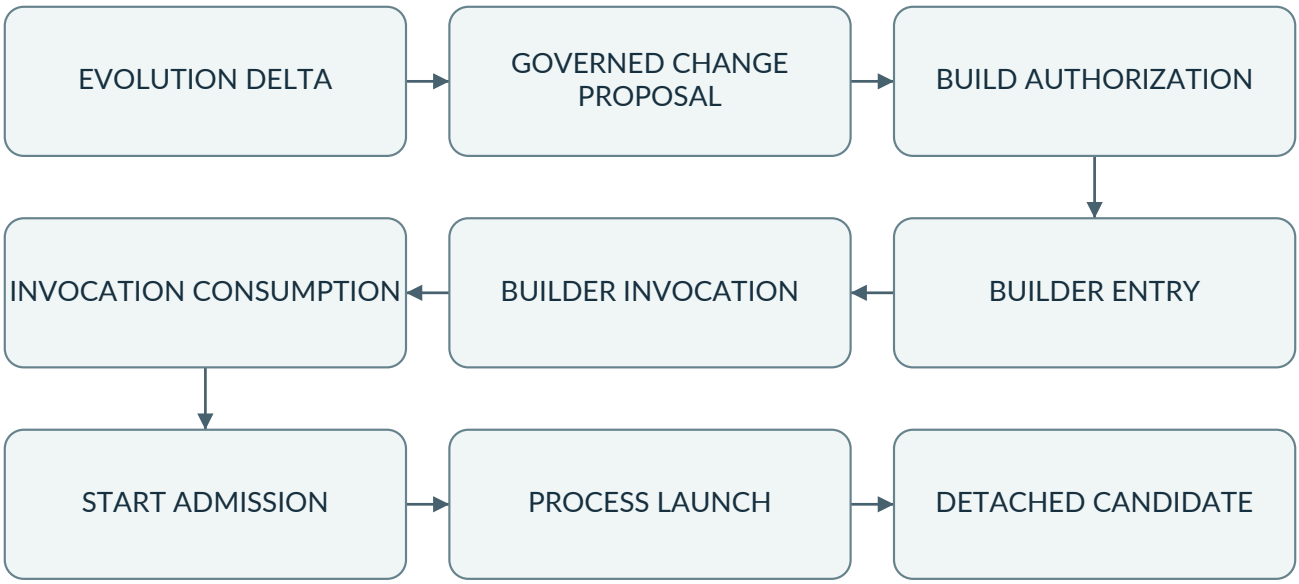
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-02 · Budowanie odłączonego kandydata

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Przepływ rozdziela propozycję zmiany od samego uruchomienia buildera. To istotne dla generatywnego systemu: opis potrzebnego kodu nie może sam uruchamiać dowolnego procesu. Związanie invocation i jego konsumpcji ogranicza możliwość niejawnego powtórzenia tego samego uruchomienia.

W atlasie wynikiem tej ścieżki jest kandydat odłączony od kanonicznego stanu, a nie merge lub deployment. Rozwinięcie musi wskazać miejsce pracy, zakres wejściowych źródeł, tożsamość buildera oraz oryginalny limit skutku. Kontrpróba powinna zmienić co najmniej jedną z tych tożsamości.



AT2-F02 · FLOW-02 / Budowanie odłączonego kandydata. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

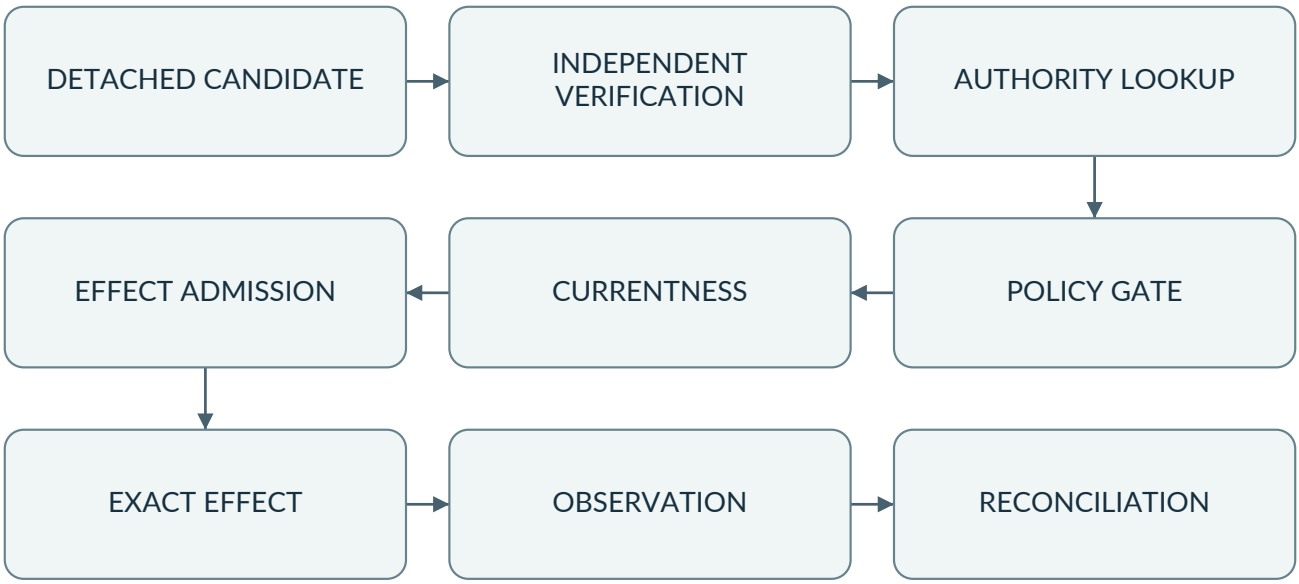
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-03 · Od kandydata do rozliczonego skutku

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Ta ścieżka oddziela niezależną weryfikację kandydata od sprawdzenia uprawnienia w momencie skutku. Zaliczenie testów nie eliminuje możliwości driftu źródeł, wygaśnięcia decyzji albo zmiany zasobu. Postconditions dotyczą rzeczywistego wyniku, nie deklarowanego celu.

W widoku temporalnym kandydat należy do pracy teraźniejszej, po której pozostaje zamrożony zapis testu i dopuszczenia. Aktualna interpretacja tego wyniku może później ulec zmianie, lecz nie wolno przepisać historycznego admission. Niepewny skutek wymaga rekonsyliacji przed kolejną generacją.



AT2-F03 · FLOW-03 / Od kandydata do rozliczonego skutku. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

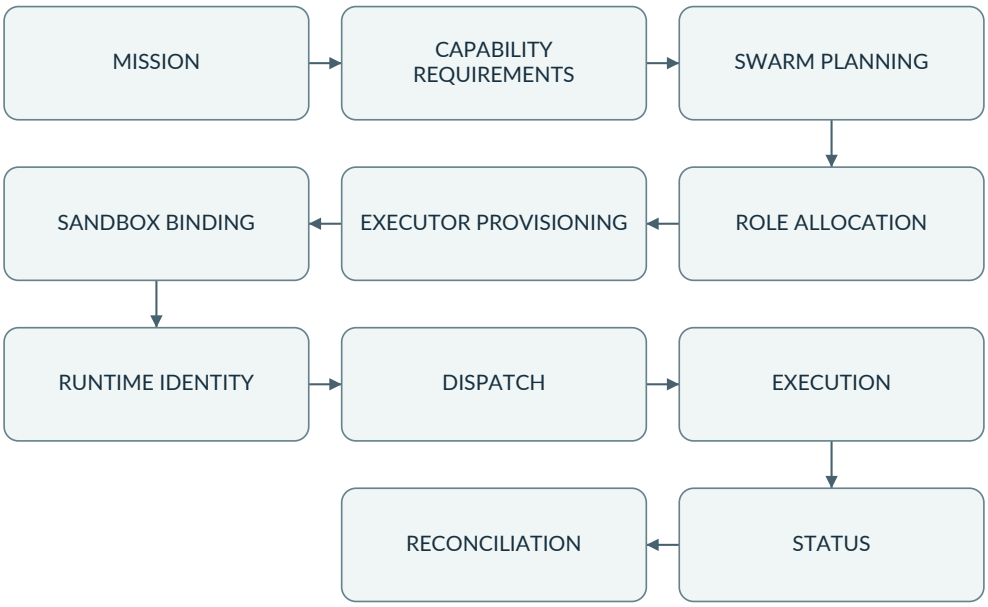
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-04 · Organizacja roju i wykonania

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Misja wyznacza wymagania zdolności, z których wynikają role i zasoby. Flota logiczna nie jest tym samym co liczba kontenerów. Każdy executor potrzebuje tożsamości i powiązania ze środowiskiem, zanim stanie się odbiorcą konkretnego przydziału.

Rozwinięcie powinno pokazywać zarówno planowaną, jak i materialną topologię. Przypisanie roli verifiera innemu kontenerowi nie dowodzi odrębnej fizycznej domeny awarii. Wartości material_target oraz capability requirements należą do dokładnej wersji misji i nie mogą być zmieniane jako zabieg wizualizacyjny.



AT2-F04 · FLOW-04 / Organizacja roju i wykonania. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

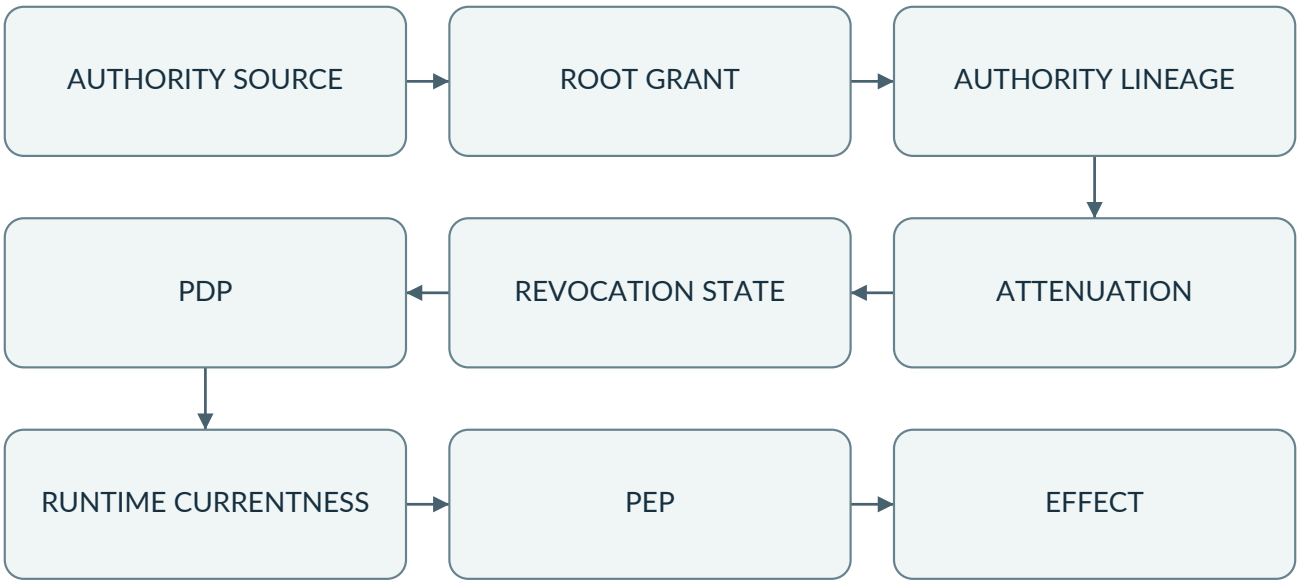
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-05 · Pochodzenie uprawnienia

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Authority ma oryginalne źródło i ograniczaną genealogię. Proces poznawczy może proponować działanie, ale nie rekonstruuje brakującego grantu z języka naturalnego. Revocation i currentness są częścią sprawdzanego stanu, nie stałą właściwością raz zaliczonego testu.

W dokumencie każda implementacja kontroli powinna mieć powiązanie z ownerem decyzji i rzeczywistą klasą efektu. Wzbogacenie grafu semantycznego nie zmienia tej ścieżki. Kontrprzykładem dla wadliwej integracji jest przyjęcie reprezentacji ALLOW bez weryfikacji oryginalnego decision evidence.



AT2-F05 · FLOW-05 / Pochodzenie uprawnienia. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

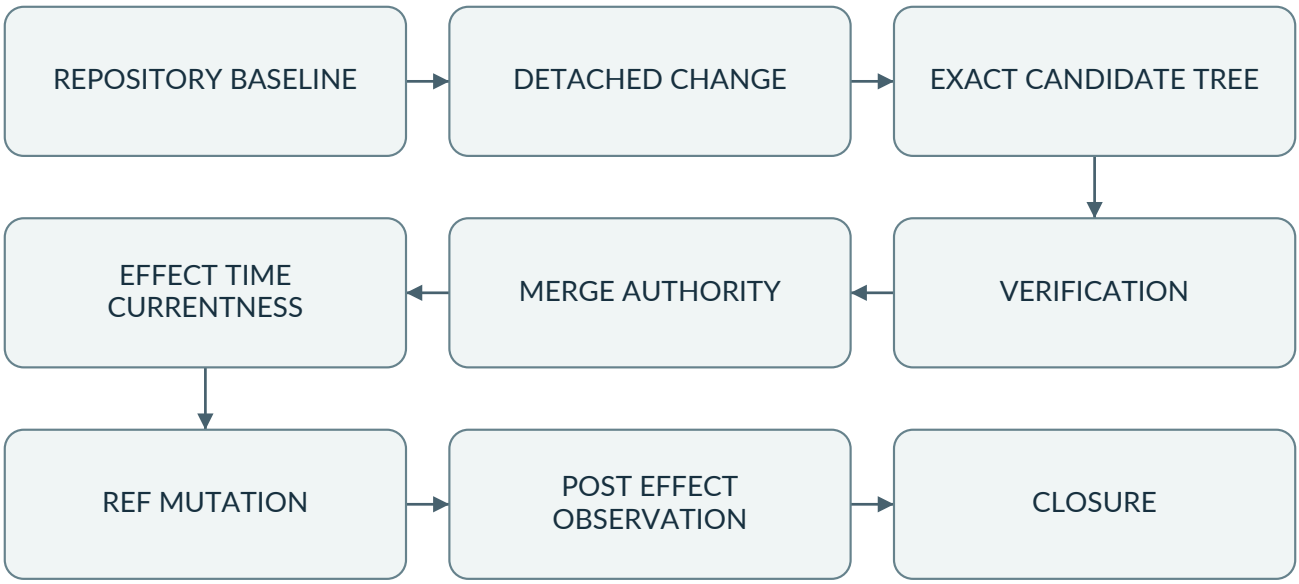
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-06 · Zmiana referencji repozytorium

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Merge jest efektem na referencji Git, nie synonimem budowania ani wdrożenia. Exact candidate tree, głowy gałęzi i topologia rodziców muszą zostać rozliczone dla konkretnego momentu. Zmiana HEAD unieważnia przeniesienie starszego exact-head CI na nową rewizję.

Przypadek writeups pokazuje, że nowy commit może zachować to samo drzewo pierwszego rodzica. Dlatego odbiór zmiany musi osobno odpowiadać na pytanie o zawartość i o genealogię. Jest to materialna podstawa temporalnego modelu R2, a nie metafora oderwana od struktury Git.



AT2-F06 · FLOW-06 / Zmiana referencji repozytorium. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

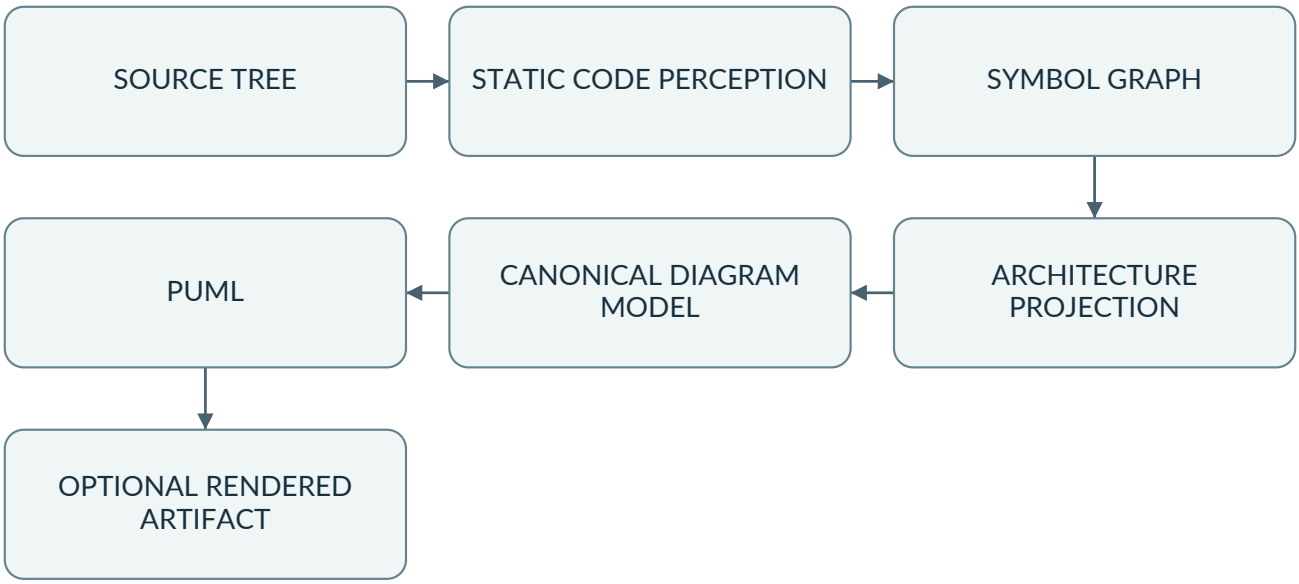
Źródła: S02, R1, G02, G03 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-07 · Percepcja kodu i obraz architektury

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

To najbardziej bezpośredni kontrapunkt drzewa Git i AST. Źródłowe bajty są normalizowane do grafu symboli, następnie wiązane z modelem architektonicznym i obrazem. Sam diagram pozostaje pochodną, której aktualność zależy od źródła i generatora.

Nowe testy R2 dotyczą wybranych właściwości Code Perception, a nie kompletnego wykonania wszystkich kroków FLOW-07 na całej federacji. W szczególności źródłowy static call candidate nie jest runtime target. Opcjonalny render nie może zmienić tej klasy dowodu przez dobór koloru lub strzałki.



AT2-F07 · FLOW-07 / Percepcja kodu i obraz architektury. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

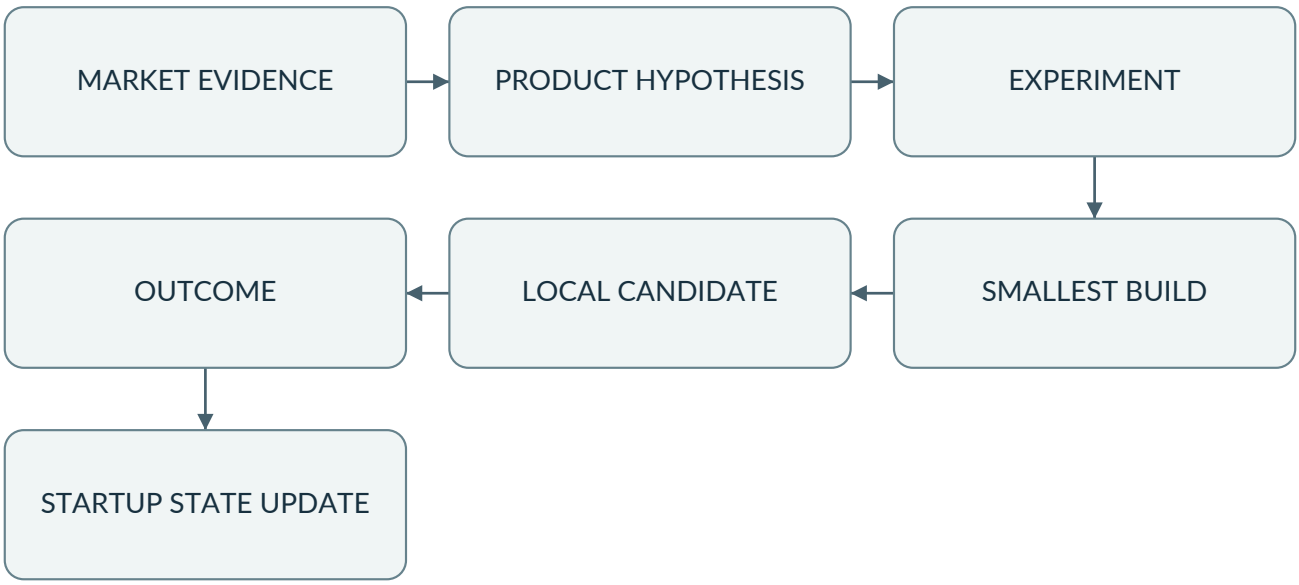
Źródła: S02, R1, T01, T02 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-08 · Eksperyment produktowy

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Źródłowy katalog obejmuje także pętlę hipotezy produktowej. Jej logika jest zbieżna z kierunkiem organicznym: z obserwacji wynika mały eksperyment, a jego rezultat wpływa na dalszą strategię. Nie należy jednak utożsamiać katalogu przepływu z potwierdzonym bieżącym badaniem rynku.

W przyszłym rozwinięciu trzeba wskazać źródła market evidence i oddzielić oczekiwanie wartości od wyniku eksperymentu. Najmniejszy użyteczny build powinien rozstrzygać ważną niepewność, nie produkować plik dla samego spełnienia licznika. Jego niepowodzenie może zmienić przyszły cel bez degradacji już zweryfikowanego CORE.



AT2-F08 · FLOW-08 / Eksperyment produktowy. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

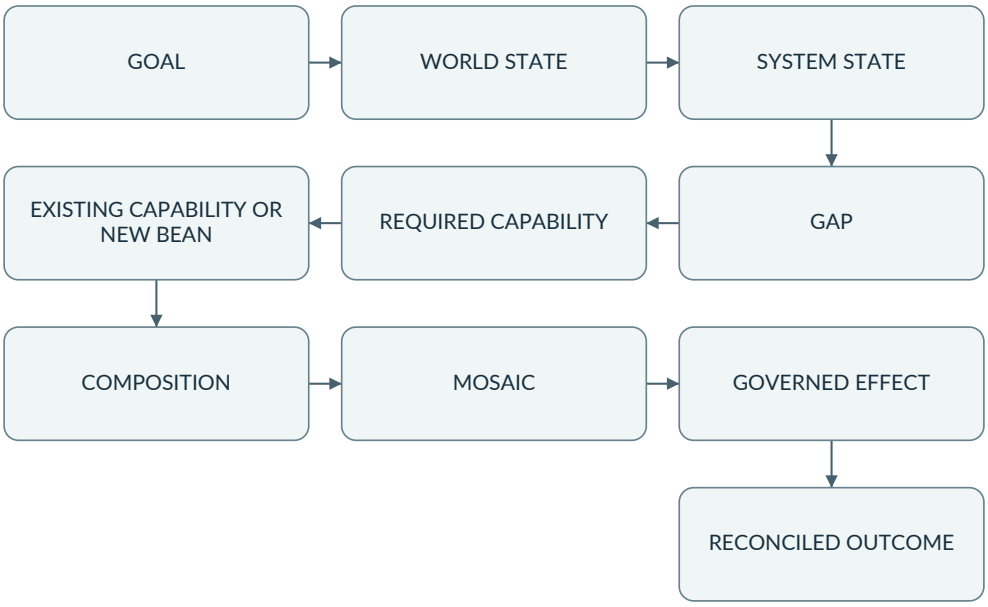
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-09 · Potrzeba, Bean i mozaika

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Przepływ wiąże cel z rzeczywistą luką, a nie z arbitralnym żądaniem nowego modułu. Dobór istniejącej zdolności lub projektowanie nowej są alternatywami tego samego procesu. Kompozycja musi zachować zależności, kompatybilność oraz rozdział ról.

Obecne eksperymenty B0 i R22F stanowią materiał do badania ograniczonej generatywności, nie pełny dowód uniwersalnej fabryki. Rozwinięcie FLOW-09 wymaga jawnego wiązania specyfikacji z realizacją oraz materiałem końcowym. Druga niezależna rodzina artefaktu jest przyszłym testem reużywalności mechanizmu, nie automatycznym skutkiem tego diagramu.



AT2-F09 · FLOW-09 / Potrzeba, Bean i mozaika. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

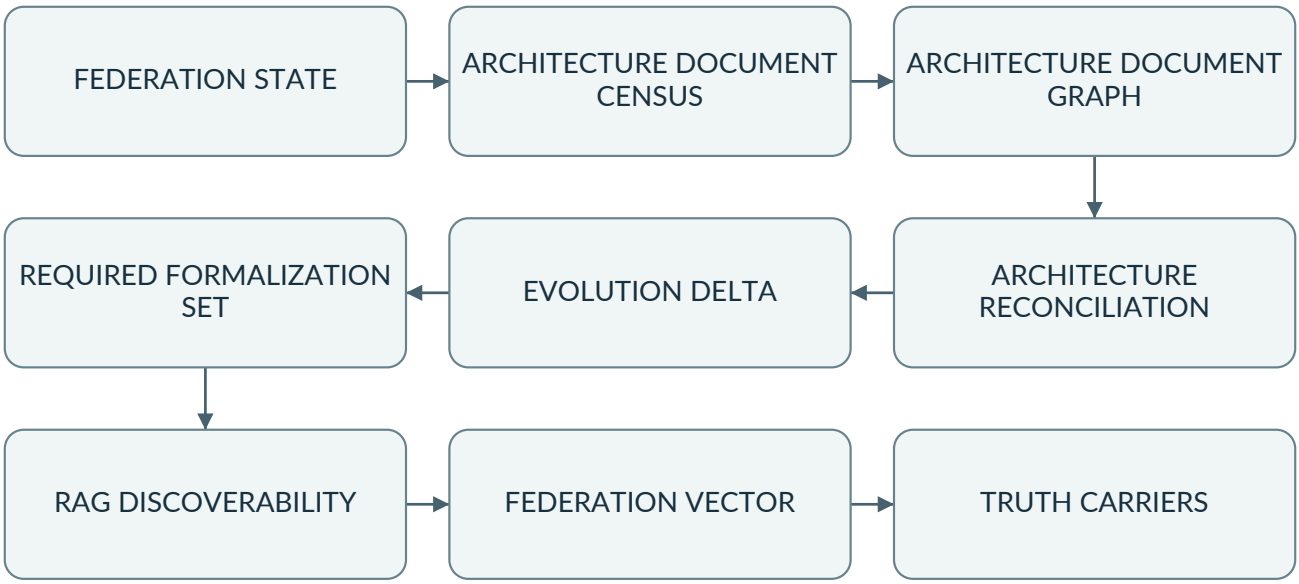
Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

FLOW-10 · Homeostaza architektury

Porządek zadeklarowany w native FLOW_SPECS; rozwinięcie dokumentacyjne, nie nowe wykonanie E2E.

Dokumentacja uczestniczy w cyklu rozwoju jako wynik i ograniczone wejście, a nie dodatkowy issuer. Zmiana źródeł może unieważnić zależne projekcje, entrypointy i kontekst RAG. Global-owner overlay pomaga rozliczyć nowszy ai_platform nad starszą bazą federacji.

R2 zachowuje regułę carrier-last i rozróżnia spójność routingu od prawdziwości każdej tezy w README. Kolejny dokument wizualny powinien powstawać z modelu, którego źródła i niepewności są zidentyfikowane. Zgodność dziesięciu nazw flow nie dowodzi wykonania dziesięciu żywych procesów.



AT2-F10 · FLOW-10 / Homeostaza architektury. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Granica modelu	Kolejność semantyczna źródła; wiring i runtime oceniane oddzielnie.
Perspektywa czasu	Cel / aktywny epizod / zapis dowodu łączone przez tożsamości.

Źródła: S02, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Nazwy powinny stabilizować znaczenie

Zmiana etykiety ma sens, gdy usuwa wieloznaczność bez zerwania tożsamości i konsumentów.

Istniejący LION_NAMING_STANDARD określa role nazw, konwencje modułów, klas, schematów i gałęzi oraz zakaz przepisywania niezmiennej historii. R2 proponuje stosowanie tego słownika jako mapy aliasów, nie serię masowych rename. Szczególnie należy zachować historyczną nazwę cyber_lion, dopóki jej konsumenci wymagają tej przestrzeni importów.

W interfejsie człowiek może widzieć polską nazwę opisową, podczas gdy API używa istniejącego identyfikatora. Wspólny słownik powinien wiązać obie postaci. Nowa nazwa semantyczna nie może łączyć różnych stanów, na przykład qualification, admission i effect. Dla publicznego symbolu przenoszonego do innego modułu potrzebna jest karta migracji i okres kompatybilności.

Nazwa użytkowa	Czytelny opis roli.
Identyfikator kontraktu	Stały klucz w określonej wersji.
Alias	Jawny historyczny lub kompatybilny odpowiednik.
Migracja	Konsumenci, importy, tests, rollback i source refs.

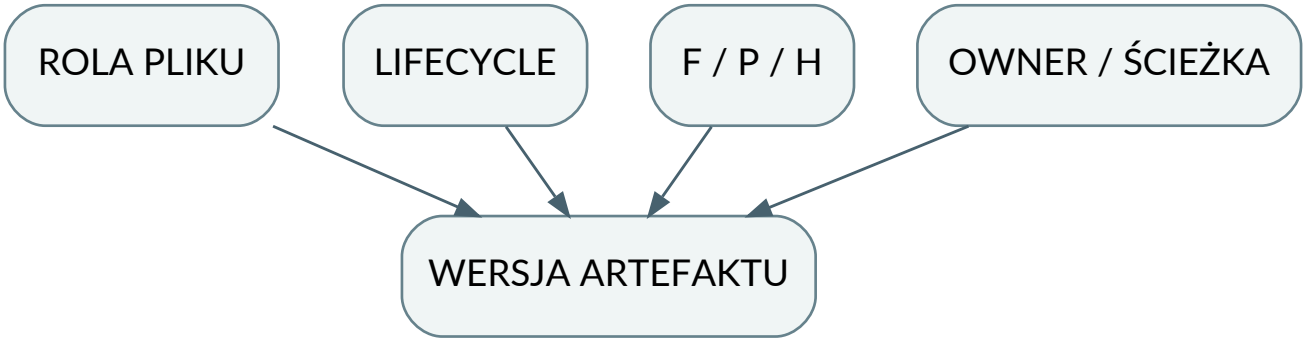
Źródła: S10, R1, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Kategorie nie są osiami czasu

Katalog archive może zawierać wiedzę użyteczną teraz, a plik target może nadal opisywać niezrealizowaną przyszłość.

Podział SOURCE, SPEC, GENERATED, RUNTIME STATE i RESEARCH ARTIFACT określa funkcję nośnika. Podział przyszłość–teraźniejszość–przeszłość określa rolę obiektu w danym epizodzie. Te osie nie są zamienne. Kontrakt z poprzedniej epoki może być nadal aktualną podstawą nowego eksperymentu, a najnowszy wygenerowany raport może dotyczyć wyłącznie historycznego cutoff.

R2 odradza utworzenie trzech wielkich katalogów future/present/past jako jedynej organizacji produktu. Trzy horyzonty lepiej realizować przez widoki, metadane i relacje czasowe. Fizyczna struktura powinna odzwierciedlać ownerów, granice importów, magazynów i deploymentu. Dzięki temu reinterpretacja archiwum nie wymusza przenoszenia oryginalnego pliku.



AT2-G25 · Ortogonalne kategorie opisu. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Funkcja	Źródło, kontrakt, projekcja, test, stan lub dowód.
Dojrzałość	Eksperyment, kandydat, formalizacja, integracja.
Czas	Rola F/P/H względem epizodu i cutoff.
Miejsce	Właściciel techniczny i fizyczny nośnik.

Źródła: S03, S10, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Docelowa organizacja repozytorium

Pierwsza normalizacja jest logiczna: rozlicza role i zależności przed fizycznym przenoszeniem kodu.

W ai_platform kontrakty pozostają przy contracts, orkiestracja misji przy mission_control, rozmowy przy app_coordination, a percepcja i formalizacja przy istniejących ownerach enterprise oraz architecture_projection. LION zawiera normy, architekturę, RAG i historyczne dossier. browser_broker stanowi inną powierzchnię wykonania niż bibliotekowy moduł Python.

Dla następnej wersji dokumentacji proponujemy widoki public_interfaces, implementation, research, evidence i history jako indeksy nad rzeczywistymi ścieżkami. Nie muszą być nowymi katalogami runtime. Przenosiny mają sens dopiero po zbadaniu import closure, package manifests, workflow, instalatorów i odwołań w dokumentacji. Stare v1_4 nie może zostać w całości zaklasyfikowane jako martwe archiwum: nadal routowane są tam konkretne kontrakty.

contracts / schemas	Typy i kompatybilność publiczna.
enterprise / process / MC	Istniejąca realizacja procesu i kontroli.
tests / experiments	Regresja, kontrpróby i ograniczone badania.
LION / docs / RAG	Wersjonowane wyjaśnienia i projekcje.
deploy / broker / tools	Dokładne entrypointy i granice pakietów.

Źródła: S10, S16, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Karta migracji symbolu i pliku

Zmiana ścieżki jest deltą architektury, nawet gdy zawartość bloba się nie zmienia.

Próba P04 pokazała, że ta sama funkcja przeniesiona z a.py do b.py otrzymuje inny lokalny symbol ID. Architektura musi więc zachować relację przeniesienia, a nie wywodzić ciągłość tylko z hasha implementacji. Ta sama treść może być skopiowana, rozdzielona albo mieć nowego ownera; każde zdarzenie ma inne znaczenie.

Proponowana karta migracji zawiera starą i nową ścieżkę, source refs, identyfikację symboli, powód, typ zmiany, odbiorców, okno kompatybilności i testy. Jej stan nie staje się APPLIED przez samo zatwierdzenie projektu. Po przyszłym wykonaniu trzeba odczytać rzeczywiste drzewo i zweryfikować, czy zmiana nie naruszyła importów, skrótów Windows, konfiguracji WSL albo payloadów instalatora.

Przed	Repo / commit / path / symbol.
Po	Proponowana realizacja i alias.
Wpływ	Importy, konfiguracja, narzędzia, wiedza i runtime.
Odbiór	Exact diff, consumer tests, readback i plan wycofania.

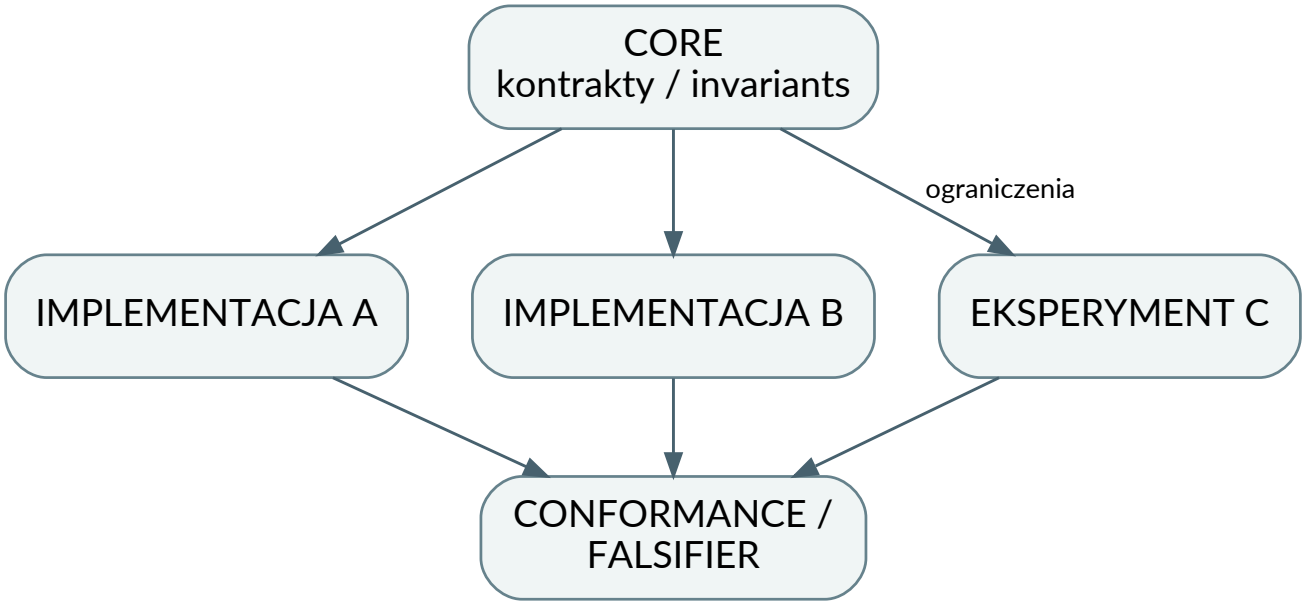
Źródła: T02, S10, R1 | PROJEKT R2 / AS-IS OGRANICZONY

CORE jako minimalny zestaw trwałych zobowiązań

Stabilny rdzeń ma chronić sens kolejnych eksperymentów, a nie zamykać ich przestrzeń.

Kandydatami CORE są tożsamość i provenance, typowane granice semantyczne, kontrakt procesu, kontrola skutku, kompozycja zdolności i mechanizm formalizacji. R1 wskazał je jako odpowiedzialności do kwalifikacji, nie jako globalnie stabilny produkt. Każda część ma odrębny etap dojrzałości.

R2 proponuje utrwalić najpierw te właściwości, których naruszenie uniemożliwia ocenę eksperymentu: źródłową identyfikację, zachowanie unknown, exact parent binding oraz rozróżnienie wyniku i efektu. Reprezentacje i algorytmy mogą pozostać polimorficzne. Promocja do CORE wymaga realnych konsumentów i negatywnych prób, a nie arbitralnej wspólności nazw.



AT2-G26 · CORE i polimorficzne realizacje. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Zachowaj	Sens kontraktu i warunki dowodu.
Wymieniaj	Implementacje spełniające ten sam jawny zakres.
Wersjonuj	Breaking changes i migracje.
Nie mieszaj	Źródłowa stabilność, deployment i authority.

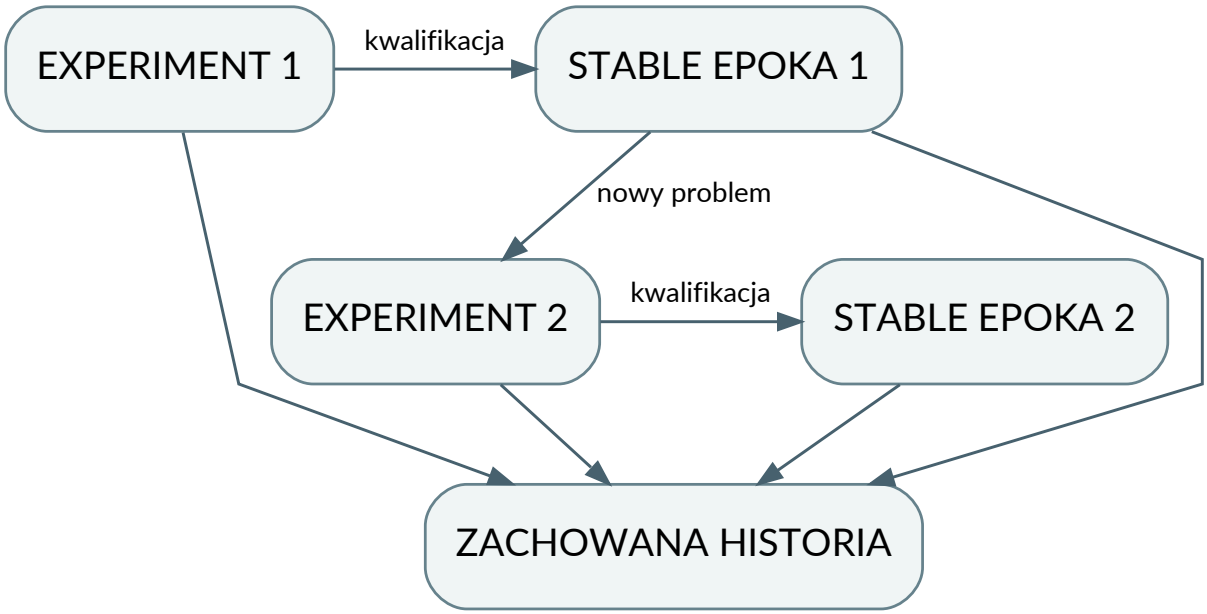
Źródła: R1, S03, S10 | PROJEKT R2 / AS-IS OGRANICZONY

Od stable do następnego experimental

Nowy kontrprzykład otwiera nową deltę, nie unieważnia całego dorobku projektu.

Warunkowy profil STABLE FOR EPOCH oznacza, że dany zakres ma ownera, wersję, rozpoznanych konsumentów, wyniki testów i rozliczone krytyczne zależności. Nie jest dodatkowym native runtime enum. Kolejna generacja może eksperymentować obok tego zakresu, dopóki nie narusza jego utrwalonych ograniczeń.

Jeśli nowe zadanie wykracza poza gramatykę B0 albo granice operatora SHD, trzeba jawnie rozszerzyć przestrzeń badania. Eksperyment powinien porównać silny baseline, nową reprezentację i transfer, zachowując dane negatywne. Wynik może uzasadnić nowy moduł, inny adapter albo pozostawienie obecnej architektury. Projekt generatywny nie musi przyjmować każdej wygenerowanej przyszłości.



AT2-G27 · Epoki stabilizacji. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Podstawa	Zweryfikowany zakres i powtarzalny rollback.
Eksploracja	Nowy problem poza dotychczasową granicą.
Selekcja	Koszt, dowody, użyteczność i falsyfikator.
Powrót	Nowa stabilna epoka albo zachowana odmowa promocji.

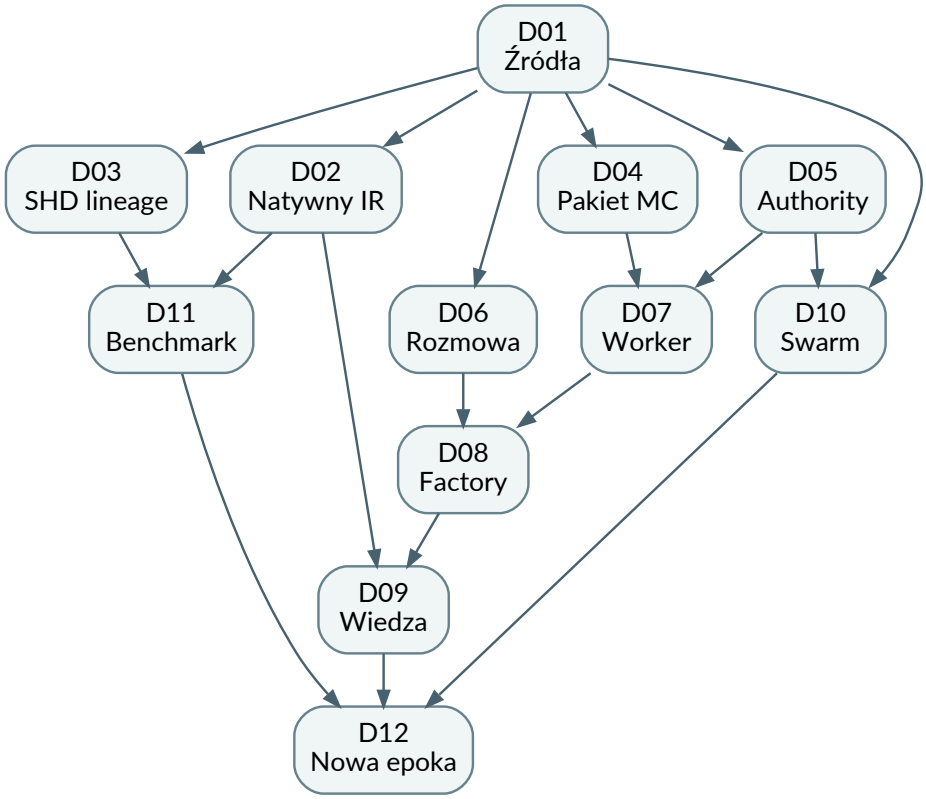
Źródła: R1, A-SHD, S03 | PROJEKT R2 / AS-IS OGRANICZONY

Mapa przejścia: od R1 do następnej epoki

Karty D01–D12 zachowują tożsamość wcześniejszego planu, lecz R2 doprecyzowuje ich zależności informacyjne.

Plan jest projektem przyszłych przyrostów, nie listą uruchomionych misji. D01 ustala baseline konkretnego zakresu; nie oznacza, że każdy wąski eksperyment musi czekać na pełny odczyt całej federacji. D02–D06 mogą rozwijać się niezależnie po domknięciu własnych źródeł. D07 i D08 wymagają oddzielnych oryginalnych dopuszczeń.

R2 dodaje do tego planu uporządkowanie hierarchii, tożsamości Git–AST i czasu. Nie tworzy nowego wykonawczego DAG obok ai_platform. Każda przyszła misja powinna mieć skończony primary artifact, negatywny test, odczyt wyniku i checkpoint. Nowy PDF ani test lokalny nie zmienia stanu Dxx na wykonany. Węższy CORE może zostać zakwalifikowany bez benchmarku SHD, ale bez twierdzenia o wykazanej korzyści tego mechanizmu.



AT2-G28 · DAG przyrostów D01–D12. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

R1 → R2	Zachowane IDs i lineage kart.
Nowa delta	Dokładniejszy model poziomów, czasu i percepcji.
Implementacja	Osobny zakres realizowany u istniejących ownerów.
Odbiór	Dowód właściwy dla konkretnego przyrostu.

Źródła: R1, T02, A-R6, A-SHD | PROJEKT R2 / AS-IS OGRANICZONY

D01 · Uzgodniony inwentarz i mapa źródeł

Wektor Git i ledger dla dokładnego zakresu przyrostu, mapowanie runtime→pakiet; szerszy frontier U01/U02 pozostaje jawny.

R2 dostarcza pięć byte-verified plików Code Perception, nowe odczyty dwóch repozytoriów i mały pakiet źródeł historycznych. Nie zastępuje nimi pełnego mianownika wszystkich gałęzi. Następny dokument musi kontynuować ewidencję odczytów, rozróżniając dokładny source ref od aktywnego checkoutu.

Przy normalizacji należy powiązać drzewo Git, rekord pliku, symbol AST i komponent widoku. Nie ma potrzeby ponownie analizować identycznych blobów dla każdej gałęzi; trzeba jednak rozliczyć delty i warunki zgodności metody. Pierwszym artefaktem powinien być zakresowany rejestr źródeł i konsumentów, nie nowa ogólna roadmapa.

Owner	ai_platform/LION + repozytoria
Zależności	Baseline zakresu
Predykat odbioru	Odtwórz liczniki i rozlicz każdą referencję krytyczną dla przyrostu; niezależna praca nie czeka na pełny odczyt pozostałej federacji.
Otwarty zakres	U01, U02, U09, U14
Granica skutku	READ_ONLY_RESEARCH
Zachowanie historii	Zachowanie poprzedniego snapshotu; nowy superseding research record.

Źródła: R1, A-R6, A-SHD, T02 | TO-BE / DESIGN_NOT_EXECUTED

D02 · Jawny właściciel materializacji reprezentacji

Adapter między actual source spans, rodzicami i natywnym IR; bez nowego schedulera.

Wiązanie reprezentacji powinno sprawdzać rzeczywistych rodziców, zakres source spans, epokę oraz integralność końców relacji. Poprawny format digestu nie jest wystarczający. P06 pokazuje analogiczną granicę w czystym builderze AST: poprawne bytes i deklarowana tożsamość nie zastępują pozyskania danych z właściwego źródła.

Przyrost należy umieścić przy istniejącym ownerze reprezentacji. Jego odbiór obejmuje podmianę rodzica, fałszywy namespace, niezgodny cutoff oraz utratę unknown constraint. Docelowy adapter ma zwracać natywny obiekt z właściwą semantyką, nie JSON o podobnych nazwach pól.

Owner	ai_platform/contracts + architecture_projection
Zależności	D01
Predykat odbioru	Właściwe typy i rodzice, graph endpoint closure, złe hashe/epoki/relacje odrzucane.
Otwarty zakres	U03
Granica skutku	ISOLATED_SOURCE_PROPOSAL
Zachowanie historii	Wyłączenie nowego adaptera; powrót do dotychczasowej projekcji kontekstu.

Źródła: R1, A-R6, A-SHD, T02 | TO-BE / DESIGN_NOT_EXECUTED

D03 · Zamknięcie luki lineage prototypu SHD

Dodatni generation, niepuste potwierdzone korzenie i przepływ unknown aż do konsumenta.

R1 zarejestrował trzy akceptowane mutacje niepoprawnego lineage prototypu SHD. Te wyniki pozostają historyczną podstawą karty; R2 nie poprawił prototypu. Odbiór powinien odrzucić generation ≤ 0 i brak wymaganych, rzeczywistych referencji, jednocześnie zachowując częściowo rozpoznane ograniczenia.

Trzeba odróżnić naprawę walidatora od wykazania generatywnego transferu. Po naprawie można ponownie uruchomić istniejący suite i dyskryminujące kontrpróby. Dopiero osobny benchmark oceni, czy operator pomaga na nowych rodzinach zadań. Sam wzrost liczby PASS nie jest dowodem przewagi poznawczej.

Owner	Oryginalny owner prototypu / później ai_platform
Zależności	D01
Predykat odbioru	F-SHD-01..03 przestają przechodzić; F-SHD-04 nie pozwala zgubić ograniczenia.
Otwarty zakres	U10, U11
Granica skutku	ISOLATED_SOURCE_PROPOSAL
Zachowanie historii	Zachować R1 i wyniki negatywne; nowa wersja, bez przepisywania dowodów.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D04 · Odbiór domknięcia pakietu Mission Control

Exact file/import closure i jawna diagnostyka phase-zero import failures.

Odczyt dwóch zgodnych plików Mission Control nie zamyka całego deployment package. Karta wymaga dokładnego manifestu, rzeczywistej ścieżki importu oraz wszystkich zależności fazy poznawczej. Brak pliku w jednej nested lokalizacji nie uzasadnia jeszcze diagnozy jego globalnej nieobecności.

Rozwinięcie L5/L6 powinno wskazać source symbol → package member → process environment → actual import. Próba kontrolna z brakującą zależnością musi dać jednoznaczną diagnostykę gotowości zadania. Badanie kopii pakietu jest odrębne od wdrożenia poprawki, które wymaga osobnego dopuszczenia.

Owner	Oryginalny installer / Mission Control
Zależności	D01
Predykat odbioru	Wszystkie wymagane moduły/piny zgodne; missing dependency jawnie BLOCKED; health nie maskuje gotowości misji.
Otwarty zakres	U04
Granica skutku	SOURCE_THEN_SEPARATE_DEPLOYMENT
Zachowanie historii	Osobny backup/rollback oryginalnego pakietu po dopuszczeniu; brak zmian teraz.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D05 · Oryginalne źródła PDP, authority i replay

Zidentyfikowany prawdziwy issuer; jeśli brak, kandydat źródłowy bez mintowania trust.

Nowy model architektury nie może stać się zastępczym issuerem. Należy rozliczyć oryginalne źródło decyzji, zakres, czas, revocation i trwałą konsumpcję replay. Historyczne wpisy o zużytej zgodzie nie są materiałem do odtworzenia grantu dla następcy.

Wizualizacja musi odróżniać reprezentację decyzji od jej autorytatywnego producenta. Powiązanie PDPResultBinding z wynikiem rzeczywistego ownera wymaga testu w granicy zaufania. W tym atlasie nie odczytywano kluczy i nie zmieniano polityk; karta pozostaje projektem wymagającym aktualnych danych przed skutkiem.

Owner	Oryginalne governance / enterprise
Zależności	D01
Predykat odbioru	Oddzielny owner decyzji; nieprzekazywanie consumed grant; podpis, input, czas i revocation sprawdzone.
Otwarty zakres	U05
Granica skutku	ORIGINAL_GOVERNANCE_REQUIRED
Zachowanie historii	Nie odtwarzać grantu; utrzymać HOLD i zachować lineage decyzji.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D06 · Ciągłość rozmowy i rzeczywista ścieżka operatora

Jedna ścieżka dispatch, przygotowanie ≠ wysłanie, exact epoch i oddzielne digests.

Ciągłość rozmowy obejmuje kontekst, epokę, lane, sesję, request i faktyczne wysłane bajty. Pierwszy etap przyrostu powinien odtworzyć automat prepared/sent/responded/reconciled oraz odpowiednie negatywne warianty. Panel ma pokazywać niepewny skutek bez zachęcania do ślepego resend.

W R1 proces Electron wskazywał release PR454, a body PR456 odnosiło się do innego head niż odczytana rewizja. Tych historycznych wyników nie promujemy na bieżącą wersję. Native GUI, skrót operatora i sesja wymagają oddzielnego odbioru po kwalifikacji właściwego źródła.

Owner	app_coordination + browser_broker
Zależności	D01
Predykat odbioru	Nowe testy źródłowe timeout/restart/late receipt; dopiero osobny native odbiór nowej sesji.
Otwarty zakres	U06, U07, U14
Granica skutku	SOURCE_THEN_SEPARATE_NATIVE_ACCEPTANCE
Zachowanie historii	Zachować profil, oryginalny skrót i journal; bez blind replay.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D07 · Kwalifikacja jednego rzeczywistego workera

Osobny HELD→READY dla jednego workera; bez skutku materiałowego.

Kwalifikacja workera nie powinna od razu produkować artefaktu. Ma wykazać tożsamość, dostępność zdolności, prawidłowe wejście, lease oraz trwałe przejście HELD → READY w dokładnym zakresie. R2 zachowuje rozdział roli logicznej i materialnej instancji.

Wymagana niezależność obserwacji zależy od klasy skutku. Dwa kontenery nad wspólnym kernelem nie są dwiema fizycznymi domenami. Odbiór powinien jawnie określić akceptowany profil oraz negatywny wariant zmienionego lease, generacji lub źródła. To przyszła kwalifikacja runtime, nie wykonany test tego dokumentu.

Owner	Mission Control / cooperative providers
Zależności	D04, D05
Predykat odbioru	Exact źródła, lease, typy, trusted input i oddzielne readback; zero artefaktów w kwalifikacji.
Otwarty zakres	U08
Granica skutku	SEPARATE_EXACT_ADMISSION
Zachowanie historii	Wycofać gotowość; nie czyścić historycznego skutku jako recovery.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D08 · Pierwszy skończony materialny odbiór Factory

Rejestracja/START osobne. Dla obecnego programu zachować 32-provider readiness; mały proof wymaga odrębnego kontraktu.

Pierwszy odbiór Factory musi dać użyteczny materialny artefakt, nie tylko zgodną opowieść o jego powstaniu. Dla obecnego programu zachowane są wymagania material_target=32; wariant minimalny wymaga własnego kontraktu. Rejestracja i aktywacja pozostają odrębnymi operacjami.

Powinny zostać związane: intencja, kontekst SaaS/LOCAL, wybrana specyfikacja, source identity, executor, admission, zapisane bytes, odczyt verifiera i końcowa rekonsyliacja. Negatywny wariant musi sprawdzać niepewny wynik przed receipt. Samo istnienie pliku po próbie nie wystarcza do wykazania pełnej genealogii wykonania.

Owner	Application Factory / operator
Zależności	D06, D07
Predykat odbioru	Exact artifact bytes, rzeczywisty receipt i verifier; brak podwójnego efektu przy UNKNOWN.
Otwarty zakres	U04, U05, U06
Granica skutku	SEPARATE_MISSION_AND_EFFECT_ADMISSION
Zachowanie historii	Rekonsyliacja i ewentualna osobno dopuszczona kompensacja.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D09 · Sprzężenie artefaktu z następną epoką wiedzy

Wynik i kontrprzykład wracają jako evidence do ProjectReality; bez drugiego truth store.

To przyrost domykający sens pętli L0. Wynik artefaktu i kontrprzykład muszą wrócić do istniejącego modelu rzeczywistości oraz wiedzy, z zachowaniem tożsamości epizodu. Nie należy budować drugiego truth store tylko dlatego, że potrzebna jest nowa projekcja.

W R2 historyczny rekord pozostaje niezmienny, a nowa ocena jest osobną interpretacją. Test ma wykluczyć użycie przyszłego dowodu w starszym cutoff, zagubienie odrzuconego kandydata i promocję niezweryfikowanej odpowiedzi. P11 pokazuje tę zasadę jedynie na instrumentowanym modelu badawczym, nie na natywnej bazie LION.

Owner	architecture_projection + evidence owners
Zależności	D02, D08
Predykat odbioru	Stary kandydat pozostaje; nowa generacja ma rodziców i rewalidację; negatywna próba nie staje się PASS.
Otwarty zakres	U03, U13
Granica skutku	SOURCE_THEN_BOUNDED_INTEGRATION
Zachowanie historii	Supersession, nie kasowanie historii; odwracalna projekcja.

Źródła: R1, A-R6, A-SHD, T02 | TO-BE / DESIGN_NOT_EXECUTED

D10 · Kwalifikacja kontraktu federacyjnego swarm

Jawne mapowanie wspólnych kontraktów i lokalnego admission; scheduler misji pozostaje w ai_platform.

Granica ai_platform–swarm wymaga aktualnych producentów i konsumentów kontraktu. Wspólne nazwy nie wystarczą: trzeba porównać preconditions, postconditions, retry, replay i rodzaj wyniku. Adapter nie może konstruować ALLOW jako substytutu oryginalnej decyzji.

Wariant cienkiej projekcji jest punktem startowym, nie nakazem przeniesienia globalnego schedulera. Osobny provider klastra jest uzasadniony dopiero po wykazaniu luki odpowiedzialności. Odbiór kontraktowy można przygotować offline; provisioning i żywa telemetria są dalszym, oddzielnie dopuszczonym etapem.

Owner	ai_platform adapter + swarm local runtime
Zależności	D01, D05
Predykat odbioru	Brak self-issued ALLOW, zgodność pre/postconditions, nonce persistence u właściwego ownera, brak rozszerzenia scope.
Otwarty zakres	U02, U08
Granica skutku	OFFLINE_FIRST_THEN_SEPARATE_PROVIDER_ACCEPTANCE
Zachowanie historii	Zachować stary adapter w compatibility window; bez przenosin wszystkich repo.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D11 · Preregistered benchmark reprezentacji i transferu

Silny baseline B, native C i D, oryginalny A tylko gdy rzeczywiście działa.

Benchmark powinien rozdzielić wpływ uporządkowania tych samych danych, ekstrakcji semantyki i transferu operatora. Silny baseline tekstowy musi otrzymać porównywalną wiedzę i budżet. Przypadki sparafrazowane z jednego scenariusza nie są niezależnymi replikacjami.

R2 nie uruchomił modeli. Docelowy protokół wymaga prerejestracji podziału rodzin, referencji, falsyfikatorów i sposobu oceny kosztu całego poprawnie zweryfikowanego zadania. Wynik zerowy lub negatywny powinien pozostać w archiwum tak samo jak sukces. Dopiero taki eksperyment może uzasadnić promocję SHD do wybranego zakresu CORE.

Owner	Właściciel badania SHD
Zależności	D02, D03
Predykat odbioru	Ślepe rodzinne splits, błędy NLU w end-to-end, uncertainty i koszt poprawnego wyniku; false analogy negatives.
Otwarty zakres	U11
Granica skutku	SEPARATE_MODEL_EVALUATION_SCOPE
Zachowanie historii	Przerwać badanie według limitów; zachować wyniki zerowe/negatywne.

Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

D12 · Ustabilizowana epoka i ponowne otwarcie eksperymentów

Per-component profile STABLE FOR EPOCH; RAG tylko po akceptacji; nowy eksperyment nie zmienia starych dowodów.

Zamknięcie epoki obejmuje stabilizowane granice i ich konsumentów. Nie powinno tworzyć globalnego zielonego statusu ponad materiał dowodowy. Każda domena zachowuje osobne lifecycle, currentness, verification, authority, execution i deployment.

Następny eksperyment korzysta z tej podstawy jako rodzica. Gdy zmienia się kontrakt, dokumentacja i projekcje wymagają źródłowej rewalidacji; stare wyniki nie znikają. R2 proponuje wydanie kolejnego dokumentu rozwijającego jedną domenę w pełnym L2–L6, zamiast kolejnego szerokiego atlasu bez zwiększenia głębokości odczytu.

Owner	Oryginalni właściciele CORE / formalization
Zależności	D09, D10, D11
Predykat odbioru	Regresja kontraktów, migracja/recovery, spójne źródła i odczyt efektu; brak globalnej kwalifikacji ponad zakres.
Otwarty zakres	U01, U02, U12, U13
Granica skutku	SEPARATE_RELEASE_KNOWLEDGE_PROMOTION
Zachowanie historii	Poprzednie wydanie nadal odtwarzalne; następna epoka ma nowy wektor i lineage.

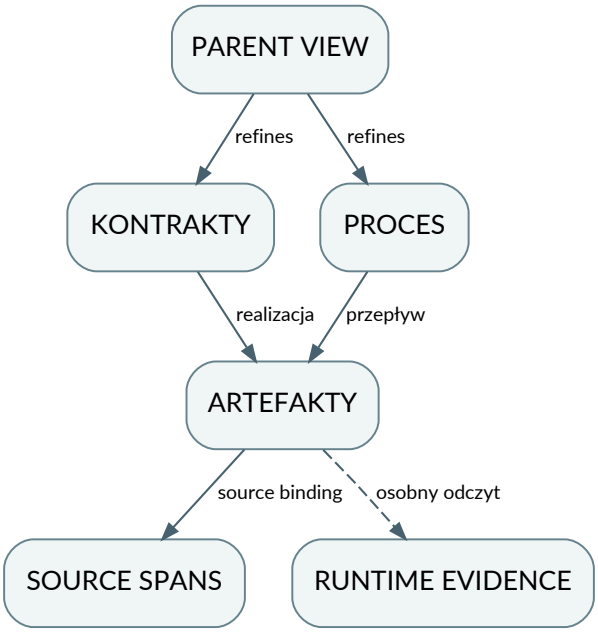
Źródła: R1, A-R6, A-SHD | TO-BE / DESIGN_NOT_EXECUTED

Widok nadrzędny i ścieżka powiększenia

Dokument powinien umożliwiać przejście od sensu pętli do dokładnego źródła bez zmiany języka opisu.

Każdy widok ma identyfikator, rodzica, zakres, poziom, źródła i status. Powiększenie nie oznacza pokazania wszystkich nazw plików naraz. Odsłania odpowiedzialności, potem kontrakty i porty, dalej sekwencję oraz artefakty, a na końcu kod i obserwację. Pozwala to uniknąć grafu, w którym wszystkie rodzaje zależności mają identyczną strzałkę.

W R2 source tree, AST, semantyczny model oraz runtime są widokami powiązanymi, lecz nie równoważnymi. Kliknięcie w przyszłym explorerze może otworzyć source span, test lub snapshot, ale nie może automatycznie promować źródłowej deklaracji do obserwacji. Atlas przygotowuje kontrakt takich przejść; nie implementuje interaktywnego Architecture Studio.



AT2-G29 · Kontrakt rozwinięcia. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Rodzic	Pytanie o całość i zachowane porty.
Dziecko	Nowy szczegół z jawnym zakresem.
Dowód	Exact source lub czas obserwacji.
Agregacja	Status rodzica zależy od właściwej tezy, nie średniej kolorów.

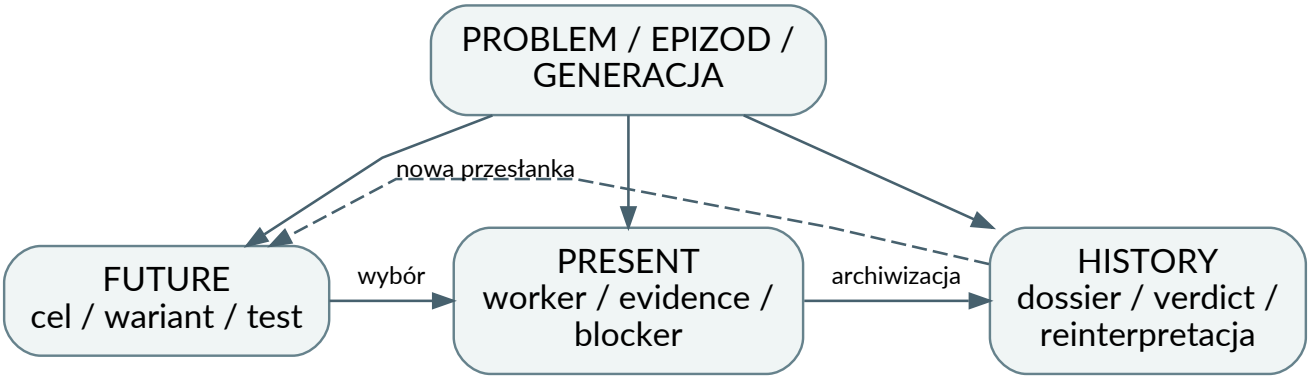
Źródła: R1, S04, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Panel trzech horyzontów

Przyszłość, teraźniejszość i przeszłość są zsynchronizowanymi widokami nad wspólną tożsamością problemu.

W panelu przyszłości operator powinien widzieć cele, alternatywy, constraints i oczekiwane testy rozstrzygające. W teraźniejszości — aktywny epizod, role, wersję kandydata, dowody, blokady i stan skutku. W przeszłości — zamrożone dossier, wyniki i późniejsze reinterpretacje. Przejście między panelami zachowuje ID problemu i generacji.

Każdy stan widoczny w GUI musi określać, czy pochodzi z zapisu, świeżej obserwacji czy propozycji. Historyczny worker nie jest aktywnym zasobem, a przyszła gałąź nie jest wdrożeniem. Interfejs powinien wyjaśniać powód WAIT i granicę uprawnień. Prezentowany układ jest projektem informacyjnym R2, nie odczytem rzeczywistego wyglądu okna na MOON.



AT2-G30 · Panel trzech horyzontów. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

FUTURE	Cel / wariant / przewidywanie / warunek próby.
PRESENT	Misja / kandydat / worker / evidence / blocker.
HISTORY	Oryginał / verdict / supersession / późniejsza interpretacja.
Wspólny klucz	Problem, epizod, generacja i cutoff.

Źródła: R1, S05, S07 | PROJEKT R2 / AS-IS OGRANICZONY

Kontrakt następnego dokumentu

Kolejny tom ma zwiększyć rozdzielczość wybranego fragmentu, nie powielać wszystkie ogólne diagramy.

Dla wybranej domeny należy odtworzyć pełny zestaw dokumentów, implementacji, konfiguracji i testów jej producentów oraz konsumentów. Każda karta R2 może stać się rodzicem szczegółowej specyfikacji: interfejs, automat stanu, przypadki błędów, model danych, source spans i wymagany runtime proof. Nie należy rozpoczynać od nadawania nowych nazw modułom.

Warunkiem odbioru następnego tomu jest dwukierunkowa identyfikowalność: od wymagania do kontraktu, kodu i testu oraz od zmienionego symbolu do wpływu na misję i federację. Jeżeli dokument nie obejmuje części tej ścieżki, ma ją oznaczyć i zachować zależną decyzję otwartą. Ilość stron nie zastępuje zwiększenia pokrycia źródeł.

Wejście	Wybrany parent view i exact baseline.
Zakres	Pełna closure istotnych producentów i konsumentów.
Wyjście	Szczegółowy kontrakt rozwinięcia i testy.
Następny priorytet	U01 wraz z wiring U03 i package closure U04.

Źródła: R1, T02 | PROJEKT R2 / AS-IS OGRANICZONY

Aktualizacja atlasu jako przyrost wiedzy

Zmiana jednego źródła unieważnia zależne twierdzenia, nie wymaga usunięcia całego dokumentu.

Wspólny model kart wskazuje ich źródła i figury. Po zmianie źródła można ustalić, które widoki wymagają rewalidacji. Jeśli zmienia się tylko commit lineage przy tym samym tree, aktualizacji może wymagać genealogia i exact-head kwalifikacja, ale nie każda treść AST. Jeśli zmienia się parser, należy osobno rozliczyć deltę instrumentu.

Nowy PDF ma relację następstwa do poprzedniego wydania i jawny rejestr korekt. Zmiana interpretacji nie usuwa starej obserwacji. Przed użyciem jako kanonicznego wejścia RAG potrzebna jest decyzja istniejącego ownera wiedzy. Sam plik PDF, nawet z poprawnym hashem i stu stronami, pozostaje dokumentem badawczo-projektowym.



AT2-G31 · Aktualizacja atlasu. Widok semantyczny R2; strzałki nie są domyślnym dowodem runtime.

Źródło zmienione	Znajdź zależne karty i grafy.
Twierdzenie	Rewaliduj zakres oraz klasę dowodu.
Wydanie	Zachowaj predecessor i changelog.
RAG	Osobna promocja, bez podmiany bootstrapu.

Źródła: S06, S17, R1 | PROJEKT R2 / AS-IS OGRANICZONY

Nowe wykonanie: 45 testów Code Perception

Ten wynik dotyczy konkretnych bajtów modułu, kontraktów i testu — nie całego ekosystemu.

Uruchomiono cyber_lion.tests.test_code_perception w lokalnym, ograniczonym pakiecie. Pięć plików porównano przez Git-framed SHA-1 z blobami odczytanymi dla d367e47c…. Suite zakończył się wynikiem 45 testów OK i returncode=0 w Python 3.13.5. Użyto pustych lokalnych __init__.py; nie odtworzono całej inicjalizacji produktu.

Zestaw sprawdza między innymi shadowing, global writes, control-flow barriers, mutable attributes, nierozwiązane call targets, integralność bloba i semantykę grafu. Wynik potwierdza te assertions w tej kopii. Nie dowodzi pełnego import closure Mission Control, aktualności floty, benchmarku modeli ani bezpieczeństwa każdej możliwej klasy źródła. Log i rejestr hashy są częścią pakietu R2.

Suite	cyber_lion.tests.test_code_perception
Wynik	45 / 45; returncode 0.
Źródło	5 plików byte-equal względem GitHub pinned blobs.
Środowisko	Python 3.13.5; lokalny pakiet testowy, bez credentials.
Wykluczone	Pełne unittest repozytorium i produkcyjny runtime.

Źródła: T01, S11, S12, S13 | NOWY WYNIK OFFLINE / ZAKRESOWANY

Git i AST: zmiana treści a zmiana tożsamości

Jawne próby P01–P12; wynik nie rozszerza zakresu dowodu ponad zastosowaną metodę.

Próby zostały zapisane w osobnym instrumencie badawczym i wykonane na syntetycznych wejściach. Dziesięć pierwszych używa oryginalnego build_code_graph, dwie ostatnie sprawdzają tylko postulowany model R2. Każdy rekord zawiera warunki porównania i wynik; PASS oznacza spełnienie oczekiwania konkretnej próby, nie ogólną bezbłędność systemu.

Najważniejsze rozdzielenia są operacyjne: tożsamość slotu logicznego nie jest hashem implementacji; integralność bloba nie jest dowodem członkostwa w zewnętrznym tree; statyczna kandydatura nie jest runtime proof. Model temporalny wymaga odrębnego odbioru na rzeczywistych store'ach, zanim będzie można mówić o zachowaniu tych własności przez całe LION.

P01 · PASS	Inny commit, ten sam tree: digest źródłowy inny, tree-semantic równy.
P02 · PASS	Komentarz: blob i zakres zmienione, ID symbolu i AST niezmienione.
P03 · PASS	Zmiana return: ID symbolu zachowane, AST zmienione.
P04 · PASS	Rename: blob równy, lokalny symbol ID inny.
P05 · PASS	Namespace: local ID równy w dwóch repo; qualified key różny.
P06 · PASS	Pure builder: dwa poprawne bloby pod tą samą deklaracją tree.

Źródła: T02, T01, S11, S12 | NOWE PRÓBY / NATIVE FIXTURES + MODEL BADAWCZY

Statyczny dowód i projekt czasu

Jawne próby P01–P12; wynik nie rozszerza zakresu dowodu ponad zastosowaną metodę.

P07–P10 rozdzielają obecność symbolu, kandydaturę statycznego wywołania i rzeczywistą tożsamość wykonawczą. Obecność definicji pod warunkiem nie czyni jej dostępną na każdej ścieżce. Również rozpoznany kandydat statyczny pozostaje odrębny od runtime proof. Negatywna kontrola podmiany bloba oraz kontrola kolejności wejść dotyczą integralności i determinizmu samego instrumentu.

P11 i P12 są minimalnymi przykładami modelu badawczego: pierwszy zachowuje oryginalne bajty podczas późniejszej reinterpretacji, drugi sprawdza kierunek rozwinięcia i unikalność identyfikatorów. Nie kwalifikują natywnego magazynu pamięci ani całej hierarchii LION. Dlatego 45 testów modułu, dziesięć nowych prób buildera i dwa przykłady projektu mają odrębne zakresy, choć wszystkie zapisano w jednym pakiecie dowodowym.

P07 · PASS	Definicja warunkowa: symbol obecny, wywołanie nierozwiązane.
P08 · PASS	Static target obecny; runtime target nadal UNRESOLVED.
P09 · PASS	Podmiana bytes pod starym blob ID odrzucona.
P10 · PASS	Kolejność wejść nie zmienia projekcji.
P11 · PASS	Model badawczy czasu: archive zachowane, future evidence wykluczone.
P12 · PASS	Model badawczy hierarchii: poprawny kierunek refines.

Źródła: T02, T01, S11, S12 | NOWE PRÓBY / NATIVE FIXTURES + MODEL BADAWCZY

Co pozostaje poza zamkniętym rozpoznaniem

Atlas ma pełną strukturę projektową, ale nie ma pełnego empirycznego pokrycia dosłownie całego ekosystemu.

R1 raportował 1917 blobów, mechaniczny skan 1813 plików oraz 982 parsowalne pliki Python. Ewidencja pełnego odczytu dokumentów tekstowych obejmowała 9 z 359. R2 zachowuje te liczby jako wynik R1. Odzyskanie 268 historycznych plików i nowe testy Code Perception nie zamykają pozostałych 350 dokumentów ani całej historii gałęzi.

W tym wydaniu nie odtworzono wszystkich repozytoriów, dynamicznych wywołań, konfiguracji hostów, retencji pamięci, aktywnej sesji SaaS i materialnego odbioru Factory. Nowe odczyty Git i lokalnych worktree są zakresowane. Przyszły plan nie może zależeć od nieprzeczytanego źródła tak, jakby było potwierdzone. Globalny formalization closure pozostaje niewydany.

R1: pełny tekst	9 / 359, nie zastąpiono scanem.
R2: pakiet historyczny	268 plików pozyskanych; nie kompletny current tree.
R2: dokładne testy	5 plików byte-verified, 45 testów oraz 12 prób.
Federacja / runtime	U02–U14 częściowo nadal otwarte.
Status	AS-IS PARTIAL; TO-BE jest warunkowym projektem.

Źródła: R1, A01, T01, T02, H01, H02 | JAWNA GRANICA KOMPLETNOŚCI

Checkpoint po R2

Następna praca ma zwiększyć głębokość źródłową, nie ponawiać odczytów i testów bez zmiany wejścia.

Pierwszy krok to kontynuacja U01 od przypiętego drzewa i istniejącej ewidencji. Równolegle należy prześledzić U03 — rzeczywiste dynamiczne wiązania reprezentacji — oraz U04 — pełny package/import closure Mission Control. Wyniki 45 testów i 12 prób można zachować, dopóki odpowiadające im źródła i metoda się nie zmienią.

Nowe badanie powinno sprawdzić drift ai_platform, writeups i dokładnych peerów w wybranym zakresie. Dla writeups bazą nowego odczytu jest 88d8f27b..., nie stary wektor R1. Warstwy L0–L6, katalog kart i źródła diagramów pozostają materiałem projektowym do rozwijania. Normalizacja nazw, konsolidacja gałęzi, deployment i uruchomienie Factory wymagają osobnych zadań i dopuszczeń.

Zachowana delta	Nowy atlas, modele, diagrams, źródła testów i odczyty.
Nie wykonano	Zmian Git, konfiguracji, misji, modeli i uprawnień.
Kontynuacja	U01 + scoped U03/U04, z delta-reacquisition.
Zasada	Nie mylić zakończenia PDF z zakończeniem badania świata.

Źródła: R1, G01, G02, T01, T02 | PUNKT KONTYNUACJI / BEZ AKTYWACJI

Źródła R1 — A-RAG

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

R1 · Raport badawczy R1, 59 stron [ZAŁĄCZNIK / HISTORYCZNY WYNIK]. Zakres: Pełny tekst, modele i wyodrębnione załączniki; wyniki R1 nie są nowym testem R2. AS-IS PARTIAL; 9/359 pełnego tekstowego odczytu w ledgerze R1.

LION_ECOSYSTEM_ARCHITECTURE_RESEARCH_R1_20261014.pdf | SHA-256: afa1673388f27eb53eb93f143368bad9a80a55429bf40539f1cd48bb3041580b

A01 · Odzyskany historyczny pakiet źródeł [PAKIET ŹRÓDŁOWY / 04.10.2026]. Zakres: 268 plików; CRC i hashe członków; wybrane kontrakty i manifesty. Nie pełny current tree; 5 plików testowych zweryfikowano oddzielnie.

source_transfer_small.zip | SHA-256: d46a63fd9b197f1672c6d52919086c4dc110e261324a48550445d32f9b5ea6f8

A-RAG · Wybrane wydanie RAG 1.5 [WERSJONOWANA WIEDZA]. Zakres: Odczyt orientacji 00,01,02,03,29; manifest i pozostałe źródła jako materiał wersjonowany. Nie aktualność runtime; bootstrap R9 pozostaje osobną relacją integracyjną.

lion-rag30-v1.5-r1 / 00–29

Źródła A-R6 — S01

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

A-R6 · Program autonomicznej implementacji R6 [PROJEKT WEJŚCIOWY]. Zakres: Plan, DAG i historyczna kwalifikacja struktur z R1. Nie aktywowano programu ani zawartego promptu.

LION_AUTONOMOUS_IMPLEMENTATION_R6_2026-10-13(9).zip

A-SHD · SHD oraz Semantic Pre-Model Filter R1 [EKSPERYMENTY WEJŚCIOWE]. Zakres: Pakiety, wyniki R1 i plan natywnej integracji. W R2 nie wykonano ponownie 39/42/24 ani benchmarku modeli.

LION_SHD_FEDERATION_RESEARCH_DESIGN_R1(6).zip; LION_SEMANTIC_PRE_MODEL_FILTER_R1(5).zip

S01 · Instrukcje i publiczne entrypointy [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Źródła z odczytów GitHub poprzedniej próby na tym samym pinned HEAD; zachowane w kontekście. Nie przeprowadzono ponownej pełnej kontroli runtime.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/AGENTS.md;
https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/AGENTS.md;
https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/AGENTS.md;
https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/tools/AGENTS.md;
https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/README.md;
https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/README.md

Źródła S02 — S04

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S02 · Piętnaście warstw i dziesięć FLOW [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Cały pokazany plik: ARCHITECTURE_LAYERS, FLOW_SPECS, canonical_flows. Deklaracja modelu nie jest wykonanym E2E.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/architecture_projection/flows.py

S03 · Protokół generowania i ewolucji [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełny tekst źródłowy pokazany przez GitHub. Norma i projekt procesu; literalne przykłady nie są poleceniami uruchomienia.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/enterprise/GENERATION_EVOLUTION_PROTOCOL.md

S04 · FullArchitectureModel [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełny pokazany kod; source-bound element specs, 15 warstw, 10 flows. Część przypisań i statusów jest deklaratywna.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/architecture_projection/full_architecture.py

Źródła S05 — S07

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S05 · Project Reality → Evolution Path [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełny pokazany dokument źródłowy. SOURCE_BOUND_SUCCESSOR; nie cały działający pipeline.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_5/PROJECT_REALITY_EVOLUTION_PATH_R1.md

S06 · Homeostaza i federacyjna architektura [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełne teksty pokazane przez GitHub w przerwanej próbie. AS-IS i TARGET rozdzielone; repozytoryjna deklaracja nie zastępuje obserwacji.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_5/README.md; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_5/FEDERATED_ARCHITECTURE_AS_IS.md; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_5/FEDERATED_ARCHITECTURE_TARGET.md

S07 · Cognition i epizod evidence-bound learning [PAKIET HISTORYCZNY / DEKLAROWANY REF 0e24fd48]. Zakres: Wybrane dokumenty i kontrakty w historycznym pakiecie A01. Źródło historyczne; pełnego obecnego consumer path nie zamknięto.

https://github.com/DonkeyJJLove/ai_platform/blob/0e24fd481697bf85fd5c4a4fdb93c39a757ff463/LION/architecture/v1_5/COGNITIVE_EVOLUTION_INTEGRATION.md; https://github.com/DonkeyJJLove/ai_platform/blob/0e24fd481697bf85fd5c4a4fdb93c39a757ff463/cyber_lion/contracts/evidence_bound_learning_episode.py

Źródła S08 — S10

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S08 · Scaffold, semantyka i EnterpriseGraph [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Kontrakty w A01 i R1; enterprise_graph byte-verified względem current pinned blob. Lokalna walidacja nie jest weryfikacją każdej zewnętrznej referencji.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/contracts/semantic_scaffold.py; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/contracts/semantic_relevance.py; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/contracts/enterprise_graph.py

S09 · ArchitectureCompiler i EvolutionFitness [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Opis aktualnej obecności źródłowej oraz ograniczeń; ustalenia R1. Nie nowy test compile_selected ani dowód żywej kompozycji.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_5/README.md

S10 · Normy nazewnictwa, statusów i layoutu [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełne źródłowe obiekty przytoczone w wątku i R1. Nie pomiar spełnienia przez wszystkie pliki.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/standards/LION_NAMING_STANDARD.json; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/standards/LION_STATUS_MODEL.json; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/standards/LION_VERSION_MODEL.json; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/standards/LION_REPOSITORY_LAYOUT_STANDARD.json

Źródła S11 — S13

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S11 · Natywny indekser Code Perception [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Całe pobrane bytes, przegląd kodu i użycie w T01/T02; blob 272df05e773cf76038ca9a99a4fb1fb5c9708198. Nie wykonano build_from_git całej federacji.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/enterprise/code_perception.py | SHA-256: 4f385f32bb19e008bd7cd805e315108e75640ee666cd7e44f85c879d933d8a5f

S12 · Kontrakty CodeGraph i runtime target proof [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Całe bytes z A01; blob eb987fac554d5c457e4ff20b3349b9488cd6901d zweryfikowany względem GitHub. Runtime proof wymaga odrębnego obserwatora, nie powstaje z AST.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/contracts/code_perception.py | SHA-256: d91fe2f1a43e14e53ec9052b337f0de4dfd50dc657e3d7521418f054fe95b22f

S13 · Typowana granica odczytu Git [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Całe pobrane bytes; blob 28b29eff73782d4c8f6826fa194cc78f61c6866e. Test T01 nie kwalifikuje całego subprocess/runtime boundary.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/enterprise/code_perception_git_boundary.py | SHA-256: ef74b8ebada0377b2768b16554272ebe3982b48e2d67dc562df0a4374e4ddf89

Źródła S14 — S16

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S14 · Subject digest i truth carriers [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pokazany zakres: SubjectEntry, CARRIER_PATHS, canonical_subject_payload, subject_digest i currentness. Nie pełny odczyt wszystkich końcowych walidatorów pliku.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/architecture_projection/truth_plane.py

S15 · Full Symbol Census i testy [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełny kod odczytany partiami przez GitHub w poprzedniej próbie. Nie uruchomiono tego suite w R2; statyczna referencja testu nie jest coverage.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/architecture_projection/full_symbol_census.py; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber_lion/tests/test_architecture_projection_full_symbol_census.py

S16 · Manifest ai_platform [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Pełny odczyt GitHub w aktualnym wznowieniu; blob 46e9d26cba916c5dfd10bbbb31695b25e5a056d9. Deklaracje eksportów i capabilities nie są pomiarem aktywnych usług.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/cyber-lion.repository.json

Źródła: S14, S15, S16 | INDEKS ŹRÓDŁOWY / CZYTAJ Z ZAKRESEM

Źródła S17 — G01

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

S17 · Formalization Kernel [PAKIET HISTORYCZNY / DEKLAROWANY REF 0e24fd48]. Zakres: Pełny dokument dostępny i odczytany w historycznym pakiecie A01; rejestr formalizacji ujęty w R1. Nie wykonano formalization closure na całym current tree.

https://github.com/DonkeyJJLove/ai_platform/blob/0e24fd481697bf85fd5c4a4fdb93c39a757ff463/LION/architecture/v1_5/FORMALIZATION_KERNEL.md

S18 · Process Contract Plane i Authorization Lifecycle [ŹRÓDŁO REPOZYTORYJNE / ZAKRES W OPISIE]. Zakres: Źródła R1 i orientacji projektu; warunki LPCL/1.2 oraz rozdział intent/binding/effect. Nie aktualny grant; starsza ścieżka katalogowa nie znaczy martwego kontraktu.

https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_4/LION_PROCESS_CONTRACT_PLANE.md; https://github.com/DonkeyJJLove/ai_platform/blob/d367e47cf4f72e450cc212908b2cdfed14ceb805/LION/architecture/v1_4/AUTHORIZATION_LIFECYCLE_CONTRACT.json

G01 · Aktualny odczyt ai_platform master [NOWY ODCZYT GITHUB]. Zakres: HEAD d367e47cf4f72e450cc212908b2cdfed14ceb805 TREE przypiętego źródła: b92133d258b6fbf2c7f1d16d992e08d2a3609a0c

https://api.github.com/repos/DonkeyJJLove/ai_platform/git/ref/heads/master

Źródła G02 — H01

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

G02 · Writeups: merge i obecna genealogia [NOWY ODCZYT GITHUB]. Zakres: HEAD 88d8f27b...; TREE ff15167945e1a7230ee8f8ddf41f00a748c6bcda; obaj parents. Nie odczytano wszystkich plików HCL ani całej historii.

https://api.github.com/repos/DonkeyJJLove/writeups/git/commits/88d8f27b5fac48812ac266d990c7639fc5d1e93d

G03 · Writeups: pierwszy parent tego samego tree [NOWY ODCZYT GITHUB]. Zakres: TREE ff15167945e1a7230ee8f8ddf41f00a748c6bcda równy G02. Wykazano równość tree, nie test funkcjonalny produktu.

https://api.github.com/repos/DonkeyJJLove/writeups/git/commits/c3fd319b205088df9fca3f9b1400e27e31834f17

H01 · Lokalny checkout MOON [NOWA OBSERWACJA / SENTINELX]. Zakres: sentinel_project_snapshot: detached 95634907d2e7,1268 tracked,334 untracked. Nie canonical master; dirty stan nie został zmieniony.

/srv/lion-e4-candidate-r1

Źródła H02 — T02

Lokalizatory, tożsamości i ograniczenia odczytu. Identyfikatory S/G/H/T nie są natywnymi statusami LION.

H02 · Lokalny wariant cooperative-publication-r4 [NOWA OBSERWACJA / SENTINELX]. Zakres: sentinel_project_snapshot: detached95f44213ca85,1635 tracked,9 staged,+1404. Do testu wybrano tylko pliki osobno zgodne z pinned blobami.

/srv/lion-e4-candidate-r1/cooperative-publication-r4

T01 · 45 testów Code Perception [NOWY WYNIK OFFLINE]. Zakres: 45 tests OK, returncode0;5 source files + empty local initializers; Python3.13.5. Nie cały unittest ai_platform.

tests/T-R2-CODE-PERCEPTION.log | SHA-256: f1770be0ebc0483b5566320b7b9b06def08a10615a731eda302992a4805d47c1

T02 · 12 prób Git–AST–czas [NOWE PRÓBY BADAWCZE]. Zakres: P01–P10: native builder fixtures; P11–P12: proposed research model;12PASS. Nie ma runtime/model invocation; PASS dotyczy oczekiwań konkretnych prób.

tests/probe_git_ast_time.py; evidence/new_git_ast_time_probes.json | SHA-256: 6b175259d78b13e3665dfd594e2c7d8d6968d474d397ebadfe63f89e17357356

Obiekty C01 — C06

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C01 · ai_platform	DonkeyJJLove/ai_platform | DOM05 | INTEGRATED_SOURCE. Globalna własność kontraktów, procesów, formalizacji i federacyjnej wiedzy.
C02 · swarm	DonkeyJJLove/swarm | DOM07 | SOURCE_CONTRACT. Lokalne kontrakty execution mesh i adapter runtime.
C03 · chunk-chunk	DonkeyJJLove/chunk-chunk | DOM03 | VERSIONED_ROLE. Semantyka procesu i trajektorii.
C04 · glitchlab	DonkeyJJLove/glitchlab | DOM03 | VERSIONED_ROLE. Analiza delt i niezmienników.
C05 · HA2D	DonkeyJJLove/HA2D | DOM02 | VERSIONED_ROLE. Snapshot i pamięć kontekstowa.
C06 · hipotezy_nadawcze_LLM	DonkeyJJLove/hipotezy_nadawcze_LLM | DOM03 | VERSIONED_ROLE. Hipotezy, kontrhipotezy i plan eksperymentu.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C07 — C12

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C07 · mosaic_lab_pro.py	DonkeyJJLove/mosaic_lab_pro.py | DOM03 | VERSIONED_ROLE. Projekcje grafów struktur i topologii.
C08 · sbom	DonkeyJJLove/sbom | DOM04 | VERSIONED_ROLE. Tożsamość artefaktów i provenance.
C09 · SymulacjaKaskadySieciowej	DonkeyJJLove/SymulacjaKaskadySieciowej | DOM03 | VERSIONED_ROLE. Modele symulacyjne, scenariusze i wrażliwość.
C10 · writeups	DonkeyJJLove/writeups | DOM02 | VERSIONED_ROLE. Korpus badań i retencja publikacji.
C11 · Semantic owners / FormalizationRegistry	ai_platform | DOM05 | SOURCE_CONTRACT. Własność pojęć oraz zakres powierzchni formalizacji.
C12 · SemanticScaffoldIR	ai_platform/contracts | DOM02 | SOURCE_CONTRACT. Reprezentacja encji, relacji, hipotez, kontrhipotez i niewiedzy.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C13 — C18

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C13 · SemanticAtom / Delta / Relevance	ai_platform/contracts | DOM02 | SOURCE_CONTRACT. Twierdzenia z korzeniami dowodowymi i ograniczony widok konsumenta.
C14 · ProjectReality / ClaimProjection	ai_platform/architecture_projection | DOM01 | SOURCE_CONTRACT. Wiązanie istniejących obserwacji, bez drugiej bazy prawdy.
C15 · EvolutionFitness / ArchitectureCompiler	ai_platform/architecture_projection | DOM03 | PARTIAL_SOURCE. Wybór opisanego kandydata oraz formalizacja przed admission.
C16 · CapabilityNeed / Resolver	ai_platform/contracts + enterprise | DOM03 | SOURCE_CONTRACT. Zapotrzebowanie powiązane z Gap i reuse lub nowa specyfikacja.
C17 · Bean composition	ai_platform/enterprise | DOM03 | SOURCE_CONTRACT. Dobór minimalnego dopuszczalnego zestawu BeanDescriptor.
C18 · LPCL / PhaseExecutionContract	ai_platform/process_language + contracts | DOM05 | INTEGRATED_SOURCE. Opis faz, capabilities, ceiling, warunków aktualności i completion.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C19 — C24

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C19 · Mission Control V3	ai_platform/tools + mission_control | DOM05 | RUNNING_BOUNDED_OBSERVATION. Odtworzenie stanu i sterowanie istniejącymi misjami.
C20 · Canonical scheduler / ledger	ai_platform/mission_control | DOM05 | SOURCE_AND_STORED_STATE. Wybór misji i trwałe assignments, generacje, leases, receipts.
C21 · Native LOCAL phase zero	ai_platform/mission_control | DOM05 | INSTALLED_BYTES_MATCH. Poznanie przed bindingiem 32 MD dla dokładnej aktywowanej misji.
C22 · Conversation / SaaS broker / ingress	ai_platform/app_coordination + browser_broker | DOM05 | PARTIAL_RUNTIME. Trwała tożsamość rozmowy, routing i rekonsyliacja odpowiedzi.
C23 · Electron operator shell	ai_platform/browser_broker | DOM05 | PROCESS_OBSERVED. Interfejs operatora i osadzona sesja SaaS.
C24 · Original authority / PDP	ai_platform/enterprise | DOM06 | SOURCE_OWNER_RUNTIME_UNKNOWN. Źródło uprawnień, polityka i aktualna dopuszczalność.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C25 — C30

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C25 · RuntimeAdmission / currentness	ai_platform/enterprise | DOM06 | SOURCE_OWNER_RUNTIME_UNKNOWN. Wiązanie decyzji, tożsamości, replay i czasu skutku.
C26 · Cooperative material worker	ai_platform/mission_control + enterprise | DOM07 | NOT_CURRENTLY_PROVEN. Ograniczona materializacja na podstawie aktualnego admission.
C27 · Artifact / verifier / reconciliation	ai_platform/mission_control + enterprise | DOM04 | SOURCE_AND_HISTORICAL_EVIDENCE. Rzeczywiste bajty, odczyt niezależnej roli, rozliczenie skutku.
C28 · RAG / architecture knowledge	ai_platform/LION | DOM02 | VERSIONED_KNOWLEDGE. Pamięć źródłowa i projekcje dla kolejnej epoki, nie żywy stan runtime.
C29 · SHD / pre-model prototype	Załączniki operatora | DOM03 | EXPERIMENTAL_OFFLINE. Ograniczona gramatyka i propozycje operatorów semantycznych.
C30 · Windows MOON / wspólny substrat WSL	Operator infrastruktury | DOM07 | OBSERVED_SHARED_KERNEL. Środowisko hostujące powiązane instancje wykonawcze.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C31 — C36

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C31 · LION-AUTH-LAB	Operator infrastruktury | DOM07 | OBSERVED. Mission Control i brama operatora.
C32 · MOON WSL	Operator infrastruktury | DOM07 | OBSERVED. Repozytoria, worktree i Docker.
C33 · LAB-DEBIAN	Operator infrastruktury | DOM07 | OBSERVED. Control plane, tunel i laboratoryjny producer TEST_ONLY.
C34 · LAB-UBUNTU	Operator infrastruktury | DOM07 | OBSERVED_LIMITED. Agent obserwacyjny; brak potwierdzonej aktywnej floty w odczycie usług.
C35 · Dify: 15 uruchomionych kontenerów	NIEUSTALONY_WŁAŚCICIEL_INTEGRACJI | DOM07 | ADJACENT_UNRESOLVED. Odkryty sąsiedni stos aplikacyjny.
C36 · R24: 32 kontenery	Historyczny cohort LION | DOM07 | STOPPED_OBSERVED. Poprzednia flota, nie bieżąca zdolność wykonania.

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Obiekty C37 — C42

Klasyfikacja domen w R2 jest widokiem analitycznym. Nie zmienia źródłowych ownerów ani historycznych statusów.

C01–C37 zachowano z modelu R1, z jego ograniczeniami odczytu; C38–C42 dodają do atlasu istniejące mechanizmy źródłowe. Tylko wybrany zakres Code Perception otrzymał w tym wznowieniu nowy dowód offline. Kolumna domeny określa główną rolę w widoku, nie jedyny możliwy udział w pętli.

Każdy obiekt może zostać rozwinięty przez kontrakty, proces, nośniki, kod i obserwację. Rozwinięcie do L6 wymaga odrębnego dowodu runtime. Udział w future/present/history zależy od wersji i epizodu, nie od stałego przypisania repozytorium do jednego horyzontu.

C37 · Monitoring LION	Automations / operator | DOM08 | CONFIGURATION_OBSERVED. Dwa odczytowe watchery, nie scheduler implementacji.
C38 · Code Perception	ai_platform/enterprise | DOM01 | SOURCE_AND_OFFLINE_TEST. SourceIdentity + bloby → CodeGraph
C39 · Git read boundary	ai_platform/enterprise | DOM01 | SOURCE_BOUNDARY. Pięć typowanych operacji odczytu Git
C40 · FullSymbolCensus	ai_platform/architecture_projection | DOM01 | SOURCE_READ_NOT_EXECUTED. Spis symboli i statycznych referencji
C41 · Truth subject / carriers	ai_platform/architecture_projection | DOM01 | PARTIAL_SOURCE_READ. Source subject digest i bieżące nośniki
C42 · Federated architecture knowledge	ai_platform/LION + tools | DOM01 | SOURCE_DECLARATION. Census, graf i rekonsyliacja dokumentacji

Źródła: R1, S02, S04, T01 | KATALOG DOKUMENTACYJNY / CZĘŚCIOWO HISTORYCZNY

Rozliczona zmiana jako jednostka architektury

Podstawowy produkt LION ma dwie części: zmienioną zdolność i wiedzę uzasadniającą jej kwalifikację.

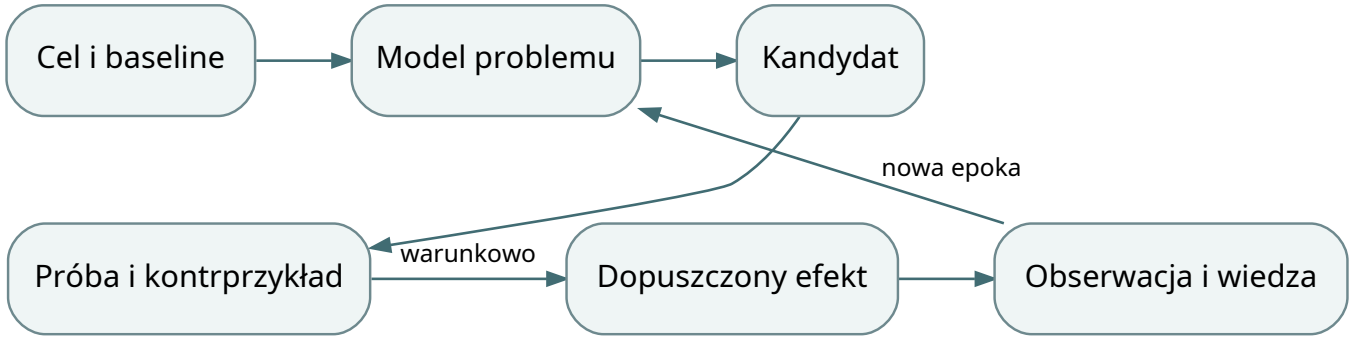
Gdy fabryka tworzy nowy moduł, nie wystarcza wskazać pliku wynikowego. Trzeba jeszcze odtworzyć potrzebę, wykorzystane źródła, rozważone ograniczenia, wersję budującego mechanizmu i wynik weryfikacji. Dopiero taki komplet pozwala ponownie użyć modułu w innym problemie. W tym sensie jednostką ewolucji jest relacja pomiędzy zmianą a jej rozliczeniem, nie sam commit.

W źródłowym protokole po ograniczonym wykonaniu następują receipt, obserwacja oraz kandydat pamięci. Każdy etap ma inny przedmiot: receipt opisuje raport przebiegu, obserwacja sprawdza świat, a pamięć opisuje to, czego wolno na tej podstawie użyć ponownie. Zlanie etapów prowadziłoby do samopotwierdzającej fabryki, która uznaje własne deklaracje za wynik.

Formalizowany tutaj epizod E wiąże intencję I, baseline B, kandydatów C, próby T, rezultat O i rozstrzygnięcie K. Brak materialnego skutku nie przekreśla epizodu: odrzucony kandydat może dostarczyć użytecznego kontrprzykładu. Nie wolno jednak wnioskować o zmianie możliwości wykonawczych wyłącznie z przyrostu wiedzy opisowej.

Przykład analityczny · Defekt w wyniku poprawnego zapisu

Worker zapisał plik pod właściwą ścieżką, lecz verifier wykazał niezgodny digest. Efekt istnieje, natomiast cel nie jest odebrany. Epizod powinien zachować wersję kandydata, treść kontrprzykładu i następny krok. Utworzenie poprawionej generacji nie może nadpisać starego receipt. Ten przypadek rozdziela materialną aktywność od zwiększenia zdolności systemu.



R3-G01 · Epizod rozliczonej zmiany. Model źródłowo-analityczny; nie nowa obserwacja runtime.

```
E = (I, B, C, T, O, K)
zamknięcie poznawcze != przyjęcie implementacji
przyjęcie implementacji != deployment
```

Źródła: R3-S009, R3-S044. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dwa rodzaje uczenia organizacji

Zmiana reprezentacji i zmiana wykonawczego mechanizmu mogą następować niezależnie.

Jeżeli test ujawni brak kontroli endpointów w grafie, można od razu poprawić wiedzę o zakresie walidatora. To nie wymaga zastosowania patcha. Wiedza zmienia się przez nową interpretację istniejących bajtów oraz przez dodanie kontrprzykładu. Naprawa implementacji jest drugim zadaniem, wymagającym własnego kandydata, testu i integracji.

Ten rozdział wyjaśnia organiczny charakter LION bez przypisywania mu nieudowodnionego uczenia wag modelu. Organizacja może zachowywać nowe przykłady, reguły, plany i ograniczenia, choć żaden parametr sieci neuronowej nie został zmieniony. Efekt pojawi się dopiero wtedy, gdy następny proces rzeczywiście odczyta tę wiedzę i zastosuje ją w odpowiednim zakresie.

W R3 natywny test R3-T16 zmienia kwalifikację lokalnej funkcji validate, lecz nie wprowadza nowego checkera produktu. W dokumentacji zapisujemy więc dwa różne następstwa: korektę twierdzenia i proponowaną deltę kodu. Ich identyfikatory nie mogą być wspólne, ponieważ różnią się wejściami, właścicielem i kryterium ukończenia.

Przykład analityczny · Korekta wiedzy przed patchem

Nowy kontrprzykład ogranicza wcześniejsze twierdzenie o walidatorze. Można zaktualizować dokument wiedzy z relacją CONTRADICT lub SUPERSEDE, pozostawiając kod bez zmian. Dopiero osobna implementacja i test nowego zachowania uzasadnią zamknięcie defektu produktu. W ten sposób system nie musi czekać z poznaniem własnego ograniczenia do zakończenia deploymentu.

Zmiana wiedzy	Nowe ograniczenie dotychczasowego twierdzenia.
Zmiana kodu	Osobny następca podlegający kwalifikacji.
Uczenie wag	Nie wykonano i nie wywiedziono z historii rozmów.

Ślad implementacyjny · RelevanceProjection

projection_id	str; pole wymagane
graph_ref	str; pole wymagane
graph_digest	str; pole wymagane
consumer_ref	str; pole wymagane
required_capability_refs	Tuple[str, ...]; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 127–159; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065, R3-S066, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Cel ogranicza poszukiwanie, nie zapisuje całej drogi

Strategia jest zmiennym modelem dochodzenia do postcondition, a nie niezmiennym skryptem.

GoalContract i MissionIntent mają różne zadania. Cel wskazuje oczekiwaną własność stanu; intencja zapisuje zakres konkretnej pracy i powiązanie komunikacyjne. W modelu ewolucyjnym jedno wymaganie może prowadzić do kilku kandydatów. Kandydat nie staje się celem tylko dlatego, że był pierwszą wymyśloną implementacją.

Załóżmy, że potrzebna jest audytowalna odpowiedź z modelu. Można poprawić istniejący adapter, zbudować nowy adapter zgodnego providera lub odroczyć zadanie do czasu odzyskania jego sesji. Ocena wariantu musi uwzględniać aktualną wykonalność, koszt i zachowanie transportu narzuconego dla requestu. Nie wolno potajemnie zastąpić wymaganego SaaS innym kanałem i ogłosić realizacji tego samego zadania.

Zmiana strategii jest prawidłowa, jeżeli zachowuje cel albo jawnie tworzy jego nową wersję. Historyczny wynik pozostaje oceniany względem wcześniejszego wymagania. W ten sposób przyszłość może się przeorganizować bez przepisywania przeszłości i bez zamiany nieudanej próby w sukces przez późniejszą zmianę kryterium.

Przykład analityczny · Zmiana trasy przy zachowanym celu

Provider wskazany w planie jest niedostępny. Alternatywa może być oceniona pod względem tych samych postconditions, ale tylko jeśli mandat dopuszcza zmianę transportu i profilu. Gdy wymagana jest natywna sesja SaaS, zastępcza odpowiedź z lokalnego modelu jest innym wynikiem. Adaptacyjność strategii nie znosi dokładności zakresu.

Ślad implementacyjny · MissionIntent

intent_id	str; pole wymagane
mission_ref	str; pole wymagane
conversation_ref	str | None; pole wymagane
source_envelope_ref	str; pole wymagane
source_envelope_digest	str; pole wymagane

Wybrane pola z mission_intent.py, klasa w liniach 36–84; całość deklaracji: 21 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S053, R3-S040, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Selekcja, planowanie i scheduling

Trzy mechanizmy rozwiązują trzy różne problemy: wartość, kolejność logiczną i dostępność wykonania.

Dynamic Evolution Fitness jest w atlasie źródłowo opisaną selekcją kandydata. Nie posiada przez to lease’u workera ani kursora uruchomionej misji. ArchitectureCompiler formalizuje wybrany CandidateDesign; LPCL opisuje skończony proces; scheduler wykorzystuje aktualny stan jego wykonania. Ich połączenie wymaga jawnych rekordów przekazania.

Przykładowy najwyżej oceniony przyrost może polegać na stworzeniu brakującego providera. Takiego przyrostu nie wolno blokować żądaniem, aby provider istniał już przed rozpoczęciem jego projektowania. Jednocześnie misja zależna od realnego działania providera musi na niego poczekać. Różne typy zależności muszą być widoczne w jednym modelu, ale nie mogą być identyczną krawędzią.

Praktyczny rozdział daje możliwość postępu niezależnych prac: formalizacja interfejsu może trwać podczas oczekiwania na zasób. Nie znosi to kontroli zmian. Nowy wariant z innym zakresem lub modelem nie dziedziczy automatycznie dopuszczenia wcześniejszego programu.

Przykład analityczny · Projektowanie brakującego wykonawcy

Faza A opisuje kontrakt nowego adaptera; faza B go buduje; faza C wykorzystuje. Wymaganie działającego adaptera przed A tworzyłoby cykl zależności. Wymaganie go przed C jest poprawne. Graf implementacji musi zatem oznaczać rodzaj krawędzi, zamiast przenosić jeden globalny READY na wszystkie etapy.

Fitness	Który kandydat ma teraz największą wartość?
Program	Jakie stany pośrednie prowadzą do odbioru?
Scheduler	Który dopuszczony krok może wykonać się teraz?

Źródła: R3-S003, R3-S015. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dlaczego nie powstaje jeden nadrzędny agent

Wspólny sens procesu nie wymaga jednego wykonawcy ani jednego magazynu wszystkich stanów.

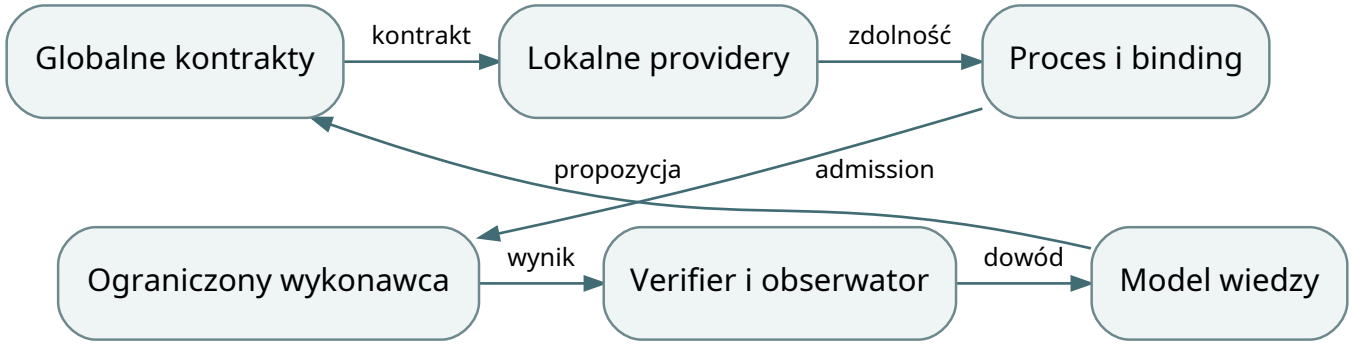
W manifestach występują role globalnego właściciela kontraktów oraz lokalnych providerów. To nie jest sprzeczne z rozproszonym wykonaniem: właściciel pojęcia określa jego znaczenie, a konkretna implementacja może działać w innym repozytorium i środowisku. Dzięki temu zmiana dostawcy zdolności nie musi zmieniać definicji całego procesu.

Jednocześnie federacja nie daje prawa do nieograniczonego rozproszenia odpowiedzialności. Dwa niezależne kursory sterujące tym samym assignmentem mogą stworzyć podwójny efekt. Dwa odczytowe widoki tego assignmentu nie tworzą takiego problemu. Odrębność właścicieli powinna więc wynikać z domeny zapisu i semantyki przejścia, nie z samej liczby usług.

Fundamentalną naturę LION trafniej opisuje organizacja kontrolowanej zmiany niż antropomorficzne centrum. Model poznawczy może być wymieniany; zobowiązania procesu pozostają przypisane do jego kontraktów. Architektura musi pozwalać wskazać, kto odpowiada za każde przejście nawet wtedy, gdy aktualny agent przestaje istnieć.

Przykład analityczny · Dwóch obserwatorów, jeden writer

Dwa panele mogą prezentować ten sam assignment bez konfliktu ownership. Dwa niezależne schedulery zmieniające jego stan wymagają natomiast protokołu spójności lub jednoznacznego podziału domen. Rozproszenie widoków jest łatwiejsze niż rozproszenie prawa do postępu. Projekt powinien wskazać, które krawędzie tylko czytają, a które mogą zapisać stan.



R3-G02 · Rozdzielenie odpowiedzialności. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S017, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Minimalna formalizacja domknięcia

Predykat odbioru odnosi się do konkretnej tezy, a nie do liczby zielonych elementów.

Przyjęcie kandydata można opisać jako koniunkcję wymaganych własności: tożsamości wejść, zgodności kontraktu, zakresu testu, aktualności zależności i obserwacji wyniku. Zbiór wymaganych własności zależy od rodzaju tezy. Parser reprezentacji i provider materialnego zapisu nie mają identycznego kryterium odbioru.

Z faktu, że dziewięć z dziesięciu modułów działa, nie wynika gotowość całej ścieżki. Dziesiąty może być jedynym konsumentem wyniku. Równie błędne byłoby unieważnienie lokalnego testu parsera dlatego, że pełna fabryka nie jest wdrożona. Dokumentacja potrzebuje predykatów per zakres oraz agregacji zależnej od grafu krytycznych portów.

Proponowany zapis nie jest nowym silnikiem wykonania. Jest językiem opisania, jakie dowody ma zgromadzić istniejący proces. Nowy dokument może przyjąć własną kartę jako domkniętą badawczo, pozostawiając odrębny status produktu i nierozstrzygnięte pytania o runtime.

Przykład analityczny · Usunięcie krytycznej krawędzi

W grafie pozostawiono wszystkie komponenty, ale usunięto consumer wyniku verifiera. Liczba zielonych węzłów się nie zmieniła, jednak pętla nie może się zamknąć. Test modelu powinien to wykryć przez zależność completion od odrębnej obserwacji. Prosty procent zaimplementowanych modułów nie wykryje tej utraty.

```
Accept(claim, epoch) = all(required_predicates(claim))
UNKNOWN(required predicate) -> brak kwalifikacji tej tezy
nie: readiness = średnia(statusy komponentów)
```

Ślad implementacyjny · CandidateBinding

candidate_digest	str; pole wymagane
head	str; pole wymagane
tree	str; pole wymagane
verification_digest	str; pole wymagane

Wybrane pola z formalization_closure.py, klasa w liniach 15–21; całość deklaracji: 4 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S015, R3-S049. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Horyzont czasu jest rolą, nie lokalizacją

Ten sam blob może jednocześnie wspierać historyczny opis i nowy eksperyment.

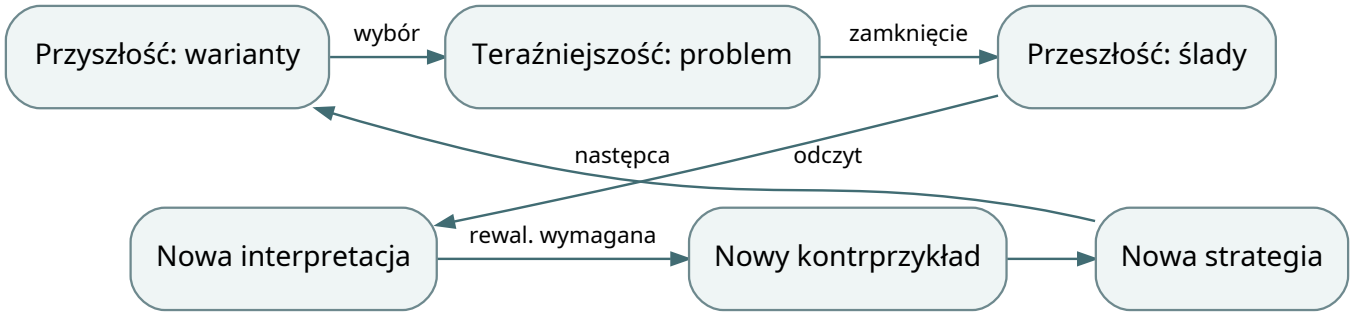
Przypisanie pliku do katalogu archive nie odbiera mu możliwości udziału w aktualnym badaniu. Historyczny test może stać się falsyfikatorem nowego kandydata, a stary szkic źródłem odrzuconej alternatywy. Rola temporalna wynika z relacji do epizodu oraz z cutoff wiedzy, nie wyłącznie z daty pliku.

Przyszłość, teraźniejszość i przeszłość należy dlatego modelować jako projekcje wspólnego zbioru identyfikowalnych artefaktów. Przyszły cel ma już materialny zapis, ale opisuje możliwość. Obecny wynik może być prowizoryczny. Historyczny rezultat może otrzymać nową interpretację, mimo że bajty jego logu są niezmienne.

Wizualizacja powinna pozwalać przełączyć perspektywę bez przenoszenia plików. Oddzielny widok pokaże, co było dostępne wykonawcy wtedy, a drugi — co wiemy teraz o tamtej decyzji. Przełączenie widoku nie zmienia statusu źródła ani nie stanowi dowodu użycia późniejszej wiedzy we wcześniejszym przebiegu.

Przykład analityczny · Archiwalny test w nowym badaniu

Ten sam plik testowy należy do przeszłej epoki i jednocześnie może sprawdzać nowego kandydata. Oryginalny wynik pozostaje historyczny; nowe wykonanie ma własny log i wersję środowiska. Nie należy przestawiać dawnej daty testu na dzisiejszą tylko dlatego, że plik ponownie wykorzystano.



R3-G03 · Trzy role informacji. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S065. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Czas zdarzenia a czas dostępu do dowodu

Porządek poznawczy wymaga czegoś więcej niż chronologii logów.

Dokument COGNITIVE_EVOLUTION_INTEGRATION rozdziela event_time, observation_time, ingestion_time, available_to_model_at, input_cutoff i decision_time. Każda data odpowiada na inne pytanie. Artefakt może powstać przed decyzją, ale zostać zaindeksowany i udostępniony modelowi już po niej.

Ocena historycznej racjonalności nie powinna korzystać z informacji niedostępnej w chwili wyboru. Późniejszy błąd może dowodzić nieskuteczności decyzji, lecz nie musi dowodzić pominięcia wiedzy, której wykonawca jeszcze nie miał. W federacji dochodzą opóźnienia transportu oraz rozbieżności zegarów, dlatego causation i identyfikatory rodziców uzupełniają znaczniki czasu.

Proponowany kontrakt replay przechowuje wszystkie istotne granice i jawne luki. Gdy available_to_model_at nie jest znane, nie zastępuje się go datą commitu. Można zbudować ograniczoną rekonstrukcję treści źródeł, ale nie pełne odtworzenie kontekstu decyzji.

Przykład analityczny · Spóźniona indeksacja

Dokument powstał przed decyzją, ale do modelowego kontekstu trafił po niej. Jego data utworzenia nie dowodzi, że agent go zignorował. Do takiego wniosku potrzebny jest ślad dostępności i rzeczywiście wybrana projekcja. Replay powinien uwzględniać cutoff konsumenta, nie tylko timestamp źródła.

event_time	Czas zdarzenia będącego przedmiotem opisu.
available_to_model_at	Najwcześniejszy potwierdzony dostęp konsumenta.
input_cutoff	Granica dopuszczonego wejścia tej decyzji.
decision_time	Moment wyboru, nie czas późniejszego raportu.

Źródła: R3-S026. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dane spóźnione nie zmieniają wstecz wejścia

Rekonsyliacja powinna tworzyć nowy widok, zachowując stary stan wiedzy.

Rozważmy receipt, który dociera po timeout. W czasie timeoutu wynik był niepewny; po odczycie receipt można rozpocząć dodatkową weryfikację. Nie wolno jednak zmienić historycznego requestu tak, aby wyglądał jak zakończony przed terminem. To naruszałoby zarówno analizę przyczyn awarii, jak i pomiar opóźnienia.

Ta sama zasada dotyczy badań: nowy kontrprzykład nie usuwa wcześniejszego zielonego suite'u. Zmienia zakres poprawnego twierdzenia o nim. Do rekordu oceny dodaje się następstwo, źródło nowego wyniku i uzasadnienie, dlaczego wcześniejsza generalizacja jest już zbyt szeroka.

W strukturze dokumentacji każde takie przejście powinno wskazywać oryginał, dawną ocenę i następcę. Można wtedy odtworzyć zarówno błąd, jak i sposób jego rozpoznania. Jest to realna wartość ewoluującej przeszłości: wzrost wiedzy o historii bez manipulowania świadectwami.

Przykład analityczny · Receipt po timeout

Odpowiedź dotarła po zakończeniu pierwszego okna oczekiwania. Nowy rekord może rozstrzygać późniejszą wiedzę o skutku, ale nie usuwa wcześniejszego stanu niepewności. Odbiór musi zachować oba czasy i zapobiec przypisaniu receipt do nowszej próby o innym attempt_id.

```
record A: wynik nieznany przy cutoff t1
record B: receipt odebrany przy t2 > t1
B REFINES ocenę A; nie nadpisuje wejścia A
```

Ślad implementacyjny · SemanticDelta

delta_id	str; pole wymagane
base_graph_digest	str; pole wymagane
operation	str; pole wymagane
subject_ref	str; pole wymagane
predicate	str; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 75–100; całość deklaracji: 11 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dossier aktywnego problemu

Niearchiwizowana wiedza robocza musi mieć własną strukturę, zanim uzyska stan końcowy.

SemanticScaffoldIR przechowuje odniesienia do hipotez, kontrhipotez, falsyfikatorów i unknowns. Dossier w R3 jest proponowanym widokiem łączącym te rekordy z kandydatami, próbami oraz odpowiedzialnością za następny krok. Nie oznacza dodatkowej bazy prawdy.

Aktywne dossier nie może być wyłącznie folderem z najnowszymi plikami. Powinno pokazać wersję problemu, zakres dopuszczonych pytań i przyczynę zmiany hipotezy. Wynik nieprzyjęty nadal jest wartościowym śladem. Wynik utracony lub nieodtwarzalny musi pozostać widoczną luką, zamiast znikać podczas porządkowania repozytorium.

Archiwizacja zamyka określony epizod i jego zestaw referencji. Późniejsze badanie może go otworzyć jako źródło nowego epizodu. W ten sposób współistnieją stabilne dossier i zmienna przestrzeń eksperymentów, a operator nie musi wybierać między całkowitym zamrożeniem pracy a nieśledzalnym chaosem.

Przykład analityczny · Nierozstrzygnięty epizod

Badanie kończy się z dwoma nadal zgodnymi wyjaśnieniami. Dossier powinno zachować oba, wskazać brakujący eksperyment i zakres istniejących wyników. Archiwizacja nie wymaga wymuszenia zwycięskiej hipotezy. Taki negatywny lub częściowy wynik może być pełnym rezultatem zadanego ograniczonego badania.

Tożsamość	Problem, generacja, rodzic, owner.
Praca	Warianty, próby, ograniczenia, blokady.
Zamknięcie	Oryginalne artefakty, ocena, zakres niewiedzy.

Ślad implementacyjny · SemanticScaffoldIR

scaffold_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane

Wybrane pola z semantic_scaffold.py, klasa w liniach 33–76; całość deklaracji: 25 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S066, R3-S044. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Stable jest profilem lokalnym

Trwałość wybranej semantyki nie gwarantuje trwałości implementacji i wdrożenia.

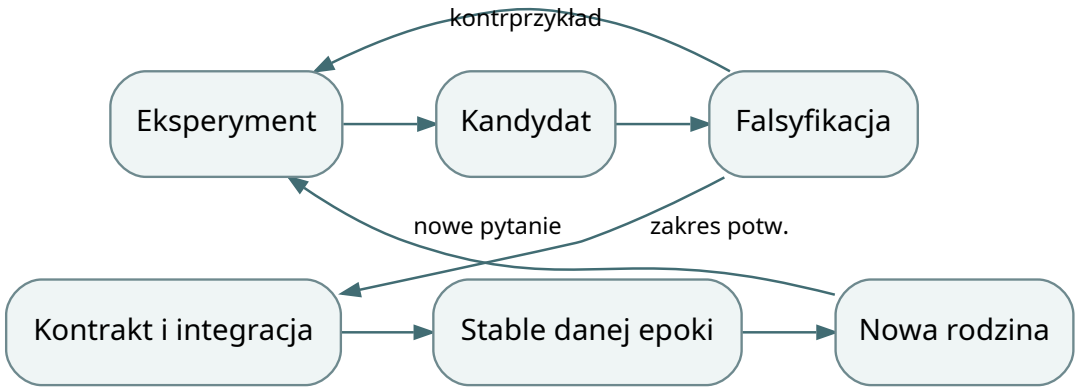
W źródłach rozdzielono lifecycle, currentness, execution, verification i deployment. Kontrakt może być użyteczny jako ustabilizowana baza dla testów, choć provider pozostaje kandydatem. Odwrotnie: uruchomiony proces może obsługiwać starszą wersję danych i nie spełniać dzisiejszego interfejsu.

Proponowany profil stable powinien mieć zakres, wersję i warunki zakończenia. Kontrprzykład nie musi usuwać całego profilu; może zawęzić go do przypadków nadal poprawnych. Zmiana konsumenta może wymagać nowej kwalifikacji bez zmiany pierwotnego bloba. To stabilność relacji w danej epoce, a nie absolutna etykieta przypisana plikowi.

Praktyczna korzyść jest zasadnicza: kolejny eksperyment wie, na czym może się oprzeć. Jeżeli narusza publiczną semantykę, powstaje nowa wersja albo adapter. Jeżeli zmienia tylko wewnętrzny algorytm przy zachowaniu kontraktu, testy regresyjne pomagają wykazać ciągłość.

Przykład analityczny · Stabilny parser, nowy provider

Ten sam format danych może pozostać stabilny, gdy zmienia się executor. Potrzebna jest nowa kwalifikacja bindingu i runtime, ale niekoniecznie nowa wersja parsera. Zmiana semantyki timeoutu providera może jednak wymagać korekty kontraktu procesu. Reguła migracji powinna rozstrzygać to jawnie.



R3-G04 · Stabilizacja lokalnego zakresu. Model źródłowo-analityczny; nie nowa obserwacja runtime.

```
StableForEpoch = zakres + wersja + konsumenci + dowody
kontrprzykład -> rekwalifikacja zakresu
nowy deployment -> osobny odbiór
```

Źródła: R3-S032, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Pamięć nie może zatwierdzać samej siebie

Refleksyjność systemu wymaga rozdzielenia mechanizmu oceny i przedmiotu oceny.

ai_platform wytwarza dokumenty o ai_platform, a więc model może stać się źródłem następnej propozycji. Jeśli nowy opis sam stanowiłby jedyny dowód swojej poprawności, powstałaby zamknięta pętla autoreferencji. Niezmienny baseline oraz dokładny model transformacji pozwalają przerwać to utożsamienie.

W praktyce trzeba zachować wersję narzędzia, które oceniało kandydata, i wersję narzędzia będącego kandydatem. Nowy walidator nie może sam wydać sobie świadectwa skuteczności tylko dlatego, że własny test przechodzi. Użyteczne są kontrprzykłady z wcześniejszej epoki, odrębna implementacja kontrolna i jawnie zamrożone oczekiwania.

W dokumencie R3 analogicznie zachowano cały poprzedni atlas. Nowe rozstrzygnięcie nie usuwa wcześniejszego statusu PARTIAL. Rozbudowana narracja może dostarczyć formalizacji i lepszej mapy zależności, ale dopiero dodatkowe źródła lub testy mogą zamknąć konkretną wcześniejszą lukę.

Przykład analityczny · Walidator oceniający następcę

Nowa wersja checkera została przetestowana przez własny zmodyfikowany suite. To niewystarczające, jeśli jednocześnie usunięto dawny falsyfikator. Odbiór powinien zachować kontrolę względem wcześniejszego znanego przypadku oraz uzasadnić zmianę oczekiwania. Nie można uzyskać pewności przez edytowanie miary sukcesu pod kandydata.

Mechanizm badania	Wersja, wejścia i ograniczenia.
Przedmiot zmiany	Osobny kandydat i jego baseline.
Kontrola	Niezależny w wymaganym zakresie falsyfikator.

Ślad implementacyjny · GovernedChangeProposal

schema_version	str; pole wymagane
proposal_id	str; pole wymagane
epoch_id	str; pole wymagane
source_delta_id	str; pole wymagane
source_delta_digest	str; pole wymagane

Wybrane pola z governed_change_proposal.py, klasa w liniach 64–121; całość deklaracji: 18 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S003, R3-S051. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

L0–L6: rozwijanie jednej odpowiedzialności

Przejście w dół dodaje szczegół, ale nie może dodawać ukrytego prawa do działania.

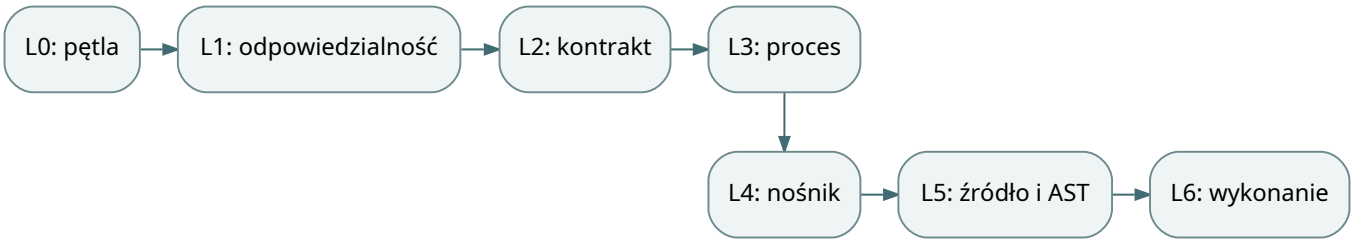
Na poziomie L0 komponent oznacza wkład w rozliczenie zmiany. Na L1 pojawia się właściciel obowiązku, na L2 kontrakt, na L3 przebieg, na L4 nośnik danych, na L5 implementacja, a na L6 konkretna obserwacja. Taki porządek pozwala przejść od sensu procesu do miejsca w kodzie bez uznawania katalogu za pełną definicję roli.

Przykład CommunicationEnvelope pokazuje potrzebę ostrożności. Na L1 wspiera komunikację, na L2 określa ścisły model danych, na L5 ma metodę validate. Nie wynika stąd L6 dostarczenia wiadomości. Rozwinięcie musi wskazać brakujący transport i jego dowody, zamiast rozciągnąć poprawność schematu na całą drogę.

W R3 relacja refines jest identyfikatorem dokumentacyjnym. Rodzic i dziecko zachowują źródła, ograniczenia oraz zakres statusu. Jeden element może mieć kilka rozwinięć: perspektywę semantyczną, czasową i infrastrukturalną. Nie tworzy to kilku konkurencyjnych ownerów tego samego stanu.

Przykład analityczny · Powiększenie jednego węzła

Element komunikacji na L1 rozwija się na kopertę, routing, invocation i delivery. Jeśli w rozwinięciu pojawia się możliwość zapisu do świata, musi mieć osobne uzasadnienie authority. Powiększenie grafiki nie może po cichu poszerzać zakresu roli nadrzędnej.



R3-G05 · Hierarchia rozwinięcia. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S043. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Natywne warstwy nie są numerami przybliżenia

Piętnaście domen, trzy płaszczyzny i siedem poziomów opisują różne osie.

Źródłowy flows.py deklaruje ARCHITECTURE_LAYERS, m.in. CODE_PERCEPTION, AUTHORITY_AND_EFFECT i EVOLUTIONARY_EPOCH. Manifesty repozytoriów posługują się INF, SEM i MAND. Atlas dodaje L0–L6 jako skalę dokumentacji. Gdy te osie zostaną zlane, powstaje pozorna sprzeczność liczby warstw zamiast użytecznej mapy.

Domena CODE_PERCEPTION ma aspekt celu, kontraktu, danych, kodu i wykonania. Nie należy jej na stałe umieszczać wyłącznie na L5. Podobnie authority jest widoczna na poziomie filozofii systemu i na poziomie konkretnego admission. Jedna etykieta katalogu nie pokrywa całej roli.

W docelowym generatorze widok powinien deklarować osie filtracji. Czytelnik wybiera domenę, szczegółowość, epokę i klasę dowodu. Takie filtrowanie jest operacją prezentacji, nie zmianą danych źródłowych. Zbiór ukrytych węzłów pozostaje dostępny jako kontekst ograniczenia widoku.

Przykład analityczny · Domena widoczna na wielu poziomach

AUTHORITY_AND_EFFECT można opisać na L0 jako zasadę separacji, na L2 jako kontrakt, a na L6 jako konkretny admission. To trzy widoki jednej domeny. Przypisanie jej wyłącznie do poziomu infrastruktury zgubiłoby najważniejszy niezmiennik ogólnego modelu.

ARCHITECTURE_LAYERS	Natywne domeny modelu architektury.
INF / SEM / MAND	Płaszczyzny funkcjonalne manifestów.
L0–L6	Stopień rozwinięcia dokumentacji.
F / P / H	Rola informacji względem epizodu.

Źródła: R3-S003, R3-S033. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Port jest ważniejszy niż podobieństwo nazw

Adekwatne zaszeregowanie wymaga zgodności wejścia, wyjścia i odpowiedzialności.

Dwa pliki o nazwie context mogą pełnić odmienne funkcje: jeden składa snapshot, drugi dobiera prompt, a trzeci zapisuje historię sesji. Ontologia architektury nie powinna łączyć ich na podstawie słowa. Potrzebny jest rzeczywisty typ payloadu i konsument, który potrafi go przyjąć.

Z drugiej strony te same zdolności mogą występować pod różnymi historycznymi nazwami. Wtedy normalizacja wprowadza jawny alias i wspólną definicję roli, zachowując oryginalną ścieżkę. Migracja kodu jest osobnym krokiem: alias dokumentacyjny nie zmienia importu ani runtime routing.

Dla każdego portu należy odnotować wymagania danych, wersję, dopuszczalne niepewności i typ błędu. Jeżeli wyjście jest jedynie reference-only, następny komponent musi wykonać własne rozwiązanie referencji. Bez tego kreska między dwoma zgodnie nazwanymi obiektami byłaby brakującym adapterem ukrytym w diagramie.

Przykład analityczny · Dwa podobne Context

Jeden obiekt przechowuje źródła, drugi gotowy prompt, a trzeci checkpoint rozmowy. Mapowanie wymaga ustalenia, co każdy consumer zakłada o danych. Wspólna nazwa nie tworzy interoperacyjności. Dopiero świadomy adapter może odróżnić utratę informacji od dopuszczalnej kompresji.

Ślad implementacyjny · CompositionBeanBinding

bean_id	str; pole wymagane
spec_digest	str; pole wymagane
implementation_digest	str; pole wymagane
provider_family	str; pole wymagane
resource_units	int; pole wymagane

Wybrane pola z bean_composition.py, klasa w liniach 34–46; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S039, R3-S059. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Agregacja stanu rodzica

Gotowość funkcji wynika z krytycznych zależności, nie z uśrednienia statusów.

Karta nadrzędna Factory obejmuje poznanie, przygotowanie, kwalifikację, BUILD i VERIFY. Każdy etap może mieć osobny status i osobny zakres źródeł. Poprawna agregacja powinna wyjaśnić, czy pytamy o obecność programu, możliwość jego zarejestrowania, czy o odebrany materialny wynik.

Jeżeli brakuje konsumenta wiążącego dwa kontrakty, uzupełnienie liczby testowanych klas nie rozwiązuje luki. Predykat rodzica jest zależny od konkretnego przejścia. Można jednocześnie uznać zintegrowaną semantykę za realny dorobek i pozostawić pełny obieg wykonania nierozstrzygnięty.

Przyszły interfejs powinien pokazywać uzasadnienie statusu w formie minimalnego podgrafu. Operator zobaczy wtedy blokującą krawędź, jej ownera i rodzaj wymaganej pracy. Nie musi zgadywać, czy czerwony stan oznacza błąd kodu, brak danych, brak zasobu czy poprawną odmowę.

Przykład analityczny · Health bez zdolności misji

Serwer odpowiada na health, lecz nie ma odpowiedniego capability binding. Panel nie powinien prezentować tego jako gotowej Factory. Jednocześnie działanie health pozostaje prawdziwym wynikiem obserwacji usługi. Agregacja musi zachować oba stany zamiast wybierać jeden globalny kolor.

```
gotowy_program = parse AND semantic_contracts
gotowy_krok = binding AND currentness AND admission
ukończony_cel = observed_postconditions AND reconciliation
```

Źródła: R3-S003, R3-S015. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Macierz widoków zamiast jednego gigantycznego grafu

Szczegółowość powinna być rozwijana tam, gdzie zmienia rozstrzygnięcie.

Graf obejmujący wszystkie obiekty jest dobry jako indeks maszynowy, ale często zły jako pojedynczy rysunek. Setki strzałek utrudniają rozpoznanie granicy odpowiedzialności. Podwidok powinien odpowiadać na jedno pytanie i wskazywać, których relacji celowo nie pokazuje.

Dla komunikacji warto oddzielić tożsamość wiadomości, sekwencję delivery, projekcję kontekstu oraz aktualne instancje providera. Wspólne identyfikatory pozwalają przechodzić między tymi widokami. Ukrycie węzła wykonawczego w ogólnym obrazie jest dopuszczalną abstrakcją; ukrycie granicy authority zmieniłoby sens.

R3 utrzymuje oryginalne 41 widoków R2 i dodaje rozwinięcia o własnej klasie. Nie przelicza ich liczby na liczbę dowodów. Ten sam test może wspierać kilka widoków, ale pozostaje jednym wykonaniem. Podpis rysunku musi mówić, czy przedstawia kod, model źródłowy, interpretację, czy TO-BE.

Przykład analityczny · Widok bez payloadów

Diagram pokazuje tożsamość wiadomości i nie wyświetla jej prywatnej treści. Jest to dopuszczalna abstrakcja, jeśli zachowuje payload_ref, zakres i regułę dostępu. Nie jest natomiast dowodem, że pominięta treść spełnia ograniczenia. Jej ocena należy do innego widoku i konsumenta.

Widok struktury	Co jest czym i do kogo należy?
Widok sekwencji	Co poprzedza co i pod jakim warunkiem?
Widok dowodu	Które przejście rzeczywiście potwierdzono?

Źródła: R3-S003, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Przykład rozwinięcia: zapis artefaktu

Opis od celu do bajtów ujawnia, gdzie przebiega prawdziwa granica odbioru.

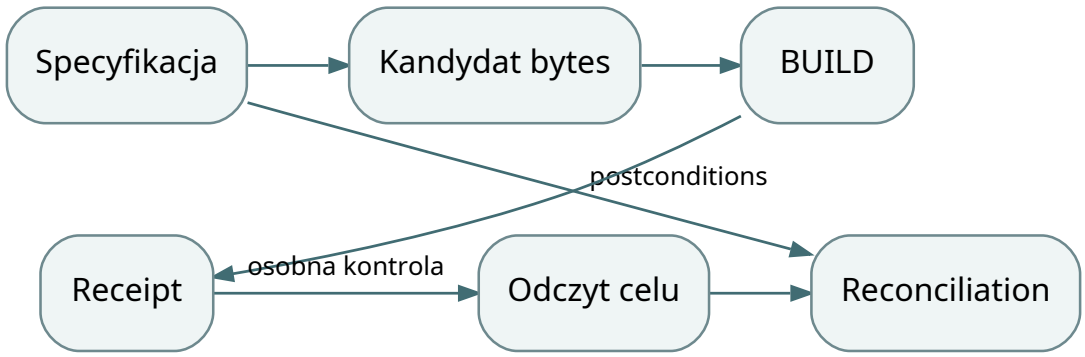
Na L0 chodzi o uzyskanie zweryfikowanej zdolności. L1 przypisuje osobno budowanie i weryfikację. L2 wymaga kontraktu pliku, referencji producenta i tożsamości wyniku. L3 definiuje BUILD przed VERIFY. L4 wskazuje nośnik i receipt; L5 konkretny writer; L6 odczytane bajty.

Każdy poziom wyklucza inny błąd. Zgodny model na L2 może zostać skierowany do złego workspace na L6. Poprawny plik może nie odpowiadać celowi L0. Samo SHA-256 wiąże bajty, lecz nie ocenia wartości rozwiązania. Potrzebna jest semantyczna relacja postcondition do zadanego problemu.

Formalizując ten przykład nie dopisujemy nowego workera do runtime. Wyznaczamy kontrakt przyszłego dowodu i miejsca jego źródłowych ownerów. R1 zawiera historyczny stan konkretnej Factory, ale R3 nie wykonuje ponownego materialnego eksperymentu.

Przykład analityczny · Ten sam hash, inny cel

Plik ma oczekiwane bajty, ale użytkownik wymagał innego rodzaju artefaktu. Integralność może być poprawna przy niespełnionym celu. Odbiór semantyczny musi odnieść wynik do specyfikacji, nie tylko do wcześniej obliczonego hash. Te dwie kontrole powinny zachować odrębne predykaty.



R3-G06 · Dwa odbiory jednego artefaktu. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S038, R3-S063, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Tożsamość drzewa a genealogia commitu

Równa zawartość nie oznacza identycznej historii.

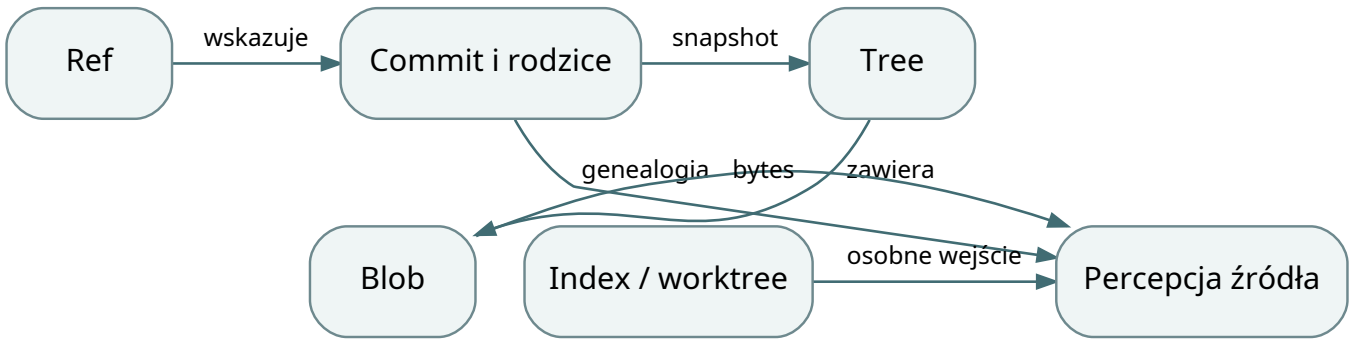
Commit łączy tree z metadanymi i rodzicami. Dwa commity mogą mieć tę samą zawartość plików, ale inne pochodzenie. Atlas pokazuje taki przykład w writeups: integracja historii może zachować drzewo, a zarazem poszerzyć osiągalną genealogię badań. To nie jest sprzeczność ani automatyczna zmiana kodu.

Dla percepcji statycznej można wówczas otrzymać niezmienioną projekcję zależną od tree. Dowody zależne od dokładnego commitu i procesu publikacji nadal pozostają inne. Ten sam wynik testu treści nie jest automatycznie świadectwem zatwierdzenia nowej genealogii.

W dokumentacji warto więc przechowywać trzy relacje: commit wskazuje tree, tree zawiera blob oraz ocena dotyczy określonej wersji i pytania. Bez tego zmiana wyłącznie historyczna może niepotrzebnie wywołać pełną analizę AST albo przeciwnie — zostać przeoczona przy kontroli provenance.

Przykład analityczny · Merge historii bez zmiany plików

Przy tym samym tree można zachować wynik analizy zależnej od treści, ale nie należy przypisywać nowemu commitowi dowolnego starszego CI. Genealogia i zgoda publikacyjna są innymi faktami. Wpisanie ich w jeden status CURRENT zacierałoby powód rewalidacji.



R3-G07 · Git: zawartość i genealogia. Model źródłowo-analityczny; nie nowa obserwacja runtime.

```
commit_A != commit_B
tree(commit_A) == tree(commit_B)
nie wynika: identyczna genealogia lub identyczne CI
```

Źródła: R3-S003, R3-S020. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Referencja ruchoma i baseline nieruchomy

Gałąź służy nawigacji; odbiór wymaga rozpoznania konkretnego obiektu.

W czasie badania master ai_platform ponownie wskazał d367e47c. Ta obserwacja ma własny czas i nie pozwala przypisać identycznej tożsamości przyszłemu odczytowi. Nazwa branch może pozostać niezmienna, gdy jej zawartość już się przesunęła.

Przed porównaniem kandydata trzeba związać zarówno jego head, jak i base. Szczególnie w stacked PR jedna zmiana może zawierać kod poprzednika, który nie jest jeszcze zintegrowany z master. Sama obecność funkcji w kandydującej gałęzi nie nadaje jej statusu globalnie przyjętego rozwiązania.

Rozwinięcie dokumentacji pozostaje bezpieczne, jeśli wyraźnie wskazuje pinned source i oddzielny zapis świeżej referencji. W przyszłym automatycznym procesie drift powinien unieważniać zależną ocenę, nie wszystkie niezależne wyniki. W tym zadaniu nie wykonywano merge, resetu ani usuwania gałęzi.

Przykład analityczny · Gałąź przesunięta podczas odczytu

Pierwszy plik odczytano przed zmianą master, drugi po niej. Wspólna nazwa branch nie tworzy atomowego snapshotu. Źródła trzeba przypiąć do konkretnych commitów albo jawnie oznaczyć mieszane okno obserwacji. Dopiero wtedy wiadomo, czy powstały model kiedykolwiek odpowiadał jednemu stanowi.

Ref	Wskazanie odczytane w konkretnej chwili.
Baseline	Dokładny commit/tree analizowanego przypadku.
Candidate	Własny head/base i odrębny zakres kwalifikacji.

Źródła: R3-S014, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Working tree jako aktywna teraźniejszość

Nieopublikowane zmiany są realnymi artefaktami, lecz nie należą automatycznie do kanonicznego źródła.

Raport R1 odnotował różne checkouty i stany lokalnych worktree. Ich znaczenie nie sprowadza się do „nieporządku”. Mogą przechowywać eksperymenty, które jeszcze nie otrzymały własnego commitu, albo przygotowane delty oczekujące na ocenę. Kasowanie ich dla estetyki usuwałoby materiał procesu.

Przy budowaniu modelu AS-IS należy jednak rozdzielić lokalny plik od bloba wskazanego przez master. Parser uruchomiony na aktualnym katalogu roboczym może zbadać inną treść niż ta opisana w metadanych raportu. Dlatego każdy rekord analizy powinien wskazywać, czy źródłem był committed blob, index, working tree czy zewnętrzny załącznik.

W normalizacji proponujemy najpierw dyspozycje i genealogy: aktywny kandydat, zachowany eksperyment, źródłowo zintegrowana delta lub materiał nierozstrzygnięty. Dopiero później można projektować migrację ścieżek. Nie wolno rekonstruować nieistniejącego commitu z samego hasha pliku.

Przykład analityczny · Staged eksperyment

Lokalny index zawiera poprawkę, której nie ma w remote master. Test na worktree może ją uwzględnić, mimo że raport wskazuje tylko remote HEAD. Kontrola exact bytes ujawnia różnicę. Nie należy kasować poprawki, lecz zachować jej własną tożsamość i zakres testu.

Committed blob	Treść związana z dokładnym Git object.
Index	Przygotowany stan możliwego commitu.
Worktree	Bieżące bajty pracy lokalnej.
Załącznik	Osobny nośnik, nie domyślny fragment repozytorium.

Źródła: R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Typowana granica odczytu Git

Kontrola wejścia do percepcji jest częścią architektury, nie tylko wygodą implementacyjną.

CodePerceptionGitBoundary udostępnia zamknięty zestaw operacji: resolve commit, resolve tree, list tree, read blob i hash stdin. W samym indekserze nie przyjmuje dowolnego argv dostarczonego przez analizowany tekst. Odczyt tworzy lokalny request digest i obserwację wyniku procesu.

Zawężenie interfejsu zmniejsza przestrzeń niejednoznaczności między „zbadaj źródło” a „wykonaj polecenie z repozytorium”. Nie jest jednak uniwersalnym dowodem bezpieczeństwa hosta. Kod zależy od rzeczywistego środowiska Git i systemu, a limity rozmiaru są sprawdzane po zebraniu wyniku subprocess. Te granice należy zachować w opisie.

Właściwy consumer powinien nadal porównać commit/tree oraz blob i rozmiar. Poszczególne metody nie zastępują pełnej ścieżki pochodzenia. Np. odczyt obiektu po SHA nie wystarcza do wykazania jego przynależności do deklarowanego drzewa bez uprzedniego inwentarza.

Przykład analityczny · Odczyt obiektu spoza drzewa

Blob jest dostępny lokalnie i poprawnie zahashowany, ale nie należy do zadeklarowanego snapshotu. Dopiero porównanie z list_tree może wykazać ten problem. Integralność treści i przynależność do source set są dwoma niezależnymi wymaganiami.

```
RESOLVE_COMMIT -> RESOLVE_TREE -> LIST_TREE
-> READ_BLOB -> HASH_STDIN -> porównanie bytes
operacje tylko odczytowe != pełna atestacja hosta
```

Źródła: R3-S021, R3-S020. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Currentness jest funkcją przedmiotu oceny

Jeden zielony hash nie powinien zastępować wszystkich wymiarów aktualności.

truth_plane.py wyodrębnia subject z dokładnego drzewa, przy zachowaniu zamkniętego zbioru carrier paths. Celem jest uniknięcie samoodniesienia pliku, który miałby deklarować własny przyszły hash. Carriers są zapisywane po stabilizacji źródeł właściwej epoki.

Aktualność deklaracji zależy od jej subject. Zmiana dokumentu może unieważnić wiedzę architektoniczną, choć nie zmienia binarnego artefaktu workera. Zmiana konfiguracji runtime może unieważnić admission, choć Git tree pozostało identyczne. Nie istnieje jeden globalny token czasu ważny dla każdego rodzaju decyzji.

W nowym modelu każdy stan CURRENT wskazuje dokładne źródła, kryterium i zakres. Po drift wybieramy zależny podgraf rewalidacji. Nie tworzymy serii pustych commitów po to, aby stale poprawiać numer HEAD w dokumencie, ani nie pomijamy nowych non-carrier źródeł w obliczeniu subject.

Przykład analityczny · Zmiana konfiguracji bez nowego kodu

Binary i Git tree pozostały te same, ale executor otrzymał inny workspace. Wtedy źródłowa projekcja może pozostać aktualna, a runtime binding wymaga ponownej oceny. To praktyczny powód wielowymiarowego modelu currentness.

Git currentness	Zgodność właściwego ref/head/tree.
Model currentness	Zgodność źródeł, generatora i projekcji.
Runtime currentness	Zgodność instancji, konfiguracji i dopuszczenia.

Źródła: R3-S003, R3-S018. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Genealogia jako informacja o sposobie powstania

Historia jest potrzebna do interpretacji zmiany, lecz nie zastępuje jej mechanicznego diffu.

Commit message może wyjaśniać intencję autora, ale nie dowodzi, że kod rzeczywiście ją realizuje. Diff wskazuje zmianę bajtów. Test może ujawnić skutki w określonym środowisku. Razem dają znacznie lepszą rekonstrukcję niż jeden z tych nośników używany w izolacji.

W przypadku homeostazy dokumentacji istotne jest odnalezienie błędu starszego manifestu, poprawki overlay i kontrpróby niezgodnego tree. Dopiero taki łańcuch uzasadnia twierdzenie, że logika została zmieniona w odpowiedzi na konkretny problem. Sam tytuł „fix” nie wystarcza.

Git pełni więc podwójną rolę: pamięci materialnych stanów i powierzchni percepcji dla następnej iteracji. Analiza znaczenia jest osobną pracą. Jeżeli brakuje części historii lub payloadu, model zachowuje UNKNOWN zamiast budować płynną, lecz nieudokumentowaną opowieść o intencjach.

Przykład analityczny · Tytuł fix bez rozstrzygającego testu

Commit deklaruje usunięcie awarii, lecz diff zmienia jedynie tekst raportu. Analiza nie powinna przejąć deklaracji jako skutku. Należy odszukać zmieniony mechanizm i test odróżniający stare zachowanie od nowego. Brak takiego dowodu pozostaje widoczny.

```
intencja autora -> deklaracja historyczna
diff -> zmiana treści
próba -> wynik w zadanym zakresie
rekonstrukcja -> jawna synteza tych źródeł
```

Źródła: R3-S003, R3-S023. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

AST porządkuje strukturę, nie dowodzi zachowania

Natywny Code Perception jawnie zachowuje nierozwiązane cele wywołań.

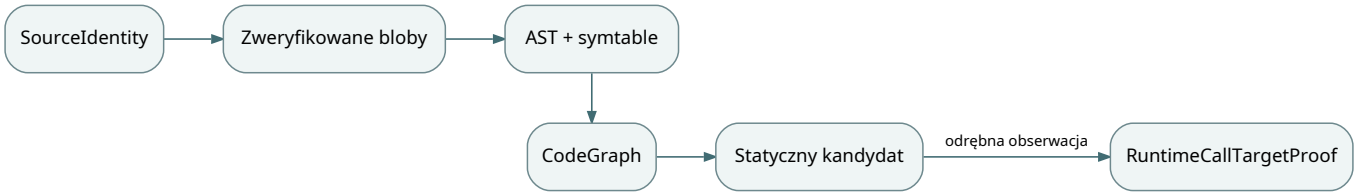
Python ast i symtable umożliwiają odtworzenie definicji, zakresów nazw, importów oraz syntaktycznych wywołań. W źródle LION CALLS ma klasę STATIC_CALL_EVIDENCE. Wskazany target_node_id pozostaje kandydatem źródłowym, a nie świadectwem konkretnego callable wybranego podczas przyszłego uruchomienia.

To rozróżnienie jest kluczowe dla samoanalizy systemu. Kod zawierający nazwę funkcji nie musi do niej dojść w danym przebiegu. Alias, parametr, redefinicja, warunek i dynamiczny dispatcher mogą zmienić wybór. Indekser powinien wówczas zachować relację nierozwiązaną zamiast domyślać się najbardziej podobnego symbolu.

W atlasie AST jest kontrapunktem drzewa Git: tree identyfikuje położenie i bajty, AST ujawnia organizację programu, a obserwacja runtime potwierdza użycie. Trzy reprezentacje mogą być powiązane, ale nie wolno redukować jednej do drugiej.

Przykład analityczny · Import a wykonanie

Graf wskazuje IMPORTS między modułami. Nie wynika z tego, że konkretny handler został wywołany w misji. Statyczna zależność jest materiałem do śledzenia, a exact runtime target potrzebuje osobnej obserwacji powiązanej z krawędzią i wydaniem.



R3-G08 · Od bajtów do dowodu wywołania. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S020, R3-S041. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Tożsamość symbolu a wersja jego struktury

Nazwa, lokalizacja i treść są oddzielnymi wymiarami zmiany.

Natywny identyfikator symbolu korzysta z path, qualified_name i kind. Semantic_digest opiera się na ast.dump bez atrybutów pozycyjnych. Zmiana ciała funkcji przy tej samej ścieżce może zachować node_id, ale zmienić digest. Przeniesienie pliku może zmienić node_id mimo tych samych bajtów funkcji.

W nazwie semantic_digest nie należy doszukiwać się dowodu pełnej równoważności semantycznej programów. Jest to hash konkretnej reprezentacji składniowej. Różne AST mogą zachowywać się równoważnie w danym zakresie, a identyczny AST może zachowywać się inaczej w innym środowisku zależności.

Normalizacja nazw powinna zatem przechowywać relacje alias, moved_from i supersedes z dowodem. Nie wolno uznać, że podmiana nazw jest zawsze kosmetyczna. Zmienia importy, refleksję, ścieżki konfiguracji, a czasem publiczny protokół narzędzi.

Przykład analityczny · Komentarz i zachowanie

Zmiana komentarza może zmienić blob, ale pozostawić AST funkcji bez zmiany. Z kolei ten sam AST użyty z innymi zależnościami może zachować digest przy innym zachowaniu. Dobór miary podobieństwa musi odpowiadać pytaniu o treść, strukturę albo wykonanie.

```
node_id = digest(path, qualified_name, kind)
semantic_digest = SHA256(ast.dump(...))
semantic_digest != dowód równoważności zachowania
```

Źródła: R3-S020, R3-S022. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Zakres leksykalny ogranicza domysły

Ten sam token nazwy nie zawsze odnosi się do tej samej funkcji.

Test z parametrem target pokazuje cień rzucany na funkcję modułu o tej samej nazwie. Wywołanie target() wewnątrz caller(target) musi pozostać nierozstrzygnięte, ponieważ argument może wskazywać dowolny obiekt. Indekser używa modelu scope, a nie globalnego dopasowania tekstu.

Podobny problem pojawia się dla lokalnych przypisań i nonlocal. Zewnętrzna funkcja jest realnym symbolem, lecz nie wynika stąd, że zostanie wywołana w badanym miejscu. Uczciwy graf może zawierać wiele nierozwiązanych krawędzi. Ich liczba nie powinna automatycznie być interpretowana jako defekt architektury.

Przy rozwijaniu analizy potrzebny jest osobny model wiązań dla danego rodzaju dynamiki. Każde zwiększenie precyzji wymaga nowych kontrprzykładów. Usunięcie UNKNOWN przez optymistyczne przypisanie targetu poprawiłoby estetykę diagramu kosztem prawdziwości.

Przykład analityczny · Parametr maskujący nazwę

Wywołanie funkcji przekazanej jako argument nie ma jednego statycznego targetu. Nie należy przypisywać mu funkcji o tej samej nazwie w module. Taki przykład pozwala sprawdzić, czy indekser zachowuje granicę wiedzy zamiast maksymalizować liczbę pozornie rozwiązanych krawędzi.

```
def target(): return 1

def caller(target):
    return target()  # parameter: exact target UNKNOWN
```

Źródła: R3-S022, R3-S020. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Definicja obecna nie musi być aktywna

Percepcja źródeł musi rozróżniać byt syntaktyczny od wiązania osiągalnego w danym przebiegu.

W próbie z if False funkcja target istnieje jako węzeł AST, ale blok definiujący nie jest wykonywany. Natywny test wymaga zachowania symbolu i nierozwiązanej krawędzi CALLS. Podobnie importy w if, loop, try, match i with nie tworzą bezwarunkowego wiązania na każdej drodze.

Z takiej ostrożności wynika ważna reguła dla analizy całego repozytorium. Znalezienie klasy providera nie dowodzi jej rejestracji, importowalności ani aktywności. Do AS-IS runtime potrzebny jest call path, konfiguracja oraz obserwacja. Sam indeks symboli pozostaje wartościowy, lecz rozstrzyga inne pytanie.

W projektowanym procesie każda luka powinna zostać przypisana do poziomu: brak symbolu, brak jawnego konsumenta, dynamiczny binding lub niepotwierdzony przebieg. Tylko pierwszy z tych przypadków uzasadnia bezpośredni wniosek o nieobecności implementacji w zbadanym zbiorze.

Przykład analityczny · Nieosiągalna definicja

Funkcja wewnątrz if False jest w źródle, lecz nie zostanie związana przez ten blok. Graf powinien zachować fakt definicji i odrębność runtime. Usunięcie symbolu zgubiłoby strukturę źródła, a rozwiązanie CALLS udawałoby wykonanie.

```
if False:
    def target(): return 1

def caller(): return target()
# symbol istnieje; runtime binding nie jest potwierdzony
```

Źródła: R3-S022. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Atrybuty i dispatch polimorficzny

Polimorfizm wymaga zachowania niepewności wyboru, dopóki nie ma właściwego dowodu wiązania.

Wywołanie self.target() zależy od rzeczywistego obiektu, dziedziczenia i możliwych modyfikacji atrybutu. Natywny mechanizm pozostawia takie cele nierozwiązane. Testy obejmują override, staticmethod, classmethod, descriptor oraz przypisanie nowej funkcji do atrybutu klasy.

Ta własność nie jest przeszkodą dla polimorfizacji federacji. Przeciwnie: chroni różnicę między wspólnym kontraktem a konkretną realizacją. Architektura może wiedzieć, jakiego zachowania wymaga od providera, nie wiedząc statycznie, która jego instancja wykona zadanie w przyszłości.

Dla dokładnego dowodu runtime potrzebne są identyfikator instancji, wersja artefaktu, źródłowa krawędź i odrębna obserwacja celu. Takie wiązanie nie powinno być produkowane przez sam indeks źródeł, bo indeks nie uczestniczył w wykonaniu.

Przykład analityczny · Podmieniony atrybut

C.target może zostać zastąpione inną funkcją przed wywołaniem. Sam tekst C.target(...) nie rozstrzyga wyboru. Test powinien sprawdzać, że indekser nie traktuje takiej relacji jako exact runtime proof bez osobnego modelu bindingu.

Ślad implementacyjny · RuntimeCallTargetProof

proof_id	str; pole wymagane
repository	str; pole wymagane
source_commit_sha	str; pole wymagane
source_tree_sha	str; pole wymagane
code_graph_digest	str; pole wymagane

Wybrane pola z code_perception.py, klasa w liniach 264–334; całość deklaracji: 13 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S020, R3-S041. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Census nie jest kompletnym modelem architektury

Mechaniczna kompletność plików nie oznacza semantycznego domknięcia wszystkich zależności.

FullSymbolCensus oczekuje dokładnego zbioru committed Python paths oraz HEAD/TREE. Może policzyć symbole, adnotacje, docstringi i statyczne referencje z testów. Test odwołujący się do nazwy funkcji nie dowodzi jej wykonania ani pokrycia gałęzi. Taką granicę deklaruje sam kod narzędzia.

R1 raportował 982 pliki Python parsujące się do AST, ale tylko dziewięć pełnych dokumentów tekstowych w swoim ledgerze. Te liczby dotyczą innych mianowników. R3 nie przenosi ich na nowe globalne twierdzenie o pełnym przeczytaniu projektu. Odczyt wybranych źródeł i nowe testy kontraktów stanowią dokładną, ograniczoną deltę.

Pełniejszy następny etap powinien rozliczyć nie-Python entrypointy, konfigurację routingu i dynamicznych konsumentów. Nie wymaga to porzucenia istniejącego AST. Wymaga dodania osobnych warstw dowodu i spójnych referencji między nimi.

Przykład analityczny · Plik nieparsowalny

Źródło z błędem składni nie znika z census. Pozostaje rekordem PARSE_ERROR z tożsamością bajtów. W przeciwnym przypadku mianownik malałby właśnie o najtrudniejsze przypadki, tworząc fałszywą stuprocentową kompletność.

Census	Czy wszystkie zadane pliki i symbole ujęto?
Call graph	Jakie zależności wynikają z modelu statycznego?
Pokrycie testów	Co naprawdę wykonano podczas konkretnego testu?
Architektura	Jaki sens i odpowiedzialność mają te relacje?

Źródła: R3-S003, R3-S022. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Metamodel, stan i widok zadania

Trzy reprezentacje rozwiązują trzy różne rodzaje problemu poznawczego.

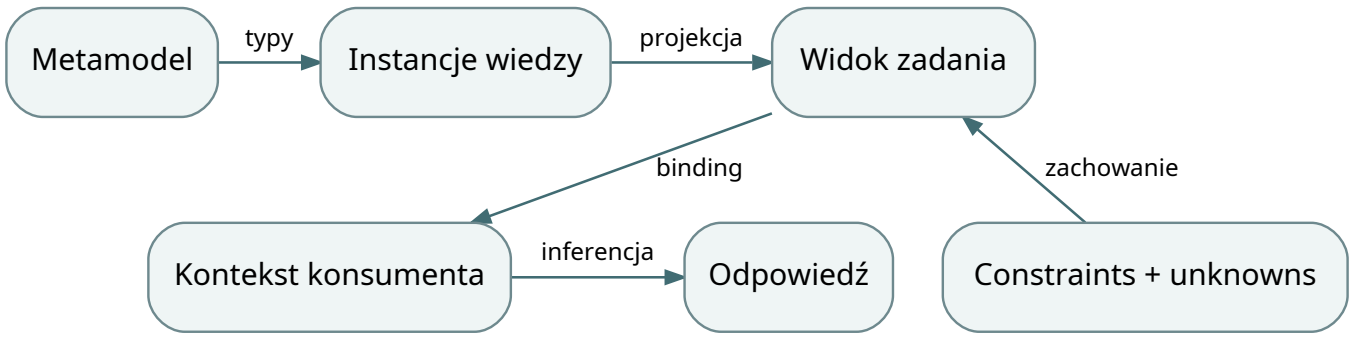
Metamodel określa, co oznacza hipoteza, dowód, relacja i capability. Graf stanu przechowuje konkretne instancje tych typów. Widok zadania wybiera fragment potrzebny konsumentowi. Jeśli połączymy te poziomy, prywatna hipoteza jednego agenta może niepostrzeżenie stać się definicją obowiązującą całą federację.

W istniejącym zestawie kontraktów rolę typowanego scaffoldingu pełni SemanticScaffoldIR, a reprezentację tezy z genealogią — SemanticAtom. RelevanceGraph wiąże refs, natomiast RelevanceProjection ogranicza widok dla konsumenta. Żaden z tych obiektów nie jest samodzielną bazą prawdy ani źródłem uprawnień.

Rozwój ontologii może zatem przyjmować dwie postacie. Częsta aktualizacja instancji wiedzy nie wymaga zmiany definicji typu. Zmiana samego słownika relacji lub semantyki statusu jest zmianą kontraktu, wymagającą migracji konsumentów i osobnej formalizacji.

Przykład analityczny · Nowa relacja w metamodelu

Jeżeli dodano predykat oznaczający nowy typ zależności, stare konsumery mogą go nie rozumieć. To zmiana kontraktu, nie zwykłe dodanie instancji w grafie. Potrzebna jest wersja, mapowanie i jawny sposób obsługi nieznanej relacji.



R3-G09 · Trzy poziomy ontologii. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S066, R3-S065. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Hipoteza potrzebuje konkurenta

Wartość eksperymentu zależy od tego, co potrafi rozróżnić, a nie od samego wyniku PASS.

HypothesisSpec w repozytorium hipotez rozdziela claim, assumptions, observable_consequences i falsifiers. To właściwy kierunek formalizacji: wynik powinien odnosić się do przewidywania, którego niespełnienie rzeczywiście zawęża lub odrzuca tezę. Zgodna odpowiedź na dowolny przykład nie jest jeszcze rozstrzygającym testem.

Dla fabryki konkurencyjne wyjaśnienie może brzmieć: mechanizm nie generuje nowej logiki, lecz wybiera z zadanej gramatyki. Nie jest to automatycznie wada; staje się wadą opisu wtedy, gdy wynik ograniczonej syntezy zostaje nazwany uniwersalnym transferem. Eksperyment musi jawnie wskazać przestrzeń, w której rozstrzyga.

W docelowej dokumentacji każda silna teza powinna mieć kartę alternatywy. To pozwala zachować nie tylko sukces, ale również powody, dla których inne wyjaśnienia pozostały możliwe. Taka wiedza jest szczególnie użyteczna dla następnej generacji agentów planujących nowe testy.

Przykład analityczny · Wynik zgodny z dwiema hipotezami

Eksperyment dał PASS zarówno dla modelu generalizującego, jak i wybierającego ze skończonej tabeli. Nie rozstrzyga więc między nimi. Kolejna próba powinna zmieniać właściwość, względem której przewidywania są różne, zamiast powtarzać podobny przykład.

Teza	Wspólny mechanizm działa w zdefiniowanym zakresie.
Alternatywa	Sukces wynika wyłącznie z dostarczonej gramatyki.
Próba rozdzielająca	Nowa rodzina i kontrola pochodzenia oczekiwanych wyników.

Źródła: R3-S013, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Relacja w scaffoldzie: forma i referent

Spójny digest nie wykazuje jeszcze istnienia każdego końca krawędzi.

ScaffoldRelation sprawdza identyfikatory subject_ref, predicate, object_ref i evidence_refs. Główne validate kontroluje unikalność relation_id oraz hashe rodziców. Nie sprawdza jednak, czy subject_ref i object_ref należą do entity_refs. R3-T16 odtworzył możliwość sealed scaffold z końcem spoza deklarowanego zbioru.

Wniosek musi pozostać precyzyjny: odkryto granicę lokalnego walidatora. Nie wykazano, że żaden wyższy konsument nie wykonuje kontroli ani że powstał nieautoryzowany efekt. Rozstrzygnięcie integracyjne wymaga prześledzenia faktycznej ścieżki i ustalenia, gdzie zgodnie z architekturą ma następować referential closure.

Proponowana korekta może polegać na walidacji relacji względem jawnego rejestru encji oraz rzeczywistych rodziców, zamiast na samym rozszerzeniu regexu. Zmiana musi respektować przypadki referencji zewnętrznych. Nie należy odrzucać poprawnego federacyjnego obiektu tylko dlatego, że nie został skopiowany do lokalnego payloadu.

Przykład analityczny · Zewnętrzna referencja poprawna

Naprawa edge closure nie może wymagać, aby każda encja federacyjna była skopiowana do lokalnego payloadu. Poprawny zewnętrzny ref powinien mieć resolver i dowód. Dangling ref powinien zostać odrzucony. Oba przypadki są potrzebne do kwalifikacji nowego adaptera.

```
R3-T16: relation.subject_ref spoza entity_refs
local validate -> zaakceptowany kształt
wniosek: edge closure wymaga osobnego właściciela
nie: wykazany bypass runtime
```

Ślad implementacyjny · SemanticScaffoldIR

scaffold_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane

Wybrane pola z semantic_scaffold.py, klasa w liniach 33–76; całość deklaracji: 25 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S066, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dowody wspólnego pochodzenia

Niezależność jest własnością genealogii, nie liczby zgodnych zdań.

SemanticAtom.independent_root_count liczy różne wartości root_ref. Dwa evidence_refs mające ten sam korzeń dają w R3-T11 wynik jeden. Jest to użyteczna kontrola duplikacji deklarowanej genealogii, lecz nie weryfikuje ona prawdziwości samych root_ref.

Jeżeli dwie odpowiedzi modeli powstały z tego samego tekstu i tego samego wyniku narzędzia, ich zgodność nie stanowi dwóch obserwacji świata. Mogą być niezależnymi próbami interpretacji wspólnego wejścia, co ma inną wartość badawczą. Należy zatem rozdzielić niezależność źródła, procesu interpretacji i wykonawcy.

W przyszłym widoku wiedzy operator powinien móc rozwinąć korzenie dowodu i zobaczyć punkty współzależności. Ukrycie genealogii za jednym procentem confidence prowadziłoby do pozornej pewności narastającej wyłącznie przez kopiowanie materiału.

Przykład analityczny · Dwa streszczenia jednego logu

Modele niezależnie streszczają ten sam wynik narzędzia. Powstają dwie interpretacje, ale jeden korzeń obserwacji. Ich zgodność może mieć wartość dla stabilności interpretacji, nie dla zwiększenia liczby niezależnych pomiarów świata.

Ślad implementacyjny · EvidenceInstance

evidence_ref	str; pole wymagane
root_ref	str; pole wymagane
content_digest	str; pole wymagane
observed_at	str; pole wymagane
currentness	str; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 31–36; całość deklaracji: 5 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065, R3-S025, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Relewancja nie jest prawdziwością

Selekcja kontekstu musi zachować ograniczenia, nawet gdy usuwa nieistotne szczegóły.

RelevanceProjection.validate_against sprawdza tożsamość grafu, podzbiory widocznych węzłów i krawędzi oraz zbiór wymaganych capabilities. R3-T12–T15 odtworzyły poprawne przyjęcie widoku i odrzucenie rozszerzeń. Ten zakres jest mocniejszy niż sama walidacja kształtu digestu.

Nie znaczy to jednak, że wybrany podzbiór jest merytorycznie wystarczający. Można usunąć z kontekstu istotny zakaz i nadal zachować formalny podzbiór. Potrzebny jest więc dodatkowy kontrakt zachowania constraints i unknowns związanych z pytaniem. W R3 to propozycja testów konsumenta, nie wykazana właściwość całej ścieżki.

Budżet zasobów uzasadnia redukcję kontekstu, lecz nie uzasadnia ukrytej zmiany znaczenia. Projekcja powinna raportować, co pominięto i dlaczego. W przeciwnym przypadku model otrzymuje uproszczony świat bez informacji, że utracił warunek konieczny do poprawnej decyzji.

Przykład analityczny · Poprawny podzbiór bez zakazu

Projekcja usuwa węzeł zawierający warunek zgody. validate_against nadal może potwierdzić podzbiór, ale semantyka zadania ulega zmianie. Potrzebny jest test zachowania constraints, nie tylko zakaz dopisywania nieznanych węzłów.

Ślad implementacyjny · RelevanceProjection

projection_id	str; pole wymagane
graph_ref	str; pole wymagane
graph_digest	str; pole wymagane
consumer_ref	str; pole wymagane
required_capability_refs	Tuple[str, ...]; pole wymagane

Wybrane pola z semantic_relevance.py, klasa w liniach 127–159; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S065, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Generatywność ontologii podlega wersjonowaniu

Nowe pojęcie powinno wyrastać z problemu, a nie z potrzeby wypełnienia pustej warstwy diagramu.

W toku eksperymentu może ujawnić się brak rozróżnienia, tak jak oddzielenie fazy od kontraktu wykonania w PCD-0001. Poprawka ontologiczna polega wtedy na wprowadzeniu bytu, który umożliwia wyrażenie wcześniej niewidocznego stanu. To lepsze uzasadnienie niż podobieństwo do popularnego frameworka.

Nowa definicja powinna wskazywać, które pytanie rozstrzyga, jakim kontrprzykładem została wymuszona i kto będzie jej konsumentem. Jeżeli nowy byt nie zmienia żadnej walidacji, zależności ani sposobu interpretacji, może być tylko aliasem dokumentacyjnym. Alias jest dopuszczalny, ale nie powinien udawać nowego modułu produktu.

Wspólny CORE stanowi w takim ujęciu pamięć wypracowanych rozróżnień. Nie musi być ostateczny. Zmiana pojęcia wymaga jednak jawnej relacji do wcześniejszej wersji, aby następny agent nie połączył payloadów o podobnej nazwie i różnych znaczeniach.

Przykład analityczny · Nowe pojęcie bez konsumenta

Dodany typ doskonale opisuje planowaną funkcję, ale nie jest używany poza testem. Może być wartościowym kontraktem kandydata. Nie należy jednak nazywać go działającym modułem procesu. Właściwy plan wskazuje brakujące przejście do konsumenta.

```
kontrprzykład -> brakujące rozróżnienie
-> definicja -> owner -> consumer -> test
-> wersja kontraktu -> migracja wiedzy
```

Ślad implementacyjny · FormalizationArtifact

artifact_id	str; pole wymagane
path	str; pole wymagane
schema_or_type	str; pole wymagane
semantic_role	str; pole wymagane
semantic_owner_concept	str; pole wymagane

Wybrane pola z formalization_registry.py, klasa w liniach 96–165; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S015, R3-S050, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Pamięć nie mieści się w jednym store

Źródła, przebieg wykonania i interpretacja mają różnych właścicieli zapisu.

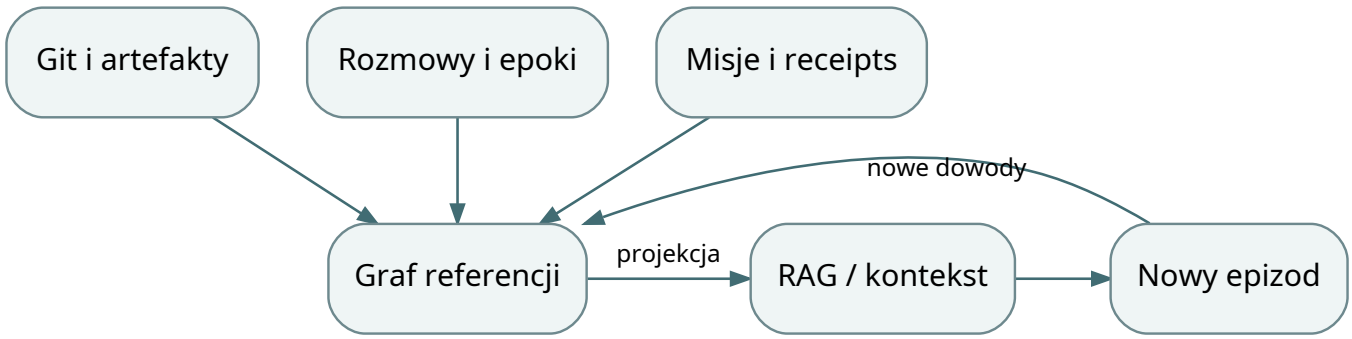
Git przechowuje wersje źródeł i dokumentów. Baza rozmów przechowuje wiadomości, lane’y i bindingi. Mission Control utrzymuje proces, assignments i receipts. RAG udostępnia wersjonowaną wiedzę, a grafy wiążą jej pochodzenie. To wzajemnie potrzebne, lecz niezamienne formy pamięci.

Umieszczenie wszystkich rekordów w jednym technicznym systemie nie usunęłoby różnic semantycznych. Nadal trzeba wiedzieć, kto może zmienić stan misji, kto tylko dodać interpretację i jaki snapshot był widoczny konkretnemu modelowi. Podobnie osobne bazy nie gwarantują niezależnych źródeł dowodu, jeżeli kopiują ten sam rekord.

Docelowy model pamięci powinien zatem najpierw wyznaczać odpowiedzialność, cykl życia i reguły odczytu. Dopiero potem wybiera implementację magazynu. R3 nie proponuje drugiego nadrzędnego store dla ProjectReality; rozwija referencyjne wiązanie istniejących nośników.

Przykład analityczny · Ta sama informacja w dwóch magazynach

Kopia transcriptu w RAG nie staje się nowym źródłem tego zdarzenia. Powinna zachować odniesienie do canonical message i własną datę indeksacji. Tylko wtedy można odróżnić nowy fakt od nowej lokalizacji starej treści i poprawnie policzyć genealogiczne korzenie dowodu.



R3-G10 · Pamięć jako kompozycja. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S044, R3-S005. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

HA2D: snapshot jest świadectwem kontekstu

Zapis kontekstu nie oznacza automatycznego zatwierdzenia pamięci organizacyjnej.

ContextSnapshot z HA2D wiąże source_repository, source_commit, epoch, cutoff_ref, uporządkowane sources i payload_digest. Walidator wymaga dodatniej całkowitej epoki, unikalnych źródeł i zgodnego digestu. Każde źródło niesie odrębną klasę epistemiczną, np. OBSERVED albo HYPOTHESIS.

Pola authority_effect i memory_commit_effect pozostają NONE. To ważne: poprawny snapshot może być materiałem wejściowym, lecz sam nie promuje wiedzy. Jego payload_digest nie jest dowodem, że przyszły model otrzymał całą treść snapshotu. Potrzebne jest osobne wiązanie projekcji i rzeczywiście wysłanych bajtów.

W federacji taki kontrakt nadaje się do przenoszenia kontekstu bez kopiowania całej historii. Odbiorca musi jednak rozwiązać referencje i sprawdzić, czy odpowiadają zadanej epoce. Sam poprawny format SHA nie potwierdza istnienia ani dostępności wskazanego commitu.

Przykład analityczny · Snapshot poza cutoff

Źródło w ContextSnapshot zostało zarejestrowane po granicy decyzji. Poprawny digest nie usuwa tego konfliktu czasowego. Odbiorca powinien odmówić użycia rekordu w historycznej projekcji albo oznaczyć ją jako późniejszą reinterpretację. Nie wolno dopisywać późniejszej wiedzy do wcześniejszego wejścia modelu.

Źródło	Repozytorium, commit i lista ContextSource.
Czas	epoch oraz cutoff_ref.
Zakres skutku	authority_effect=NONE; memory_commit_effect=NONE.
Odbiór	Osobne użycie kontekstu przez rzeczywistego konsumenta.

Źródła: R3-S011. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Selekcja historii ma realne limity

W kandydacie PR #456 cały transcript i kontekst przekazany modelowi nie są tym samym.

Odczytane linie 1–220 conversation_chat.py zawierają _canonical_history. Funkcja wybiera rekordy oznaczone canonical_context i role user/assistant. Treść każdego ogranicza do 1400 znaków, rozważa ostatnich osiem i buduje wynik do 4200 znaków. To konkretny mechanizm ograniczania wejścia w tym źródle.

Nie wolno rozciągać tej obserwacji na każdą ścieżkę LION ani na dowolny model. Dotyczy wskazanej funkcji w dokładnym kandydacie. Pokazuje jednak fundamentalną różnicę między trwałą pamięcią rozmowy a jej chwilową projekcją. Starszy zakaz może pozostać w bazie, lecz nie znaleźć się w wybranym fragmencie.

Projekt rozwinięcia powinien związać wynik selekcji z krytycznymi constraints i dołączyć informację o pominięciu. Samo zwiększenie okna nie jest pełnym rozwiązaniem, bo dowolny skończony budżet wymaga selekcji. Trwałe reguły roli i aktualny kontekst zadania potrzebują jawnego, wersjonowanego wiązania.

Przykład analityczny · Starsze ograniczenie poza oknem

Istotny zakaz znajduje się w dziewiątej od końca wiadomości. Funkcja wybierająca ostatnich osiem może go pominąć. Problemem nie jest sama liczba osiem, lecz brak gwarancji przeniesienia ograniczenia. Przyszły kontrakt powinien wskazywać osobny nośnik trwałych reguł albo jawnie raportować niepełny kontekst.

```
_canonical_history w PR456:
canonical_context; role user/assistant
limit treści rekordu: 1400 znaków
wybór: ostatnie 8; budżet łączny: 4200 znaków
zakres odczytu: linie 1–220, nie cały plik
```

Źródła: R3-S007, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Epizod uczenia wiąże ślady, nie przejmuje kolejek

EvidenceBoundLearningEpisode opisuje, jak wynik może wrócić do następnej generacji.

Kontrakt epizodu wiąże wiadomości, invocations, assignments, wyniki, obserwacje i rekonsyliację. Nie powinien zastępować oryginalnych ownerów delivery ani wykonywania. Dzięki referencjom da się analizować, które źródła rzeczywiście uczestniczyły w próbie i gdzie powstał kontrprzykład.

Przekształcenie epizodu w EvidenceObservation trzeba interpretować z uwzględnieniem przedmiotu obserwacji. Można potwierdzić, że dany epizod i jego rekordy istnieją, bez potwierdzenia prawdziwości każdej tezy modelu umieszczonej w odpowiedzi. Etykieta OBSERVED nie powinna propagować się bezwarunkowo do wszystkich treści rodziców.

W następnym etapie fabryki taki epizod może zasilać zmianę planu, test regresyjny lub kandydacką regułę. Każda promocja musi wskazywać nowy zakres użycia. Wynik z jednej gramatyki nie jest domyślnie wiedzą o całej klasie aplikacji.

Przykład analityczny · Epizod zawiera symulację

Rekord epizodu został rzeczywiście zapisany, lecz jego wynik pochodzi z modelu symulacyjnego. OBSERVED może opisywać istnienie zapisu, a nie materialny skutek symulowanego świata. Consumer pamięci musi zachować pierwotną result_class i nie promować jej przez samą zmianę nośnika.

Ślad implementacyjny · EvidenceBoundLearningEpisode

episode_id	str; pole wymagane
model_release_ref	str; pole wymagane
causal_group_ref	str; pole wymagane
task_family	str; pole wymagane
episode_class	str; pole wymagane

Wybrane pola z evidence_bound_learning_episode.py, klasa w liniach 39–65; całość deklaracji: 26 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S044, R3-S064. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Retencja, archiwizacja i dostępność

Zachowany hash bez zachowanych bajtów nie jest pełnym nośnikiem reprodukcji.

Indeks RAG może przechowywać identyfikator archiwum, ścieżkę i payload hash, lecz nie pełny payload. To przydatna mapa źródeł, ale nie wystarczy do odtworzenia eksperymentu. Podobnie manifest ZIP może być poprawny, choć sam ZIP nie jest już dostępny w aktywnym środowisku.

Właściwy projekt retencji określa, które oryginały trzeba zachować, gdzie są replikowane i jak sprawdza się możliwość odczytu. Nie wszystkie dane wymagają wiecznej retencji: część może podlegać polityce prywatności lub usuwania. Trzeba wtedy jawnie zachować status niedostępności zamiast udawać, że hash zastępuje materiał.

R3 przechowuje dokładny załącznik R2 i pakiet komunikacji jako źródła, a nowe modele jako pochodne. Nie twierdzi, że odtworzono wszystkie historyczne archiwa LION. Rejestr braków jest częścią wiedzy, nie błędem składu dokumentu do ukrycia.

Przykład analityczny · Hash istnieje, payload zniknął

Indeks potwierdza dawną tożsamość pliku, ale nośnik jest niedostępny. Odtwarzalność jest wtedy ograniczona. Można odwołać się do historycznego raportu i jawnie oznaczyć brak oryginału; nie można odtworzyć bajtów z hasha ani uznać samego indeksu za pełne archiwum.

Integrity	Czy dostępne bajty zgadzają się z manifestem?
Availability	Czy oryginał nadal można odczytać?
Reproducibility	Czy są też wejścia, zależności i metoda?
Retention	Kto i przez jaki okres zachowuje materiał?

Źródła: R3-S023, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Promocja wiedzy jako osobna transformacja

Zapis kandydata, jego przyjęcie i użycie w nowym kontekście nie są jednym zdarzeniem.

Nowy kontrprzykład można zapisać natychmiast jako wynik badania. Żeby stał się normą, potrzebne jest rozstrzygnięcie właściciela i aktualizacja właściwego kontraktu albo reguły. Żeby norma wpłynęła na działanie, musi zostać dostarczona do konsumenta i poprawnie egzekwowana.

W dokumentacji warto śledzić trzy krawędzie: produced, accepted i consumed. Ich połączenie tworzy drogę uczenia organizacji. Brak ostatniej krawędzi wyjaśnia sytuację, w której system ma poprawny opis problemu, a mimo to powtarza stary błąd. Zwiększanie objętości RAG nie rozwiązuje automatycznie braku konsumenta.

Wydanie R3 samo pozostaje kandydatem wiedzy referencyjnej. Nie zmienia bootstrapu, rejestru formalizacji ani bieżącej polityki wykonania. Właściwy kolejny krok może polegać na przyjęciu wybranego rozstrzygnięcia, a nie na automatycznej promocji wszystkich opisów TO-BE.

Przykład analityczny · Wiedza przyjęta, lecz nieużyta

Reguła została dodana do dokumentacji, ale aktualny bootstrap jej nie wczytuje. Organizacja posiada poprawny materiał, a executor nadal powtarza błąd. Diagnoza dotyczy konsumenta i actual context, nie konieczności napisania jeszcze dłuższej instrukcji o tej samej treści.

```
wynik badania -> kandydat wiedzy
-> przyjęcie w zadanym zakresie
-> wersja nośnika -> rzeczywisty odczyt konsumenta
-> obserwowana zmiana działania
```

Ślad implementacyjny · FormalizationClosureRecord

closure_id	str; pole wymagane
candidate	CandidateBinding; pole wymagane
formalization_manifest_digest	str; pole wymagane
artifact_results	Tuple[ArtifactResult, ...]; pole wymagane
semantic_owner_uniqueness	str; pole wymagane

Wybrane pola z formalization_closure.py, klasa w liniach 46–104; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S006, R3-S049. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

PCD-0001: ontologia rodzi się z braku wykonania

Semantycznie poprawna faza może pozostawać niewykonalna.

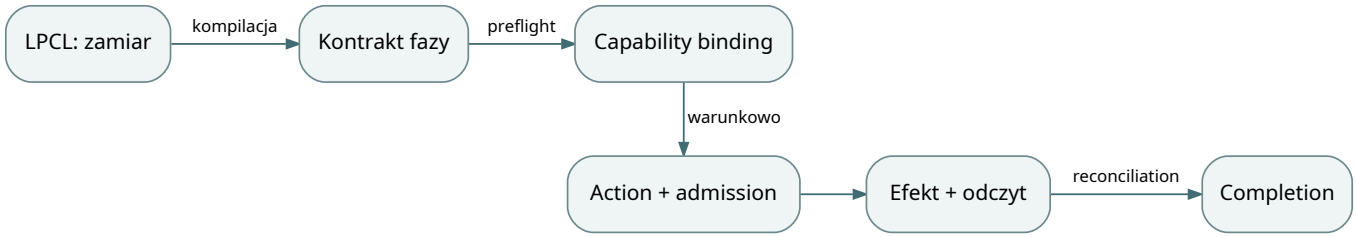
Process Contract Plane zachowuje kontrprzykład REPAIR_EXECUTION_BINDER, który osiągnął CAPABILITY_NOT_AVAILABLE mimo sensownej fazy LPCL. Z tego wyprowadzono rozróżnienie pomiędzy intencją i kontraktem wykonania. Nie chodziło o poprawę nazwy, lecz o możliwość reprezentowania wcześniej niejawnego wymagania.

PhaseExecutionContract opisuje klasę dopuszczalnego wykonania, effect ceiling, wymagane dowody, currentness i completion predicates. Capability binding wybiera aktualną implementację. Action IR opisuje konkretny ograniczony krok. Każdy z tych bytów może istnieć bez zakończonego efektu.

Ten przypadek jest mocnym źródłowym przykładem kształtowania logiki przez działanie: napotkana niemożność zmieniła model procesu. R3 nie odtwarza historycznej awarii runtime, lecz analizuje jej utrwalony opis i istniejący kontrakt. To rozdzielenie chroni przed dopisaniem własnego eksperymentu do cudzej historii.

Przykład analityczny · Znacząca, lecz niewykonalna faza

Faza REPAIR ma cel i opis, ale nie ma sposobu spełnienia completion. Należy utworzyć jawny kontrakt i odkryć capability, a nie polecić modelowi „postaraj się bardziej”. Nowa ontologia ujawnia brak techniczny, który wcześniej był ukryty pod sensowną nazwą.



R3-G11 · Od fazy do completion. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S015, R3-S056. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Leksyka LPCL nie może odgadywać uprawnień

Polecenie REPAIR opisuje zamiar, a nie automatyczny grant do mutacji.

Parser tekstu rozpoznaje składnię programu. Kompilacja semantyczna tworzy kontrakty faz, a preflight ocenia ich wykonalność. Te działania nie są tożsame. Tekst może być syntaktycznie poprawny i nie zawierać wystarczających informacji o capability lub completion.

Zgodność wsteczna LPCL/1.1 przez LEGACY_INFERRED_SAFE ma zachować bezpieczną interpretację z effect_ceiling=NONE. Nie wolno na podstawie słów FIX, IMPLEMENT lub MIGRATE wyprowadzić szerszej zgody. To ważny przykład konstrukcyjnego ograniczenia tego, co język procesu może oznaczać.

W docelowym edytorze operator powinien widzieć zarówno błąd składni, jak i brak kontraktu wykonania. Model generujący program może proponować uzupełnienie, ale serwer powinien sam odtworzyć semantykę z dokładnego lpcl_text. Nie należy ufać dostarczonemu przez klienta obiektowi pochodnemu jako źródłu autorytetu.

Przykład analityczny · Słowo MIGRATE w danych

Historyczny dokument zawiera polecenie migracji. Parser badania powinien traktować je jako treść źródła, dopóki operator nie uruchomi dokładnego programu. Sama obecność słowa wykonawczego nie może stać się Action IR ani uprawnieniem do zmiany konfiguracji.

Ślad implementacyjny · CanonicalProcessIR

canonical_bytes	bytes; pole wymagane
process_digest	str; pole wymagane

Wybrane pola z process_ir.py, klasa w liniach 333–360; całość deklaracji: 2 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S015, R3-S057. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Preflight ujawnia rodzaj brakującej zdolności

VALID_UNBOUND_WAITING jest informacją o procesie, a nie ukrytym INVALID.

Kontrakt procesu rozdziela READY_BOUND, VALID_WITH_DYNAMIC_BINDING, WAITING_FOR_CAPABILITIES i INVALID. Faza może być poprawna, choć jeszcze nie znaleziono jej wykonawcy. Takie oczekiwanie jest dozwolone tylko wtedy, gdy jawna polityka dopuszcza dynamic discovery.

W źródle rozdzielono trwały kontrakt, aktualny binding i runtime execution spec. Dzięki temu zmiana providera nie musi przepisywać semantycznego celu fazy. Jednocześnie nie wolno przyjąć nowej implementacji bez sprawdzenia wersji, scope i aktualności.

Warunkiem użytecznego WAIT jest znana przyczyna oraz właściciel następnego kroku. Jeżeli capability nie istnieje, proces może zaprojektować jej budowę w odrębnym zadaniu. Nie powinien bez końca odpytywać rejestru, który sam z siebie nie dostarczy brakującego kodu.

Przykład analityczny · Rebind po utracie providera

Semantyczny kontrakt fazy pozostaje aktualny, lecz jego instancja wykonawcza wygasła. Nowy binding może wybrać innego zgodnego providera po sprawdzeniu identity i currentness. Nie wolno zachować starego lease’u jako dowodu dostępności nowej instancji.

Poprawność	Czy faza ma zgodny kontrakt semantyczny?
Dostępność	Czy istnieje aktualna implementacja zdolności?
Dopuszczenie	Czy konkretny efekt może zostać wykonany?
Postęp	Jaki warunek rzeczywiście blokuje następny krok?

Ślad implementacyjny · PhaseExecutionContract

mission_id	str; pole wymagane
phase_id	str; pole wymagane
ordinal	int; pole wymagane
execution_class	str; pole wymagane
capability_classes	tuple[str, ...]; pole wymagane

Wybrane pola z phase_execution_contract.py, klasa w liniach 70–158; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S015, R3-S056. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

VERIFY_BEFORE_REPAIR jako ochrona przed dryfem planu

Najpierw sprawdź rzeczywiste postcondition, potem projektuj brakującą zmianę.

W trakcie długiego procesu inny dopuszczony przyrost może już naprawić wskazany problem. Stary plan pozostaje historycznie uzasadniony, ale jego ponowne wykonanie mogłoby być zbędne albo szkodliwe. Process Contract Plane nakazuje dla VERIFY_THEN_REPAIR sprawdzić aktualny stan przed mutacją.

Jeżeli wymagane predykaty są spełnione, faza może zakończyć się przez odczytowe evidence i reconciliation. Brak nowego zapisu nie oznacza wtedy braku realizacji celu. Jeżeli predykaty nie są spełnione, należy wyznaczyć najmniejszą brakującą capability lub Action IR.

Ta zasada jest szczególnie użyteczna w federacji z wieloma kandydatami i worktree. Chroni przed globalnym restartem audytu oraz przed wprowadzaniem drugiej naprawy tego samego defektu. Nie daje jednak prawa do korzystania ze starego wyniku weryfikacji po zmianie jej źródeł.

Przykład analityczny · Cel naprawiony przez inny przyrost

Przed wykonaniem starego planu nowe źródło wskazuje, że postconditions są już spełnione. Poprawnym wynikiem może być odczytowa reconciliation. Ponowna mutacja nie zwiększyłaby jakości dowodu, a mogłaby stworzyć konflikt z rzeczywistym stanem.

```
observe(current_postconditions)
if all_required_true:
    reconcile_without_mutation()
else:
    propose_smallest_missing_change()
```

Źródła: R3-S015. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Receipt nie zamyka celu fazy

Potwierdzenie działania wykonawcy i spełnienie wymagania to odrębne twierdzenia.

Worker może poprawnie zapisać plik, ale zapisać nieodpowiednią treść albo pod inną ścieżką. Receipt opisze zgłoszony rezultat działania. Completion wymaga odrębnej oceny postconditions, a reconciliation wiąże zamiar, raport i obserwację.

W programie Factory sama odpowiedź modelu nie powinna zaspokajać predykatu gotowości materialnego providera. Analogicznie health procesu nie dowodzi import closure, a istniejący plik nie dowodzi zgodności jego bajtów z kandydatem. Każda teza potrzebuje adekwatnej obserwacji.

W przyszłym panelu należy pokazać dwa osobne etapy: wykonawca zgłosił wynik oraz cel został zweryfikowany. Dzięki temu operator rozpozna niepewny skutek po utracie ACK i nie uruchomi ślepego retry. Końcowy status fazy powinien wynikać z jawnych predykatów, a nie z ostatniego tekstu SUCCESS.

Przykład analityczny · Poprawny zapis, niewłaściwy adres

Executor zwrócił sukces, ale plik trafił do innego workspace. Receipt jest realnym raportem, lecz completion nie powinno przejść. Niezależny odczyt miejsca docelowego rozstrzyga błąd, którego nie wykryje samo ponowne parsowanie zgłoszenia workera.

Ślad implementacyjny · RuntimeObserverTrustBinding

source_id	str; pole wymagane
source_instance_id	str; pole wymagane
source_implementation_digest	str; pole wymagane
trust_anchor_id	str; pole wymagane
trust_anchor_digest	str; pole wymagane

Wybrane pola z runtime_reconciliation.py, klasa w liniach 35–41; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S015, R3-S063. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Restart jest rekonstrukcją, nie nową misją

Trwałe identyfikatory pozwalają kontynuować pracę bez podwójnego efektu.

Po ponownym uruchomieniu serwera trzeba odtworzyć kontrakty, bindingi i niedokończone przejścia. Nie wolno automatycznie traktować braku odpowiedzi jako nieistnienia wcześniejszego efektu. Oryginalny assignment i jego idempotency semantics wyznaczają dozwoloną ścieżkę recovery.

Jeżeli operacja jest nieidempotentna, należy najpierw ustalić stan celu. Dla samej percepcji można bezpiecznie powtórzyć odczyt w nowej epoce. Dla mutacji trzeba zachować identity kandydata, admission i obserwacji, aby wynik nie został przypisany do innego uruchomienia.

Atlas wskazuje kolejność startu Mission Control jako osobną ścieżkę od logiki ewolucyjnej. R3 ją rozwija pojęciowo, ale nie wykonuje restartów hosta. Test restart-durability musi w przyszłości objąć rzeczywisty przebieg przez granicę restartu, a nie sam heartbeat usługi po starcie.

Przykład analityczny · Restart pomiędzy send i ACK

Po wznowieniu ledger nie zawiera odpowiedzi, ale zewnętrzny skutek mógł już zajść. System powinien odtworzyć stan celu i starego attempt, zamiast tworzyć nowy send. Odrębna próba wymaga uzasadnienia, dlaczego poprzednia nie zakończyła się materialnym skutkiem.

Odczyt	Nowa obserwacja tej samej intencji.
Niepewny zapis	Najpierw ustalenie rzeczywistego skutku.
Nowy kandydat	Nowa generacja i odrębne dopuszczenie.

Źródła: R3-S003, R3-S015. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Od potrzeby do kompozycji

Fabryka rozwiązuje brak zdolności przez reuse albo jawnego kandydata.

CapabilityNeed wiąże potrzebę z konkretnym Goal i Gap. BeanSpec opisuje zdolność, którą można katalogować i zestawiać z innymi. Kompozycja sprawdza zależności i wymagania portów. Wspólny mechanizm nie musi oznaczać jednego sztywnego workflow dla wszystkich problemów.

Kiedy istniejąca zdolność wystarcza, tworzenie nowej implementacji może tylko zwiększyć dług i liczbę ownerów. Gdy brakuje elementu, generator powinien stworzyć specyfikację kandydata, a nie udawać dostępnego providera. Ta różnica jest ontologicznym warunkiem generatywnej fabryki.

R3 traktuje więc reuse jako pełnoprawny wynik ewolucji. Nowe możliwości mogą powstawać przez inną kompozycję już stabilnych komponentów. Taką zmianę również trzeba ocenić, ponieważ nowa topologia zależności może zmienić obserwowalność, ryzyko i zakres awarii.

Przykład analityczny · Reuse zamiast nowej klasy

Katalog zawiera zdolność spełniającą wszystkie wymagania Gap. Nowy generator nie powinien tworzyć duplikatu tylko dlatego, że celem misji było „rozwijanie” systemu. Użyteczny rozwój może polegać na poprawnym zestawieniu istniejących elementów i zachowaniu mniejszego kosztu utrzymania.

Ślad implementacyjny · CapabilityNeed

need_id	str; pole wymagane
gap_digest	str; pole wymagane
goal_digest	str; pole wymagane
required_capability	str; pole wymagane
required_inputs	Tuple[str, ...]; pole wymagane

Wybrane pola z capability_need.py, klasa w liniach 25–51; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S040, R3-S037, R3-S039. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

BeanSpec i BeanCandidate nie są zamienne

Specyfikacja opisuje możliwość, kandydat wiąże ją z konkretnym wytworem.

BeanSpec zawiera wymagania roli i interfejsów. BeanCandidate wnosi tożsamość proponowanej realizacji, pochodzenie oraz kwalifikację. R2 zachował ważną korektę wcześniejszego schematu: strzałka BeanCandidate do CompositionEngine nie jest sama w sobie dowodem natywnego interfejsu, bo kompozytor pracuje na opisie zdolności i descriptorach.

Formalizując drogę kandydata należy wskazać przejście od zweryfikowanej implementacji do pozycji katalogu. Taki krok musi zachować wersję kontraktu, digest artefaktu, zależności i wynik odbioru. Sam wpis do katalogu nie powinien materializować procesu ani wydawać credentials.

W ten sposób fabryka unika dwóch błędów: uznania samego projektu za zdolność wykonawczą i utraty historii po promocji kandydata. Wersja przyjęta do katalogu pozostaje powiązana z oryginalnym epizodem, kontrprzykładami i warunkami ponownej kwalifikacji.

Przykład analityczny · Kandydat przed rejestracją

Plik implementacji istnieje i przeszedł test, ale nie jest jeszcze przyjętą pozycją katalogu. Consumer może go analizować jako kandydata, nie jako gwarantowaną capability. Promocja wymaga osobnego powiązania specyfikacji, wersji i wyników kwalifikacji.

Ślad implementacyjny · BeanCandidate

candidate_id	str; pole wymagane
bean_id	str; pole wymagane
spec_digest	str; pole wymagane
implementation_digest	str; pole wymagane
builder_identity_digest	str; pole wymagane

Wybrane pola z bean_candidate.py, klasa w liniach 45–90; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S037, R3-S038, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Kompozycja sprawdza całość, nie tylko części

Zgodne osobno komponenty mogą stworzyć niezgodny układ.

Interfejs wyjściowy jednego Beana musi odpowiadać potrzebie drugiego. Zamknięcie zależności, budżet zasobów oraz obecność verifiera są własnościami zestawu. Dodatkowe połączenie może naruszyć założenie izolacji, choć nie zmienia żadnego pojedynczego modułu.

Dlatego polimorfizacja wymaga zastępowalności zachowania, nie wyłącznie podobnego JSON. Inny provider może mieć inną semantykę timeoutu albo inaczej traktować powtórzenie requestu. Jeżeli kontrakt tego nie ujmuje, pozornie poprawna zamiana może stworzyć podwójny skutek.

Stabilizacja powinna obejmować rodziny zgodnych implementacji i testy negatywne granic. Nie oznacza to konieczności jednego centralnego executora. Właściwym centrum jest semantyka kompozycji, a wykonanie może pozostać federacyjne i wyspecjalizowane.

Przykład analityczny · Zgodne porty, różne retry

Dwa providery przyjmują podobny JSON, ale jeden bezpiecznie deduplikuje request, a drugi powtarza skutek. Nie są zastępowalne dla tego samego kontraktu procesu. Interfejs musi obejmować zachowanie przy błędzie, a nie tylko typy normalnego wejścia i wyjścia.

Typy	Zgodność wejścia i wyjścia.
Zależności	Brak niejawnego brakującego poprzednika.
Zasoby	Budżet i wymagany profil wykonawcy.
Weryfikacja	Odrębny odbiór w odpowiedniej domenie zaufania.

Ślad implementacyjny · CompositionRequest

composition_id	str; pole wymagane
mission_id	str; pole wymagane
goal_digest	str; pole wymagane
required_capabilities	Tuple[str, ...]; pole wymagane
external_allowed_capabilities	Tuple[str, ...]; pole wymagane

Wybrane pola z bean_composition.py, klasa w liniach 49–74; całość deklaracji: 14 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S039, R3-S046. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Mosaic jako cykl organizacji pracy

Topologia robocza może powstać i zniknąć bez utraty dowodu epizodu.

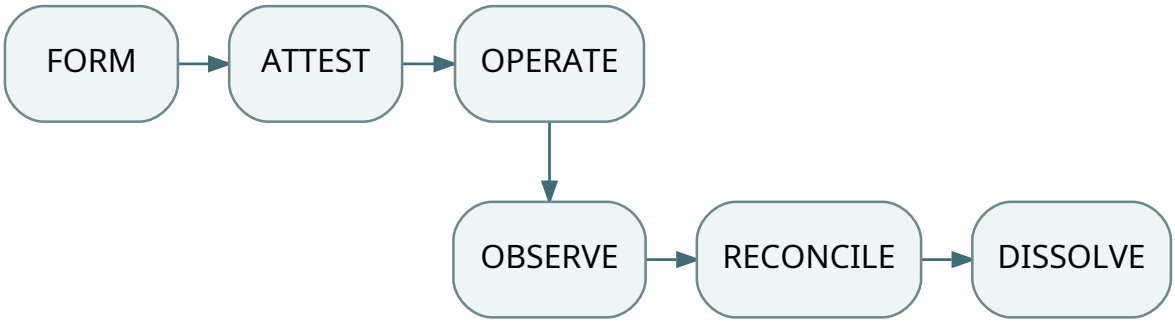
Mosaic opisuje organizację wybranych zdolności i jej kolejne fazy. FORM, ATTEST, OPERATE, OBSERVE, RECONCILE i DISSOLVE mają różne znaczenie. Rozwiązanie składu po zadaniu nie powinno usuwać lineage ani zapisanych wyników.

To naturalny mechanizm organicznego systemu: część struktury jest trwała jako kontrakt, a część efemeryczna jako kompozycja dla problemu. Nie należy utożsamiać każdej komórki Mosaic z jednym kontenerem. Materializacja zależy od konkretnego execution domain i osobnego bindingu.

W projektowanym interfejsie trzeba zatem pokazywać planowany skład, atestowane instancje, aktywne wykonanie i zachowane artefakty. Jeden licznik dronów nie rozróżnia tych stanów. Kryterium rozwiązania kompozycji musi także uwzględniać niezakończone obserwacje oraz skutki wymagające rekonsyliacji.

Przykład analityczny · Rozwiązanie roju z pending receipt

Instancje kończą pracę, lecz odbiór skutku pozostaje nierozstrzygnięty. DISSOLVE nie powinno usuwać zapisów umożliwiających późniejszą reconciliation. Efemeryczność wykonawców jest zgodna z trwałością evidence, jeśli oba cykle życia są rozdzielone.



R3-G12 · Życie kompozycji. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S055. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dwie rodziny problemów a transfer

Eksperyment B0 jest użyteczny właśnie wtedy, gdy zachowuje własne granice.

B0 pokazuje wspólną organizację syntezy dla normalizacji tekstu i progowego zliczania telemetrii. Zdefiniowana gramatyka pozwala porównywać kandydatów i sprawdzać wynik na holdout. R22F ujawnia odrzucenie rodziny danych spoza tej gramatyki.

Zmiana oczekiwanej wartości holdout sprawdza, czy niezgodny wynik zostanie zatrzymany. Nie jest samodzielnym dowodem braku zapamiętywania ani wycieku. Analogicznie trzecia nieobsługiwana rodzina nie dowodzi niemożliwości dalszego rozwoju. Takie rozróżnienie zostało już utrwalone w R1 i nie jest nowym wykonaniem B0 w R3.

Kolejny eksperyment powinien osobno zmierzyć rozszerzenie języka, jakość wyboru i transfer operatora. Kontrole muszą wskazywać, które elementy mechanizmu są współdzielone, a które dopisano dla nowego zadania. Dopiero wtedy wiadomo, co naprawdę zostało ustabilizowane.

Przykład analityczny · Trzecia rodzina po rozszerzeniu gramatyki

Dopisanie operatorów dla float-list może dać poprawny wynik, ale nie dowodzi jeszcze transferu bez zmian workflow. Raport powinien wskazać, które reguły dodano ręcznie i co rzeczywiście pozostało wspólne. Inaczej generalizacja zostanie zawyżona przez zmianę badanego mechanizmu.

B0	Dwie rodziny w ograniczonej przestrzeni syntezy.
Negatywny holdout	Kontrola niepromowania niezgodnego wyniku.
R22F	Granica deklarowanej obsługi danych.
Nowy cel	Rozdziel rozszerzenie gramatyki od ogólnego transferu.

Źródła: R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Materialny proof ma własny program

Mniejszy eksperyment nie może po cichu usuwać bramek istniejącej Factory.

Źródłowy program Application Factory obejmuje poznanie, preactivation, pełną gotowość providerów, BUILD i VERIFY. Historyczny plan R6 przewiduje wcześniejszą kwalifikację pojedynczego workera. To kompatybilne etapy budowania dowodu, ale nie identyczny kontrakt misji.

Jeżeli potrzebny jest minimalny artefakt na jednym workerze, należy zaprojektować jego dokładny program i predykaty. Nie wolno ogłosić wykonania pełnej Factory po usunięciu jej wymagania full-fleet tylko w narracji. Liczba workerów jest częścią konkretnego zakresu, nie uniwersalnym warunkiem inteligencji.

Odbiór powinien wskazywać rzeczywiste bajty, odrębnego verifiera, kontrprzykład kontrolny i drogę wyniku do wiedzy. R3 dostarcza dokumentację tych przejść, nie materializuje floty i nie przekształca historycznych zer w bazie R1 w nową obserwację dzisiejszego runtime.

Przykład analityczny · Canary a pełny program

Test jednego workera jest wartościowym poprzednikiem pełnego odbioru, lecz nie spełnia automatycznie predykatu gotowości 32 providerów. Mniejszy program powinien mieć własne completion, a nie udawać wykonania większej misji po usunięciu jej trudniejszego etapu.

```
minimalny proof -> własny kontrakt + admission + readback
pełna Factory -> wszystkie jej właściwe predykaty
nie: usunięcie bramki w opisie = wykonanie programu
```

Źródła: R3-S005, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Manifest otwiera drogę, adapter ją realizuje

Eksport semantyczny jest deklaracją możliwej współpracy, nie śladem wywołania.

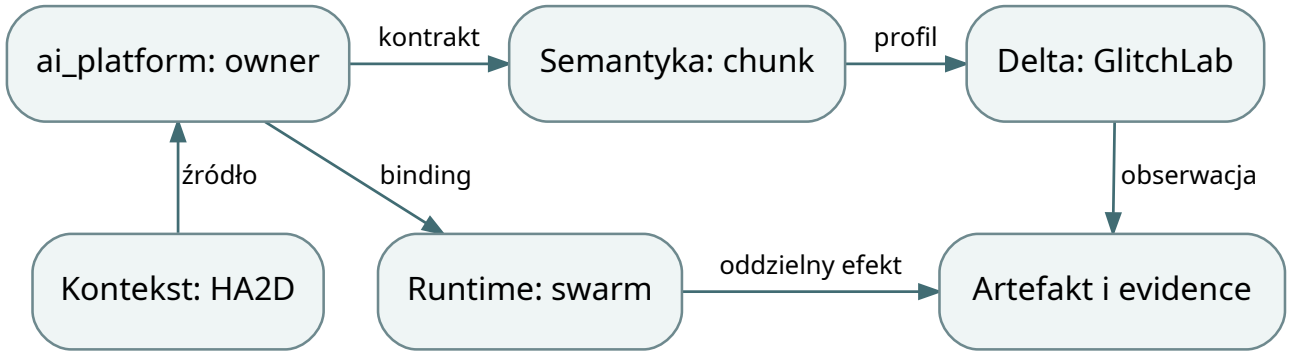
Każdy peer ma własną rolę, manifest i lokalne artefakty. Globalny model w ai_platform organizuje ich odpowiedzialności bez kopiowania całego kodu. To pozwala zachować samodzielność repozytoriów oraz wspólny język zależności.

Rzeczywista interoperacyjność wymaga jednak konkretnego producenta, payloadu, konsumenta i testu. Deklaracja semantic_imports nie dowodzi, że odpowiednia funkcja została wywołana w misji. W nowym dokumencie takie krawędzie otrzymują osobną klasę: deklaracja, źródłowy adapter, test integracji albo obserwacja runtime.

Wybór całego podgrafu zależy od problemu. Analiza zmiany kodu może używać GlitchLab i Code Perception, a nie potrzebować symulatora. Generowanie kontekstu może wykorzystywać HA2D bez uruchamiania floty. To selektywna kompozycja, nie stała kolejność dziewięciu usług.

Przykład analityczny · Eksport bez importującego consumera

Manifest peera deklaruje zdolność, ale nie odnaleziono żadnego rzeczywistego wywołania w centrali. Stan można sklasyfikować jako kontrakt lub provider źródłowy. Integracja pozostaje osobnym zadaniem. Brak consumer proof nie usuwa wartości lokalnej implementacji.



R3-G13 · Selektywna kompozycja federacyjna. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S017, R3-S023, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

chunk-chunk: semantyka procesu

Kontrakt ProcessSemanticsSpec przechowuje wersję soczewki, nie wykonuje jej za cały system.

Pełny odczyt contracts/process_semantics.py pokazuje uporządkowane osie T,S,R,E,I,F,A,P,D oraz listy bridge_ids i consumer_refs. Wymaga runtime_identifier_policy=INTERNAL_ONLY i braku authority oraz execution effect. Wiąże source_ref, source_digest i protocol_version.

Taki kontrakt nadaje się do identyfikacji semantycznego profilu użytego w projekcji. Nie dowodzi automatycznie jakości ekstrakcji ani kalibracji dziewięciu osi. Opis draftu HMK9D zawiera symbole i wartości domyślne, które powinny być odróżnione od wyników pomiaru.

W integracji z LION najlepiej zachować osobny referencjonowany obiekt. Native CommunicationEnvelope nie wymaga nowych dowolnych pól. Zmiana profilu HMK powinna wpływać na digest właściwej projekcji, a nie na identyfikatory zewnętrznego wątku czy źródło authority.

Przykład analityczny · Nowa wersja soczewki

Zmiana bridge_ids albo protocol_version powinna zmienić tożsamość semantycznego profilu. Nie należy zachowywać starego spec_digest dla wygody cache. Jednocześnie zewnętrzny thread i canonical conversation nie muszą przez to otrzymywać nowych tożsamości, jeśli kontrakt migracji zachowuje ich ciągłość.

Tożsamość	source_ref, source_digest, protocol_version.
Struktura	Dokładna kolejność dziewięciu osi i bridge IDs.
Ograniczenie	INTERNAL_ONLY; effects NONE.

Źródła: R3-S016, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

GlitchLab: obserwacja delty

Znormalizowana zmiana staje się materiałem oceny, ale nie samym dopuszczeniem.

observe_git_range rozwiązuje base i head, pobiera nazwy zmienionych plików oraz binary diff, następnie wytwarza fingerprint i wynik analizy inwariantów. DeltaObservation wiąże też process_semantics_ref i digest. To konkretny przykład wspólnej semantyki między repozytoriami.

Wynik invariant_block nie jest produkcyjnym PDP. Źródło zachowuje authority_effect i mutation_effect NONE. Wartość tego providera polega na dostarczeniu obserwacji struktury delty do dalszej decyzji. Caller musi zadbać o dokładny baseline i właściwe znaczenie progów.

Implementacja tymczasowo podmienia resolver _guess_repo_root i przywraca go w finally. To istotny szczegół przy projektowaniu współbieżnego użycia; sam finally nie stanowi dowodu izolacji równoległych wywołań. R3 odnotowuje zależność do kwalifikacji, nie ogłasza wykrytej awarii runtime.

Przykład analityczny · Analiza pustej zmiany treści

Dwa różne commity mogą mieć identyczne tree. Provider porównujący base i head nie powinien być interpretowany jako dowód materialnej zmiany kodu tylko dlatego, że commity są różne. Wynik diffu i jego zakres są odrębną informacją od genealogii.

Wejście	Repozytorium oraz dokładne base/head.
Wynik	Diff hash, fingerprint, histogram, score, block.
Kontrakt	Obserwacja nie mutuje repozytorium i nie daje authority.
Pytanie otwarte	Izolacja przy równoległych wywołaniach providera.

Źródła: R3-S010. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

swarm: granica założenia upstream

Adapter danych nie jest niezależnym wystawcą decyzji.

W odczytanym adapt_to_swarm_request budowany jest RuntimeAdmissionRequest z RuntimeIdentityBinding, RequestedRuntimeEffect i PDPResultBinding. Ten ostatni otrzymuje decision=PDPDecision.ALLOW z założenia, że upstream był już governowany. Funkcja nie przeprowadza w tym miejscu pełnej obserwacji oryginalnego źródła zgody.

To wymaga precyzyjnego opisu. Docstring o braku nadawania authority oznacza rolę adaptera, nie dowód zewnętrznej autentyczności każdej dostarczonej polityki. Integracja musi zachować producenta upstream, jego tożsamość i sprawdzenie. Nie można uznać poprawnie złożonego obiektu ALLOW za samodzielną zgodę.

Nie wykazano tutaj nieautoryzowanego efektu. Zidentyfikowano kontrakt zależności, który musi być uwzględniony w projekcie CORE. Z tego wynika argument przeciw przenoszeniu globalnego authority ownera do swarma: lokalny provider powinien konsumować związane decyzje, a nie domyślać się ich z danych.

Przykład analityczny · ALLOW z nieudokumentowanego upstream

Adapter potrafi zbudować poprawny obiekt danych z polityki wejściowej. Bez powiązania jej z oryginalnym źródłem zgody kwalifikacja pozostaje niepełna. Rozstrzygający test musi objąć konsumenta upstream evidence, a nie wyłącznie udaną konstrukcję RuntimeAdmissionRequest.

```
upstream decision evidence
-> jawny adapter tożsamości i effect request
-> lokalny admission
PDPResultBinding(ALLOW) != oryginalny grant
```

Źródła: R3-S019. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Graf, tożsamość bajtów i symulacja

mosaic, sbom i symulator odpowiadają na odmienne pytania o ten sam artefakt.

Pakiet komunikacyjny wiąże mosaic_lab_pro.py z projekcjami grafu, sbom z identyfikacją bajtów i SymulacjaKaskadySieciowej z deterministycznym fixture symulacji. Te trzy funkcje mogą wspierać jedną zmianę, ale wynik każdej ma inną klasę epistemiczną.

Graf może pokazywać zależności, lecz nie dowodzi wykonania każdej krawędzi. Identyfikator artefaktu wiąże bajty, lecz nie ocenia ich przydatności. Symulacja pozwala sprawdzić konsekwencje założeń modelu, lecz nie jest pomiarem produkcyjnego środowiska. Brak takiego rozdzielenia prowadziłby do sztucznego wzmocnienia dowodu.

W R3 te role są rekonstruowane z załączonego modelu oraz dostępnych manifestów. Nie odtworzono ponownie wszystkich implementacji i testów peerów. Projekt interoperacyjności pozostawia dla nich osobne karty odbioru, zamiast przenosić jeden zielony wynik z centrali na całą federację.

Przykład analityczny · Symulowana topologia

Graf przedstawia działanie planowanej floty na syntetycznym scenariuszu. Jest to evidence o modelu, nie o uruchomionych kontenerach. Tożsamość seed, parametrów i założeń powinna pozostać w źródłach, aby późniejsza wizualizacja nie nadała symulacji statusu obserwacji świata.

mosaic	Struktura i projekcja.
sbom	Identyfikacja i pochodzenie artefaktu.
Symulator	Konsekwencje jawnych założeń modelu.
Runtime	Odrębna obserwacja faktycznego skutku.

Źródła: R3-S001, R3-S023. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

ChangeSet jako bezpieczna transformacja federacji

Kolejność migracji ma zachowywać pośrednie stany kompatybilności.

Protokół generowania opisuje cross-repository ChangeSet z deltami, kolejnością zależności, compatibility window, rollout i rollback. To właściwy model dla rozwoju federacyjnego: wspólna zmiana nie musi oznaczać równoczesnego przepisania wszystkich repozytoriów.

Najpierw powinien powstać uzgodniony kontrakt oraz ewentualny adapter zgodności, potem provider, następnie consumer i obserwacja działania. Przejście dual-read albo dual-write wymaga własnego uzasadnienia, szczególnie gdy zapis ma skutek nieidempotentny. Nie jest uniwersalnym poleceniem do stosowania przy każdej migracji.

W dokumentacji każdy peer otrzymuje dyspozycję UPDATE, VALIDATE_ONLY albo NOT_APPLICABLE w związanym zakresie formalizacji. Nie należy dopisywać zmian do repozytorium, którego rola nie jest dotknięta. R3 zachowuje ten kierunek jako plan, bez publikacji ChangeSet.

Przykład analityczny · Migracja z oknem zgodności

Provider publikuje nową wersję, a consumer nadal korzysta ze starej. Właściwy ChangeSet może utrzymać adapter przez określony czas. Przed wycofaniem starej ścieżki potrzebny jest dowód, że zależni konsumenci zakończyli migrację. Sam nowy release nie rozstrzyga tej kwestii.

Ślad implementacyjny · RepositoryFormalizationDisposition

repository	str; pole wymagane
baseline_head	str; pole wymagane
baseline_tree	str; pole wymagane
disposition	str; pole wymagane
required_artifact_ids	Tuple[str, ...]; pole wymagane

Wybrane pola z federated_formalization_binding.py, klasa w liniach 30–58; całość deklaracji: 5 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S009, R3-S048. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Koperta nie jest przesyłką

CommunicationEnvelope opisuje tożsamość i pochodzenie, a nie zakończone delivery.

W natywnym kontrakcie payload bytes pozostają na zewnątrz, wskazane przez payload_ref i payload_digest. Koperta ma własny envelope_digest, sender, recipient set, correlation i opcjonalne powiązania conversation oraz mission. To wystarcza do sprawdzenia integralności reprezentacji, nie do odczytania jej treści z dowolnego store.

R3-T10 pokazuje, że poprawnie sformatowana, lecz nierozwiązana referencja może przejść validate. Funkcja nie wykonuje pobierania. Właściwy adapter musi sprawdzić rzeczywiste bajty oraz związek z odbiorcą. Jeżeli tylko przepisze ref do innej kolejki, granica dowodowa pozostaje otwarta.

Zaletą takiej architektury jest rozdzielenie semantyki od transportu. Ten sam rodzaj wiadomości może być przenoszony różnymi dopuszczonymi mechanizmami, lecz konkretne żądanie nadal zachowuje narzucony transport i własną tożsamość. Wymiana medium wymaga jawnej decyzji, nie cichego fallbacku.

Przykład analityczny · Koperta wskazuje brakujący payload

Próba R3-T10 zachowuje poprawną strukturę, lecz nie odczytuje referenta. Odbiór komunikacji musi więc osobno rozwiązać payload i porównać hash. Nie wolno uznać wywołania sealed za dowód, że właściwa treść została wysłana lub w ogóle istniała w magazynie.

Ślad implementacyjny · CommunicationEnvelope

schema_id	str; pole wymagane
envelope_id	str; pole wymagane
sender_ref	str; pole wymagane
recipient_refs	tuple[str, ...]; pole wymagane
conversation_ref	str | None; pole wymagane

Wybrane pola z communication_envelope.py, klasa w liniach 115–218; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S043, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Ścisły schemat wymusza kompozycję

Opcjonalna holomozaika nie może naruszać istniejącej koperty przez dopisanie przypadkowych kluczy.

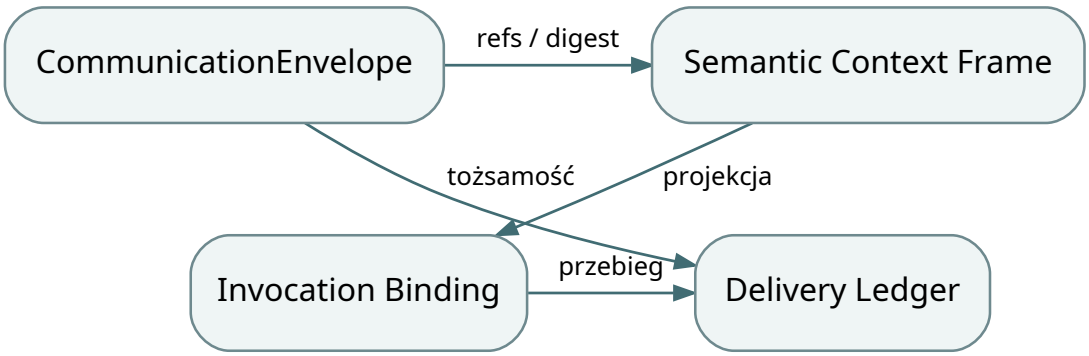
communication_envelope_from_mapping wymaga dokładnego zbioru pól. Nowa próba R3-T04 dodaje hmk9d do poprawnego mappingu i otrzymuje odrzucenie. Jest to zgodne z rolą natywnego kontraktu: rozszerzenia semantyczne mają być odrębnymi obiektami albo wersjonowaną zmianą schematu.

Projekt holomozaikowy powinien więc wiązać CommunicationEnvelope, profil semantyczny, routing invocation i ledger. Każdy z nich zachowuje własny digest. Dzięki temu można zmienić soczewkę interpretacji bez udawania zmiany zewnętrznego thread ID albo przyznania nowej authority.

Taki podział zapobiega także mieszaniu dojrzałości. Natywna koperta może być przetestowana, podczas gdy ekstrakcja HMK pozostaje eksperymentalna. Wspólne pudełko JSON z jednym SUCCESS ukryłoby tę różnicę i utrudniło lokalną stabilizację.

Przykład analityczny · Nowe pole semantyczne

Dopisanie hmk9d do natywnej koperty łamie exact-field parser. Właściwy kierunek to osobny profil połączony referencją albo jawna nowa wersja kontraktu. Nie należy osłabiać parsera do przyjmowania dowolnych kluczy tylko po to, by pojedynczy eksperyment zaczął przechodzić.



R3-G14 · Cztery obiekty komunikacji. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S043, R3-S002, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Rozmowa bez misji ma sens architektoniczny

Konsola diagnostyczna nie powinna wymagać fikcyjnej aktywnej misji.

CommunicationEnvelope dopuszcza conversation_ref i mission_ref jako opcjonalne. R3-T01 odtworzył przypadek rozmowy bez misji. Jednocześnie MissionIntent w swoim natywnym kontrakcie posiada wymagany mission_ref. To nie jest powód do podstawienia sztucznego identyfikatora, lecz do jawnego rozdzielenia rozmowy i projektu misji.

Pakiet komunikacji proponuje rolę CONSOLE_OBSERVE dla pytań o świat i źródła. Taka rozmowa może być audytowalna, źródłowo związana i ograniczona do odczytu. Dopiero kiedy powstaje projekt procesu, należy utworzyć właściwą intencję i semantyczny program.

W docelowym modelu trzeba wskazać adapter przejścia pomiędzy tymi stanami. Nie należy deklarować, że cały łańcuch MissionIntent jest już używany dla każdej wypowiedzi ad hoc. Referencja nullable w kopercie i required w innym kontrakcie oznacza różnicę domen, którą trzeba zachować.

Przykład analityczny · Konsola diagnostyczna

Operator pyta o różnicę źródeł bez żądania mutacji. Rozmowa może mieć poprawną kopertę z mission_ref=None. Dopiero projekt misji tworzy właściwe wymagane powiązanie MissionIntent. Fikcyjny identyfikator misji zacierałby tę granicę i utrudniał późniejszy audyt.

```
conversation exists; mission may be absent
projekt misji -> MissionIntent z poprawnym mission_ref
nie: mission_id fikcyjne tylko dla zgodności parsera
```

Ślad implementacyjny · CommunicationEnvelope

schema_id	str; pole wymagane
envelope_id	str; pole wymagane
sender_ref	str; pole wymagane
recipient_refs	tuple[str, ...]; pole wymagane
conversation_ref	str | None; pole wymagane

Wybrane pola z communication_envelope.py, klasa w liniach 115–218; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S043, R3-S053, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Causation i correlation nie są synonimami

Identyfikatory porządkują zdarzenia, ale ich obecność nie dowodzi przyczyny.

correlation_ref łączy rekordy należące do wspólnego przebiegu. causation_ref wskazuje bezpośredniego poprzednika w deklarowanej genealogii. Koperta odrzuca self-causation, co odtworzono w R3-T08. Nie sprawdza jednak globalnie wszystkich możliwych cykli czy prawdziwości odwołania.

W delivery ledger powiązanie odpowiedzi powinno zachować conversation, epokę, lane, request i correlation. Inaczej opóźniony receipt może zostać przypisany do nowej rozmowy albo kolejnej generacji. Sam identyczny tekst pytania nie jest kluczem deduplikacji.

Dla analizy naukowej związek przyczynowy wymaga też mechanizmu i adekwatnego dowodu. Dwa rekordy następujące po sobie w logu mogą być skorelowane przez wspólny trigger. Dlatego opis grafu musi rozróżniać formalną krawędź causation od empirycznie potwierdzonej przyczyny awarii.

Przykład analityczny · Podobny tekst dwóch requestów

Dwa pytania mają identyczną treść, ale inne message_id i epoch. Odpowiedź jednego nie może zamknąć drugiego przez dopasowanie tekstu. Correlation i causation wskazują właściwy przebieg; actual payload digest uzupełnia, a nie zastępuje tę tożsamość.

Correlation	Wspólny przebieg lub grupa logiczna.
Causation	Deklarowany rodzic konkretnego zdarzenia.
Mechanizm	Źródło lub obserwacja wyjaśniająca przejście.

Ślad implementacyjny · CommunicationEnvelope

schema_id	str; pole wymagane
envelope_id	str; pole wymagane
sender_ref	str; pole wymagane
recipient_refs	tuple[str, ...]; pole wymagane
conversation_ref	str | None; pole wymagane

Wybrane pola z communication_envelope.py, klasa w liniach 115–218; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S043, R3-S007, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Adresat, participant i provider

Kto otrzymuje zadanie nie musi być tym samym bytem co silnik inferencji.

Pakiet komunikacji wyraźnie rozdziela logical drone, material worker i model provider. Participant może utrzymywać rolę w misji, podczas gdy kolejne wywołania korzystają z tego samego współdzielonego modelu. Z drugiej strony ta sama rola może otrzymać inną dopuszczoną realizację poznawczą.

Identity rozmowy nie powinna więc pochodzić z nazwy modelu. Provider_session_ref opisuje konkretne wiązanie usługi; binding_epoch wskazuje wersję relacji. Zmiana session nie musi tworzyć nowego celu misji, ale wymaga zachowania historii i oceny ciągłości kontekstu.

W przyszłym panelu trzeba pokazać zarówno rolę uczestnika, jak i rzeczywisty backend. Licznik 32 workerów nie oznacza 32 niezależnych modeli lub domen dowodowych. Tożsamości są polimorficzne tylko w granicach jawnego kontraktu, nie przez arbitralną zamianę etykiet.

Przykład analityczny · Jedna rola, zmieniony backend

Participant recenzenta zachowuje cel, ale nowy model ma inny tokenizer i tool schema. Potrzebna jest jawna zmiana bindingu i aktualizacja model_release_ref. Nie należy traktować drugiego wykonania jako identycznego eksperymentu tylko dlatego, że rola w panelu ma tę samą nazwę.

Ślad implementacyjny · CognitiveSessionBinding

binding_id	str; pole wymagane
scope_type	str; pole wymagane
mission_id	str | None; pole wymagane
thread_id	str; pole wymagane
endpoint_ref	str; pole wymagane

Wybrane pola z cognitive_invocation.py, klasa w liniach 42–54; całość deklaracji: 13 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S042, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Fanout zamraża zbiór odbiorców

Zmiana składu roju nie może retroaktywnie zmienić adresatów wcześniejszej wiadomości.

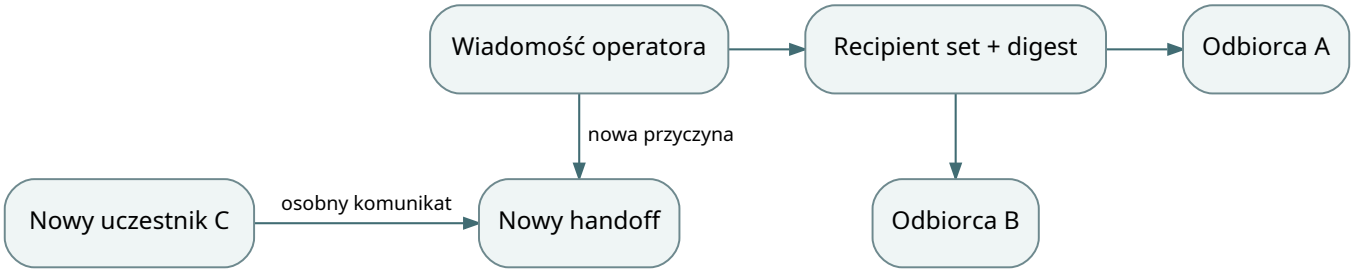
Pakiet opisuje operator_messages i operator_message_deliveries z fanout_id oraz recipient_set_digest. Ich rolą jest utrwalenie, do kogo skierowano konkretny komunikat. Później dołączony worker nie powinien automatycznie stać się odbiorcą starego polecenia.

Różnica między broadcastem do obecnego składu a wiadomością skierowaną do konkretnego zestawu jest ważna dla replay i audytu. Nowa konfiguracja może wymagać osobnego handoff albo synchronizacji wiedzy. Nie wolno ukrywać takiej operacji jako ponownego odczytu starego fanoutu.

W R3 nie wykonano operacyjnego testu fanoutu. Opis pozostaje wyprowadzony z pakietu komunikacyjnego. Natomiast testy koperty rzeczywiście potwierdzają canonical sort i odrzucenie duplikatów, co jest komplementarną kontrolą lokalnej reprezentacji odbiorców, nie pełnym dowodem delivery.

Przykład analityczny · Worker dołączony po fanout

Nowy MD nie był w zamrożonym recipient set. Nie powinien otrzymać starego polecenia automatycznie pod tym samym fanout_id. Wymagany handoff tworzy odrębny komunikat z własną przyczyną i zakresem. Deduplikacja nie może maskować tej nowej operacji.



R3-G15 · Zamrożony zbiór adresatów. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S002, R3-S043, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

HMK-9D nie jest nowym protokołem sieciowym

Soczewka semantyczna działa przed transportem i musi zachować tożsamość jego obiektów.

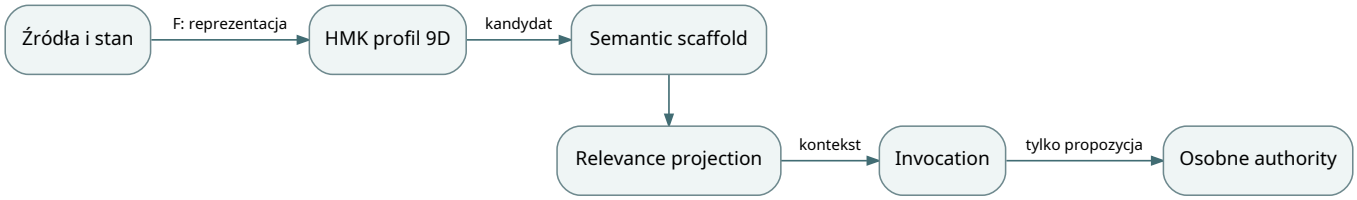
W pakiecie HMK-9D opisano jako kompresję F:S→Σ i politykę g:Σ→A. Kompozycja H(s)=g(F(s)) jest modelem reprezentacyjnym. Nie zawiera sama w sobie implementacji HTTP, MCP, SQLite ani canonical broker receipt. Natywny ProcessSemanticsSpec zachowuje effects NONE.

Właściwe miejsce tej soczewki jest przy organizacji kontekstu i rozpoznaniu struktury problemu. Może pomóc określić skalę, role, opóźnienie lub stopień domknięcia propozycji. Każda uzyskana cecha wymaga jednak definicji ekstrakcji i zakresu interpretacji.

Nie należy przypisywać soczewce fizycznej gwarancji kontroli agenta. Wysoki wynik osi D albo funkcji mandatu nie nadaje uprawnień. Authority nadal pochodzi z osobnego ownera, a skutek wymaga runtime admission i obserwacji.

Przykład analityczny · Profil semantyczny bez transportu

HMK frame może zostać poprawnie skonstruowany, lecz wiadomość nadal nie dotarła do modelu. Rysunek powinien pokazać odrębne stany reprezentacji i delivery. Zmiana jakości kompresji nie naprawia zatrzymanego consumera ani niewłaściwego bridge.



R3-G16 · Soczewka HMK i granica skutku. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S002, R3-S016. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dziewięć osi wymaga dziewięciu definicji pomiaru

Wektor ma sens tylko wtedy, gdy wiadomo, co reprezentuje każda współrzędna.

Oś T dotyczy czasu i rytmu, S sensu, R relacji, E obciążenia poznawczego, I tożsamości, F funkcji, A abstrakcji, P przewidywania, a D domknięcia decyzji. W draftowym protokole występują proponowane metryki i zakresy. To opis ontologiczny, nie zbiór empirycznie skalibrowanych sensorów.

W przyszłym eksperymencie należy osobno wskazać cechy obserwowalne i interpretacyjne. Liczba odnośników cross-module może być policzalna. „Spójność semantyczna” wymaga modelu oceny i walidacji. Zestawienie ich w jednym wektorze nie czyni wszystkich osi równie wiarygodnymi.

Normalizacja nazw powinna także rozróżnić literę F jako operator kompresji i F jako oś funkcji oraz S jako state space i S jako sens. W R3 nie zmieniamy oryginalnej notacji źródła; dodajemy nazwy kwalifikowane w widoku, aby uniknąć przypadkowej kolizji pojęć.

Przykład analityczny · E jako umowna miara

Spadek deklarowanej energii poznawczej w wektorze nie jest pomiarem zużycia energii elektrycznej. Metryka wymaga własnej definicji i kalibracji. W przeciwnym przypadku ten sam symbol łączyłby nieporównywalne wielkości i produkował pozornie fizyczne uzasadnienie decyzji.

T / S / R	czas / sens / relacja
E / I / F	obciążenie poznawcze / tożsamość / funkcja
A / P / D	abstrakcja / przewidywanie / domknięcie
Uwaga	Symbole draftu nie są skalibrowanym pomiarem ani grantem.

Źródła: R3-S002, R3-S012. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Kompresja ma zachowywać relacje krytyczne

Mniejszy payload jest użyteczny tylko wtedy, gdy nie znika warunek poprawnej decyzji.

Operator CORE_PERIPH selekcjonuje istotę problemu, a EDGE_PATIENCE reguluje granulację. To propozycje organizacji reprezentacji. Ich wartość należy sprawdzać przez zachowanie relacji, negacji, zakresu, modalności i provenance, nie tylko przez zmniejszenie liczby tokenów.

Przykładowo skrócenie „można zaprojektować zapis po odrębnej zgodzie” do „można zapisać” usuwa najważniejsze ograniczenie. Formalnie krótszy komunikat może pozostać spójny językowo, a mimo to prowadzić do innej klasy działania. Taki kontrprzykład powinien być podstawą testu adaptera.

W docelowej kompozycji kompresja powinna emitować mapę tego, co zachowano i pominięto. Nie zawsze potrzebny jest pełny tekst oryginału w modelowym wejściu, ale krytyczne constraints i droga do źródła muszą pozostać rozpoznawalne. Wynik ograniczonego parsera nie może ukrywać UNKNOWN jako braku zakazu.

Przykład analityczny · Kompresja pomija warunek zgody

Przejście z „zaprojektuj zmianę” do „wykonaj zmianę” nie jest zwykłym skróceniem tekstu. Zmienia modalność i klasę skutku. Negatywny test powinien rozpoznawać właśnie takie przekształcenie, nawet gdy końcowy komunikat pozostaje płynny i semantycznie podobny.

```
oryginał: projekt zapisu, dopiero po odrębnej zgodzie
wadliwa kompresja: zapis dozwolony
zachowanie: PROPOSAL_ONLY + authority_ref_required
test: negacja / modalność / scope / cutoff / roots
```

Ślad implementacyjny · SemanticScaffoldIR

scaffold_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane

Wybrane pola z semantic_scaffold.py, klasa w liniach 33–76; całość deklaracji: 25 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S002, R3-S066, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Locus–Medium–Mandat

Adres, kanał i zakres roli muszą pozostać rozłączne nawet w jednym semantycznym widoku.

Pakiet opisuje LOCUS_MEDIUM_MANDATE jako most ustalający kto, gdzie i w jakiej funkcji uczestniczy w procesie. Jego odpowiednikiem technicznym nie jest jedno nowe pole role, lecz złożenie participant_ref, provider binding, transportu oraz ograniczeń zadania.

Konsola może analizować problem bez aktywnej misji. Recenzent może otrzymać kod do oceny, ale nie zgodę na merge. Materialny worker może być uprawniony do konkretnego zapisu, lecz nie do redefinicji celu. Wspólne słowo „agent” nie oddaje tych różnic.

Przy projektowaniu bootstrapu rozmowy należy przekazać rolę jako wersjonowaną specyfikację i powiązać ją z actual input. Nie wolno inferować mandatu z tego, że użytkownik jest zalogowany na konto albo że aktywna karta ma nazwę projektu. Tożsamość roli jest częścią requestu i jego źródeł.

Przykład analityczny · Rola z tytułu karty

Nazwa okna „security” nie nadaje modelowi prawa do operacji administracyjnych. Rola musi być związana z requestem, zadaniem i źródłami. Medium określa transport, mandat zakres pracy, a właściwa authority dopuszczenie konkretnego skutku. Są to odrębne komponenty wiązania.

Locus	Uczestnik i jego konkretne powiązanie.
Medium	Zarejestrowany transport dla requestu.
Mandat	Cel i granice zadania; nie automatyczny grant.

Ślad implementacyjny · InvocationIntent

invocation_id	str; pole wymagane
parent_request_ref	str; pole wymagane
message_ref	str; pole wymagane
payload_digest	str; pole wymagane
binding_ref	str; pole wymagane

Wybrane pola z cognitive_invocation.py, klasa w liniach 56–62; całość deklaracji: 12 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S002, R3-S042. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Cztery obiekty projektowanego złożenia

Wspólność komunikacji nie wymaga jednego monolitycznego schematu.

Pakiet proponuje złożenie canonical envelope, semantic context frame, routing/invocation binding oraz delivery/effect ledger. Każdy obiekt ma własną odpowiedzialność i lifecycle. To spójne z oddzieleniem komunikacji, poznania, authority i skutku w źródłach LION.

Wariant ten umożliwia niezależny rozwój. Profil HMK może zmienić sposób selekcji kontekstu, nie zmieniając tożsamości odbiorcy. Adapter transportu może zostać wymieniony po dopuszczonej migracji, nie zamieniając historycznych wiadomości. Ledger nadal musi rozliczać dokładne requesty i receipts.

Nazwy proponowanych stanów, takie jak RESPONSE_OBSERVED lub CANONICAL_DELIVERED, należy mapować do istniejących stanów konkretnego źródła. Nie wolno przepisać ich do natywnych enumów tylko dlatego, że dobrze porządkują diagram. R3 zachowuje je jako język projektu.

Przykład analityczny · Wymiana soczewki bez zmiany rozmowy

Nowy profil HMK zmienia projekcję, ale canonical conversation może trwać dalej. Potrzebny jest nowy projection digest i jawny checkpoint wiedzy, nie ciche nadpisanie starego payloadu. To przykład niezależnej ewolucji reprezentacji przy zachowanej tożsamości procesu.

Envelope	Semantyczna tożsamość wiadomości i payload ref.
Context frame	Treść, constraints i profil semantyczny.
Invocation binding	Odbiorca, provider, session, rzeczywiste wejście.
Ledger	Dostarczenie, odpowiedź oraz osobno effect/reconciliation.

Źródła: R3-S002, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

LCCP jest nazwą agregatu logicznego

Porządek dokumentacji nie powinien dublować aktywnych ownerów systemu.

Załącznik proponuje nazwę LION Communication & Cognitive Coordination Plane. Wyraźnie zastrzega, że nie jest to wdrożony monolit, nowa kolejka ani drugi scheduler. Naturalny moduł obejmuje istniejące odpowiedzialności app_coordination, mission_control, contracts, enterprise i browser_broker.

R3 używa tej nazwy wyłącznie jako widoku kompozycji. Można wskazać porty, zależności, niezmienniki i kryteria odbioru bez przemieszczania plików. Fizyczna struktura kodu wymaga osobnej analizy importów, konfiguracji i właścicieli zapisu.

Najbardziej użyteczne następne przyrosty dotyczą rozstrzygających luk: bootstrap binding, preprepare admission, rozliczenie pojedynczego claimu i zachowanie kontekstu. Jeżeli istniejący owner potrafi je realizować po lokalnej korekcie, tworzenie nowej usługi byłoby niepotrzebnym zwiększeniem złożoności.

Przykład analityczny · Nowa nazwa nie jest nową usługą

LCCP może scalać dokumentacyjnie siedem odpowiedzialności, nie tworząc ósmego ownera zapisu. Jeżeli implementacja nowego monolitu dublowałaby kolejkę i scheduler, należałoby wykazać korzyść oraz plan migracji. Sama estetyka diagramu nie stanowi takiego uzasadnienia.

```
LCCP = widok kompozycji istniejących odpowiedzialności
nie: nowa baza, kolejka, scheduler lub authority owner
każdy brak -> konkretny istniejący consumer i predykat
```

Źródła: R3-S002, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

LOCAL: od intencji do zapisu odpowiedzi

Nawet lokalne wywołanie wymaga zachowania tożsamości i rzeczywistego wejścia.

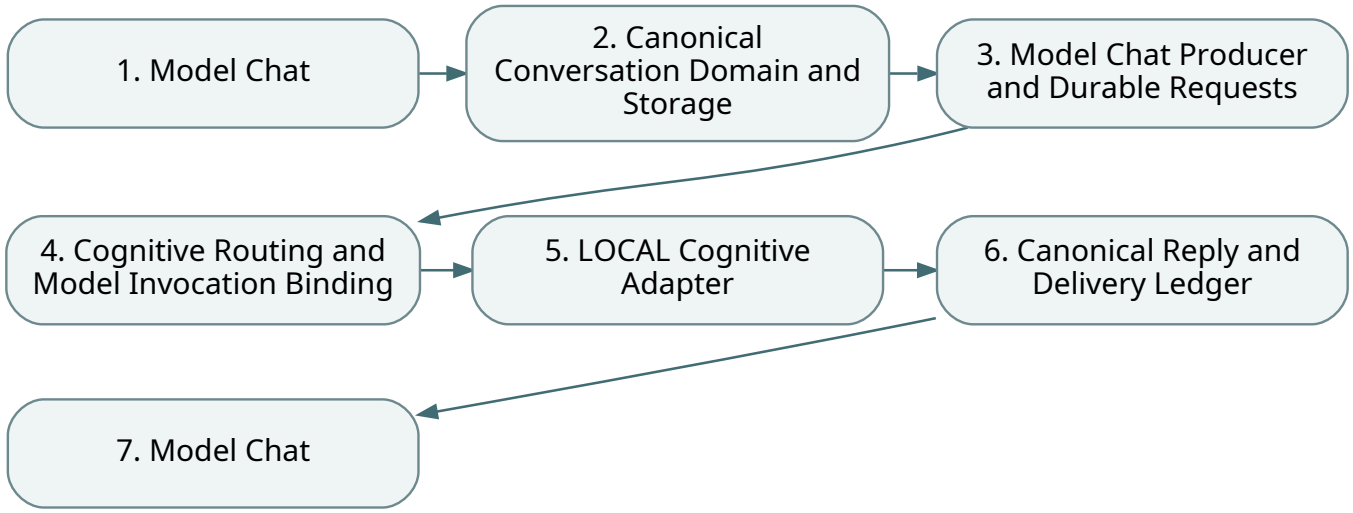
Pakiet komunikacyjny przedstawia drogę od panelu przez canonical conversation, chat producer, invocation i local model adapter do delivery ledger. Model lokalny jest backendem poznawczym. Samo istnienie procesu nie dowodzi wykonania wskazanego requestu ani wykorzystania oczekiwanego kontekstu.

Odbiór powinien związać conversation_id, binding_epoch, lane, correlation, shared_context_digest, projection_digest i actual payload bytes. Odpowiedź musi wrócić do tej samej kanonicznej epoki. W przeciwnym przypadku można otrzymać prawidłowy tekst przypisany do niewłaściwej generacji problemu.

R3 nie wykonuje inferencji. Opisuje ścieżkę wskazaną przez źródłowy model kandydata. Nowe testy reprezentacji pomagają kwalifikować jej wejścia, ale nie mogą zastąpić przyszłego pomiaru rzeczywistego roundtripu LOCAL.

Przykład analityczny · Lokalny backend zwraca niepusty tekst

Taki wynik dowodzi odpowiedzi procesu, ale nie użycia właściwego kontekstu ani przypisania do konkretnej epoki. Odbiór potrzebuje identity i actual input bytes. Test jakości treści oraz test poprawnego delivery powinny mieć oddzielne kryteria.



R3-G21 · model chat local. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S001, R3-S054. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

SaaS: ostatni adapter pozostaje szczególny

Natywna sesja przeglądarkowa nie jest zamienna z API tylko dlatego, że oba kanały zwracają tekst.

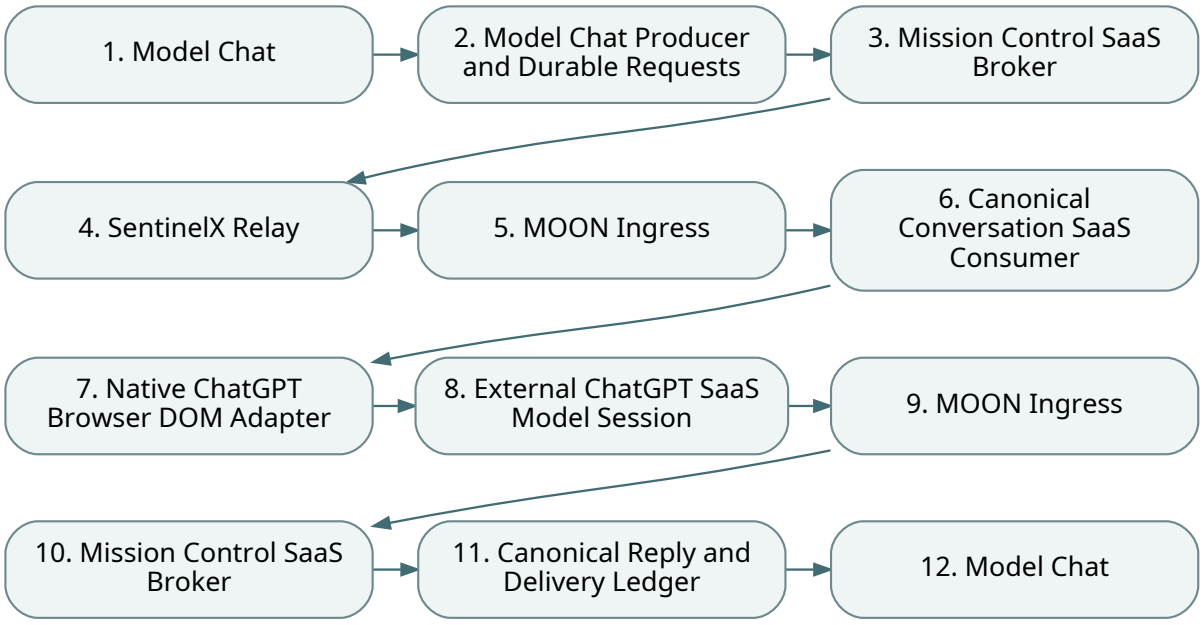
Załącznik opisuje ścieżkę przez broker, SentinelX relay, turn ingress, browser consumer i DOM dostawcy. HTML jest ostatnią powierzchnią adaptera, nie kanonicznym protokołem LION. Treść w polu tekstowym albo utworzony turn_id nie dowodzą przyjęcia wiadomości przez model.

Historyczne obserwacje pakietu mówią o dziewięciu pending requests, zatrzymanym consumerze i pięciu pytaniach PERSISTED. W R3 nie odczytywano tych aktywnych stanów ponownie. Są materiałem do projektu recovery, a nie świeżą diagnozą hosta. Żadnego z pending turns nie zakończono.

Przyszły odbiór powinien zachować dokładny bridge, zewnętrzny thread ID, potwierdzenie wysłanych bytes, broker receipt i canonical response. Wymiana transportu wymaga zgodności z żądaniem operatora. Nie wolno ogłosić powodzenia natywnego SaaS po cichym użyciu innego kanału.

Przykład analityczny · Utworzony wątek bez odpowiedzi

Zewnętrzny thread może istnieć mimo nieudanego send. Poprawny bridge nie jest końcowym dowodem roundtripu. Należy odtworzyć stan requestu i odpowiedzi, nie tworzyć kolejnej rozmowy w nadziei, że stara sama zniknie z historii.



R3-G22 · model chat saas. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S002, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

DUAL: dwa legs, nie podwojona prawda

Deterministyczne połączenie odpowiedzi wymaga osobnego rozliczenia obu przebiegów.

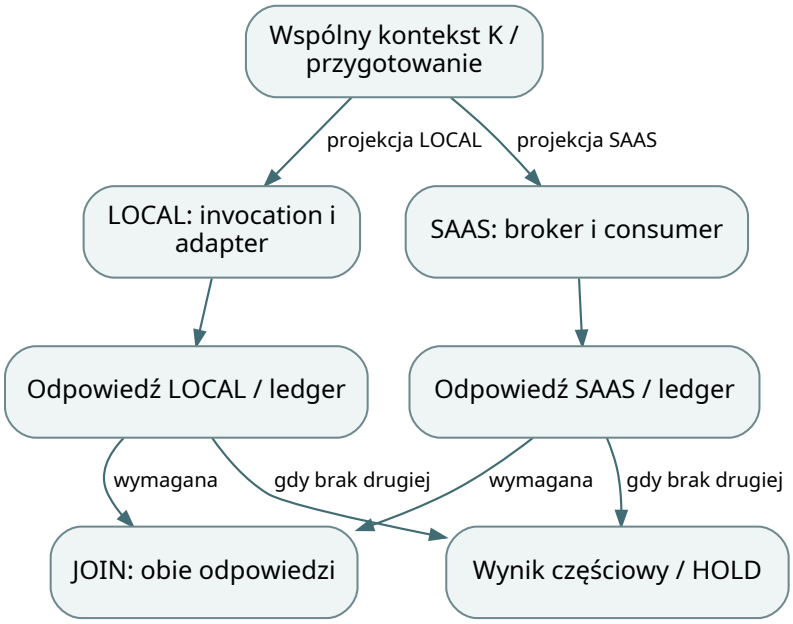
DUAL oznacza LOCAL(K) i SAAS(K) z wspólnym kontekstem logicznym K. Projekcje oraz rzeczywiście wysłane payloady mogą być różne. Wspólny digest kontekstu nie oznacza identycznych bytes, tak jak dwa providery nie oznaczają dwóch niezależnych obserwacji źródła.

Join powinien nastąpić dopiero po odnalezieniu właściwych response rows obu legs. Jeżeli jeden leg pozostaje niepewny, wynik całości powinien zachować ten fakt. Nie wolno przedstawiać odpowiedzi LOCAL jako gotowej współpracy dwóch modeli tylko dlatego, że przygotowano także request SaaS.

Przy dalszych eksperymentach należy mierzyć trzy rzeczy osobno: zgodność wejściowej wiedzy, różnice interpretacji i niezależność źródeł. Zgodność odpowiedzi przy jednym wspólnym materiale może poprawić ocenę stabilności interpretacji, ale nie tworzy dodatkowego dowodu świata.

Przykład analityczny · LOCAL gotowy, SaaS w HOLD

Join DUAL pozostaje niepełny. Panel może pokazać poprawną lokalną odpowiedź i osobną przyczynę braku drugiego legu. Nie wolno podpisywać wyniku jako uzgodnionego przez dwa modele ani ukrywać HOLD przez ciche przełączenie route na LOCAL.



R3-G23 · DUAL: dwa legs i warunek złączenia. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S001, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Preprepare admission: dwie sensowne polityki

Zatrzymany consumer nie powinien generować niekontrolowanego backlogu, ale intencja użytkownika także nie powinna znikać.

Pakiet proponuje SaaSPrepareAdmission przed trwałym przygotowaniem requestu. Celem jest uniknięcie nowych USER/PERSISTED i broker requests, gdy consumer jest zatrzymany albo istnieje nierozliczony claim. To konkretny wariant projektowy, nie już działający mechanizm.

Należy porównać go z wariantem trwałego zapisu intencji bez uruchomienia delivery. W pierwszym HOLD nie tworzy requestu; w drugim tworzy odrębny INTENT_PENDING z jasną informacją dla operatora. Te polityki mają inne gwarancje trwałości i nie mogą korzystać bez uzgodnienia z tego samego znaczenia PERSISTED.

Sam odczyt readiness przed zapisem nie usuwa wyścigu czasowego. Consumer może się zatrzymać natychmiast później. Docelowy kontrakt potrzebuje ograniczonej ważności, atomowego warunku albo ponownej oceny przez właściciela dispatch. R3 projektuje te wymagania, nie zapisuje nowego admission ani nie modyfikuje bazy.

Przykład analityczny · Readiness zmienia się po kontroli

Consumer był gotowy podczas precheck, ale zatrzymał się przed dispatch. Kontrakt musi wyjaśniać, jak długo ważna jest obserwacja i kto rozstrzyga wyścig. Preprepare bez takiego wiązania redukuje backlog, lecz nie dowodzi atomowej gwarancji dostępności.

Wariant A	HOLD: brak nowego requestu, jawna odpowiedź operatorowi.
Wariant B	Trwała intencja, osobny stan oczekiwania bez send.
Wspólny warunek	Brak udawania delivery i brak niejawnego retry.
Ryzyko	Ready-check bez bindingu nie eliminuje zmiany stanu.

Źródła: R3-S002, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Bootstrap nowego wątku jako kontrakt wejścia

Prompt transportowy get/complete nie jest pełnym dowodem wczytania roli i wiedzy.

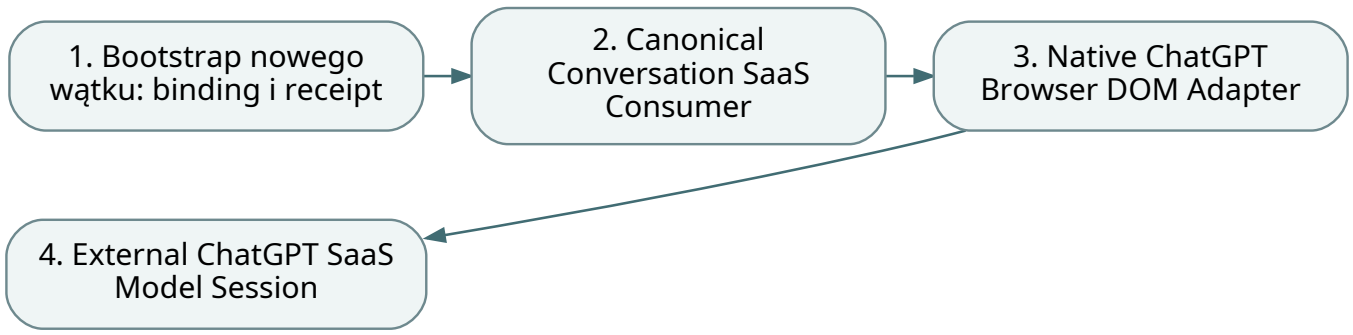
Załącznik rozpoznaje brak osobnego wersjonowanego ConversationBootstrapBinding i ThreadBootstrapReceipt. Obecny canonicalPrompt według analizy pakietu wiąże transport tury, ale nie pełne zasady roli, wydania RAG i tool fences. Są to proponowane brakujące kontrakty, nie źródłowo wdrożone moduły.

Bootstrap powinien wskazywać mały, relewantny zestaw źródeł oraz ich tożsamości. Nie wymaga dumpu całego archiwum do pierwszej wiadomości. Wymaga natomiast odróżnienia przygotowanego tekstu, potwierdzonego send i rzeczywistego pobrania kontekstu. Sama deklaracja modelu „przeczytałem” nie zastępuje śladu użytego narzędzia.

Dla istniejącego wątku odświeżenie wiedzy jest nową epoką lub jawnym checkpointem, a nie udawanym pierwszym uruchomieniem. Każda zmiana wydania lub zakresu roli wymaga oceny wpływu na kontekst i granice zadania. Uprawnienia nie wynikają z treści bootstrapu.

Przykład analityczny · Bootstrap przygotowany, lecz nieprzyjęty

W magazynie znajduje się poprawny prompt z referencją RAG. Dopóki nie ma śladu send i wejścia konsumenta, nie wiadomo, czy model poznał rolę. ThreadBootstrapReceipt powinien mieć dokładnie określony zakres: transportowy ACK i potwierdzony intake nie są identycznymi faktami.



R3-G24 · new saas thread bootstrap. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S001, R3-S006. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

SEND_UNKNOWN wymaga adjudykacji pojedynczego requestu

Niepewność wysłania nie daje prawa do ponowienia ani masowego zamknięcia kolejki.

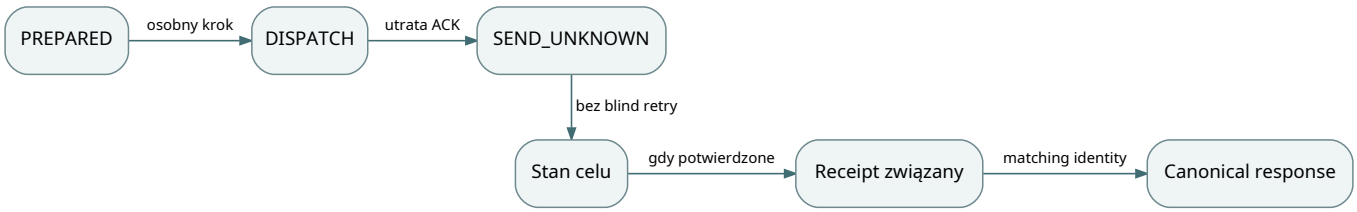
Jeżeli adapter stracił ACK po kliknięciu send, dwa wyjaśnienia pozostają możliwe: dostawca przyjął wiadomość albo jej nie otrzymał. Ślepe powtórzenie może wytworzyć duplikat. Sam timeout nie rozstrzyga, która historia zaszła.

Pakiet proponuje PerRequestClaimAdjudicator nad istniejącymi ownerami. Jego rolą byłoby powiązanie claimu, turnu, bridge, zewnętrznego śladu i canonical ledger dla jednego requestu. Decyzja o retry wymaga dowodu odpowiedniego do klasy operacji, a nie globalnego założenia, że wszystkie stare pending są martwe.

W R3 zachowujemy tę propozycję i jej historyczne przesłanki, ale nie wykonujemy żadnego completion. Analiza kolejki w przyszłości musi mieć dokładny zakres i nie naruszać cudzych turnów. Właściwy wynik może pozostać UNKNOWN, gdy źródła nie umożliwiają rozstrzygnięcia.

Przykład analityczny · Powtórzenie po niepewnym send

Ponowienie może stworzyć drugą odpowiedź albo drugi efekt. Adjudykacja dotyczy jednego requestu i wszystkich jego źródeł, a nie globalnej oceny „kolejka stara”. Brak rozstrzygnięcia jest poprawnym stanem, gdy nie ma wiarygodnego źródła potwierdzenia.



R3-G17 · Niepewność dostarczenia. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S002, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Prawo do skutku jest osobnym obiektem

Szeroka przestrzeń propozycji może współistnieć z wąską przestrzenią dopuszczonych działań.

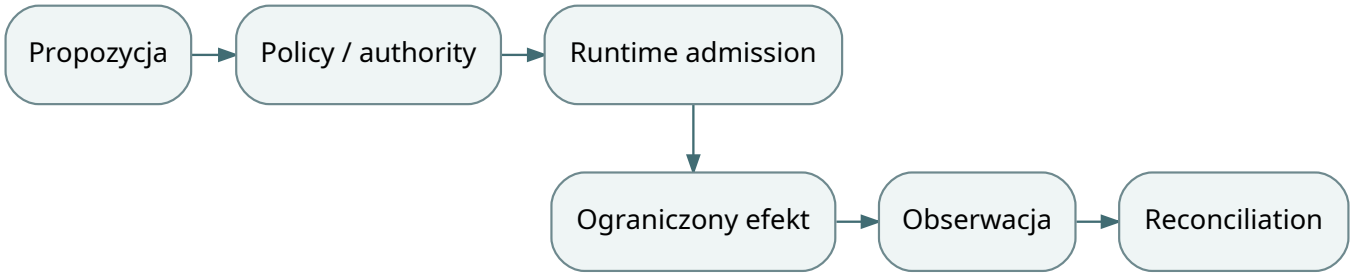
LLM może tworzyć hipotezy i kandydatów, ale nie jest źródłem production authority. Kontrakt wiadomości, scaffold i plan nie powinny przenosić uprawnienia przez samą treść. W native envelope i semantic contracts effects NONE są jawnie walidowane.

R3-T06 i R3-T07 odtworzyły odrzucenie deklaracji skutku oraz przyznania uprawnień w kopercie. To kontrola lokalnego modelu danych. Pełna ochrona zależy też od tego, czy wszystkie ścieżki materialnego wykonania przechodzą przez właściwy owner decyzji i runtime admission.

Z tej różnicy wynika charakter CORE: stabilizować należy granice i ich konsumentów, nie tylko instrukcję dla modelu. Model może proponować szerszą możliwość, ale jej materializacja wymaga aktualnego, dokładnego i odwoływalnego dopuszczenia.

Przykład analityczny · Pole authority w modelowym output

Dodanie ALLOW do tekstu lub JSON modelu nie powinno zmienić efektywnej władzy. Lokalny parser może odrzucić niedozwolone pole, lecz pełny test powinien również sprawdzić, czy późniejszy consumer nie tworzy z prose nowego grantu.



R3-G18 · Granice kontroli skutku. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S043, R3-S025, R3-S004. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Action IR materializuje konkretny zamiar

Typowana akcja ogranicza znaczenie propozycji, ale nadal nie jest permission.

PhaseExecutionContract opisuje dopuszczalną klasę wykonania. Action IR wskazuje konkretny przedmiot, parametry i scope kroku. Nie należy traktować ich zamiennie: jedno wymaganie fazy może prowadzić do kilku alternatywnych akcji, a samo powodzenie akcji nie musi spełniać celu fazy.

Właściwa ścieżka ma zachować bieżący context, identity kandydata, źródła authority i dokładny requested effect. Zmiana targetu, modelu, zakresu lub payloadu może zmienić przedmiot wcześniejszej zgody. Nie wolno przenosić dopuszczenia na następcę tylko dlatego, że jego nazwa jest podobna.

R3 opisuje tę mechanikę na podstawie źródeł i kontraktów. Nie tworzy Action IR przeznaczonego do aktywnego dispatchu. Przykłady formalne są dokumentacyjne i nie zawierają produkcyjnych credentials, nonce ani rzeczywistych pending requestów.

Przykład analityczny · Podmiana targetu po selekcji

Kandydat oceniono dla jednego zasobu, a akcja wskazuje inny. Właściwy binding musi wykryć zmianę przed admission. Podobieństwo nazwy pliku lub tożsamość modelu nie zachowują ważności poprzedniej zgody na inny przedmiot skutku.

Ślad implementacyjny · CanonicalActionIR

canonical_bytes	bytes; pole wymagane
payload_digest	str; pole wymagane

Wybrane pola z action_ir.py, klasa w liniach 286–313; całość deklaracji: 2 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S034, R3-S035, R3-S015. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Runtime admission jest zależne od instancji

Ta sama implementacja uruchomiona w innym środowisku może wymagać innej kwalifikacji.

Runtime identity obejmuje wykonawcę, sandbox, workspace, artefakt i aktualne wiązania. Zgodny kontrakt wejścia nie wystarcza, jeśli zapis trafi do innego zasobu albo proces użyje innej wersji. Admission powinno dlatego wiązać konkretną instancję z konkretnym żądaniem skutku.

Kontrola czasu ma znaczenie również po decyzji policy. ALLOW nie dowodzi, że zasób nadal jest dostępny i zgodny w momencie efektu. Currentness i replay state powinny być oceniane przez właściwych właścicieli, a nie rekonstruowane z modelowego podsumowania.

Historyczna poprawność testu admission pozostaje wartościowa, ale nie nadaje zgody na nowy eksperyment hostowy. W obecnej pracy wszystkie testy były lokalnymi próbami kontraktów. Nie wystawiano produkcyjnych grantów, nie używano key custody i nie uruchamiano workerów.

Przykład analityczny · Nowy sandbox z tym samym kodem

Zgodność implementacji nie dowodzi zgodności wykonania. Inny workspace, sieć lub mount może zmienić zakres ryzyka. Odbiór runtime powinien związać konfigurację i instancję, nie kopiować starego statusu po samym porównaniu bloba.

Ślad implementacyjny · CurrentnessSourceTrustBinding

source_id	str; pole wymagane
source_instance_id	str; pole wymagane
source_implementation_digest	str; pole wymagane
trust_anchor_id	str; pole wymagane
trust_anchor_digest	str; pole wymagane

Wybrane pola z runtime_currentness.py, klasa w liniach 35–41; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S062, R3-S061. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Niezależność verifiera ma kilka poziomów

Odrębna rola logiczna nie gwarantuje odrębnej domeny błędu.

Kompozycja może wymagać innego identyfikatora lub rodziny implementacji verifiera. To istotna ochrona przed oczywistym samopotwierdzeniem. Dwa procesy mogą jednak nadal dzielić host, kernel, model, magazyn albo źródło oczekiwanej odpowiedzi.

Raport R1 opisywał wspólny substrat WSL kilku instancji administracyjnych. R3 nie odnawia tej obserwacji, lecz zachowuje jej znaczenie metodologiczne: liczba nazw hostów nie jest liczbą niezależnych domen fizycznych. Adekwatny poziom niezależności zależy od ryzyka tezy.

W projekcie odbioru należy jawnie zadeklarować, czy wymagana jest niezależność kodu, danych, interpretacji, procesu czy infrastruktury. Nie trzeba postulować pełnej separacji fizycznej dla każdego testu lokalnego. Trzeba natomiast unikać przedstawiania słabszego profilu jako mocniejszego.

Przykład analityczny · Verifier na wspólnym źródle

Dwie implementacje sprawdzają wynik względem tego samego błędnego expected value. Mogą zgodnie dać PASS. Wymagany profil niezależności powinien uwzględniać pochodzenie referencji, nie tylko różne procesy i identyfikatory wykonawców.

Ślad implementacyjny · HostBootObservation

host_id	str; pole wymagane
hostname	str; pole wymagane
boot_id	str; pole wymagane
virtualization_class	str; pole wymagane
evidence_digest	str; pole wymagane

Wybrane pola z failure_domain_evidence.py, klasa w liniach 23–29; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S046, R3-S047, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Obserwowalność uczestniczy w kontroli

System nie powinien zachowywać pełnej mocy działania po utracie wymaganej wiedzy o skutku.

GENERATION_EVOLUTION_PROTOCOL wiąże wymaganą obserwowalność z authority. Zmiana zmniejszająca widoczność skutku wymaga odpowiedniej degradacji lub odrębnej kwalifikacji. Monitoring nie jest więc wyłącznie dekoracyjnym dashboardem.

Praktycznym przykładem jest niepewny zapis pliku albo send. Jeżeli wykonawca zgłosił wynik, ale nie ma możliwości odczytu celu, nie można bezwarunkowo przejść do kolejnej zależnej mutacji. Brak obserwacji powinien mieć jawny stan i ścieżkę rekonsyliacji.

Nie wynika z tego, że każdy brak telemetrii zatrzymuje całą federację. Blokowane są skutki zależne od wymaganej obserwacji, a niezależna analiza może trwać. Taki model pozwala utrzymać postęp bez udawania globalnego bezpieczeństwa lub globalnej martwoty.

Przykład analityczny · Utrata odczytu celu

Po zgłoszeniu zapisu observer nie może odczytać pliku. Stan zależnej fazy pozostaje niepewny i nie uzasadnia kolejnej mutacji. Niezależne badanie dokumentacji może trwać. Tak działa zakresowa degradacja, a nie globalne zatrzymanie całej federacji.

```
wymagana obserwacja utracona
-> zdegraduj zależną możliwość działania
-> zachowaj niezależne prace poznawcze
-> odtwórz źródło i rozlicz niepewny skutek
```

Ślad implementacyjny · RuntimeObserverTrustBinding

source_id	str; pole wymagane
source_instance_id	str; pole wymagane
source_implementation_digest	str; pole wymagane
trust_anchor_id	str; pole wymagane
trust_anchor_digest	str; pole wymagane

Wybrane pola z runtime_reconciliation.py, klasa w liniach 35–41; całość deklaracji: 6 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S009, R3-S063. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Safety i liveness są różnymi pytaniami

Poprawna odmowa może chronić system i jednocześnie ujawniać brak drogi do realizacji celu.

Fail-closed chroni przed wykonaniem bez wystarczającego dowodu. Nie gwarantuje, że system kiedykolwiek osiągnie cel. Jeśli każda faza oczekuje na capability, której nikt nie buduje, bezpieczeństwo odmowy nie rozwiązuje problemu użyteczności.

Właściwa architektura powinna rozróżniać brak zgody od braku implementacji i braku informacji. Dla każdej klasy istnieje inny następny krok: odrębne dopuszczenie, budowa adaptera albo pozyskanie źródła. Nowy moduł nie może dziedziczyć authority tylko dlatego, że ma usunąć blokadę.

W planie transformacji należy więc zapisywać zarówno niezmienniki safety, jak i warunki zakończenia oraz właściciela postępu. Uzasadniony WAIT ma rozpoznany predykat; nierozstrzygnięta możliwość badawcza pozostaje UNKNOWN; jawny brak drogi staje się kandydatem zmiany architektury.

Przykład analityczny · Bezpieczna pętla bez postępu

Każdy tick poprawnie odmawia wykonania z powodu brakującej capability, ale żaden proces jej nie tworzy. Safety może być zachowane przy braku liveness. Następny plan powinien wskazać konkretny przyrost implementacyjny, nie tylko częstszy monitoring tego samego WAIT.

Safety	Niedopuszczalny skutek nie zostaje wykonany.
Liveness	Istnieje realizowalna droga do zakończenia zakresu.
Diagnoza	Informacja / implementacja / zasób / uprawnienie.

Źródła: R3-S015, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Najpierw nazwać znaczenie, potem zmieniać ścieżkę

Normalizacja jest poprawką modelu odpowiedzialności, a nie kosmetyką katalogów.

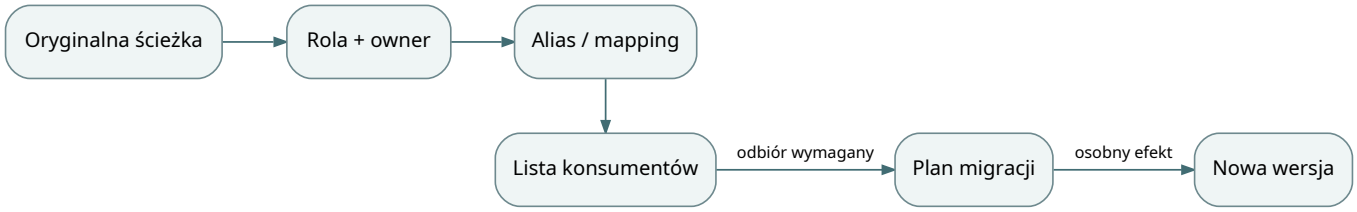
Historyczne nazwy Cyber-Lion i ai_platform współistnieją z LION jako nazwą systemu. Kod nadal wykorzystuje przestrzeń cyber_lion. Zmiana etykiety marketingowej nie uzasadnia masowego rename importów, kontraktów i nośników aktualności.

W pierwszym kroku warto wprowadzić kanoniczną nazwę pojęcia, aliasy historyczne i dokładnych ownerów. Można wtedy uporządkować dokumentację bez naruszenia działań produktu. Fizyczna migracja powinna wynikać z realnego kosztu utrzymania i mieć listę konsumentów, test oraz rollback.

R3 zachowuje oryginalne nazwy typów i stanów. Nowe nazwy, np. profil stable danej epoki albo LCCP jako agregat, są jawnie dokumentacyjne. Dzięki temu przyszły generator nie spróbuje importować klasy, która istnieje wyłącznie na rysunku.

Przykład analityczny · Alias dokumentacyjny a import

Zmiana nazwy roli w atlasie nie powoduje zmiany ścieżki Python. Jeśli później następuje rename, trzeba uwzględnić również konfiguracje, refleksję i testy. Mapowanie aliasów pozwala zachować historię, ale nie zastępuje technicznej migracji zależnych konsumentów.



R3-G19 · Normalizacja bez utraty lineage. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S017, R3-S030, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Dojrzałość nie jest pojedynczą osią

Źródło, kontrakt, integracja i deployment mogą mieć różny stan.

LION_STATUS_MODEL rozdziela kilka wymiarów oceny. Jest to odpowiedź na częsty błąd: przejście testu źródłowego zostaje przedstawione jako produkcyjna gotowość. W rzeczywistości test mógł dotyczyć parsera, a brakujący adapter nadal uniemożliwia użycie wyniku.

Dla każdej sekcji CORE proponujemy kartę z wersją kontraktu, źródłem implementacji, zakresem testów, stanem konsumentów i osobną kwalifikacją runtime. Pozwala to stabilizować realny dorobek bez czekania na globalne zakończenie projektu.

Nowy kontrprzykład aktualizuje właściwy wymiar. Błąd lineage w prototypie nie musi unieważniać wszystkich jego testów gramatyki. Może natomiast blokować użycie wyników jako źródłowo związanej wiedzy. Taka kwalifikacja jest dokładniejsza niż całkowite PASS albo FAIL modułu.

Przykład analityczny · Kontrakt stabilny, provider kandydacki

Można oprzeć nowe testy na przyjętym formacie, pozostawiając materialne wykonanie w stanie nieodebranym. Te dwa statusy nie są sprzeczne. Dokumentacja powinna umożliwić ich jednoczesny odczyt zamiast wymuszać jedną etykietę dojrzałości całego modułu.

Kontrakt	Czy znaczenie i ograniczenia są jednoznaczne?
Implementacja	Czy kod realizuje zbadany zakres?
Integracja	Czy właściwy consumer rzeczywiście korzysta z wyniku?
Deployment	Czy konkretna instancja została odebrana?

Źródła: R3-S032, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

CORE jako zachowane rozróżnienia

Najtrwalszą częścią architektury są granice, które przetrwały kontrprzykłady.

Źródła stale rozdzielają proposal, authority, admission, effect, observation i reconciliation. Podobnie zachowują conversation, mission, provider i worker jako różne tożsamości. Te rozróżnienia nadają się do formalizacji wspólnego CORE, ponieważ obejmują wiele realizacji procesu.

Nie oznacza to, że całe katalogi contracts lub enterprise są już produkcyjnie stabilne. Każdy typ potrzebuje własnej kwalifikacji oraz sprawdzenia rzeczywistych rodziców i konsumentów. Nowe próby R3 pokazały różnicę między lokalnym kształtem a relational closure.

Promocja do CORE powinna zachować źródło problemu, konkretny falsyfikator, semantykę interfejsu i warunki wersjonowania. Dzięki temu następna generacja może wymienić implementację bez ponownego odkrywania wszystkich wcześniejszych błędów.

Przykład analityczny · Nowy typ bez regresji

Publiczny schemat został dodany, ale nie ma testu błędnego rodzica ani zakresu konsumenta. To kandydat do CORE, nie domknięta podstawa. Wartość abstrakcji wynika z tego, jakie rozróżnienie zachowuje podczas następnej zmiany, a nie z samej ogólności nazwy.

```
CORE = znaczenie + niezmiennik + owner + consumer
+ odtwarzalny test + pochodzenie + reguła rewizji
nie: katalog zawierający każdy ważnie nazwany plik
```

Ślad implementacyjny · CommunicationEnvelope

schema_id	str; pole wymagane
envelope_id	str; pole wymagane
sender_ref	str; pole wymagane
recipient_refs	tuple[str, ...]; pole wymagane
conversation_ref	str | None; pole wymagane

Wybrane pola z communication_envelope.py, klasa w liniach 115–218; całość deklaracji: 16 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S004, R3-S043, R3-S066, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Struktura katalogów jako widok funkcji i dojrzałości

Jeden folder może zawierać normy, kandydatów i historyczne evidence.

Rozpoznany układ ai_platform rozdziela LION jako dokumentację, standardy i wiedzę; cyber_lion jako kontrakty i implementacje; browser_broker jako adapter desktop/SaaS; tools jako entrypointy i generatory. Taki podział nie pokrywa się jeden do jednego z horyzontami przyszłości, teraźniejszości i przeszłości.

Katalog v1_4 może zawierać obowiązujący kontrakt procesu, choć część jego nośników current_state ma już rolę historyczną. Nowy katalog v1_5 może przechowywać dossier dawnego kandydata. Klasyfikacja musi dotyczyć pliku, jego relacji i epoki, nie tylko prefiksu ścieżki.

Proponowana normalizacja może najpierw przyjąć formę indeksu kategorii i jawnych odniesień. Dopiero po zidentyfikowaniu importów i odbiorców warto projektować przenosiny. W R3 żaden plik repozytorium nie został przeniesiony.

Przykład analityczny · Starszy katalog, aktualny kontrakt

Process Contract Plane pozostaje w v1_4 i jest nadal wskazywany przez bieżący AGENTS. Automatyczne przeniesienie wszystkiego z v1_4 do archiwum zgubiłoby aktywne odniesienie. Kategorię trzeba wyznaczać przez ownera i currentness, nie wyłącznie numer katalogu.

```
LION/            wiedza, standardy, formalizacja, dossier
cyber_lion/      kontrakty, logika, proces, testy
browser_broker/  określony adapter komunikacyjny
tools/           entrypointy i generatory
status pliku jest niezależny od nazwy katalogu
```

Źródła: R3-S005, R3-S015, R3-S031. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Regresja jako pamięć granic

Test negatywny utrwala to, czego nowa implementacja nie może zgubić.

Nowy wynik badania powinien dawać konkretny przypadek, który przyszły kandydat potrafi rozliczyć. R3-T04 wymusza exact-field granicę koperty; R3-T13 sprawdza zakaz rozszerzenia projekcji; R3-T16 dokumentuje brak endpoint closure lokalnego scaffoldingu. Nie wszystkie PASS mają to samo znaczenie.

Dla naprawy R3-T16 oczekiwanie testu produktu musiałoby się zmienić: niepożądana relacja powinna zostać odrzucona albo poprawnie rozwiązana jako zewnętrzna. Historyczny test badawczy nadal pozostaje świadectwem starej implementacji. Nie należy nadpisywać jego wyniku, lecz dodać test nowej wersji.

W ten sposób stabilny rdzeń nie oznacza zatrzymania eksperymentów. Zawiera rodzinę znanych granic i warunków rozszerzenia. Nowe dane mogą zakwestionować regułę, ale wymagają jawnego dowodu i migracji, a nie cichego osłabienia assertions.

Przykład analityczny · Nowe oczekiwanie po naprawie

Badawczy test odtwarzający dawny brak walidacji może nadal dokumentować historię. Test nowej wersji powinien oczekiwać odrzucenia albo poprawnego rozwiązania referencji. Nie należy nadpisywać starego logu, aby wyglądał jak rezultat późniejszej implementacji.

Badanie	Co faktycznie robi obecna implementacja?
Regresja naprawy	Jakiego zachowania wymaga nowy kontrakt?
Historia	Który wynik należy do której wersji?

Źródła: R3-S025, R3-S008. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Eksperymentalna warstwa nad stabilną podstawą

Powrót do experimental powinien zwiększać osiągalny zakres, nie usuwać odtwarzalność.

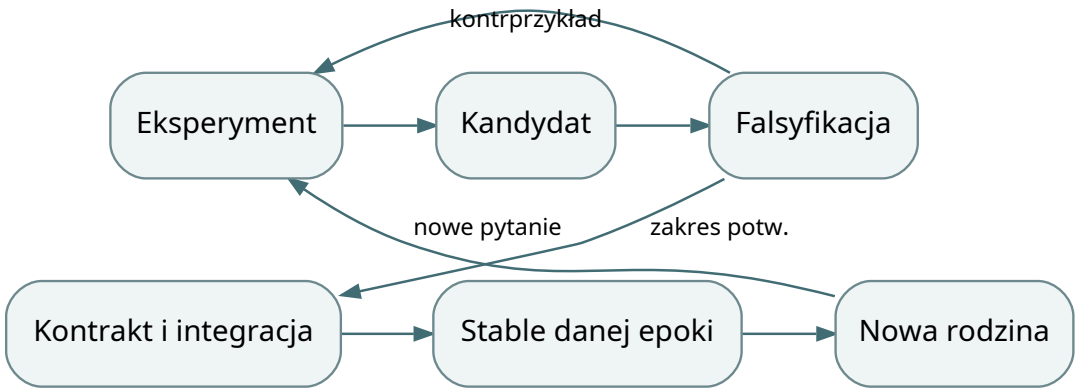
Po ustabilizowaniu właściwych kontraktów można badać nową rodzinę problemów, inną topologię kompozycji lub inny profil reprezentacji. Kandydat otrzymuje własny namespace, wersję, źródła i wynik. Nie powinien stawać się niejawnie obowiązkową zależnością startową całej aplikacji.

Zależność stable od eksperymentu wymaga szczególnej uwagi. Jeśli działa tylko nowy parser, a stary consumer zakłada jego pełną poprawność, niepewność eksperymentu przenosi się do rdzenia. Warianty powinny być jawnie wybierane przez konfigurację i kwalifikowane dla konkretnego zakresu.

W tym modelu rozwój nie jest wahaniem między chaosem i porządkiem. Jest narastającą pamięcią rozwiązań oraz otwartą przestrzenią nowych prób. Zmiana formy organizacji jest dopuszczalna tak długo, jak zachowuje tożsamość, ograniczenia i zdolność rozliczenia skutku.

Przykład analityczny · Eksperyment jako zależność startowa

Nowy parser został włączony bez wyboru wersji do podstawowego bootstrapu. Jego awaria może wtedy zatrzymać wszystkie misje. Izolacja eksperymentu wymaga jawnej konfiguracji, kontrolowanego zakresu i możliwości powrotu do poprzedniej stabilnej realizacji.



R3-G04 · Stabilizacja lokalnego zakresu. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S009, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D01–D02: źródła i natywne powiązania

Pierwsza poprawka powinna usuwać niepewność o przedmiocie dalszej pracy.

Plan R2 zaczyna od uzgodnienia źródeł i połączeń reprezentacji. To nadal uzasadnione: bez exact baseline test może dotyczyć innego kandydata niż dokument. Bez natywnych parent bindings poprawny JSON może pozostać wyłącznie szkieletem danych.

R3 wnosi dokładny odczyt wybranych kontraktów oraz nowe próby koperty i projekcji. Nie zamyka U01 pełnej dokumentacji ani U03 dynamicznych wywołań. Następny przyrost powinien wskazać wybraną krytyczną ścieżkę i udowodnić producenta oraz konsumenta każdego przejścia.

Odbiór nie powinien być deklaracją „zrobiono mapping”. Wymaga prawidłowych rodziców, zgodnych digestów, zachowanych constraints oraz negatywnego przypadku podmiany. Migracja całej struktury katalogowej nie jest warunkiem takiego pierwszego domknięcia.

Przykład analityczny · Źródła uzupełnione częściowo

Odczytano kluczowy kontrakt, ale nie jego dynamiczny resolver. Można zamknąć pytanie o shape validation i pozostawić open pytanie o referential closure. Precyzyjny checkpoint powinien zapisać oba stany; pełna dokumentacja nie wynika z ukończenia jednej karty.

D01	Dokładny source vector i rozliczone rozbieżności.
D02	Źródło → natywny obiekt → rzeczywisty consumer.
Odbiór	Brak podmiany rodzica i utraty krytycznego ograniczenia.

Ślad implementacyjny · SemanticScaffoldIR

scaffold_id	str; pole wymagane
mission_intent_ref	str; pole wymagane
mission_intent_digest	str; pole wymagane
query_plan_ref	str; pole wymagane
query_plan_digest	str; pole wymagane

Wybrane pola z semantic_scaffold.py, klasa w liniach 33–76; całość deklaracji: 25 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S003, R3-S066, R3-S065, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D03–D04: lineage prototypu i kompletność pakietu

Różne rodzaje braków wymagają różnych testów.

R1 odtworzył ograniczenia generation/provenance w prototypie SHD. Jego naprawa wymaga zmiany lokalnej walidacji i testu rzeczywistych rodziców. Nie jest tym samym zadaniem co kompletność wdrożonego pakietu Mission Control.

Dla import closure trzeba ustalić źródłowy manifest wydania i rzeczywistą ścieżkę importu. Brak modułu w jednym sprawdzonym katalogu nie daje globalnego dowodu nieobecności. Szerokie przechwytywanie wyjątku może utrudniać diagnostykę, ale należy tę hipotezę sprawdzić na dokładnym entrypoincie i środowisku.

R3 nie stosuje żadnej z napraw. Dostarcza rozróżnienie predykatów i danych potrzebnych do następnego zadania. Dzięki temu test parsera nie zostanie błędnie użyty jako dowód naprawy deployu, a health serwera jako dowód pełnej poprawności lineage.

Przykład analityczny · Dwie niezależne naprawy

Lineage prototypu i import closure Mission Control mają różne przyczyny, testy i właścicieli. Umieszczenie ich w jednej wielkiej zmianie utrudnia odbiór i rollback. Lepiej zachować skończone delty połączone tylko rzeczywistymi zależnościami.

D03	Walidacja generacji, roots i użytych źródeł.
D04	Pełny manifest wydania, import path i ekspozycja błędu.
Zakaz skrótu	Lokalny PASS nie zastępuje odbioru innej powierzchni.

Źródła: R3-S003, R3-S005. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D05–D06: authority i ciągłość rozmów

Przywrócenie komunikacji nie może odtwarzać zgody z historycznego tekstu.

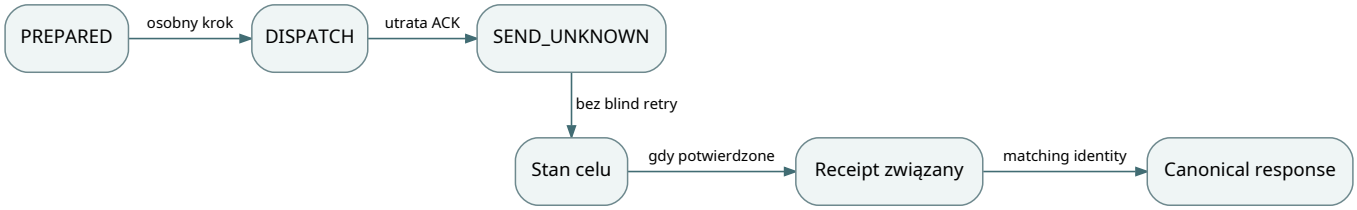
Oryginalne authority oraz replay state mają własnych ownerów. Zmiana transportu lub kandydata nie dziedziczy ważności wcześniejszego dopuszczenia. Plan musi najpierw ustalić, jakie źródło zgody dotyczy dokładnego skutku i czy nadal jest aktualne.

Ciągłość rozmów obejmuje osobno bootstrap, bridge, epokę, claim, rzeczywiste bytes i odpowiedź. Pakiet holomozaikowy dodaje konkretne propozycje preprepare oraz per-request adjudication. Są to uzupełnienia D06, a nie powód do masowego opróżnienia kolejki.

Odbiór powinien uwzględnić kontrolnie opóźniony receipt, stary binding, duplikat oraz SEND_UNKNOWN. Warto oddzielić testy offline stanu od rzeczywistej sesji SaaS, której uruchomienie wymaga odrębnego mandatu. R3 nie wykonuje takiej sesji.

Przykład analityczny · Recovery nie odnawia grantu

Uzgodnienie stanu starego requestu może wyjaśnić historię, lecz nie daje zgody na nowy send lub mutację. Adjudykacja i admission to różne przejścia. Tę granicę należy zachować również wtedy, gdy operator chce szybko przywrócić działanie systemu.



R3-G17 · Niepewność dostarczenia. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S002, R3-S004. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D07–D08: pierwszy odebrany materialny obieg

Najmniejszy użyteczny artefakt powinien rzeczywiście domykać zależność rozwoju.

Kwalifikacja workera wymaga tożsamości, izolacji, aktualnego powiązania i właściwej zdolności. Dopiero potem może powstać odrębnie dopuszczony zapis. Kontrolny artefakt nie powinien być wyłącznie sztucznym plikiem, jeśli można wytworzyć mały komponent rzeczywiście potrzebny następnemu przyrostowi.

Po BUILD niezależny w wymaganym zakresie VERIFY powinien sprawdzić exact bytes, postconditions i zgodność z zadaniem. Celowo błędny kandydat jest potrzebny do wykazania, że odbiór nie zawsze daje sukces. Następna poprawiona generacja zachowuje lineage poprzedniej.

Taki obieg nie dowodzi jeszcze uniwersalnej fabryki. Dostarcza pierwszy ograniczony proof. Druga rodzina problemów powinna później sprawdzić, czy wspólny mechanizm można ponownie zastosować bez osobnego workflow skrojonego pod przykład.

Przykład analityczny · Kontrolny błędny artefakt

Verifier powinien odrzucić plik o zmienionym bajcie albo niespełnionym postcondition. Taki test rozróżnia realny odbiór od mechanizmu zawsze zwracającego PASS. Następca zachowuje własną generację i nie usuwa dowodu pierwotnej niezgodności.

D07	Realna kwalifikacja wykonawcy w zadanym zakresie.
D08	BUILD → niezależny VERIFY → reconciliation.
Kontrola	Błędny kandydat odrzucony, następca z lineage.

Ślad implementacyjny · BeanCandidate

candidate_id	str; pole wymagane
bean_id	str; pole wymagane
spec_digest	str; pole wymagane
implementation_digest	str; pole wymagane
builder_identity_digest	str; pole wymagane

Wybrane pola z bean_candidate.py, klasa w liniach 45–90; całość deklaracji: 15 pól. Wyciąg strukturalny nie dowodzi działania konsumenta.

Źródła: R3-S003, R3-S005, R3-S038. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D09–D10: wynik wraca do wiedzy i federacji

Zakończenie zapisu jest dopiero wejściem do następnej epoki.

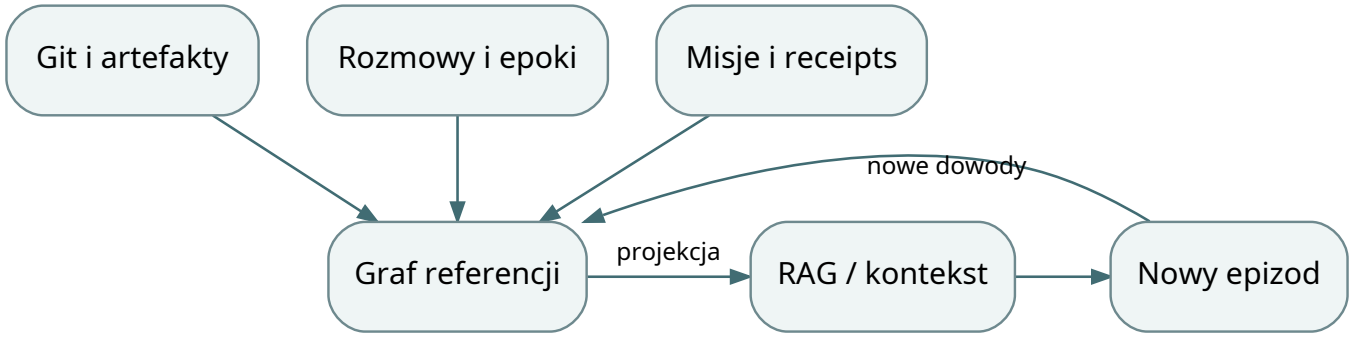
Po materialnym proof trzeba związać wynik z modelem problemu, jego ograniczeniami i zakresem ponownego użycia. Jeśli artefakt pozostanie tylko w folderze wyników, fabryka nie uzyskała jeszcze zamkniętej pętli uczenia organizacji. Potrzebny jest rzeczywisty consumer wiedzy.

Równolegle adapter federacyjny swarm powinien zostać oceniony na dokładnej granicy upstream policy i lokalnego admission. Istnienie funkcji adaptującej nie dowodzi kompletnego przekazania evidence. Należy porównać wejścia i wyjścia oraz negatywnie sprawdzić zmianę identity, scope i currentness.

Wspólny rezultat D09–D10 to zdolność do ponownego użycia artefaktu i wykonawcy w jasno ograniczonej relacji. Nie jest nim przeniesienie globalnego schedulera do nowego repozytorium ani skopiowanie wszystkich danych do jednej bazy.

Przykład analityczny · Wynik bez konsumenta pamięci

Artefakt istnieje, ale następna propozycja go nie uwzględnia. Pętla wykonania może być zamknięta, natomiast pętla uczenia organizacji nie została wykazana. Potrzebny jest ślad ponownego użycia właściwego wyniku i jego ograniczeń.



R3-G10 · Pamięć jako kompozycja. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S019, R3-S064. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

D11–D12: benchmark i stabilizowana epoka

Przewaga nowej reprezentacji wymaga porównania, a nie samej elegancji grafu.

Plan SHD zakłada porównanie istniejącej ścieżki, uporządkowanego tekstu, natywnej reprezentacji i wariantu discovery/reuse. Należy zachować porównywalną informację i budżet, oddzielić ekstrakcję od transferu oraz unikać traktowania parafraz jako niezależnych przypadków.

Stabilizacja może nastąpić dla zakresu, który przeszedł adekwatne próby i ma realnych konsumentów. Nie wymaga zakończenia wszystkich badań LION, ale nie może ukrywać krytycznej nierozstrzygniętej zależności dla przyjmowanej tezy. Deployment nadal pozostaje osobnym odbiorem.

Wydanie danej epoki powinno zawierać kontrakty, przykłady negatywne, aktualny source vector i reguły otwarcia nowego eksperymentu. Wtedy następna generacja nie zaczyna od zera, a zarazem nie jest zamknięta w granicach starej gramatyki.

Przykład analityczny · Przewaga wynikająca z większej informacji

Nowa reprezentacja otrzymała dodatkowe referencje, których nie dostał baseline tekstowy. Lepszy wynik nie izoluje wtedy efektu formatu. Benchmark powinien zachować porównywalną wiedzę i budżet, zanim przypisze przewagę samemu mechanizmowi holomozaikowemu.

```
baseline porównawczy -> test rozdzielający -> zakres efektu
-> profil stable -> warunki rewizji -> nowy eksperyment
benchmark modelowy w R3: NOT_EXECUTED
```

Źródła: R3-S003, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Panel powinien pokazywać problem, nie tylko procesy

Operator potrzebuje wiedzieć, co jest celem, co blokadą, a co już potwierdzonym rezultatem.

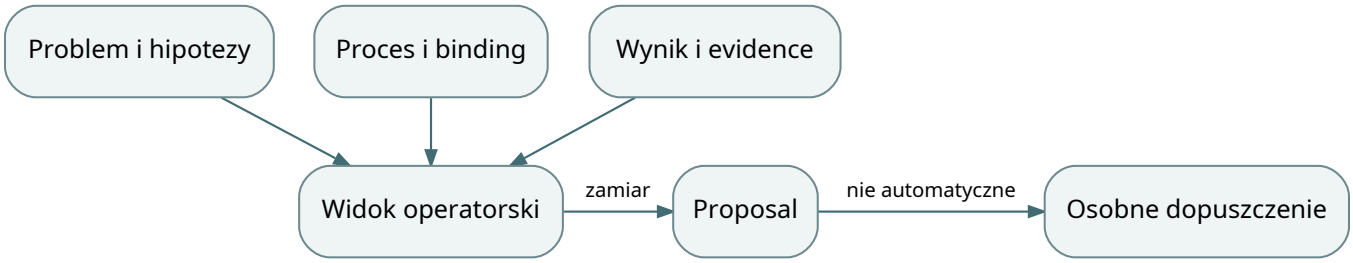
Lista uruchomionych procesów nie pokazuje, czy istnieje zarejestrowana misja ani czy jej predykaty zostały spełnione. Z kolei lista zadań bez źródeł i epok może przedstawiać stary plan jako aktualną pracę. Interfejs powinien łączyć te perspektywy bez ich utożsamiania.

W widoku problemu potrzebne są intencja, konkurencyjne hipotezy, najbliższy rozstrzygający test i aktywne ograniczenia. Widok wykonania pokazuje binding, admission, assignment i obserwowany rezultat. Widok pamięci wyjaśnia, co z wyniku zostało przyjęte do następnej epoki.

To propozycja rozwinięcia istniejących powierzchni, nie nowo wdrożony UI. Przyszły przycisk musi mieć rzeczywisty backend owner i jawne przejście stanu. Nie należy projektować zielonego przycisku „autonomia” ukrywającego kilka odrębnych dopuszczeń.

Przykład analityczny · Panel bez rozróżnienia blokad

Jeden czerwony status ukrywa, czy brakuje kodu, zasobu, wiedzy czy zgody. Każda z tych przyczyn wymaga innej reakcji. Widok powinien pokazywać minimalny blokujący podgraf oraz ownera następnego kroku, nie tylko animowany heartbeat.



R3-G20 · Panel jako projekcja trzech obiegów. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S003, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Wizualizacja jako indeks dowodów

Kliknięcie krawędzi powinno prowadzić do jej źródła i ograniczenia.

Dokumentacyjne widoki L0–L6 mają wspólne identyfikatory obiektów. Dzięki temu rysunek nie jest luźną ilustracją, lecz wejściem do konkretnego modelu. Czytelnik może odnaleźć kontrakt, źródło implementacji, próbę i nierozstrzygnięty warunek.

Każda strzałka powinna mieć klasę: deklarowana zależność, statyczne evidence, zaobserwowane przejście, inferencja albo TARGET. Linia ciągła sama nie oznacza runtime PASS. R2 już wprowadza tę granicę; R3 rozwija ją przez referencje do realnych plików i nowych testów.

W automatycznym generatorze dokumentacji warto walidować końce krawędzi, parent/refines i zgodność źródeł. Taka walidacja daje spójność modelu, nie prawdziwość każdej tezy. Dlatego testy grafu są w raporcie oddzielone od testów natywnego kodu.

Przykład analityczny · Źródło strzałki

Kliknięcie relacji prowadzi do konkretnego pliku lub obserwacji, a nie wyłącznie do innego rysunku. Jeśli dowód jest historyczny, widok pokazuje jego epokę. Projektowana krawędź nie może otrzymać statusu AS-IS przez samo dołączenie do grafu.

Obiekt	Stabilne ID, owner, źródło, wersja.
Relacja	Typ, kierunek, status i podstawa dowodowa.
Widok	Perspektywa, epoka, filtr i ukryte ograniczenia.

Źródła: R3-S003, R3-S001. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

RAG powinien przenosić ograniczenie razem z tezą

Wyrwany fragment nie może zamieniać projektu w istniejący mechanizm.

Jeżeli chunk zawiera tylko zdanie „LCCP organizuje komunikację”, może zgubić informację, że nazwa oznacza proponowany agregat logiczny. Podobnie „45 testów PASS” bez zakresu może zostać błędnie użyte jako dowód działania pełnej aplikacji. Indeksowanie wymaga więc zachowania statusu i źródła przy samym twierdzeniu.

W R3 każda nowa karta ma identyfikator, rodzica R2, źródła oraz klasę badawczo-projektową. Oryginalne strony zachowano bez przepisywania ich historycznej paginacji. Globalna nawigacja wskazuje, gdzie znajduje się dany nośnik i jaka relacja łączy go z rozwinięciem.

Przyszły test retrieval powinien sprawdzić nie tylko znalezienie słowa, lecz również poprawne odtworzenie ownera, epoki, granicy dowodu i brakującego kroku. Nowy PDF nie promuje się automatycznie do obowiązującego RAG; jego przyjęcie ma odrębnego właściciela.

Przykład analityczny · Chunk gubi słowo TARGET

Fragment RAG zawiera opis bootstrapu, lecz pomija nagłówek oznaczający go jako projekt. Agent może wtedy przypisać mu istniejącą implementację. Test retrieval powinien sprawdzać zachowanie klasy epistemicznej przy podziale dokumentu, a nie tylko trafienie nazwy komponentu.

```
chunk = teza + zakres + status + źródło + epoka
+ ograniczenie + wymagany dalszy odczyt
trafienie słowa != poprawne odtworzenie znaczenia
```

Źródła: R3-S006, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Uczciwa kompletność badania

Pełny inwentarz, pełny odczyt i pełna rekonstrukcja to różne osiągnięcia.

R3 wykorzystuje dostarczony atlas, pakiet komunikacji, historyczną kopię źródeł i wybrane ponowne odczyty GitHub. Nie zamyka wszystkich gałęzi, dokumentów, dynamicznych wywołań ani środowisk. Powiększenie PDF nie może zostać użyte jako zastępstwo dalszego badania tych obszarów.

Mianownik historycznego snapshotu wynosi 268 plików dostępnych w tym pakiecie, nie cały ekosystem. Wszystkie można zinwentaryzować mechanicznie, lecz nie należy automatycznie oznaczać ich jako pełny semantyczny odczyt. Nowe testy obejmują dokładnie trzy natywne moduły kontraktów.

Wartością wydania jest dokładniejszy model relacji i konkretnych braków. Dla tezy lokalnej można osiągnąć domknięcie przy nadal otwartym globalnym audycie. Dla deklaracji o całej produkcyjnej fabryce wymagany materiał pozostaje szerszy. Oba stany powinny być widoczne równocześnie.

Przykład analityczny · Więcej stron, ten sam frontier

Rozbudowana analiza nie zmienia liczby przeczytanych źródeł poza rzeczywistą deltą odczytu. Nie należy liczyć kart ani diagramów jako nowych obserwacji. Mianowniki dokumentacji, kodu, testów i runtime muszą pozostać niezależne.

R3 zamyka	Lokalne próby wskazanych kontraktów i integralność wydania.
R3 rozwija	Model czasu, komunikacji, fabryki i warunkowej transformacji.
R3 nie zamyka	Globalny audyt źródeł i produkcyjny E2E ekosystemu.

Źródła: R3-S023, R3-S025, R3-S003. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Pierwszy konkretny następny krok

Kontynuacja powinna uzupełniać ledger, a nie rozpoczynać kolejną ogólną rekonstrukcję.

Najpierw należy rozliczyć krytyczny podgraf reprezentacji i komunikacji, wskazując faktycznych konsumentów SemanticScaffoldIR, RelevanceProjection i bootstrapu. Nowe R3-T16 oraz R3-T10 dają konkretne pytania o referential closure i rozwiązanie payloadu.

Równolegle można kontynuować pełny odczyt dokumentacji z przypiętego drzewa, zachowując stare zakresy i nowe delty. Nie ma powodu ponownie uruchamiać niezmienionych testów załączników tylko dla zwiększenia licznika. Potrzebne są nowe dowody tam, gdzie wcześniejszy zakres był niewystarczający.

Osobna misja implementacyjna może później stworzyć minimalny adapter lub poprawkę walidacji. Musi mieć exact source binding, test negatywny i osobne dopuszczenie zapisu. Obecne wydanie nie uruchamia tej misji, nie zmienia kolejki i nie kończy cudzych requestów.

Przykład analityczny · Kontynuacja bez restartu audytu

Nowy wykonawca odczytuje checkpoint, sprawdza deltę źródeł i zaczyna od brakującego consumera. Nie powtarza całej historii dla samej orientacji. Taki przebieg wykorzystuje pamięć poprzednich epizodów, pozostawiając ich ograniczenia i statusy widoczne.

```
następny odczyt: krytyczny producer -> payload -> consumer
następny test: podmiana / brak rodzica i zachowanie UNKNOWN
następna zmiana produktu: osobny, dokładny mandat
```

Źródła: R3-S003, R3-S025, R3-S002. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

Formalizacja natury LION: wynik tej generacji

LION można opisać jako federację zmiennych realizacji nad pamięcią rozliczalnych ograniczeń.

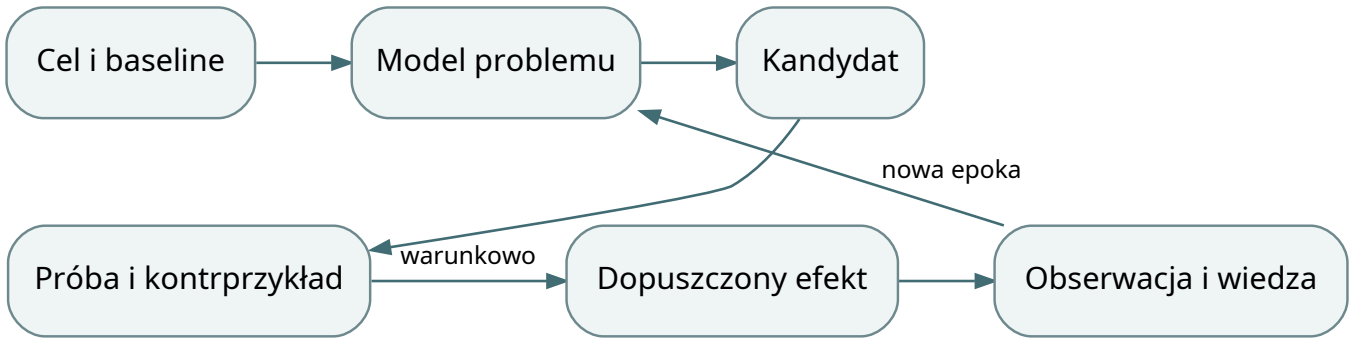
W realnych źródłach powracają te same relacje: potrzeba, reprezentacja, kandydat, próba, dopuszczenie, artefakt, obserwacja i wiedza. Nie tworzą one jeszcze udowodnionej pełnej autonomii, ale stanowią spójny materiał do formalizacji najwyższej logiki systemu.

Przyszłość jest przestrzenią wariantów i celów. Teraźniejszość rozwija model problemu oraz wytwarza dowody. Przeszłość przechowuje oryginały i kolejne interpretacje. Stable jest lokalnym profilem kwalifikacji, dzięki któremu następny eksperyment nie musi na nowo odkrywać wcześniejszych rozróżnień.

Główny wynik R3 to dokładniejsze związanie tej wykładni z plikami, kontraktami i operacyjnymi granicami komunikacji. Ostateczna formalizacja produktu wymaga dalszych konsumentów i odbiorów. Nie umniejsza to realnemu dorobkowi: system ma już język, w którym można precyzyjnie zapisać zarówno osiągnięte możliwości, jak i następną potrzebną zmianę.

Przykład analityczny · Nowa generacja z zachowanym rodzicem

R3 rozszerza R2 bez usuwania jego stron i świadectw. Analogicznie kandydat produktu powinien rozszerzać zdolności bez cichego przepisywania dawnych wyników. To praktyczna forma ciągłości: trwały ślad, nowa interpretacja i odrębnie kwalifikowany następny skutek.



R3-G01 · Epizod rozliczonej zmiany. Model źródłowo-analityczny; nie nowa obserwacja runtime.

Źródła: R3-S009, R3-S005, R3-S002, R3-S025. Pełny lokalizator i klasa dowodu: rejestr R3-S oraz model osadzony.

ANEKS / REPRODUKOWALNOŚĆ R3-A01

A01 · Tożsamość nośników i pochodzenie

To wydanie rozszerza dokładnie otrzymany egzemplarz R2, nie dowolny plik o podobnej nazwie.

Oryginalne strony są widoczne jako ciągły blok; dodatkowo osadzono oryginalne bytes R2. Wszystkie pliki archiwum komunikacyjnego zachowano bez ingerencji. Nowe modele, testy i tekst R3 są odrębnymi artefaktami pochodnymi.

Nie wywiedziono przyczyny różnicy między wcześniejszą wiadomością o 133 stronach a otrzymanym 137-stronicowym nośnikiem. Wszelkie nowe liczniki odnoszą się do aktualnie sprawdzonego załącznika. Prawdziwość tezy i integralność pliku pozostają innymi kategoriami.

R2 PDF	137 stron; 134 karty; 41 widoków w modelu
R2 SHA-256	592459765bb85962b69de2a19b8d74af04d8d352b8bc459f092f6ada680827c2
Komunikacja ZIP SHA-256	e4c8bcd56d6e82ca5409fc0b9fadb3698beb20f9c09e2556ed20c0c8ff73afd9
Snapshot źródeł	268 plików; historyczny source_transfer_small.zip
Authority / runtime	Brak zmian repozytoriów, kolejek, flot i uprawnień

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A02 · Próby R3-T01–T09

Każdy wynik dotyczy konkretnego sprawdzenia lokalnego kodu, nie pełnego procesu aplikacji.

Instrument jest zachowany jako run_evidence.py, a rezultaty jako R3_NATIVE_CONTRACT_TESTS.json. Użyto natywnych modułów ze zweryfikowanymi blobami. Poprawne i negatywne przypadki są częścią jednej jawnej serii; testy nie korzystały z aktywnych misji, produkcyjnych credentials ani inferencji.

PASS oznacza oczekiwane zachowanie w danej próbie. T10 oraz T16 dokumentują ograniczenie walidatora; nie są świadectwem usunięcia tego ograniczenia. Pozostałe wyniki nie zmieniają automatycznie ich zakresu.

R3-T01	Natywna koperta dopuszcza rozmowę bez misji. Wynik: PASS
R3-T02	Budowa kanonizuje kolejność odbiorców. Wynik: PASS
R3-T03	Duplikat adresata jest odrzucany. Wynik: PASS
R3-T04	Dowolne pole HMK dodane do koperty narusza exact-field parser. Wynik: PASS
R3-T05	Podmiana payload_digest po sealed jest wykrywana. Wynik: PASS
R3-T06	Koperta nie może deklarować skutku. Wynik: PASS
R3-T07	Koperta nie może deklarować przyznania uprawnień. Wynik: PASS
R3-T08	Self-causation jest odrzucana. Wynik: PASS
R3-T09	Roundtrip nie zmienia kanonicznych bajtów. Wynik: PASS

A03 · Próby R3-T10–T18

Każdy wynik dotyczy konkretnego sprawdzenia lokalnego kodu, nie pełnego procesu aplikacji.

Instrument jest zachowany jako run_evidence.py, a rezultaty jako R3_NATIVE_CONTRACT_TESTS.json. Użyto natywnych modułów ze zweryfikowanymi blobami. Poprawne i negatywne przypadki są częścią jednej jawnej serii; testy nie korzystały z aktywnych misji, produkcyjnych credentials ani inferencji.

PASS oznacza oczekiwane zachowanie w danej próbie. T10 oraz T16 dokumentują ograniczenie walidatora; nie są świadectwem usunięcia tego ograniczenia. Pozostałe wyniki nie zmieniają automatycznie ich zakresu.

R3-T10	Validator koperty nie pobiera payloadu: referencja o poprawnym kształcie przechodzi. Wynik: PASS
R3-T11	Dwa dowody ze wspólnym root_ref są liczone jako jeden korzeń. Wynik: PASS
R3-T12	Projekcja o poprawnym podzbiorze przechodzi validate_against. Wynik: PASS
R3-T13	Projekcja nie może dodać węzła spoza grafu. Wynik: PASS
R3-T14	Projekcja nie może poszerzyć dopuszczonych capabilities. Wynik: PASS
R3-T15	Tożsamość innego graph_digest jest odrzucana. Wynik: PASS
R3-T16	Scaffold lokalnie dopuszcza relację z końcem spoza entity_refs: closure wymaga osobnego konsumenta. Wynik: PASS
R3-T17	unknown_refs pozostają częścią payloadu i digestu. Wynik: PASS
R3-T18	Zmiana parent digest zmienia digest scaffold; nie potwierdza istnienia rodzica. Wynik: PASS

A04 · Rejestr źródeł 1/4

Każda referencja wskazuje własną epokę i zakres; historyczny snapshot nie dziedziczy currentness.

Wpisy GITHUB_PINNED_SOURCE_READ oznaczają odczyt wskazanego źródła przez GitHub; nie są dowodem deploymentu. HISTORYCZNE źródła lokalne pochodzą z załączonego pakietu. Pełne refy, bloby, hashe, zakresy i lokalizatory zapisano w osadzonym R3_SOURCE_REGISTER.json.

R3-S001	LION_GRAF_KOMUNIKACJI_I_ONTOLOGII_R2.json ref 3aaaf2e97a71 · załącznik badawczy / nie nowy runtime
R3-S002	LION_ARCHITEKTURA_KOMUNIKACJI_HOLOMOZAIKOWEJ_R2.md ref 3aaaf2e97a71 · załącznik badawczy / nie nowy runtime
R3-S003	LION_ATLAS_ARCHITEKTURY_EWOLUCYJNEJ_R2_20261014.pdf oryginał R2 / AS-IS PARTIAL
R3-S004	DonkeyJJLove/ai_platform / AGENTS.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S005	DonkeyJJLove/ai_platform / LION/architecture/v1_5/README.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S006	DonkeyJJLove/ai_platform / LION/rag/RAG_BOOTSTRAP.json ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S007	DonkeyJJLove/ai_platform / cyber_lion/app_coordination/conversation_chat.py ref 3aaaf2e97a71 · GitHub / przypięte źródło / tylko linie 1–220
R3-S008	DonkeyJJLove/ai_platform / cyber_lion/AGENTS.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S009	DonkeyJJLove/ai_platform / cyber_lion/enterprise/GENERATION_EVOLUTION_PROTOCOL.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S010	DonkeyJJLove/glitchlab / glx/federation/delta_provider.py ref 0afffd9ac6e0 · GitHub / przypięte źródło
R3-S011	DonkeyJJLove/HA2D / contracts/context_snapshot.py ref f41cd0f089b7 · GitHub / przypięte źródło
R3-S012	DonkeyJJLove/chunk-chunk / hmk9d_protocol.yaml ref b8327059360b · GitHub / przypięte źródło / ODCZYT CZĘŚCIOWY
R3-S013	DonkeyJJLove/hipotezy_nadawcze_LLM / contracts/hypothesis_spec.py ref 25b432b7a57b · GitHub / przypięte źródło
R3-S014	DonkeyJJLove/ai_platform / refs/heads/master ref d367e47cf4f7 · odczyt referencji Git
R3-S015	DonkeyJJLove/ai_platform / LION/architecture/v1_4/LION_PROCESS_CONTRACT_PLANE.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S016	DonkeyJJLove/chunk-chunk / contracts/process_semantics.py ref b8327059360b · GitHub / przypięte źródło
R3-S017	DonkeyJJLove/ai_platform / README.md ref d367e47cf4f7 · GitHub / przypięte źródło

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A05 · Rejestr źródeł 2/4

Każda referencja wskazuje własną epokę i zakres; historyczny snapshot nie dziedziczy currentness.

Wpisy GITHUB_PINNED_SOURCE_READ oznaczają odczyt wskazanego źródła przez GitHub; nie są dowodem deploymentu. HISTORYCZNE źródła lokalne pochodzą z załączonego pakietu. Pełne refy, bloby, hashe, zakresy i lokalizatory zapisano w osadzonym R3_SOURCE_REGISTER.json.

R3-S018	DonkeyJJLove/ai_platform / LION/codex/CODEX_RUNBOOK.md ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S019	DonkeyJJLove/swarm / federation/ai_platform_runtime_adapter.py ref 91b0a4174300 · GitHub / przypięte źródło
R3-S020	code_perception.py ref d367e47cf4f7 · źródło zachowane z R2
R3-S021	code_perception_git_boundary.py ref d367e47cf4f7 · źródło zachowane z R2
R3-S022	test_code_perception.py ref d367e47cf4f7 · źródło zachowane z R2
R3-S023	SOURCE_SNAPSHOT_REPORT.json historyczny inwentarz
R3-S024	40 source studies in this document synteza R3
R3-S025	R3_NATIVE_CONTRACT_TESTS.json nowy wynik prób R3
R3-S026	DonkeyJJLove/ai_platform / LION/architecture/v1_5/COGNITIVE_EVOLUTION_INTEGRATION.md ref 0e24fd481697 · snapshot historyczny
R3-S027	DonkeyJJLove/ai_platform / LION/architecture/v1_5/DOCUMENTATION_CURRENTNESS_MODEL.json ref 0e24fd481697 · snapshot historyczny
R3-S028	DonkeyJJLove/ai_platform / LION/architecture/v1_5/semantic_owners.json ref 0e24fd481697 · snapshot historyczny
R3-S029	DonkeyJJLove/ai_platform / LION/rag/RAG_BOOTSTRAP.json ref d367e47cf4f7 · lokalne bytes zgodne z pinned master
R3-S030	DonkeyJJLove/ai_platform / LION/standards/LION_NAMING_STANDARD.json ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S031	DonkeyJJLove/ai_platform / LION/standards/LION_REPOSITORY_LAYOUT_STANDARD.json ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S032	DonkeyJJLove/ai_platform / LION/standards/LION_STATUS_MODEL.json ref d367e47cf4f7 · GitHub / przypięte źródło
R3-S033	DonkeyJJLove/ai_platform / cyber-lion.repository.json ref 0e24fd481697 · snapshot historyczny
R3-S034	DonkeyJJLove/ai_platform / cyber_lion/contracts/action_ir.py ref 0e24fd481697 · snapshot historyczny

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A06 · Rejestr źródeł 3/4

Każda referencja wskazuje własną epokę i zakres; historyczny snapshot nie dziedziczy currentness.

Wpisy GITHUB_PINNED_SOURCE_READ oznaczają odczyt wskazanego źródła przez GitHub; nie są dowodem deploymentu. HISTORYCZNE źródła lokalne pochodzą z załączonego pakietu. Pełne refy, bloby, hashe, zakresy i lokalizatory zapisano w osadzonym R3_SOURCE_REGISTER.json.

R3-S035	DonkeyJJLove/ai_platform / cyber_lion/contracts/action_proposal_context.py ref 0e24fd481697 · snapshot historyczny
R3-S036	DonkeyJJLove/ai_platform / cyber_lion/contracts/architecture_formalization_manifest.py ref 0e24fd481697 · snapshot historyczny
R3-S037	DonkeyJJLove/ai_platform / cyber_lion/contracts/bean.py ref 0e24fd481697 · snapshot historyczny
R3-S038	DonkeyJJLove/ai_platform / cyber_lion/contracts/bean_candidate.py ref 0e24fd481697 · snapshot historyczny
R3-S039	DonkeyJJLove/ai_platform / cyber_lion/contracts/bean_composition.py ref 0e24fd481697 · snapshot historyczny
R3-S040	DonkeyJJLove/ai_platform / cyber_lion/contracts/capability_need.py ref 0e24fd481697 · snapshot historyczny
R3-S041	DonkeyJJLove/ai_platform / cyber_lion/contracts/code_perception.py ref 0e24fd481697 · snapshot historyczny
R3-S042	DonkeyJJLove/ai_platform / cyber_lion/contracts/cognitive_invocation.py ref 0e24fd481697 · snapshot historyczny
R3-S043	DonkeyJJLove/ai_platform / cyber_lion/contracts/communication_envelope.py ref d367e47cf4f7 · lokalne bytes zgodne z pinned master
R3-S044	DonkeyJJLove/ai_platform / cyber_lion/contracts/evidence_bound_learning_episode.py ref 0e24fd481697 · snapshot historyczny
R3-S045	DonkeyJJLove/ai_platform / cyber_lion/contracts/evolutionary_state.py ref 0e24fd481697 · snapshot historyczny
R3-S046	DonkeyJJLove/ai_platform / cyber_lion/contracts/executor_independence.py ref 0e24fd481697 · snapshot historyczny
R3-S047	DonkeyJJLove/ai_platform / cyber_lion/contracts/failure_domain_evidence.py ref 0e24fd481697 · snapshot historyczny
R3-S048	DonkeyJJLove/ai_platform / cyber_lion/contracts/federated_formalization_binding.py ref 0e24fd481697 · snapshot historyczny
R3-S049	DonkeyJJLove/ai_platform / cyber_lion/contracts/formalization_closure.py ref 0e24fd481697 · snapshot historyczny
R3-S050	DonkeyJJLove/ai_platform / cyber_lion/contracts/formalization_registry.py ref 0e24fd481697 · snapshot historyczny
R3-S051	DonkeyJJLove/ai_platform / cyber_lion/contracts/governed_change_proposal.py ref 0e24fd481697 · snapshot historyczny

A07 · Rejestr źródeł 4/4

Każda referencja wskazuje własną epokę i zakres; historyczny snapshot nie dziedziczy currentness.

Wpisy GITHUB_PINNED_SOURCE_READ oznaczają odczyt wskazanego źródła przez GitHub; nie są dowodem deploymentu. HISTORYCZNE źródła lokalne pochodzą z załączonego pakietu. Pełne refy, bloby, hashe, zakresy i lokalizatory zapisano w osadzonym R3_SOURCE_REGISTER.json.

R3-S052	DonkeyJJLove/ai_platform / cyber_lion/contracts/group_channel.py ref 0e24fd481697 · snapshot historyczny
R3-S053	DonkeyJJLove/ai_platform / cyber_lion/contracts/mission_intent.py ref 0e24fd481697 · snapshot historyczny
R3-S054	DonkeyJJLove/ai_platform / cyber_lion/contracts/model_call_v2.py ref 0e24fd481697 · snapshot historyczny
R3-S055	DonkeyJJLove/ai_platform / cyber_lion/contracts/mosaic.py ref 0e24fd481697 · snapshot historyczny
R3-S056	DonkeyJJLove/ai_platform / cyber_lion/contracts/phase_execution_contract.py ref 0e24fd481697 · snapshot historyczny
R3-S057	DonkeyJJLove/ai_platform / cyber_lion/contracts/process_ir.py ref 0e24fd481697 · snapshot historyczny
R3-S058	DonkeyJJLove/ai_platform / cyber_lion/contracts/query_plan.py ref 0e24fd481697 · snapshot historyczny
R3-S059	DonkeyJJLove/ai_platform / cyber_lion/contracts/rag_context_envelope.py ref 0e24fd481697 · snapshot historyczny
R3-S060	DonkeyJJLove/ai_platform / cyber_lion/contracts/runtime_authority_binding.py ref 0e24fd481697 · snapshot historyczny
R3-S061	DonkeyJJLove/ai_platform / cyber_lion/contracts/runtime_currentness.py ref 0e24fd481697 · snapshot historyczny
R3-S062	DonkeyJJLove/ai_platform / cyber_lion/contracts/runtime_enforcement.py ref 0e24fd481697 · snapshot historyczny
R3-S063	DonkeyJJLove/ai_platform / cyber_lion/contracts/runtime_reconciliation.py ref 0e24fd481697 · snapshot historyczny
R3-S064	DonkeyJJLove/ai_platform / cyber_lion/contracts/semantic_cloud_episode_binding.py ref 0e24fd481697 · snapshot historyczny
R3-S065	DonkeyJJLove/ai_platform / cyber_lion/contracts/semantic_relevance.py ref d367e47cf4f7 · lokalne bytes zgodne z pinned master
R3-S066	DonkeyJJLove/ai_platform / cyber_lion/contracts/semantic_scaffold.py ref d367e47cf4f7 · lokalne bytes zgodne z pinned master
R3-S067	DonkeyJJLove/ai_platform / cyber_lion/contracts/workload_identity.py ref 0e24fd481697 · snapshot historyczny

A08 · Węzły pakietu komunikacji 1/2

Zachowano identyfikatory źródłowego grafu; nie oznaczają nowo zarejestrowanych instancji.

Pakiet rozpoznaje 29 węzłów i 49 relacji. Jego statusy dotyczą własnego kandydata oraz historycznych obserwacji. R3 nie przepisuje ich na bieżący master. Cały model oryginalny jest osadzony jako część niezmienionego ZIP.

operator_shell	Electron Operator Shell AS_IS_R7_SOURCE
operator_panel	Model Chat / LPCL User Interface AS_IS_R7_SOURCE
console_router	Source-grounded Console / LOCAL Cognitive Gateway AS_IS_R7_SOURCE
conversation_domain	Canonical Conversation Domain and Storage AS_IS_R7_SOURCE
chat_producer	Model Chat Producer and Durable Requests AS_IS_R7_SOURCE
delivery_ledger	Canonical Reply and Delivery Ledger AS_IS_R7_SOURCE
communication_envelope	Canonical CommunicationEnvelope AS_IS_INTEGRATED_NON_EFFECT_CONTRACT
semantic_cloud	Semantic Intent, QueryPlan and Relevance AS_IS_INTEGRATED_CONTRACTS_NOT_E2E_ROUTE
holomosaic_hmk9d	HMK-9D Process Semantics FEDERATION_DRAFT_CONTRACT
context_memory	Context, temporal memory and provenance FEDERATION_SOURCE_CONTRACT
cognitive_invocation	Cognitive Routing and Model Invocation Binding AS_IS_R7_CONTRACT
local_model_adapter	LOCAL Cognitive Adapter AS_IS_R7_SOURCE
saas_broker	Mission Control SaaS Broker AS_IS_R7_SOURCE
sentinelx_relay	SentinelX Relay / Turn Transport OBSERVED_PREVIOUS_RUNTIME_SOURCE_SCOPE_PARTIAL
sentinelx_turn_ingress	MOON Ingress / SaaS Turn Ledger OBSERVED_PREVIOUS_RUNTIME

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A09 · Węzły pakietu komunikacji 2/2

Zachowano identyfikatory źródłowego grafu; nie oznaczają nowo zarejestrowanych instancji.

Pakiet rozpoznaje 29 węzłów i 49 relacji. Jego statusy dotyczą własnego kandydata oraz historycznych obserwacji. R3 nie przepisuje ich na bieżący master. Cały model oryginalny jest osadzony jako część niezmienionego ZIP.

browser_consumer	Canonical Conversation SaaS Consumer AS_IS_R7_SOURCE_LIVE_STOPPED
browser_dom	Native ChatGPT Browser DOM Adapter AS_IS_R7_SOURCE_UI_FRAGILE
chatgpt_provider	External ChatGPT SaaS Model Session EXTERNAL_PROVIDER_NOT_AUTONOMOUSLY_ATTESTED
lpcl_compiler	LPCL/Process Contract Compiler AS_IS_R7_SOURCE
mission_control	Mission Control Scheduler / Mission Control Plane AS_IS_R7_SOURCE
participant_protocol	Operator Protocol / Logical Drone Conversation AS_IS_R7_SOURCE
group_channel	Governed Cross-Mission Group Channel AS_IS_PROTOCOL_CONTRACT
logical_drone	Logical Drone Participant Identity AS_IS_R7_SOURCE
material_worker	Material Worker Runtime and LOCAL Inference AS_IS_R7_SOURCE
swarm_mesh	SWARM Federated Runtime Mesh FEDERATION_SOURCE_CONTRACT
authority_effects	Action/PDP Admission and Effect Observation AS_IS_SOURCE_CONTRACTS
federated_research	Source, Delta, Graph, Hypothesis, Simulation and Provenance Providers FEDERATION_SOURCE_CONTRACT
saas_preprepare_gate	Server-Side SAAS Route Admission Before Persist TARGET_NOT_IMPLEMENTED
thread_bootstrap	Project SaaS ConversationBootstrapBinding and Receipt TARGET_NOT_IMPLEMENTED

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A10 · Dziewięć przepływów komunikacji

Każda ścieżka ma osobny zakres dowodu i nie może przejąć wyniku innego przebiegu.

Powtarzający się węzeł w message_flow oznacza kolejne wystąpienie w opisie, nie nową instancję komponentu. Kolejność i rozgałęzienia wynikają z description oraz expects: DUAL ma dwa odrębne legs i join po obu odpowiedziach, nie jeden szereg wywołań. Zachowano wszystkie dziewięć przepływów; diagramy nie są nowym pomiarem runtime.

holomosaic_enrichment	TARGET_NOT_IMPLEMENTED HMK-9D semantic lens over existing envelope and relevance; not an autonomous transport layer.
mission_design_and_execution	AS_IS_CONTRACT_CHAIN_NOT_AUTOMATIC_CHAT_EFFECT Proposal of mission and phase contract precedes distinct operator authority and bounded execution.
model_chat_dual	AS_IS_R7_SOURCE_SAAS_LEG_BLOCKED Two independent LOCAL(K),SAAS(K) legs must both respond before deterministic joined result.
model_chat_local	AS_IS_R7_SOURCE Canonical LOCAL turn with durable request and return event.
model_chat_saas	AS_IS_SOURCE_LIVE_BLOCKED Native project thread through scoped SentinelX and browser DOM; R7 live has nine backlog claims and no proven new E2E response.
new_saas_thread_bootstrap	TARGET_NOT_IMPLEMENTED A source-bound first message makes ChatGPT aware of role/tool scaffolding; missing today as durable cold-start binding.
protocol_fanout	AS_IS_R7_SOURCE Operator addresses exact logical drone and material worker recipients, cognition via shared providers.
reconcile_overdue_claims	OBSERVED_DEFECT_AND_TARGET_REPAIR No blind retry: per-request readback/adjudication before admitting new request.
unbound_console_observe	AS_IS_SOURCE_ONLY Operator can ask the shared LOCAL model about repository state without creating a mission.

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A11 · Crosswalk R2 → rozwinięcia R3

Rodzic jest punktem nawigacji po znaczeniu, nie nowym ograniczeniem kodu.

Numery AT2 odnoszą się do zachowanego oryginału. Wydanie R3 nie zmienia treści poprzednich kart ani wyników ich testów. Rozwinięcia mogą zawężać interpretację lub dodawać konkretny kontrprzykład, ale otrzymują własny identyfikator i źródła.

AT2-P004	R3-D001–D006: NATURA SYSTEMU I PĘTLA EWOLUCJI
AT2-P010	R3-D007–D012: CZAS, HISTORIA I ZMIENNOŚĆ PROBLEMU
AT2-P019	R3-D013–D018: POZIOMY I ZACHOWANIE ZNACZENIA
AT2-P028	R3-D019–D024: GIT JAKO PAMIĘĆ I PERCEPCJA
AT2-P040	R3-D025–D030: AST I GRANICE PERCEPCJI KODU
AT2-P052	R3-D031–D036: ONTOLOGIA, HIPOTEZY I RELEWANCJA
AT2-P055	R3-D037–D042: PAMIĘĆ JAKO UKŁAD MAGAZYNÓW I PROJEKCJI
AT2-P061	R3-D043–D048: PROCES, FAZY I WYKONYWALNOŚĆ
AT2-P062	R3-D049–D054: FABRYKA ARTEFAKTÓW I KOMPOZYCJA
AT2-P069	R3-D055–D060: FEDERACJA I RZECZYWISTA INTEROPERACYJNOŚĆ
AT2-P058	R3-D061–D066: KOMUNIKACJA: TOŻSAMOŚĆ PRZED TRANSPORTEM
AT2-P058	R3-D067–D072: HOLOMOZAIKA JAKO ORGANIZACJA ZNACZENIA
AT2-P058	R3-D073–D078: DELIVERY, BOOTSTRAP I RECOVERY
AT2-P066	R3-D079–D084: AUTHORITY, DOPUSZCZENIE I OBSERWACJA
AT2-P090	R3-D085–D090: NAZEWNICTWO, NORMALIZACJA I CORE
AT2-P096	R3-D091–D096: TRANSFORMACJA: OD BRAKU DO PRZYROSTU
AT2-P109	R3-D097–D102: INTERFEJS, DOKUMENTACJA I NASTĘPNA ITERACJA

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A12 · Ograniczenia i kontynuacja 1/2

Nazwy obszarów zachowują sens checkpointu poprzedniego badania; nie są zamknięte samym opisaniem.

R3 dodaje szczegółową interpretację oraz lokalny wynik testowy, lecz nie zamyka globalnego AS-IS. Dla decyzji zależnej od niezbadanego obszaru należy zatrzymać tę decyzję i kontynuować niezależną pracę. Brakujące evidence nie może zostać zastąpione zgodną odpowiedzią kilku modeli.

U01 · Pełny odczyt dokumentacji ai_platform	Kontynuować ledger na dokładnym drzewie; nie utożsamiać census z odczytem.
U02 · Cała historia i peery	Rozliczyć gałęzie, unikalne bloby i brakujące oryginały; snapshot 268 nie jest całością.
U03 · Dynamiczne połączenia semantyczne	Prześledzić realny consumer i resolver referencji poza bezpośrednim AST.
U04 · Kompletność pakietu Mission Control	Sprawdzić manifest, rzeczywisty import path i sposób raportowania błędu.
U05 · Oryginalne źródło authority	Osobny, aktualny odczyt właściwego grantu, issuer i replay state.
U06 · Sesje i niepewne wysłania SaaS	Per-request adjudication w osobnym mandacie; brak globalnego retry.
U07 · Natywny GUI i intake	Odrębny odbiór bridge, bootstrapu, send i rzeczywistego kontekstu.

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A13 · Ograniczenia i kontynuacja 2/2

Nazwy obszarów zachowują sens checkpointu poprzedniego badania; nie są zamknięte samym opisaniem.

R3 dodaje szczegółową interpretację oraz lokalny wynik testowy, lecz nie zamyka globalnego AS-IS. Dla decyzji zależnej od niezbadanego obszaru należy zatrzymać tę decyzję i kontynuować niezależną pracę. Brakujące evidence nie może zostać zastąpione zgodną odpowiedzią kilku modeli.

U08 · Niezależność verifierów	Zadeklarować poziom kodu, danych, procesu i domeny infrastrukturalnej.
U09 · Relacje infrastruktury sąsiedniej	Nie przypisywać np. Dify do LION wyłącznie przez obecność hostową.
U10 · Brakujący oryginał wcześniejszego R2	Historyczny raport testów nie jest ich reprodukcją.
U11 · Benchmark reprezentacji i transferu	Nie wykonano; zaprojektować porównywalne baseline i rozstrzygające próby.
U12 · Pełny suite repozytorium	18 prób kontraktowych R3 nie zastępuje globalnych testów i exact-head CI.
U13 · Retencja i recovery pamięci	Pełna mapa stores, dostępność payloadów, odtwarzanie i migracja pozostają otwarte.
U14 · Drift kandydatów	Każdy następny efekt wymaga świeżego odczytu source/base/head i zakresu.

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A14 · Rozstrzygnięcia tej generacji

Formalizacja rozróżnia ustalony fakt, zakresowy wniosek i zaprojektowane przejście.

Poniższe rozstrzygnięcia są ograniczone do źródeł i prób R3. Nie stanowią autoryzacji zmian produktu. Szczególnie istotne są granice: koperta nie czyta payloadu, scaffold lokalnie nie domyka wszystkich referencji, a model holomozaikowy nie jest osobnym działającym transportem.

Każdy następny przyrost musi wskazać, które z tych pytań ma rozstrzygnąć i jak zachowa istniejące kontrakty. Nie należy zwiększać liczby modułów, jeśli wystarcza poprawny binding w istniejącym ownerze.

R3-DEC01	Wszystkie 137 stron R2 zachowane; dodatki mają własne ID.
R3-DEC02	L0–L6 pozostają poziomami dokumentacji, nie nowymi warstwami runtime.
R3-DEC03	HMK jako referencjonowana kompozycja; exact Envelope bez dowolnych pól.
R3-DEC04	T10: rozwiązanie payloadu wymaga odrębnego consumera.
R3-DEC05	T16: endpoint closure wymaga wyraźnego kontraktu lokalnych/zewnętrznych refs.
R3-DEC06	LCCP: logiczny agregat, nie dodatkowy scheduler.
R3-DEC07	Stable: profil zakresu i epoki, deployment osobno.
R3-DEC08	Plan transformacji nie został wykonany.

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.

A15 · Checkpoint i protokół następnego kroku

Kontynuacja rozpoczyna się od konkretnej zależności, nie od ponownego tworzenia ogólnego planu.

Pierwszy krok źródłowy: wybrać krytyczny tor Scaffold → RelevanceProjection → invocation, odnaleźć rzeczywiste resolvery rodziców i sprawdzić zachowanie constraints oraz unknowns. R3-T10 i R3-T16 wyznaczają dwa rozstrzygające pytania. Odczyt ma zachować exact source i odrębność kandydata PR456.

Równolegle można kontynuować U01 pełnej dokumentacji oraz rozliczyć U04 import closure. Nie ma powodu ponownie uruchamiać niezmienionych prób tylko dla większego licznika. Zapisuj nowe zakresy odczytu, zmiany źródeł, wnioski i pierwszy niewykonany krok.

Przyszła implementacja poprawki jest osobną misją. Wymaga własnego zakresu, dopuszczenia zapisu, negatywnych testów i odczytu skutku. To wydanie nie aktywuje kolejnego programu, nie kończy pending turns i nie promuje samoczynnie dokumentacji do RAG.

```
CHECKPOINT=LION-ATLAS-R3-SOURCE-EXPANSION-20261014
AS_IS=PARTIAL
DESIGN=CONDITIONAL
PRODUCT_CHANGES=NONE
NEW_TESTS=18 scoped native contract probes
NEXT=critical source binding + real consumer + closure test
```

Orientacja / model wydania R3. Dane wejściowe i zakresy: rejestr źródeł oraz aneksy.